PARAMETRIC HUMAN MODELS

人体参数化技术

从零基础到前沿

人体 · 人脸 · 人手 · 动物

1995 统计形状　2005 形状 / 姿态　2015 SMPL　2026 前沿

> 用少量参数，描述一个高维的人。

Dr. Dezeming

2026 年 · 第一版 · 27 章 + 4 附录 · 83 幅插图 · CC BY-NC-SA 4.0

人体参数化技术

从零基础到前沿：人体 · 人脸 · 人手 · 动物

Dr. Dezeming

2026 年 · 第一版

人体参数化技术 从零基础到前沿：人体 · 人脸 · 人手 · 动物

作者：Dr. Dezeming · 版本：2026 年 · 第一版

© 2026 Dr. Dezeming。本书文字与插图采用 CC BY-NC-SA 4.0 许可（署名—非商业性使用—相同方式共享）发布，转载请注明作者与出处。

本书为学习与研究用途编写。全部插图由作者编写的程序生成，不含任何受限的 SMPL / MANO / FLAME 模型文件；文中提及的论文版权归原作者与出版方所有。

勘误与建议：欢迎在项目仓库提交 Issue。

前 言

这本书写给三类人：刚进实验室、被导师丢了一篇 SMPL 论文的大一/大二学生；做游戏、影视、数字人，需要理解“参数化人体”到底怎么工作的工程师；以及想快速建立全局视野、不想从 1995 年读到 2025 年的研究者。

写作的起点是一个很具体的问题：网上关于 SMPL 的中文资料，大多停在“它有 10 个形状参数、72 个姿态参数”这一层。知道这两个数字，离“能读懂公式、能跑通代码、能判断一篇新论文改了什么”还有很远的距离。尤其是 PCA、协方差、线性混合蒙皮这些前置知识，很多教程默认你会，或者用一句“降维”带过。

所以本书做了三个也许有点笨的决定。

第一，把数学从零讲起。 第 2 章从“一列数”讲到最小二乘，第 3 章用 5 个二维点手算一遍 PCA。你不必先学完线性代数才能读，但读完之后，应该能自己写出 20 行 PCA 代码，并解释每一步。

第二，按“问题—旧方法—局限—新方法”讲史，而不是按论文列表讲。 每一个模型都先说明它要解决前人的什么缺陷。这样你记住的不是一堆缩写，而是一条有因果的脉络。

第三，把失败模式写进去。 每个模型不只讲“它怎么工作”，也讲“它在什么情况下会坏”。遮挡、极端姿势、尺度歧义、数据集偏差，这些内容往往比公式更能帮你在真实任务里做判断。

怎么读

- 零基础：按第 1–6 章顺序读，每章做完思考题；然后选人体、人脸、人手、动物中的一条主线深入。
- 有视觉/图形基础：第 3、5 章快速过，直接从第 7、8 章开始。
- 只想做某个对象：人脸从第 15 章、人手从第 19 章、动物从第 22 章开始；缺的数学回第 2、3 章补。
- 想动手：第 26 章的代码不依赖 SMPL 文件，随时可以插入。

约定

- 向量加粗、矩阵大写加粗、标量斜体；角度默认用弧度。
- 公式编号按章，例如式 (3.8) 指第 3 章第 8 个公式。
- 引用用方括号数字，对应附录 D。
- “SMPL”指原始的身体模型，“SMPL-X”指包含手脸的扩展，二者不混用。

说明

书中的具体数字（顶点数、关节数、参数量）均以原论文为准；不同实现或后续版本可能存在差异，读代码时以实际加载的模型为准。

目　录

前言

第一篇　地基

1　什么是「参数化人体」 1

一个看似简单的问题 1

计算机眼里的“人”：四种表示 2

“参数”到底是什么 2

参数化带来的四件事 3

四条主线，一个地基 4

一条简短的历史线 4

这本书怎么读 5

本章小结 6

思考题 6

2　数学工具箱（一）：向量、矩阵与最小二乘 7

为什么人体可以用一列数字表示 7

向量：箭头与一列数 7

矩阵：一台变形机器 9

把“求解”变成“找最小” 10

矩阵分解：把复杂变换拆成简单部分 12

这一章在整本书里的位置 12

本章小结 12

思考题 13

3　数学工具箱（二）：概率、协方差与 PCA 14

从一个“平均人”说起 14

两个变量：协方差 15

高斯分布：为什么数据像一团椭圆云 17

从原理到五步法（手算一遍） 18

特征向量为什么是“方差最大方向” 22

用 SVD 算 PCA：为什么更稳 25

保留几个主成分 28

重建误差：PCA 到底丢了什么 29
PCA 在人体上长什么样 30
本章小结 31
思考题 31

4 三维世界的表示：点、网格、旋转与相机 32

三维点与坐标系 32
旋转的矩阵写法 33
轴角：更接近直觉的写法 34
齐次坐标与刚体变换 35
相机：三维如何变成二维 36
三角网格：顶点的连接关系 37
可微渲染：把“渲染”变成可求导的函数 38
本章小结 38
思考题 38

5 骨架、蒙皮与线性混合蒙皮（LBS） 39

骨架是一棵树 39
蒙皮：让顶点跟着骨头走 40
为什么 LBS 会出问题 43
对偶四元数蒙皮：把“混合”从矩阵换成刚体变换 45
SMPL 里的 LBS 50
从 LBS 到动作捕捉 50
本章小结 50
思考题 51

6 统计形状模型：从 ASM 到 3DMM 52

一个想法：形状是可以“统计”的 52
对应：一切的起点 53
ASM：在允许的形状里搜索 54
3DMM：把统计形状搬到三维 58
从线性到非线性 59
怎么评价一个统计形状模型 59
本章小结 59
思考题 60

第二篇　人体

7　SCAPE：把「形状」和「姿态」拆开　62

在 SCAPE 之前：只有“形状”，没有“姿势”　62

SCAPE 的核心思想　63

SCAPE 的贡献与代价　65

中间十年：Hasler 与 FAUST　65

为什么 SCAPE 最终输给了 SMPL　66

本章小结　66

思考题　67

8　SMPL：一个公式统治人体　68

为什么还需要一个新模型　68

输入与输出　69

第一步：形状 blendshape　70

第二步：从形状回归关节　71

第三步：姿势 blendshape　71

第四步：LBS　72

SMPL 是怎么训练出来的　72

使用 SMPL 时最容易踩的坑　73

SMPL 的局限与影响　73

本章小结　74

思考题　74

9　SMPL 家族：从身体到手，再到脸　75

一条家族谱系　75

SMPL+H：把手接进来　76

SMPL-X：身体、手、脸的统一　77

STAR：更稀疏、更会“校正”的身体模型　77

GHUM：连续尺度与层级结构　78

2025 之后：ATLAS 与 MHR——把骨架和形状拆开　78

怎么选：一张实用表　80

家族成员的共同点　80

参数是怎么拼起来的：SMPL-X 的一次完整前向　81

换模型时要改哪些代码　82

本章小结 83
思考题 83

10 从单张图片反推人体：优化法与回归法 84

问题定义 84
路线 A：优化法 85
路线 B：回归法 86
两条路线的融合 87
为什么单图重建仍然难 88
优化法复盘：把式（10.1）拆到原子级 88
回归法的病根：歧义与“平均人” 89
本章小结 90
思考题 90

11 视频、多人、全身与度量尺度 92

逐帧重建的三个毛病 92
时序模型：VIBE 与 4D-Humans 92
从相机坐标到世界坐标 93
多人：一次前向 vs 先检测再拟合 94
度量尺度：从“看起来”到“量得准” 94
规模化：SMPLer-X 94
怎么评价视频方法 94
相机运动怎么估：从光流到世界位移 95
时序平滑：平滑了什么，又抹掉了什么 96
本章小结 97
思考题 97

12 数据集、评估指标与常见坑 98

数据集：从实验室到野外 98
五个核心指标 99
论文里常见的评估陷阱 100
一个健康的评估习惯 101
指标是怎么算出来的：五个关节走一遍 101
一个可以直接用的评估脚本 103
本章小结 104

思考题 104

13 着装人体：衣服、头发与神经渲染 105

裸体模型的三个不够 105

路线一：给 SMPL 加位移 106

路线二：隐式表面 106

路线三：神经渲染 107

3D Gaussian Splatting：新的折中 107

把“人”和“衣服/光照”分开 108

选型：什么时候用哪种表示 109

评估与难点 109

本章小结 109

思考题 110

14 人体动作与生成模型 111

动作的表示 111

姿态先验：VPoser 112

运动先验：HuMoR 112

文本生成动作：MDM 与 MotionDiffuse 112

可控性与物理约束 113

为什么动作生成难评估 114

从动作到行为 114

本章小结 115

思考题 115

第三篇 人脸

15 3DMM 深入与 Basel Face Model 117

一张照片，两个世界 117

公式：三个子空间，一套加法 118

最难的一步其实是对应 120

Basel Face Model：把论文变成基础设施 120

表情：从加法到双线性 121

反解：怎么从一张图得到参数 122

3DMM 的四面墙 123

本章小结 124

思考题 124

16 FLAME：脸、颈、表情的统一参数化 125

BFM 缺的三样东西 125

公式：模板 + 三种偏移 + 蒙皮 125

5023、4、15：三个数字怎么来的 127

数据：3800 张头、400 段 4D、33000 次扫描 128

FLAME 与 SMPL-X：缝合点在哪里 129

从 FLAME 到 DECA、EMOCA 129

四面墙与两个突围方向 131

本章小结 131

思考题 132

17 高保真人脸：细节、纹理与身份 133

线性模型到底丢了什么 133

两条数据路线：FaceVerse 与 FaceScape 134

纹理与反射率：被忽视的一半 136

身份与度量：不只是“看着像” 137

神经参数化头模型：跳出拓扑 137

评估：为什么“误差 1 毫米”不能直接比 138

本章小结 139

思考题 139

18 人脸驱动与神经头像 140

三条路线，两种矛盾 140

表情与情绪：几何对了还不够 142

语音驱动：让一张脸说话 142

重演与换脸：把 A 的表情装到 B 身上 144

3DGS 头像：把高斯钉在脸上 145

伦理与安全：可编辑性的另一面 145

本章小结 146

思考题 146

第四篇 人手

19 手的解剖与 MANO 148

手为什么这么难 148
手部解剖：从腕到指尖 150
MANO 的五个部件 150
MANO 与 SMPL：同一套机器，不同的调参 152
从 MANO 到 SMPL-X 的手 153
参数量、单位与坐标约定 154
本章小结 155
思考题 156

20 单手重建：从优化到 Transformer 157

先把问题说清楚 157
深度图时代：把拟合当成搜索 158
单张 RGB 的第一步：把模型塞进网络里 158
从参数到像素：热图、图网络与注意力 159
规模化：HaMeR 160
全身一体化：FrankMocap 与 PyMAF 161
数据从哪来：多视角标注与弱监督 162
评估指标与失败模式 162
本章小结 164
思考题 164

21 手物交互与抓取 165

为什么手和物体必须一起建模 165
数据：四种采集哲学 165
联合重建：从接触损失到类别无关 167
物理约束：穿透、接触与力闭合 167
生成抓取：先造数据，再学估计 169
无标记捕捉与高斯泼溅：MANUS 169
应用与开放问题 170
本章小结 171
思考题 171

第五篇 动物

22 SMAL：四足动物的参数化 173

为什么不能直接套 SMPL 173

SMAL 的构建：从玩具到模型 174
33 个关节与一套拓扑 175
形状空间：物种作为方向 176
配准到底在解什么 176
SMAL 的参数与代码接口 178
从玩具到真实动物：域差距 178
SMAL 的局限 179
本章小结 179
思考题 179

23 动物重建：从 SMALST 到 MagicPony 181

单图动物的四个难点 181
SMALST：用轮廓和关键点做弱监督 182
Dogs Out：把拟合放进训练循环 182
BARC：品种作为先验 183
MagicPony：从图片集合学“怎么动” 183
数据与评估 184
把这些方法写成同一个优化式 185
尺度与相机：单图只能定到“乘积” 186
失败模式 186
本章小结 187
思考题 187

24 跨物种：鸟类、鱼类与动物生成 188

从四足到鸟类、鱼类 188
跨物种：共享什么，不共享什么 189
任务阶梯：从单图到生成 190
生成动物：能做什么，不能做什么 191
应用与伦理 191
四足步态：为什么“会动”比“长得像”更难 191
把一个新物种接进现有流水线：检查清单 193
开放问题 193
本章小结 193
思考题 194

第六篇　前沿与实战

25　生成式人体：从扩散到基础模型 196

三条趋势 196

扩散模型：零基础版 197

文本到 3D：SDS 与它的后代 197

3DGS：实时的照片级化身 198

基础模型如何改变人体重建 198

一个正在形成的“人体基础模型” 199

开放问题 199

本章小结 200

思考题 200

26　动手实践：从 PCA 到拟合 201

环境准备 201

实验一：用 PCA 学一个“迷你形状空间” 201

实验二：二十行实现 LBS 202

实验三：从 2D 关键点拟合形状 203

换成真实的 SMPL 204

三个可以立刻做的练习 205

本章小结 206

思考题 206

27　研究脉络与未来 207

四次范式转移 207

不变的三角：表示、先验、拟合 208

未来可能的方向 208

给读者的建议 208

结语 209

本章小结 209

思考题 209

后 记 227

附　录

附录 A 数学补充 210

附录 B 术语中英对照 213

附录 C 论文时间线与导读 216

附录 D 参考文献 223

第一篇　地基

在谈论 SMPL 之前，先把数学、几何与形变讲清楚

第 1 章　什么是「参数化人体」
第 2 章　数学工具箱（一）：向量、矩阵与最小二乘
第 3 章　数学工具箱（二）：概率、协方差与 PCA
第 4 章　三维世界的表示：点、网格、旋转与相机
第 5 章　骨架、蒙皮与线性混合蒙皮（LBS）
第 6 章　统计形状模型：从 ASM 到 3DMM

第 1 章 什么是「参数化人体」

本章导读：这一章不写公式，只回答三个问题：为什么计算机需要一个“人体模型”？“参数化”到底是什么意思？人体、人脸、人手、动物这四条看似不同的路线，为什么能装进同一本书？读完这一章，你会得到一张全局地图，后面的章节都是在往地图上填细节。

1.1 一个看似简单的问题

请先看一个任务：给你一张普通的生活照，要求你算出照片里这个人的身高、肩宽、腿长，并且让他在电脑里做一个“转身”的动作。

对人来说，这件事平平无奇；对计算机来说，它几乎无从下手。因为照片在计算机眼里只是一个 1920×1080 的数字表格，每个格子记录红绿蓝三个亮度值。它既不知道什么是“人”，也不知道什么是“肩膀”。

更麻烦的是“转身”。如果照片里只有正面，背面长什么样？计算机没有见过，只能猜。而一个真实的 3D 扫描仪可以扫出背面，却要几十万元，还得让人站进一个布满相机的棚子里。

于是研究者换了一个思路：与其每次从零重建一个人，不如事先学好“人”这件事，把它压缩成一组可以调节的旋钮。 这就是参数化人体（parametric human model）的核心思想。

1.2 计算机眼里的“人”：四种表示

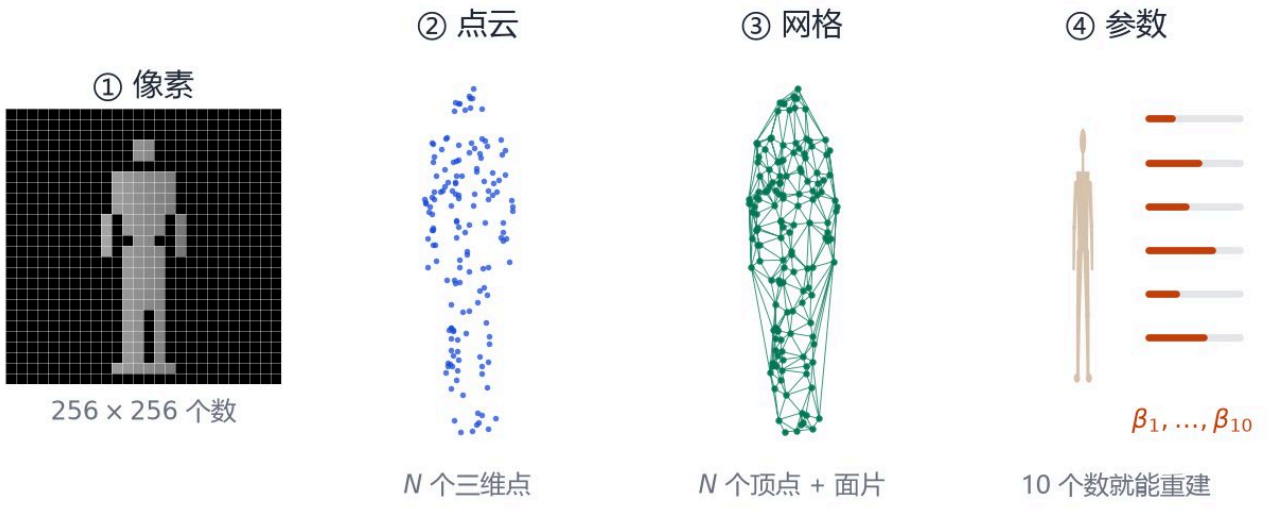


图 1-1 同一个人的四种表示。从左到右，自由度越来越低，可控性越来越强：像素有几十万个数，点云有几千个点，网格有固定的顶点与面片，而参数模型只需要十几个数。

图 1-1 展示了四种常见表示，我们逐一来看。

第一种是像素。 一张 256×256 的灰度图有 65536 个数。它能很好地“看”，却不方便“改”：想让人长高 5 厘米，应该改哪些像素？没人说得清。

第二种是点云。 用深度相机或激光扫描，可以得到人体表面的一堆三维点，每个点有 (x, y, z) 三个坐标。点云很忠实，但它没有“表面”的概念：点与点之间没有连接关系，也就无法直接计算面积、法线，更不能驱动动画。

第三种是网格（mesh）。 把点用三角形连接起来，就得到一张“皮”。网格的顶点数 N 通常是固定的（SMPL 是 6890 个），每个顶点有三维坐标，可以整体做变换、做变形。这是目前工业界最通用的表示：游戏、影视、VR 都用网格。

第四种是参数。 图 1-1 最右边只画了 10 根小滑块。滑动它们，人体的高矮、胖瘦、肩宽、腿长会一起变化。10 个数当然不可能精确描述一个人的每一个细节，但它抓住了主要矛盾：绝大多数人的身材差异，可以被少数几个方向解释。

从 65536 个数，到 6890 个顶点，再到 10 个参数——这条“降维”的路，就是本书要讲的故事。

1.3 “参数”到底是什么

先抛开人体，看一个更简单的例子。

假设你要描述一个圆。你可以列出圆周上 1000 个点的坐标，也可以只写三个数：圆心横坐标、圆心纵坐标、半径。后一种写法就是“参数化”：用少数几个有明确含义的量，生成任意多的点。

人体当然不是圆。人的形状太复杂，找不到一个像“半径”那样简单的几何量。但研究者发现了一件重要的事：

> **如果把很多很多人的三维扫描对齐、叠放在一起，那么“人与人之间的差异”其实集中在一个很小的子空间里。**

这句话是整本书的技术核心，值得慢慢解释。

想象你收集了 1000 个成年人的三维网格，每个网格都有 6890 个顶点，于是每个人可以写成一个长度为 $6890 \times 3 = 20670$ 的向量。1000 个人就是 1000 个 20670 维的向量。听起来维度高得吓人，但它们并不是杂乱无章的：身高、胖瘦、骨架比例这些因素彼此关联，导致这 1000 个向量其实挤在一个低维的“薄片”上。

这个“薄片”，数学上叫子空间。找到它的方法叫主成分分析（Principal Component Analysis, PCA），我们会在第 3 章用整整一章，从“平均数”开始讲起，保证你不需要任何线代基础也能懂。

找到子空间之后，每个人就可以写成：

$$\mathbf{x} \approx \boldsymbol{\mu} + b_1\mathbf{p}_1 + b_2\mathbf{p}_2 + \cdots + b_{10}\mathbf{p}_{10}, \tag{1.1}$$

其中 $\boldsymbol{\mu}$ 是“平均人”，$\mathbf{p}_1, \ldots, \mathbf{p}_{10}$ 是十个“变化方向”（第一主成分、第二主成分……），$b_1, \ldots, b_{10}$ 是每个人在这十个方向上的“刻度”。这十个 b 就是图 1-1 里的滑块。

顺便说一句：式 (1.1) 里的加号是向量加法，$b_1\mathbf{p}_1$ 是数乘向量。如果你现在还不知道这两个词，完全没关系，第 2 章会从“一列数”讲起。

1.4 参数化带来的四件事

为什么大家非要费劲做这件事？因为一旦有了参数，四类原本很难的任务会突然变简单。

第一，压缩。 一个原始扫描动辄几十 MB，而一组参数只有几十个浮点数，几百字节。传输、存储、做数据集都方便得多。

第二，可控。 想让人物“长高一点”“壮一点”“笑一下”，只需要改对应的参数，而不是去手工雕刻几千个顶点。动画师、游戏策划、医生都能用。

第三，可反演。 这是最关键的一点。给定一张照片，我们可以反过来问：哪一组参数，渲染出来最像这张照片？ 于是问题变成了一个优化问题：

$$\min_{\boldsymbol{\beta},\boldsymbol{\theta}} \left\| \mathrm{Render}(\mathrm{Model}(\boldsymbol{\beta},\boldsymbol{\theta})) - \mathrm{Image} \right\|^2. \tag{1.2}$$

只要模型可微、渲染可微，就能用梯度下降求解。2015 年之后的单图人体重建（HMR、SMPLify）、人脸重建（DECA、EMOCA），本质上都在解式 (1.2) 的各种变体。

第四，可生成。 有了低维参数空间，就可以在这个空间里采样、插值、做统计，甚至训练扩散模型。今天文生 3D 人体、动作生成、数字人直播，底层几乎都站着一个参数化模型。

1.5 四条主线，一个地基

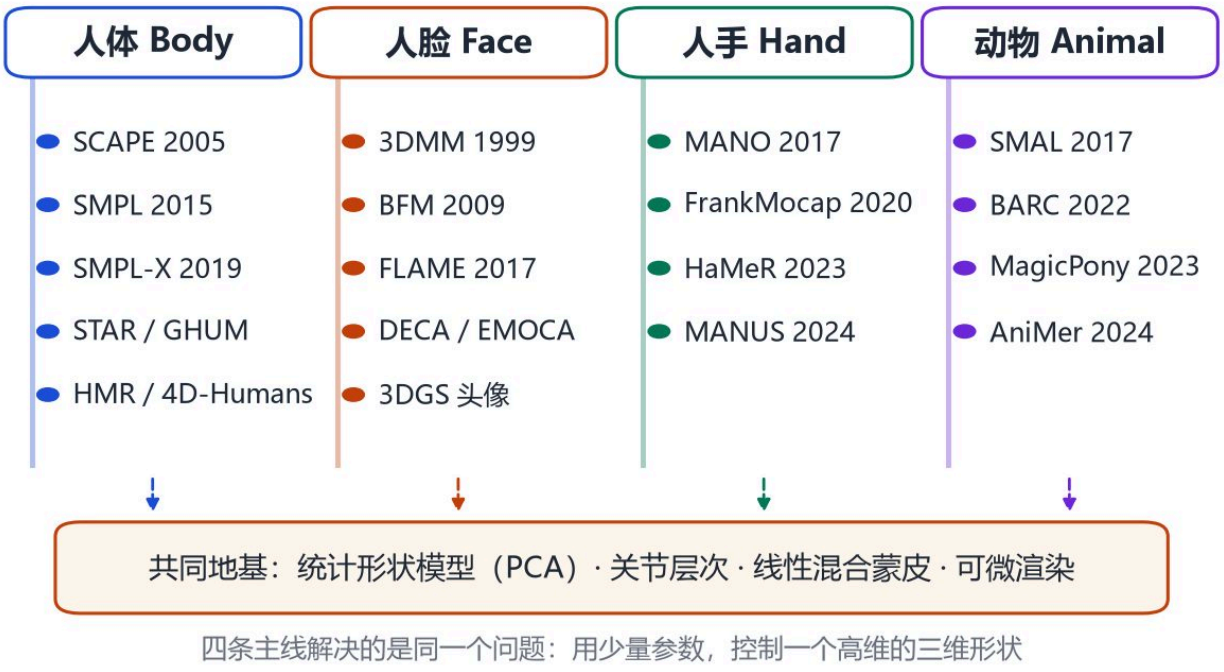


图 1-2 本书的四条技术主线。它们看起来分属不同应用，但共享同一套地基：用 PCA 建立形状空间、用关节层次描述姿态、用线性混合蒙皮把两者合起来、用可微渲染把模型和图像连起来。

本书讲四个对象：人体、人脸、人手、动物。它们各自有一批代表性模型：

- 人体：SCAPE（2005）→ SMPL（2015）→ SMPL-X（2019）→ STAR / GHUM（2020）→ ATLAS / MHR（2025，骨架与形状解耦）→ 单图与视频重建；
- 人脸：3DMM（1999）→ Basel Face Model（2009）→ FLAME（2017）→ DECA / EMOCA → 神经头像；
- 人手：MANO（2017）→ FrankMocap（2020）→ HaMeR（2023）→ 手物交互；
- 动物：SMAL（2017）→ SMALST / BARC → MagicPony（2023）→ AniMer（2024）。

如果你只记住一件事，请记住：它们不是四套互不相干的技术，而是同一套数学在四种对象上的四次落地。 人脸多了表情 blendshape，手多了 21 个关节的精细骨架，动物多了跨物种的形状对应，但“平均形状 + 形变基 + 骨架蒙皮”的骨架没有变。

这也解释了本书的编排：先用六章把地基打牢，再分别进入四条主线，最后在“前沿”一篇把它们重新汇合。

1.6 一条简短的历史线

参数化人体的历史，可以粗略分成四段。

第一段：统计形状的诞生（1990s）。 Cootes 等人提出主动形状模型（ASM），第一次用 PCA 描述一组轮廓的形状变化 [1]；Blanz 与 Vetter 在 1999 年把同样的思想搬到三维人脸，做出 3D 形变模型（3DMM）[2]。这两项工作确立了“平均形状 + 主成分”的范式。

第二段：把形状和姿态分开（2005–2015）。 Allen 等人建立了人体形状空间 [3]，Anguelov 等人的 SCAPE 第一次把“体型”和“姿势”拆成两套参数 [4]，并用三角变形处理关节处的皮肤褶皱。十年后，SMPL 用更简洁的线性混合蒙皮取代 SCAPE 的复杂变形，把模型做到了实时、可微、工业可用 [5]。

第三段：从模型到图像（2016–2020）。 SMPLify 用 2D 关节点反推 3D 人体 [6]，HMR 用神经网络直接回归参数 [7]，SPIN 把两者结合 [8]。人脸有 DECA，手有 FrankMocap，动物有 SMALST [9]。参数化模型从“表示工具”变成了“视觉系统的输出层”。

第四段：神经化与生成化（2020 至今）。 NeRF、3D Gaussian Splatting 把外观细节推到照片级；扩散模型让“用文字生成人体/动作”成为可能；视觉基础模型（SAM、DINOv2、深度大模型）又反过来提升了参数估计的鲁棒性。参数化模型没有被取代，反而成了神经渲染的“骨架”和生成模型的“接口”。

这条线在每一篇里都会重新展开，这里你只需要有一个时间感。

1.7 这本书怎么读

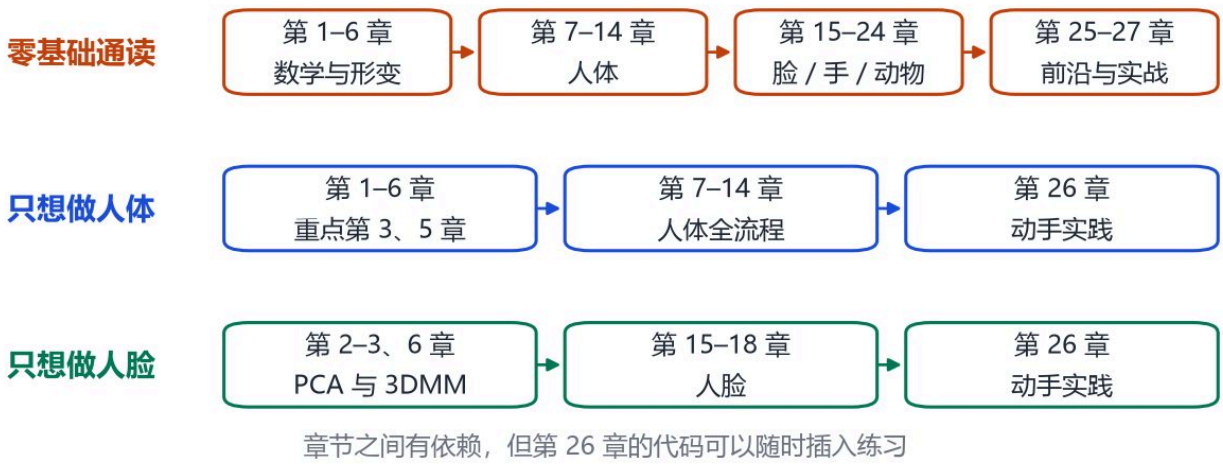


图 1-3 三条推荐阅读路线。第一次接触这个领域，建议按第一条走；已经有明确目标，可以走第二或第三条，遇到缺的数学再回补。

全书共 27 章，分六篇：

- 第一篇（第 1–6 章）地基：向量与矩阵、概率与 PCA、三维表示与旋转、骨架与蒙皮、统计形状模型。这一篇是“必须”，但可以跳读：如果你已经会 PCA，直接看第 3 章的“几何直觉”一节即可。

- 第二篇（第 7 – 14 章）人体：SCAPE、SMPL、SMPL 家族、单图/视频重建、数据集、着装与神经渲染、动作生成。这是全书的主干。
- 第三篇（第 15 – 18 章）人脸：3DMM 深入、FLAME、高保真细节、人脸驱动。
- 第四篇（第 19 – 21 章）人手：手部解剖与 MANO、单手重建、手物交互。
- 第五篇（第 22 – 24 章）动物：SMAL、动物重建、跨物种扩展。
- 第六篇（第 25 – 27 章）前沿与实战：生成式人体、亲手跑代码、研究脉络与未来。

每一章的开头都有“本章导读”，结尾有“本章小结”和几道思考题。思考题不需要写代码，但如果你能用自己的话回答，说明这一章真的读懂了。

本章小结

- 参数化人体用少量参数描述高维的三维形状，核心是“平均形状 + 若干形变方向”的线性模型。
- 它带来四个能力：压缩、可控、可反演（从图像推参数）、可生成。
- 人体、人脸、人手、动物共享同一套地基：PCA 形状空间、关节层次、线性混合蒙皮、可微渲染。
- 技术史可以分成统计形状、形状/姿态分离、模型与图像结合、神经化与生成化四段。

思考题

1. 如果一个模型只有 3 个形状参数，它能描述哪些差异？会丢失哪些差异？
2. 为什么“可微”对从图像反推参数如此重要？如果一个环节不可微，还有什么办法？
3. 人脸和人手都比人体小得多，为什么反而需要更多参数？

第 2 章 数学工具箱（一）：向量、矩阵与最小二乘

本章导读：这一章只做一件事——把“一个三维人体”翻译成一列数字，再讲清楚对这列数字做加、乘、投影、拟合分别意味着什么。你不需要学过线性代数，只要会加减乘除。读完后，你应该能看懂 $S = \bar{S} + B\beta$ 这种写法，并理解为什么“拟合”最后会变成一个求最小值的问题。

2.1 为什么人体可以用一列数字表示

一个三角形有三个顶点，每个顶点有 x, y, z 三个坐标，一共 9 个数。我们可以把它写成：

$$\mathbf{x} = (x_1, y_1, z_1,\ x_2, y_2, z_2,\ x_3, y_3, z_3). \tag{2.1}$$

这就是一个向量：把若干个数按顺序排成一列。人体网格有 6890 个顶点，就对应 $6890 \times 3 = 20670$ 个数，写成向量就是 20670 维。维度高不代表概念难，它只是“很长的一列数”。

这里有一个非常重要的约定：顶点的编号顺序必须固定。第 100 号顶点在每个人身上都必须对应“左手腕内侧”这同一个位置，否则把它们相减没有意义。后面讲的所有模型，都建立在“所有网格已经对齐、顶点一一对应”这个前提上。

2.2 向量：箭头与一列数

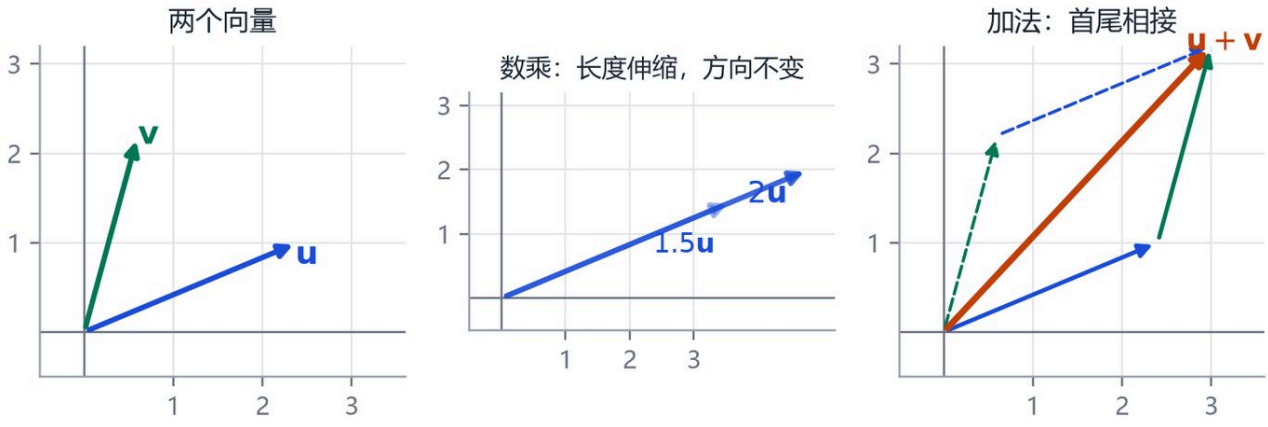


图 2-1 向量的三种基本操作。向量既可以看成一列数，也可以看成从原点出发的箭头；两种看法在计算时等价。

2.2.1 加法与数乘

两个向量相加，就是对应位置相加：

$$(1,2)+(3,-1)=(4,1).$$

几何上，这就是把第二个箭头的起点移到第一个箭头的终点，然后从原点连到新的终点（图 2-1 右）。

数乘更简单：一个数 k 乘向量，就是把每个分量都乘 k：

$$2\cdot(1,2)=(2,4).$$

几何上，方向不变，长度变成原来的 k 倍；$k<0$ 时方向反过来。

这两个操作合起来叫“线性组合”：$a\mathbf{u}+b\mathbf{v}$。整本书里所有“平均形状 + 若干形变方向”的模型，本质上都是线性组合。

2.2.2 点积：投影与相似度

两个向量的点积定义为对应分量相乘再相加：

$$\mathbf{u}\cdot\mathbf{v}=u_1v_1+u_2v_2+\cdots+u_dv_d. \tag{2.2}$$

例如 $(1,2)\cdot(3,1)=1\times3+2\times1=5$。

点积有一个几何解释，这是理解 PCA 的关键：

$$\mathbf{u}\cdot\mathbf{v}=\|\mathbf{u}\|\,\|\mathbf{v}\|\cos\theta, \tag{2.3}$$

其中 $\|\mathbf{u}\|=\sqrt{\mathbf{u}\cdot\mathbf{u}}$ 是长度，θ 是夹角。于是：

- 点积为 0 → 两向量垂直（正交）；
- 点积为正 → 方向大致相同；
- 点积为负 → 方向大致相反。

如果 $\|\mathbf{v}\|=1$，那么 $\mathbf{u}\cdot\mathbf{v}$ 恰好等于 $\mathbf{u}$ 在 $\mathbf{v}$ 方向上的投影长度。把一个向量投影到某个方向上，就是丢掉其他方向的信息，只保留这个方向上的分量。PCA 做的事情，就是找一组“最好的方向”，然后把高维数据投影上去。

2.2.3 距离

两个向量的欧氏距离是

$$\|\mathbf{u}-\mathbf{v}\|=\sqrt{\sum_i(u_i-v_i)^2}. \tag{2.4}$$

它衡量两个形状有多不一样：把两个人体网格展平成向量，算距离，就得到一个“形状差异”的数值。后面评估重建精度时常用的 MPVE、PVE，本质上就是这个距离的平均。

2.3 矩阵：一台变形机器

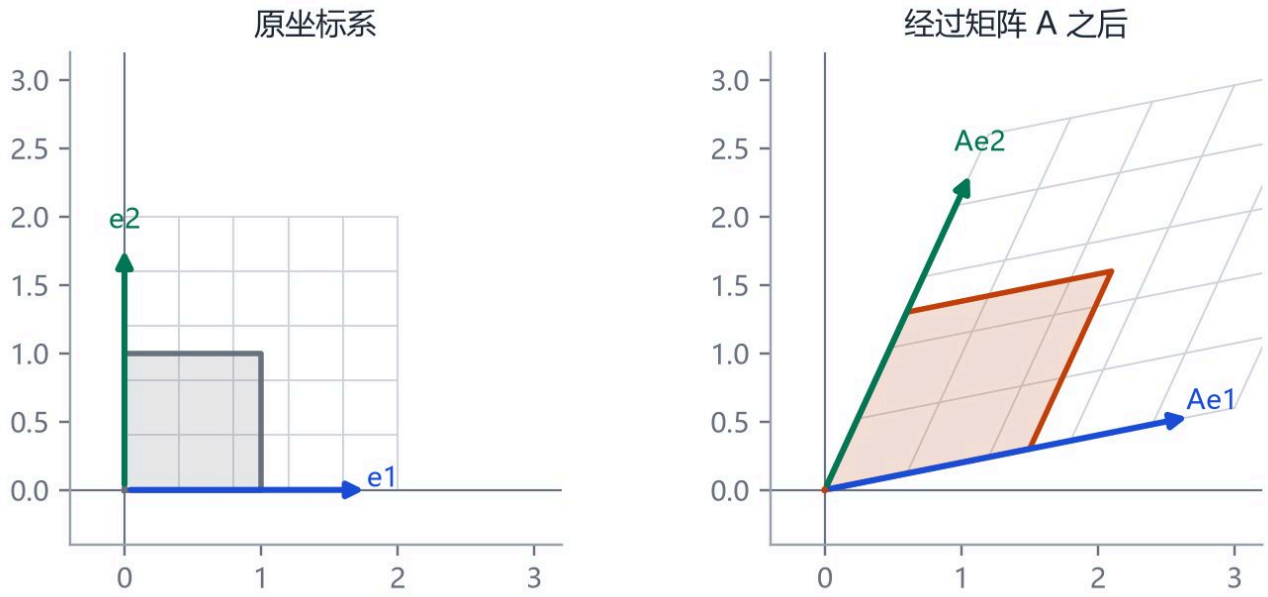


图 2-2 矩阵乘法的几何意义。左图是原来的单位正方形与坐标轴；右图是经过矩阵 A 变换后的结果。整个平面被“拉伸、旋转、错切”，网格线仍然是直线，这就是“线性变换”。

2.3.1 矩阵乘向量

一个 $m \times n$ 的矩阵就是一个 m 行 n 列的数表。矩阵乘向量，得到另一个向量：

$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \times 1 + 1 \times 2 \\ 0 \times 1 + 3 \times 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}. \tag{2.5}$$

计算规则是“行乘列”：结果的第 i 个数，等于矩阵第 i 行与向量做点积。

但更有用的看法是列的组合：

$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 1 \cdot \begin{bmatrix} 2 \\ 0 \end{bmatrix} + 2 \cdot \begin{bmatrix} 1 \\ 3 \end{bmatrix}. \tag{2.6}$$

也就是说，矩阵的每一列都是一个“基方向”，向量里的每个数告诉你“这个方向要走多远”。这正是参数化模型的写法：$B\beta$ 就是“用 β 的每一维去组合 B 的每一列”。

2.3.2 矩阵乘矩阵：复合变换

如果先做变换 A，再做变换 B，合起来就是 BA（注意顺序：右边的先作用）。矩阵乘法不满足交换律，$AB \neq BA$，因为“先旋转再平移”和“先平移再旋转”结果不同。

在人体模型里，每个关节的变换都是一个 4×4 矩阵，它同时包含旋转和平移；一长串父子关节的变换相乘，就得到从局部坐标到世界坐标的完整变换。第 4、5 章会详细讲。

2.3.3 转置、单位矩阵、逆矩阵

- 转置 $\mathbf{A}^\top$：把行和列互换。$(m\times n)$ 变 $(n\times m)$。
- 单位矩阵 $\mathbf{I}$：对角线上全是 1，其余是 0；它乘任何向量都不改变向量。
- 逆矩阵 $\mathbf{A}^{-1}$：满足 $\mathbf{A}^{-1}\mathbf{A}=\mathbf{I}$。只有方阵、且行列式不为 0 时才存在。

逆矩阵的几何意义是“撤销变换”。但实际计算中我们很少真的去求逆：数值上不稳定，通常用解方程或最小二乘代替。

2.3.4 秩与“信息量”

矩阵的秩（rank）可以理解成“变换后还剩几个独立方向”。一个 3×3 矩阵如果秩是 2，它会把整个三维空间压扁成一个平面，很多不同的输入会得到同一个输出，变换就不可逆了。

PCA 里，协方差矩阵的秩最多是 $n-1$（n 是样本数）。这解释了一个常见现象：1000 个样本最多只能张成 999 维，哪怕原始数据有 20670 维。数据本身的“有效维度”远小于表面维度，这正是降维能成立的根本原因。

2.4 把“求解”变成“找最小”

2.4.1 方程无解时怎么办

假设你有 5 个观测点，想用一条直线 $y=a+bx$ 去拟合。每个点都给你一个方程：

$$\begin{cases} a+bx_1=y_1,\\ \vdots\\ a+bx_5=y_5. \end{cases} \tag{2.7}$$

5 个方程、2 个未知数，一般无解——因为点不会恰好共线。写成矩阵形式就是 $\mathbf{A}\mathbf{w}=\mathbf{y}$，其中 $\mathbf{A}$ 是 5×2，$\mathbf{w}=(a,b)^\top$。

既然无解，就退一步：找一个 $\mathbf{w}$，让左右两边“尽量接近”。衡量接近程度最常用的方式是残差平方和：

$$L(\mathbf{w})=\sum_{i=1}^{5}\big(y_i-(a+bx_i)\big)^2=\|\mathbf{A}\mathbf{w}-\mathbf{y}\|^2. \tag{2.8}$$

这就是最小二乘（least squares）。

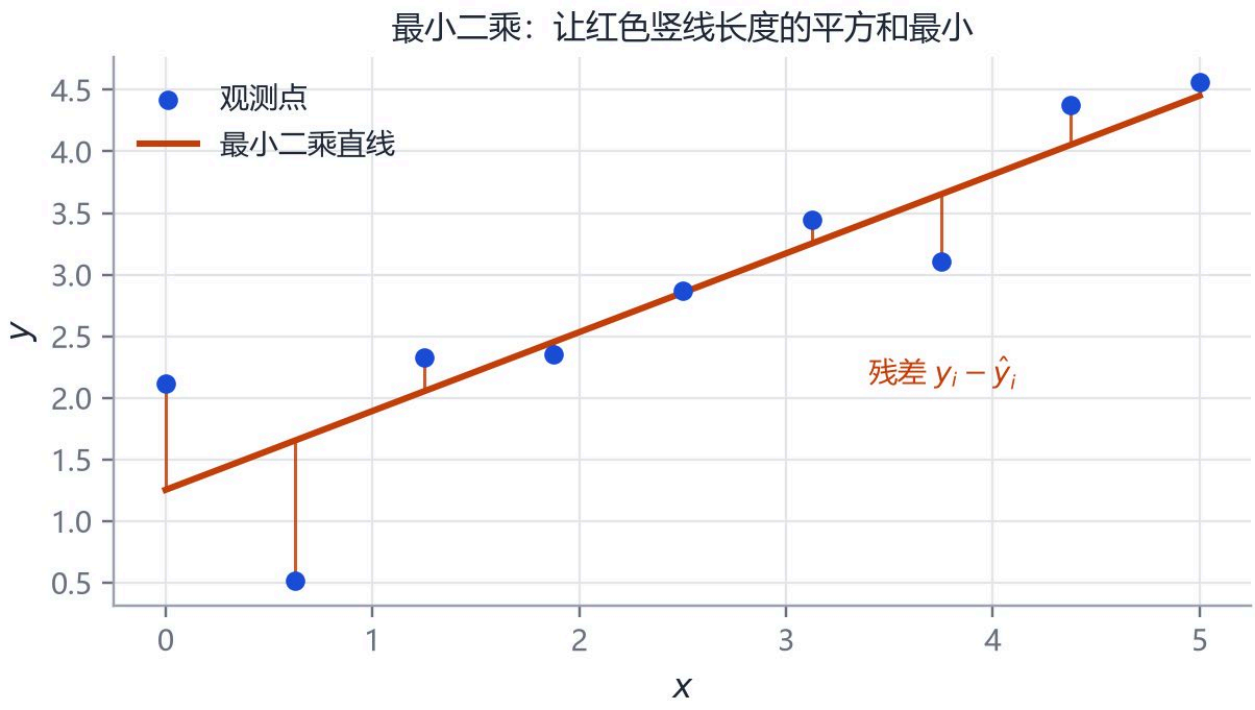


图 2-3 最小二乘的几何直觉。每个红竖线是一个残差；我们要找一条直线，让所有红竖线长度的平方和最小。注意是“平方和”，不是“绝对值和”，后者在 0 点不可导，不便于优化。

2.4.2 怎么求最小值

对 L 关于 $\mathbf{w}$ 求导并令其为 0，可以得到正规方程：

$$\mathbf{A}^\top \mathbf{A}\,\mathbf{w} = \mathbf{A}^\top \mathbf{y}. \tag{2.9}$$

如果 $\mathbf{A}^\top\mathbf{A}$ 可逆，解就是

$$\mathbf{w} = (\mathbf{A}^\top \mathbf{A})^{-1}\mathbf{A}^\top \mathbf{y}. \tag{2.10}$$

这个式子值得多看一眼：$\mathbf{A}^\top\mathbf{y}$ 把观测投影到参数空间，$(\mathbf{A}^\top\mathbf{A})^{-1}$ 再做一次校正。伪逆 $\mathbf{A}^+ = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top$ 就是“当方程无解时，给出最佳折中”的工具。

在 Python 里，你几乎不会手写这个公式，一行代码就够了：

```
import numpy as np
w, *_ = np.linalg.lstsq(A, y, rcond=None)
```

但你必须知道它在做什么：最小二乘把“解方程”换成了“最小化误差”，这是所有模型拟合、神经网络训练的共同起点。

2.4.3 加一点约束：正则化

最小二乘有一个隐患：如果某些方向几乎没有数据，$\mathbf{A}^\top\mathbf{A}$ 接近奇异，解会爆炸。解决办法是加一个“惩罚项”，让参数不要太大：

$$L(\mathbf{w}) = \|\mathbf{A}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2. \tag{2.11}$$

这叫岭回归（ridge regression），解变成

$$\mathbf{w} = (\mathbf{A}^\top\mathbf{A} + \lambda\mathbf{I})^{-1}\mathbf{A}^\top\mathbf{y}. \tag{2.12}$$

在人体重建里，这一项无处不在：$\lambda_\beta\|\boldsymbol{\beta}\|^2$ 防止形状参数跑到不合理的区域，$\lambda_\theta\|\boldsymbol{\theta} - \boldsymbol{\theta}_0\|^2$ 让姿态不要扭曲。第 10 章会看到它的具体形式。

2.5 矩阵分解：把复杂变换拆成简单部分

后面会反复用到两种分解，这里先建立直觉。

特征分解（eigendecomposition）：对方阵 $\mathbf{C}$，找一组方向 $\mathbf{p}_i$ 和对应的数 λ_i，使得

$$\mathbf{C}\mathbf{p}_i = \lambda_i\mathbf{p}_i. \tag{2.13}$$

意思是：矩阵作用在 $\mathbf{p}_i$ 上，只改变长度（乘 λ_i），不改变方向。这样的方向叫特征向量，λ_i 叫特征值。如果 $\mathbf{C}$ 是协方差矩阵，特征值就是“这个方向上的方差”。

奇异值分解（Singular Value Decomposition, SVD）：对任意矩阵 $\mathbf{X}$（不必是方阵），都可以写成

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top, \tag{2.14}$$

其中 $\mathbf{U}, \mathbf{V}$ 的列互相正交，$\mathbf{S}$ 是对角矩阵，对角线上的数叫奇异值。SVD 是 PCA 的数值实现方式，第 3 章会用它避免显式构造巨大的协方差矩阵。

2.6 这一章在整本书里的位置

你现在已经具备了三样工具：

1. 用向量表示形状；
2. 用矩阵表示变换与线性组合；
3. 用最小二乘把“拟合”变成“最小化”。

第 3 章会用这三样工具回答一个问题：给定一大堆人体，如何自动找到那组“最好的方向”？答案就是 PCA。第 5 章会把矩阵变换用在骨架上，得到线性混合蒙皮。第 8 章的 SMPL，本质上就是“PCA 形状空间 + 骨架变换 + 最小二乘拟合”的组合。

本章小结

- 形状可以展平成向量，前提是所有网格顶点一一对应。
- 矩阵乘向量是“列的线性组合”；矩阵乘矩阵是“变换的复合”。
- 点积衡量投影与相似度；正交方向互不影响。
- 方程无解时，最小二乘给出残差平方和最小的折中解；加正则项可以防止过拟合。

- 特征分解找“不被矩阵转向的方向”，SVD 把任意矩阵拆成旋转—缩放—旋转。

思考题

1. 一个 2×2 矩阵把单位正方形变成面积为 3 的平行四边形，它的行列式是多少？
2. 为什么最小二乘用平方而不是绝对值？提示：从求导和异常值两个角度想。
3. 如果两个人体网格的顶点顺序不一致，直接相减会发生什么？可以怎样补救？

第 3 章 数学工具箱（二）：概率、协方差与 PCA

> **本章导读**：这是全书最重要的一章。PCA 是 3DMM、SMPL、FLAME、MANO、SMAL 共同的心脏，但它经常被一句“降维”带过。本章从“平均身高”讲起，一步一步搭出协方差矩阵、特征向量和投影公式，并用 5 个二维点手算一遍。读完这一章，你应该能自己写出 20 行 PCA 代码，并解释每一步在做什么。

3.1 从一个“平均人”说起

假设你量了 5 个人的身高：170、175、180、165、185 厘米。你大概会立刻说出“平均 175”。这个“平均”就是均值（mean）：

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i. \tag{3.1}$$

但只报均值会丢掉信息。170 和 180 的平均是 175，175 和 175 的平均也是 175，可这两组人显然不一样。于是我们需要第二个数，描述“数据离均值有多远”。最自然的想法是把每个偏差加起来，但正负会抵消（$(-5)+5=0$），所以改成平方后再平均：

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - \mu)^2. \tag{3.2}$$

这叫方差（variance），它的平方根 σ 叫标准差（standard deviation）。方差越大，数据越分散。

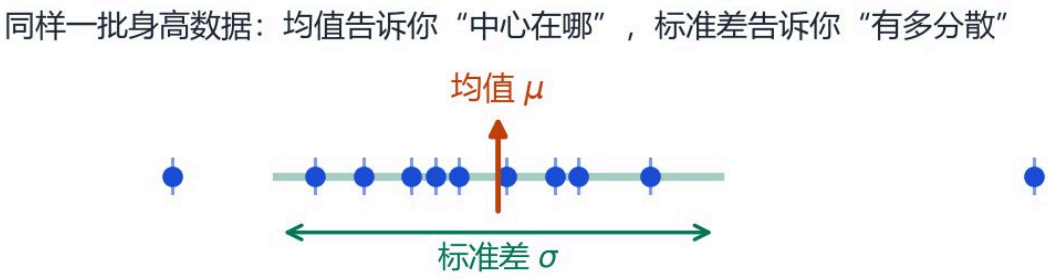


图 3-1 一维数据的均值与标准差。蓝点是观测值，橙色箭头指向均值，绿色线段是 ±1 个标准差的范围。绝大多数观测会落在这个范围内（高斯分布下约 68%）。

注意图 3-1 里最左和最右各有一个“离群点”。它们会把均值往自己那边拉，也会显著增大方差。真实的人体扫描数据里同样有离群样本（扫描失败、姿势不对、穿帮），所以做 PCA 之前通常要先清洗数据。

3.2 两个变量：协方差

人体不只有身高。假设我们同时记录身高 x_1 和肩宽 x_2。这两个量往往一起变大：高的人通常肩也宽。怎么用数字描述“一起变”？

仿照方差的思路，把两个变量的偏差乘起来再平均：

$$\operatorname{Cov}(x_1, x_2) = \frac{1}{n-1} \sum_{i=1}^{n} (x_{1i} - \mu_1)(x_{2i} - \mu_2). \tag{3.3}$$

这个量叫协方差（covariance）。它的符号很有信息量：

- 如果 x_1 高于均值时 x_2 也倾向于高于均值，乘积为正，协方差为正；
- 如果一个大另一个就小，乘积为负，协方差为负；
- 如果两者无关，正负相互抵消，协方差接近 0。

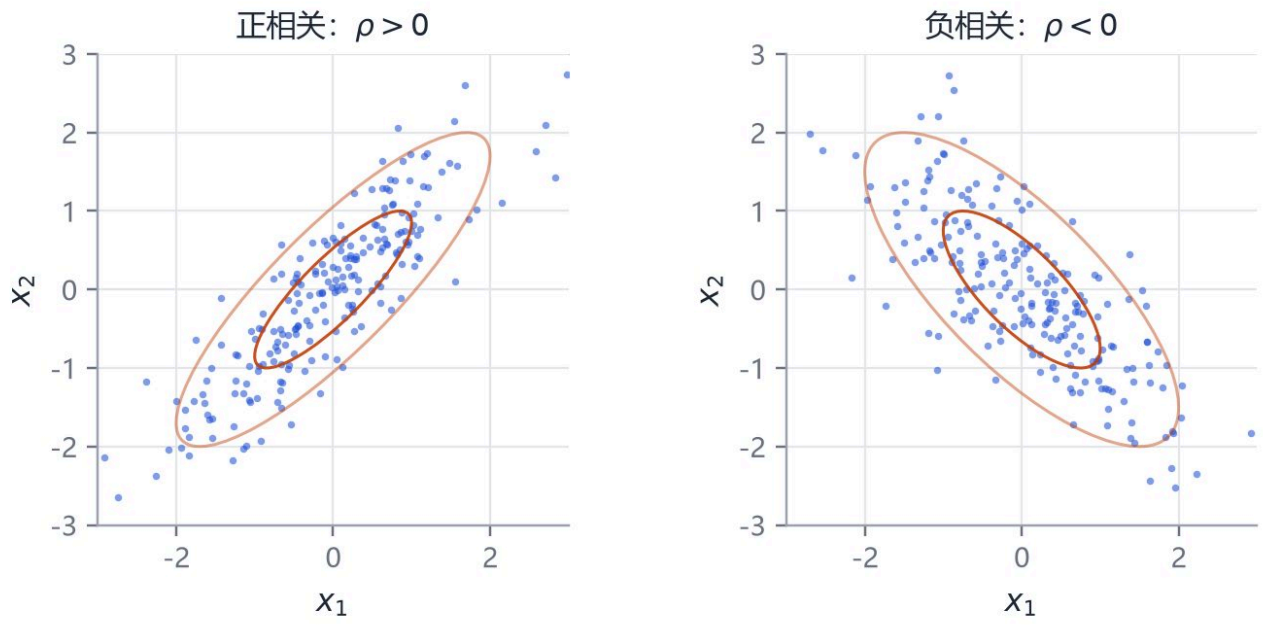


图 3-2 协方差的几何样子。左图正相关，点云沿右上方向拉长；右图负相关，沿右下方向拉长。橙色椭圆是高斯分布的 1σ 与 2σ 等高线，它的倾斜方向由协方差决定。

把协方差除以两个标准差的乘积，就得到相关系数 $\rho \in [-1, 1]$。它去掉了量纲，只保留“线性相关程度”。$\rho = 1$ 表示完全正相关，$\rho = 0$ 表示没有线性关系（注意：不一定独立）。

3.2.1 协方差矩阵

当变量有 d 个时，两两之间都有一个协方差，把它们排成一个 $d \times d$ 的表：

$$\mathbf{C} = \begin{bmatrix} \mathrm{Cov}(x_1, x_1) & \mathrm{Cov}(x_1, x_2) & \cdots & \mathrm{Cov}(x_1, x_d) \\ \mathrm{Cov}(x_2, x_1) & \mathrm{Cov}(x_2, x_2) & \cdots & \mathrm{Cov}(x_2, x_d) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(x_d, x_1) & \mathrm{Cov}(x_d, x_2) & \cdots & \mathrm{Cov}(x_d, x_d) \end{bmatrix}. \tag{3.4}$$

对角线是每个变量自己的方差，非对角线是两两协方差。这个矩阵是对称的（$\mathbf{C}^\top = \mathbf{C}$），而且“半正定”——这两个性质保证了它一定能做特征分解，且特征值非负。

写成矩阵形式更紧凑。设 $\tilde{\mathbf{X}}$ 是中心化后的数据矩阵（每一行是一个样本，每一列是一个变量，且每列均值为 0），那么

$$\mathbf{C} = \frac{1}{n-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}. \tag{3.5}$$

这个式子是 PCA 计算的起点，值得记住：协方差矩阵 = 中心化数据的内积，再除以样本数减一。

补一个常被跳过的问题：为什么协方差矩阵的秩最多是 $n-1$？

（第 2 章 2.3.4 只给了结论，这里补上原因。）

秩（rank）是“线性无关的方向数”。$\mathbf{C}$ 的秩等于中心化数据矩阵 $\tilde{\mathbf{X}}$ 的秩（乘一个非零常数不改变秩）。而 $\tilde{\mathbf{X}}$ 的每一列（每个变量）都减去了自己的均值，于是把所有样本的向量加起来正好是零向量：

$$\sum_{i=1}^{n} \tilde{\mathbf{x}}_i = \mathbf{0}. \tag{3.6}$$

这就是一条线性约束：最后一个样本可以由前面 $n-1$ 个拼出来（$\tilde{\mathbf{x}}_n = -\sum_{i=1}^{n-1} \tilde{\mathbf{x}}_i$），所以 n 个样本最多只有 $n-1$ 个独立方向。一句话：中心化把数据按掉了一个维度。

两个直接推论：

1. 协方差矩阵最多只有 $n-1$ 个非零特征值，所以样本数决定了可保留主成分数的上限（$k \le n-1$）——1000 个样本最多张成 999 维，哪怕原始数据有 20670 维；
2. 除以 $n-1$ 而不是 n，正是因为独立方向少了一个（无偏估计，3.7 节会再提到）。

3.3 高斯分布：为什么数据像一团椭圆云

如果一维数据的直方图中间高、两边低，我们常用高斯分布（正态分布）描述它：

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2} \right). \tag{3.7}$$

不用被公式吓到，它只说了三件事：以 μ 为中心；σ 越大越胖；离中心越远，概率按指数的平方下降。

推广到 d 维，高斯分布由均值向量 $\boldsymbol{\mu}$ 和协方差矩阵 $\mathbf{C}$ 完全确定：

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\mathbf{C}|^{1/2}} \exp\left(-\frac{1}{2} (\mathbf{x}-\boldsymbol{\mu})^{\top} \mathbf{C}^{-1} (\mathbf{x}-\boldsymbol{\mu}) \right). \tag{3.8}$$

指数里的 $(\mathbf{x}-\boldsymbol{\mu})^{\top}\mathbf{C}^{-1}(\mathbf{x}-\boldsymbol{\mu})$ 是一个“马氏距离”的平方（Mahalanobis distance）：它量的不是到中心的欧氏距离，而是按数据自己的尺度有多罕见。下面把这两个词（二次型、马氏距离）拆开讲清楚。

3.3.1 “二次型”是什么

先把词解释掉。给定一个对称矩阵 $\mathbf{A}$（$\mathbf{A}^{\top} = \mathbf{A}$）和一个向量 $\mathbf{x}$，下面这个式子叫 $\mathbf{A}$ 的二次型：

$$q(\mathbf{x}) = \mathbf{x}^{\top} \mathbf{A} \mathbf{x} = \sum_{i} \sum_{j} A_{ij}\, x_i x_j. \tag{3.9}$$

它之所以叫“二次”，是因为每一项都恰好是两个分量的乘积（$x_i x_j$），既没有常数项，也没有一次项。二维时展开最清楚：

$$\mathbf{x}^{\top}\mathbf{A}\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = a x_1^2 + 2b\, x_1 x_2 + c\, x_2^2. \tag{3.10}$$

（交叉项系数是 $2b$：矩阵里对称位置的两个 b 各贡献一次。）

两个例子就能看出它在干什么：

- $\mathbf{A}=\mathbf{I}$ 时，$q=x_1^2+x_2^2$，就是“到原点的距离平方”，等值线 $q=1$ 是圆；
- $\mathbf{A}=\begin{bmatrix}4&0\\0&1\end{bmatrix}$ 时，$q=4x_1^2+x_2^2$，等值线 $q=1$ 是椭圆：横向半轴 $1/2$、纵向半轴 1。也就是“横方向被放大了 4 倍”——横向稍微偏一点，q 就涨得很快。

一句话：二次型是“带权重的距离平方”，不同方向的权重不一样。当等值线是椭圆时，椭圆的轴就是 $\mathbf{A}$ 的特征向量。

3.3.2 马氏距离：把每个方向按自己的尺度归一化

回到式（3.8）指数里的那一项：

$$(\mathbf{x}-\boldsymbol{\mu})^\top\mathbf{C}^{-1}(\mathbf{x}-\boldsymbol{\mu}),\tag{3.11}$$

它正是以 $\mathbf{C}^{-1}$ 为矩阵的二次型——一个加权的距离平方。为什么用逆？因为在方差大的方向上数据本来就铺得开，除以一个大的方差，惩罚就变小。

用数字体会。设 $\mathbf{C}=\begin{bmatrix}4&0\\0&1\end{bmatrix}$（横向方差 4、纵向方差 1），则 $\mathbf{C}^{-1}=\begin{bmatrix}1/4&0\\0&1\end{bmatrix}$，于是

$$(\mathbf{x}-\boldsymbol{\mu})^\top\mathbf{C}^{-1}(\mathbf{x}-\boldsymbol{\mu})=\frac{\Delta x_1^2}{4}+\Delta x_2^2.\tag{3.12}$$

两个点到均值的欧氏距离都是 2，但含义完全不同：

偏移 Δ	欧氏距离	马氏距离²	含义
$(2,0)$：横向偏 2	2	$4/4=1$	横向本来就能散到 ±2，不算稀奇
$(0,2)$：纵向偏 2	2	$0+4=4$	纵向标准差只有 1，偏 2 已是 2σ，很少见

所以“更正常”的意思是：同样是偏离 2 个单位，落在“本来就爱散开的方向”上就不稀奇。马氏距离量的不是“离得多远”，而是“按数据自己的尺度，这个偏离有多罕见”——等价地说，把每个方向除以自己的标准差，椭圆就被拉成了圆。

这也解释了式（3.8）的等高线为什么是椭圆：$\mathbf{C}$ 正定时，二次型等于常数是一条椭圆；长轴方向就是 $\mathbf{C}$ 最大特征值对应的特征向量，半轴长度正比于 $\sqrt{\lambda_i}$（特征值越大，同样概率下能伸得越远）。这正是 PCA 的几何根源：数据云最长的方向，就是第一主成分。

3.4 从原理到五步法（手算一遍）

3.4.1 先想清楚：我们要的是什么

二维点 (x_1,x_2) 要用两个数描述。如果这些点几乎排在一条斜线上，我们希望只用一个数就大致说清“它在哪”——这就是降维。

做法是：选一条过均值 $\boldsymbol{\mu}$ 的单位方向 $\mathbf{p}$（$\|\mathbf{p}\|=1$），把每个点“投影”到这条直线上。投影点写成

$$\hat{\mathbf{x}} = \boldsymbol{\mu} + b\,\mathbf{p}, \tag{3.13}$$

其中 b 是一个标量。这个 b 就是本节的主角：它是这个点在 $\mathbf{p}$ 方向上的带符号坐标（长度为 $|b|$，正负表示在均值两侧的哪一边）。$\mathbf{p}$ 一旦定下来，一个二维点就被压缩成了一个数 b。

3.4.2 b 怎么算：一次最小二乘

b 应该取多少？自然的要求是：投影点离原样本最近。

$$\min_{b} \; \big\|\mathbf{x} - (\boldsymbol{\mu} + b\mathbf{p})\big\|^2. \tag{3.14}$$

对 b 求导并令其为零（注意 $\mathbf{p}^\top\mathbf{p}=1$）：

$$-2\,\mathbf{p}^\top\big(\mathbf{x}-\boldsymbol{\mu}-b\mathbf{p}\big)=0 \quad\Longrightarrow\quad b=\mathbf{p}^\top(\mathbf{x}-\boldsymbol{\mu}). \tag{3.15}$$

这就是式 (3.15)。所以“编码”没有任何魔法：把中心化后的样本投影到主成分方向上，量一下有多远。

3.4.3 为什么要取“方差最大”的方向

$\mathbf{p}$ 有无数种选法，凭什么选特征向量？因为下面两个标准其实是同一件事：

1. 让 b 越分散越好（投影后保留的信息最多）：最大化 $\mathrm{Var}(b)$；
2. 让重建误差越小越好：最小化 $\|\mathbf{x}-\hat{\mathbf{x}}\|^2$ 的平均。

先把“残差”说清楚。 投影点 $\hat{\mathbf{x}}=\boldsymbol{\mu}+b\mathbf{p}$ 与真实点 $\mathbf{x}$ 的差，叫残差（residual）：

$$\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}} = \mathbf{x} - \boldsymbol{\mu} - b\mathbf{p}. \tag{3.16}$$

它表示“投影没有解释掉的那部分”，重建误差就是它的长度平方 $\|\mathbf{r}\|^2$。

为什么残差与 $\mathbf{p}$ 垂直？ 因为 3.4.2 里 b 正是用“让 $\|\mathbf{r}\|^2$ 最小”定出来的，最优解满足一阶条件（也就是式 (3.15) 中间那一步）：

$$\mathbf{p}^\top\big(\mathbf{x}-\boldsymbol{\mu}-b\mathbf{p}\big) = \mathbf{p}^\top\mathbf{r} = 0 \qquad (\text{这里用了 } \mathbf{p}^\top\mathbf{p}=1). \tag{3.17}$$

内积为零就是垂直，所以 $\mathbf{r}\perp\mathbf{p}$。而 $\hat{\mathbf{x}}-\boldsymbol{\mu}=b\mathbf{p}$ 正好沿着 $\mathbf{p}$，因此 $\mathbf{r}$ 也垂直于 $\hat{\mathbf{x}}-\boldsymbol{\mu}$。几何上这就是“从一点向一条直线作垂线，垂足是最近点”——“垂直”与“最短”在这里是同一件事。

勾股关系是怎么来的？ 把“样本到均值的向量”拆成两段：

$$\mathbf{x}-\boldsymbol{\mu} = \underbrace{(\hat{\mathbf{x}}-\boldsymbol{\mu})}_{=b\mathbf{p}\text{，沿 }\mathbf{p}} + \underbrace{\mathbf{r}}_{\perp\mathbf{p}}. \tag{3.18}$$

两边平方，交叉项因为 $\mathbf{p}^\top\mathbf{r}=0$ 而整项消失：

$$\|\mathbf{x}-\boldsymbol{\mu}\|^2 = b^2\|\mathbf{p}\|^2 + 2b\,\mathbf{p}^\top\mathbf{r} + \|\mathbf{r}\|^2 = b^2 + \|\mathbf{r}\|^2. \tag{3.19}$$

勾股定理能用的唯一原因，就是残差与 $\mathbf{p}$ 垂直。 如果换一个不是最小二乘解的 b（比如随手取 $b=1$），残差就不再垂直，这个等式也就不成立。

拿一个具体点验算。 取 3.4.4 的样本 $(4,4)$：中心化后 $\mathbf{x}-\boldsymbol{\mu}=(1,1.4)$，第一主成分 $\mathbf{p}_1\approx(0.766,0.643)$，于是 $b\approx1.667$，$\hat{\mathbf{x}}-\boldsymbol{\mu}=b\mathbf{p}_1\approx(1.276,1.071)$，残差 $\mathbf{r}\approx(-0.276,0.329)$。

- 垂直：$0.766\times(-0.276)+0.643\times0.329\approx-0.211+0.212\approx0$ ✓
- 勾股：$1^2+1.4^2=2.96$；而 $b^2\approx2.78$、$\|\mathbf{r}\|^2\approx0.18$，$2.78+0.18\approx2.96$ ✓

于是两个标准严格等价。 对每个样本，式（3.19）的左边 $\|\mathbf{x}-\boldsymbol{\mu}\|^2$ 只由数据决定，与方向 $\mathbf{p}$ 完全无关；右边是“保留的 b^2”加“丢掉的 $\|\mathbf{r}\|^2$”，两者此消彼长、和固定。把 n 个样本累加、再除以 $n-1$：

$$\underbrace{\frac{1}{n-1}\sum_i\|\mathbf{x}_i-\boldsymbol{\mu}\|^2}_{=\operatorname{tr}(\mathbf{C})，\text{与 }\mathbf{p}\text{ 无关}}=\underbrace{\operatorname{Var}(b)}_{\text{保留的}}+\underbrace{\frac{1}{n-1}\sum_i\|\mathbf{r}_i\|^2}_{\text{丢掉的}}.\tag{3.20}$$

左边正好是协方差矩阵的迹 $\operatorname{tr}(\mathbf{C})=\lambda_1+\cdots+\lambda_d$（对角线之和 = 总方差）。所以“最大化 $\operatorname{Var}(b)$”和“最小化平均残差”是同一件事。而把 $b=\mathbf{p}^\top(\mathbf{x}-\boldsymbol{\mu})$ 代进去，$\operatorname{Var}(b)$ 就是二次型

$$\mathbf{p}^\top\mathbf{C}\,\mathbf{p},\qquad\|\mathbf{p}\|=1,\tag{3.21}$$

也就是瑞利商：它的最大值恰好在 $\mathbf{C}$ 最大特征值对应的特征向量处取到，最大值就是 λ_1（证明见 3.5 节）。这就是“第一主成分 = 最大特征向量”的来历：不是规定，而是“投影后最分散”这个要求逼出来的。

> 顺带回答一个常见疑问：为什么不用“让 b 的绝对值最大”？因为单个样本的 b 可正可负、还能被整体缩放，没有意义；只有**一组样本的分散程度**才是与坐标系无关的量。

3.4.4 五个步骤（把上面的话变成可算的式子）

下面用 5 个二维点走完整流程。点集是：

$$(1,1),\ (2,2),\ (3,2),\ (4,4),\ (5,4).$$

第一步：求均值。

$$\mu_1=\frac{1+2+3+4+5}{5}=3,\qquad\mu_2=\frac{1+2+2+4+4}{5}=2.6.$$

第二步：中心化。 每个点减去均值：

$$(-2,-1.6),\ (-1,-0.6),\ (0,-0.6),\ (1,1.4),\ (2,1.4).$$

第三步：算协方差矩阵。 用式（3.5）：

$$\mathbf{C}=\frac{1}{4}\begin{bmatrix}4+1+0+1+4 & 3.2+0.6+0+1.4+2.8\\ 3.2+0.6+0+1.4+2.8 & 2.56+0.36+0.36+1.96+1.96\end{bmatrix}=\begin{bmatrix}2.5 & 2.0\\ 2.0 & 1.8\end{bmatrix}.$$

第四步：特征分解。 解 $\det(\mathbf{C}-\lambda\mathbf{I})=0$：

$$(2.5-\lambda)(1.8-\lambda)-4=0 \;\Longrightarrow\; \lambda^2-4.3\lambda+0.5=0.$$

得到 $\lambda_1\approx 4.181$，$\lambda_2\approx 0.119$。对应的单位特征向量是

$$\mathbf{p}_1\approx(0.766,\,0.643)^\top,\qquad \mathbf{p}_2\approx(-0.643,\,0.766)^\top.$$

$\mathbf{p}_1$ 指向右上，正是点云拉长的方向；$\mathbf{p}_2$ 与它垂直。两个特征值加起来 4.3 正好等于矩阵的迹（对角线之和），也就是总方差。

第五步：投影与重建。 取第一个主成分，任意点 $\mathbf{x}$ 的编码就是 3.4.2 推出的那个 b（式 (3.15)）：

$$b_1=\mathbf{p}_1^\top(\mathbf{x}-\boldsymbol{\mu}),$$

重建是

$$\hat{\mathbf{x}}=\boldsymbol{\mu}+b_1\mathbf{p}_1. \tag{3.22}$$

以 $(4,4)$ 为例，中心化后是 $(1,1.4)$，$b_1=0.766\times 1+0.643\times 1.4\approx 1.667$，重建得到 $(4.28,3.67)$，与原点相差约 $(0.28,-0.33)$。这点误差就是“只用一个方向”所丢掉的细节。

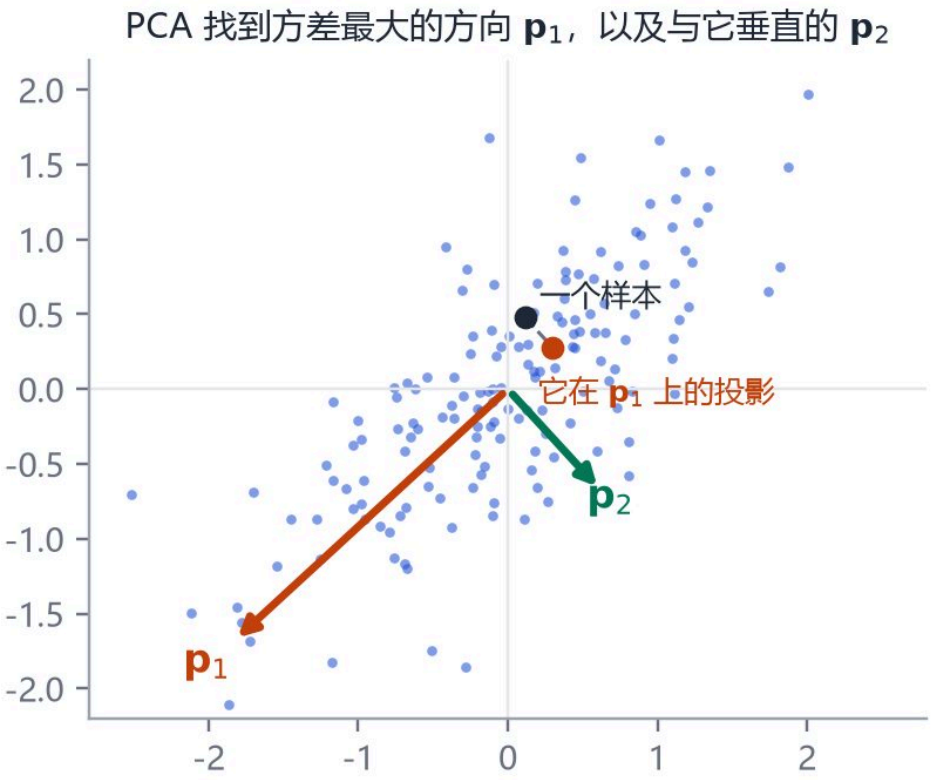


图 3-3 PCA 的几何操作。橙色箭头 $\mathbf{p}_1$ 是方差最大的方向，绿色箭头 $\mathbf{p}_2$ 与它垂直。任意样本（黑点）投影到 $\mathbf{p}_1$ 上，得到一个一维坐标 b_1；只用 b_1 就能近似原来的二维点。

把上面的流程写成代码，只有十几行：

```
import numpy as np

def pca(X, k):
    mu = X.mean(axis=0)          # 1. 均值
    Xc = X - mu                  # 2. 中心化
    C = Xc.T @ Xc / (len(X) - 1) # 3. 协方差矩阵
    w, V = np.linalg.eigh(C)     # 4. 特征分解（升序）
    idx = np.argsort(w)[::-1]
    w, V = w[idx], V[:, idx]
    P = V[:, :k]                 # 前 k 个主成分
    B = Xc @ P                   # 5. 编码
    Xr = mu + B @ P.T            #    重建
    return mu, P, w, B, Xr
```

3.5 特征向量为什么是“方差最大方向”

这一节把 3.4.3 的结论证明掉：在所有单位方向里，让投影方差最大的那个方向，就是协方差矩阵最大特征值对应的特征向量。 只用得到“矩阵乘向量”和“特征值是什么”，值得跟着走一遍。

3.5.1 先把“投影方差”写成一个矩阵表达式

设数据已经中心化（均值是 $\mathbf{0}$），样本记作 $\mathbf{x}_1,\dots,\mathbf{x}_n$。任取一个单位方向 $\mathbf{p}$，每个样本在它上面的投影（就是 3.4.2 的 b）是 $b_i=\mathbf{p}^\top\mathbf{x}_i$。

按方差的定义（式 (3.2)），这 n 个投影数的方差是

$$\frac{1}{n-1}\sum_{i=1}^{n} b_i^2=\frac{1}{n-1}\sum_{i=1}^{n}\mathbf{p}^\top\mathbf{x}_i\mathbf{x}_i^\top\mathbf{p}=\mathbf{p}^\top\Big(\frac{1}{n-1}\sum_{i=1}^{n}\mathbf{x}_i\mathbf{x}_i^\top\Big)\mathbf{p}=\mathbf{p}^\top\mathbf{C}\,\mathbf{p}. \tag{3.23}$$

三步都只是“把跟 i 无关的东西提到求和号外面”：$\mathbf{p}$ 与样本序号无关，可以提出来；括号里那一坨正好是协方差矩阵 $\mathbf{C}$ 的定义（式 (3.5)）。所以：

> **换一个方向 $\mathbf{p}$，投影方差就是 $\mathbf{p}^\top\mathbf{C}\mathbf{p}$。** 于是“找方差最大的方向”变成了“最大化一个二次型”。

3.5.2 为什么必须有 $\|\mathbf{p}\|=1$ 这个约束

不限制长度的话，把 $\mathbf{p}$ 放大 10 倍，b_i 就放大 10 倍、方差放大 100 倍——“最大值”永远不存在（越大越好），问题就没意义了。固定 $\|\mathbf{p}\|=1$，只比“朝向”、不比“长度”。

等价的写法是最大化瑞利商 $\dfrac{\mathbf{p}^\top\mathbf{C}\mathbf{p}}{\mathbf{p}^\top\mathbf{p}}$：分母正是长度平方，自己就把长度归一化了。

3.5.3 用拉格朗日乘子把约束塞进目标

有约束的优化，标准做法是引入一个乘子 λ，把约束乘进去一起求导：

$$L(\mathbf{p}, \lambda) = \mathbf{p}^\top \mathbf{C}\,\mathbf{p} - \lambda\big(\mathbf{p}^\top \mathbf{p} - 1\big). \tag{3.24}$$

对 $\mathbf{p}$ 求偏导并令其为零（$\frac{\partial}{\partial \mathbf{p}} \mathbf{p}^\top \mathbf{C}\mathbf{p} = 2\mathbf{C}\mathbf{p}$，因为 $\mathbf{C}$ 对称；推导见附录 A）：

$$2\mathbf{C}\mathbf{p} - 2\lambda \mathbf{p} = \mathbf{0} \quad \Longrightarrow \quad \mathbf{C}\mathbf{p} = \lambda \mathbf{p}. \tag{3.25}$$

这正是特征值方程：满足它的 $\mathbf{p}$ 是 $\mathbf{C}$ 的特征向量，λ 是对应的特征值。再在两边左乘 $\mathbf{p}^\top$：

$$\mathbf{p}^\top \mathbf{C}\,\mathbf{p} = \lambda\, \mathbf{p}^\top \mathbf{p} = \lambda. \tag{3.26}$$

所以 λ 就是该方向上的投影方差——特征值越大，这个方向保留的信息越多。到这里我们只证明了“极值点一定是特征向量”，还没说明为什么最大的是 λ_1，下一步补上。

3.5.4 为什么“最大”的那个一定是 λ_1

只用一个事实：对称矩阵的特征向量可以选成一组正交基（这叫谱定理）。既然它们构成基，任何单位向量都能写成它们的组合：

$$\mathbf{p} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \cdots + \alpha_d \mathbf{v}_d, \qquad \alpha_1^2 + \alpha_2^2 + \cdots + \alpha_d^2 = 1. \tag{3.27}$$

代进二次型。交叉项 $\mathbf{v}_i^\top \mathbf{C}\mathbf{v}_j = \lambda_j \mathbf{v}_i^\top \mathbf{v}_j$ 因正交而全为零，只剩

$$\mathbf{p}^\top \mathbf{C}\,\mathbf{p} = \alpha_1^2 \lambda_1 + \alpha_2^2 \lambda_2 + \cdots + \alpha_d^2 \lambda_d. \tag{3.28}$$

右边是“以 α_i^2 为权重的 λ_i 加权平均”：权重非负、加起来等于 1，所以它不可能超过最大的那个 λ_1：

$$\mathbf{p}^\top \mathbf{C}\,\mathbf{p} \le \lambda_1(\alpha_1^2 + \cdots + \alpha_d^2) = \lambda_1, \tag{3.29}$$

等号只在 $\alpha_1 = \pm 1$、其余全为 0 时成立，也就是 $\mathbf{p} = \pm \mathbf{v}_1$。

这就是“第一主成分 = 最大特征向量”的完整理由：它不是定义，而是“方差最大”这个要求逼出来的。 同理，在“与 $\mathbf{v}_1$ 垂直”的方向里重复一遍同样的论证，最大的是 λ_2 对应的 $\mathbf{v}_2$；再往下依此类推（为什么必须垂直、为什么结论还是特征向量，下一小节专门讲）。于是有三条结论：

- 主成分两两正交（因为特征向量正交）；
- 第 k 个主成分上的方差恰好是 λ_k；
- 全部特征值之和 $\lambda_1 + \cdots + \lambda_d = \mathrm{tr}(\mathbf{C})$ 就是总方差，这解释了后面“保留几个主成分”为什么能按特征值比例来算。

3.5.5 第二主成分：为什么必须与第一主成分垂直

第一主成分定下来之后，“第二主成分”该是什么？

如果什么额外要求都不加，答案很无聊：还是 $\mathbf{v}_1$（或者跟它只差一点点的方向）——它本来就是方差最大的方向。可这样得到的“第二个方向”没有提供任何新信息，两个方向几乎重合，等于把同一件事说了两遍。

所以第二步必须加一个约束：新方向要与 $\mathbf{v}_1$ 正交，即 $\mathbf{p}^\top \mathbf{v}_1 = 0$。这句话的意思是：在第一个方向解释完之后，剩下的变化里最大的那个方向。

用特征基看：一步就能看出答案

任意单位向量仍可写成 $\mathbf{p} = \alpha_1 \mathbf{v}_1 + \cdots + \alpha_d \mathbf{v}_d$。加上正交约束，就是 $\alpha_1 = \mathbf{p}^\top \mathbf{v}_1 = 0$；单位长度要求则是 $\alpha_2^2 + \cdots + \alpha_d^2 = 1$。代进二次型（交叉项照旧为零）：

$$\mathbf{p}^\top \mathbf{C}\,\mathbf{p} = \alpha_2^2 \lambda_2 + \cdots + \alpha_d^2 \lambda_d \le \lambda_2 \left(\alpha_2^2 + \cdots + \alpha_d^2\right) = \lambda_2, \tag{3.30}$$

等号在 $\alpha_2 = \pm 1$、其余为 0 时成立，也就是 $\mathbf{p} = \pm \mathbf{v}_2$。

这就是“在与 $\mathbf{v}_1$ 垂直的方向里重复一遍同样的论证”的确切含义：把求和从 $i = 2$ 开始，把 λ_2 当作这个子空间里的“最大值”，其余推导一字不改。

用拉格朗日验算：约束是“自动满足”的

把两个约束都写进目标（λ 管长度，μ 管正交）：

$$L(\mathbf{p}, \lambda, \mu) = \mathbf{p}^\top \mathbf{C}\,\mathbf{p} - \lambda\left(\mathbf{p}^\top \mathbf{p} - 1\right) - \mu\, \mathbf{p}^\top \mathbf{v}_1. \tag{3.31}$$

对 $\mathbf{p}$ 求导并令其为零：

$$2\mathbf{C}\mathbf{p} - 2\lambda \mathbf{p} - \mu \mathbf{v}_1 = \mathbf{0}. \tag{3.32}$$

两边左乘 $\mathbf{v}_1^\top$，并利用 $\mathbf{C}\mathbf{v}_1 = \lambda_1 \mathbf{v}_1$ 与 $\mathbf{v}_1^\top \mathbf{p} = 0$：

$$2\lambda_1 \underbrace{\mathbf{v}_1^\top \mathbf{p}}_{=0} - 2\lambda \underbrace{\mathbf{v}_1^\top \mathbf{p}}_{=0} - \mu \underbrace{\mathbf{v}_1^\top \mathbf{v}_1}_{=1} = 0 \quad \Longrightarrow \quad \mu = 0. \tag{3.33}$$

$\mu = 0$ 代回去，就得到 $\mathbf{C}\mathbf{p} = \lambda \mathbf{p}$：加了正交约束之后，最优解仍然是特征向量，只是不能再取 $\mathbf{v}_1$（它与自己不正交），只能取 $\mathbf{v}_2, \ldots, \mathbf{v}_d$ 之一；其中方差最大的当然是 λ_2 对应的 $\mathbf{v}_2$。

> 这里有个“自洽”之处值得注意：对称矩阵**不同特征值**对应的特征向量本来就互相正交，所以 $\mathbf{v}_2$ 天然满足我们加的约束。约束不是额外限制，而是把“**不重复计算同一份方差**”这件事写成了一条等式。

另一个视角：把已解释的部分减掉，再做一次

“在正交补里找最大方差”等价于下面这个更像算法的做法：先把每个样本在 $\mathbf{v}_1$ 上的分量减掉，

$$\mathbf{x}_i^{(1)} = \mathbf{x}_i - \left(\mathbf{v}_1^\top \mathbf{x}_i\right)\mathbf{v}_1, \tag{3.34}$$

再对残差 $\mathbf{x}_i^{(1)}$ 从头做一次 PCA。它给出的第一个方向正是 $\mathbf{v}_2$。这一步叫 deflation（去膨胀）。

第三、第四主成分同理：每次都在“与前面所有主成分都正交”的子空间里取最大方差。于是 $\mathbf{v}_1, \mathbf{v}_2, \ldots$ 两两正交，方差依次是 $\lambda_1 \ge \lambda_2 \ge \cdots$。PCA 是“逐个抽取”的过程，不是一次把方向全挑出来。

回到本章的 2 维例子

2 维时，与 $\mathbf{v}_1$ 垂直的方向只有 $\pm\mathbf{v}_2$，没有可优化的余地：第二主成分只能取 $\mathbf{v}_2$，方差 0.119。

维度越高，“正交补”空间越大（$d-1$ 维），才需要在这个空间里再做一次同样的最大化——这也是为什么 3.6 节要强调“按奇异值排序取前 k 个”：前 k 个方向恰好是这个逐个抽取过程前 k 步的结果。

3.5.6 用第 3 章的例子核一遍

3.4 的 5 个点给出 $\mathbf{C}=\begin{bmatrix}2.5 & 2.0\\ 2.0 & 1.8\end{bmatrix}$，特征值 $\lambda_1\approx 4.181$、$\lambda_2\approx 0.119$（相加 4.3，正好是迹）。把几个方向代进 $\mathbf{p}^\top\mathbf{C}\mathbf{p}$ 算一遍：

方向	投影方差
$\mathbf{v}_1\approx(0.766,\ 0.643)$	**4.181**
$(0.707,\ 0.707)$（正 45°）	4.15
$(1,\ 0)$（横轴）	2.50
$\mathbf{v}_2\approx(-0.643,\ 0.766)$	0.119

读法：$\mathbf{v}_1$ 给出最大的 4.181（就是 λ_1），$\mathbf{v}_2$ 给出最小的 0.119（就是 λ_2）；正 45° 只比最优差 0.7%，说明这团点接近圆形；而“随手取横轴”只有 2.50，比最优少了一半——坐标轴不等于主成分，这正是 PCA 要做一次旋转的原因。

数据越“扁”，λ_1 与 λ_2 拉得越开，PCA 越有用（第 3.7 节就用这个比例决定保留几个主成分）。

> **一个常见误解**：特征向量既不是“数据点最多的方向”，也不是“最长的那根线”，而是“**投影后方差最大**的方向”。对一团高斯云，它恰好是椭圆的长轴——这就是 3.3 节结尾那句话的确切含义。

3.6 用 SVD 算 PCA：为什么更稳

3.6.1 三个麻烦：存不下、算不起、误差被放大

数据矩阵 $\tilde{\mathbf{X}}$ 是 $n\times d$（n 个样本、d 个变量）。人体数据里 d 动辄两万，n 只有几千：

1. 存不下：$d\times d$ 的协方差要 $20670^2\times 8$ 字节 ≈ 3.4 GB；
2. 算不起：对 $d\times d$ 矩阵做特征分解是 $O(d^3)$；
3. 数值上更糟：先算 $\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ 会把误差“平方放大”（3.6.4 解释）。

SVD 把这三件事一起解决。

3.6.2 SVD 是什么

任意 $n \times d$ 矩阵都能分解成

$$\tilde{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top, \tag{3.35}$$

其中 $\mathbf{U}$（$n \times r$）与 $\mathbf{V}$（$d \times r$）的列都是单位正交的，$\mathbf{S}$ 是对角矩阵、对角元非负且降序排列（$r \le \min(n, d)$），这些对角元叫奇异值 $s_1 \ge s_2 \ge \cdots \ge 0$。

几何上：任何线性变换 = 先旋转（$\mathbf{V}^\top$）→ 沿坐标轴拉伸（$\mathbf{S}$）→ 再旋转（$\mathbf{U}$）。“拉伸倍数”就是奇异值。

3.6.3 为什么 SVD 就给出了主成分

把 SVD 代进协方差的定义：

$$\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}} = \frac{1}{n-1}\mathbf{V}\mathbf{S}\mathbf{U}^\top\mathbf{U}\mathbf{S}\mathbf{V}^\top = \mathbf{V}\Big(\frac{\mathbf{S}^2}{n-1}\Big)\mathbf{V}^\top. \tag{3.36}$$

（用了 $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$。）而特征分解长这样：$\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$。两式一对比就得到

$$\lambda_i = \frac{s_i^2}{n-1}, \qquad \text{主成分方向} = \mathbf{V} \text{ 的列}. \tag{3.37}$$

“SVD 能算 PCA”的全部理由就是这一步：$\mathbf{V}$ 的列就是特征向量，奇异值平方除以 $n-1$ 就是特征值；而且 SVD 天然按 $s_1 \ge s_2 \ge \cdots$ 排序，不用再自己排。

用 3.4 的例子核对：$\lambda_1 = 4.181 \Rightarrow s_1 = \sqrt{4.181 \times 4} \approx 4.09$；$\lambda_2 = 0.119 \Rightarrow s_2 = \sqrt{0.119 \times 4} \approx 0.69$。

3.6.4 “更稳”到底指什么：条件数被平方了

数值计算里衡量“误差会被放大几倍”的量叫条件数 κ。关键恒等式是

$$\kappa(\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}) = \kappa(\tilde{\mathbf{X}})^2. \tag{3.38}$$

假设 $\tilde{\mathbf{X}}$ 的最大奇异值是 1、最小是 10^{-6}，则 $\kappa(\tilde{\mathbf{X}}) = 10^6$，平方之后变成 10^{12}。双精度只有约 16 位有效数字，10^{12} 的条件数意味着最小的那几个主成分方向只剩 4 位有效数字——算出来可能歪得厉害，而小主成分恰恰对应“细节与噪声”。

SVD 直接分解 $\tilde{\mathbf{X}}$，只承受 $\kappa(\tilde{\mathbf{X}})$，不平方。这就是“更稳”的实际含义，不是玄学。

3.6.5 四个必须注意的细节

1. 一定要先中心化。 直接对未中心化的 $\mathbf{X}$ 做 SVD，得到的是“相对原点的最大伸展方向”，不是方差最大方向（等价于强行要求均值在原点）。
2. 排序。 `numpy.linalg.eigh` 给的是升序特征值，要自己反转；`numpy.linalg.svd` 给的 s 已经是降序。
3. 符号是任意的。 $\pm\mathbf{v}$ 都是合法的特征向量，所以不同库、不同机器可能整体变号。它不影响重建，但要做“可复现的基”或跨机器比对时，得自己定符号规则（例如让绝对值最大的分量为正）。

4. 除以 $n-1$ 还是 n。 SVD 自己不给方差，$s_i^2/(n-1)$ 才是无偏方差。用 n 会让“解释方差比例”的比例不变、但绝对数值对不上别处。

3.6.6 $n < d$ 时的对偶 PCA

人体数据常见 $n \ll d$（比如 3000 个扫描、20670 维）。这时先算一个 $n \times n$ 的小矩阵：

$$\mathbf{K} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top = \mathbf{U}\mathbf{S}^2\mathbf{U}^\top, \tag{3.39}$$

对 $\mathbf{K}$ 做特征分解得到 $\mathbf{U}$ 和 s_i，再用

$$\mathbf{V} = \tilde{\mathbf{X}}^\top \mathbf{U}\mathbf{S}^{-1} \tag{3.40}$$

把右奇异向量“抬”回来（$s_i = 0$ 的那几列直接丢掉）。计算量从 $O(nd^2)$ 降到 $O(n^2 d)$——样本几千、维度几万时，这就是“能跑”和“跑不动”的差别。

3.6.7 两种写法对照

```
# 写法 A：显式协方差 + 特征分解（教学用；d 大时不要用）
C = Xc.T @ Xc / (n - 1)
w, V = np.linalg.eigh(C)              # 升序
w, V = w[::-1], V[:, ::-1]            # 改成降序，与 SVD 一致

# 写法 B：SVD（sklearn.decomposition.PCA 内部就是这么做的）
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)   # s 降序
lam = s ** 2 / (n - 1)
```

两种写法得到的 V 只差各列的符号。

> **一句话总结**：PCA 要的是“方差最大的方向”，数学上等于“求协方差矩阵的特征向量”；SVD 是求这些方向最稳、最省内存的办法——**根本不需要真的把协方差矩阵造出来**。

3.7 保留几个主成分

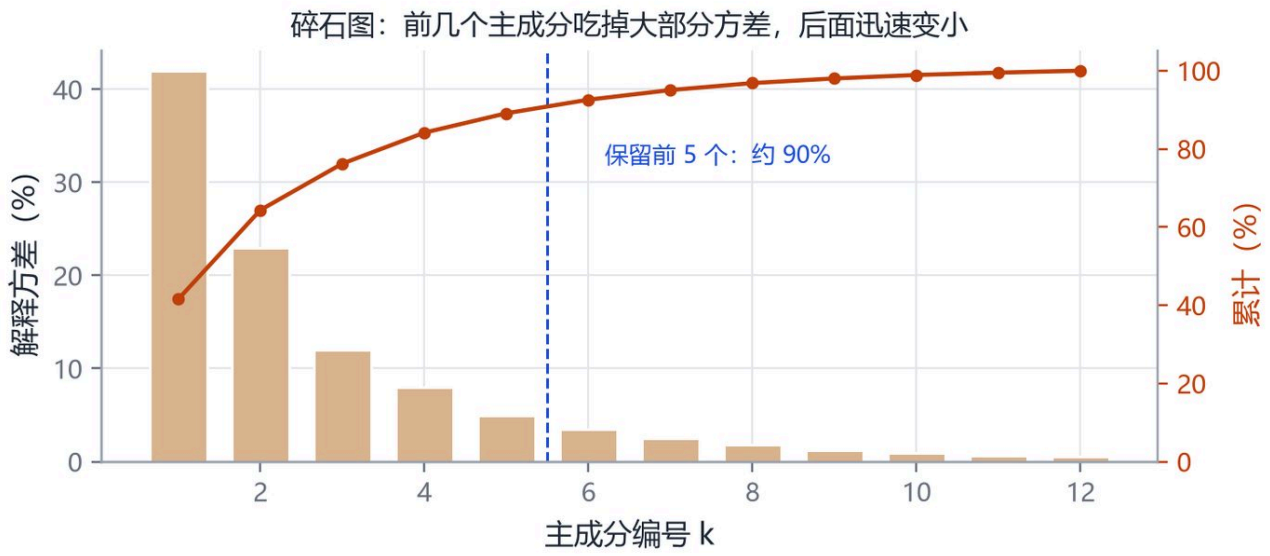


图 3-4 碎石图（scree plot）。柱状图是每个主成分单独解释的方差比例，折线是累计比例。前几个主成分吃掉大部分方差，之后迅速衰减；图中保留前 5 个约能解释 90%。

选择 k 没有唯一答案，常用三种判据：

1. 累计方差阈值：保留到累计解释方差达到 90% 或 95%。简单，但对噪声敏感。
2. 碎石图拐点：曲线从陡变缓的位置。主观，但直观。
3. 下游任务效果：用交叉验证，看 k 取多少时重建误差或识别准确率最好。

还有一条经验：如果数据里有明显的噪声，最后几个主成分往往是在拟合噪声。把它们丢掉不仅能压缩，还能去噪。

3.8 重建误差：PCA 到底丢了什么

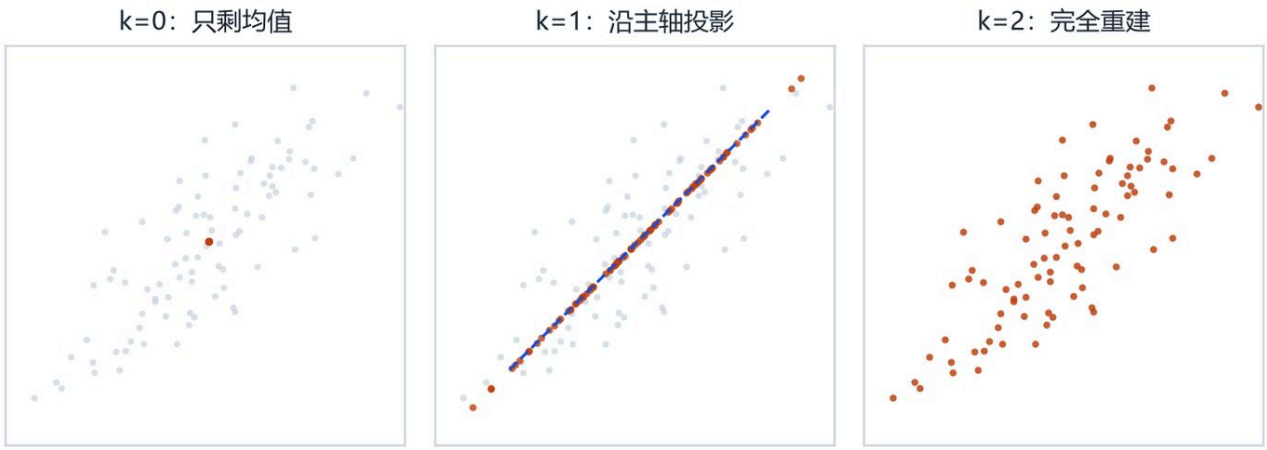


图 3-5 用不同数量的主成分重建同一批二维点。$k=0$ 时所有点都塌到均值；$k=1$ 时点被压到一条直线上；$k=2$ 时完全恢复。注意 $k=1$ 的重建点并不是“随机丢失”，而是沿垂直于主成分的方向被压扁。

重建误差可以用均方误差衡量：

$$E_k = \frac{1}{n}\sum_{i=1}^{n} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \sum_{j=k+1}^{d} \lambda_j. \tag{3.41}$$

这个等式很漂亮：丢掉前 k 个主成分之后，剩下的误差恰好等于被丢掉的那些特征值之和。所以“解释方差”与“重建误差”是同一件事的两面。

在人体形状模型里，这个性质被用来决定“10 个形状参数够不够”。SMPL 选了 10 维，是因为在它的训练数据上，前 10 个主成分已经能解释绝大部分体型差异；再往后增加维度，带来的视觉提升很小，却会让优化更容易过拟合。

3.9 PCA 在人体上长什么样

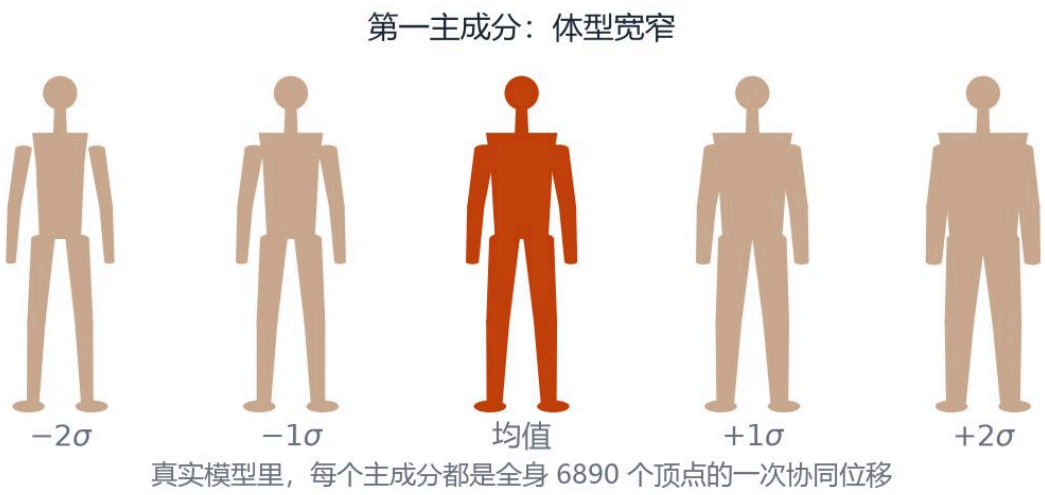


图 3-6 第一主成分的直观效果。沿第一主成分从 -2σ 走到 $+2\sigma$，人体的整体宽窄发生协同变化：不是某个顶点单独移动，而是全身 6890 个顶点一起按同一个比例改变。真实的 PCA 基向量就是这样一张“全身位移图”。

把 PCA 用在人体网格上，得到的每个主成分不是一个数，而是一张位移场：它规定了“如果这个参数加 1，全身每个顶点应该往哪个方向移动多少”。把所有主成分按列排成矩阵 $\mathbf{B}$（$20670 \times k$），就得到

$$\mathbf{x} = \boldsymbol{\mu} + \mathbf{B}\boldsymbol{\beta}. \tag{3.42}$$

这正是第 1 章式（1.1）的矩阵写法。SMPL 的 `shapedirs` 就是这样一个矩阵：$6890 \times 3 \times 10$，10 个主成分，每个都是全身的位移。

3.9.1 一个经常被忽略的细节：单位

PCA 对单位极其敏感。假设你有两个变量：身高（厘米，数值约 170）和头围（米，数值约 0.55）。身高的方差会比头围大好几个数量级，PCA 会几乎只关注身高。解决办法是先标准化：每个变量减去均值再除以标准差，让它们方差都为 1。

但对人体形状做 PCA 时，我们通常不标准化，因为三个坐标轴本来就应该是同一单位（米），而且我们希望保留“身高差异比手指粗细差异更重要”这一事实。是否标准化取决于你想让模型关注什么，没有绝对的对错。

3.9.2 四个常见的坑

坑一：忘记中心化。 不做中心化直接分解，第一主成分会指向“均值方向”，而不是变化方向。

坑二：把线性当成万能。 PCA 只能表示“沿直线方向的变化”。如果数据分布是弯的（比如人从站立到蹲下，关节角度不是线性变化），就需要核 PCA、自编码器或流形学习。人体姿态空间尤其如此，所以 SMPL 用关节旋转而不是直接对顶点做 PCA。

坑三：外插太远。 参数 β 取 $\pm 5\sigma$ 时，重建出来的形状可能已经不像人。实际系统通常把 β 限制在 ± 2 到 ± 3 个标准差内，或在优化时加正则项。

坑四：样本偏差。 如果训练数据只有年轻人，模型对老人、儿童、孕妇的形状会失真。SMPL 的训练集是 CAESAR（以欧美成年人为主），这也是它在不同人群上会有偏差的原因之一。后来的 STAR、GHUM 扩大了数据，部分缓解了这个问题。

本章小结

- 均值描述中心，方差描述分散，协方差描述“一起变”的趋势。
- 协方差矩阵把所有两两关系装进一张对称表；它的特征向量是数据云的主轴，特征值是各轴上的方差。
- PCA 五步：中心化 → 协方差 → 特征分解 → 取前 k 个 → 投影/重建。
- 特征值就是投影方差，丢掉的特征值之和就是重建误差。
- 在人体上，每个主成分是一张全身位移场；$\mathbf{x} = \boldsymbol{\mu} + \mathbf{B}\boldsymbol{\beta}$ 是所有参数化形状模型的共同形式。
- 注意单位、中心化、线性假设、外插范围和训练集偏差。

思考题

1. 如果所有样本都相同，协方差矩阵是什么？特征值呢？
2. 为什么 PCA 的主成分彼此正交？如果去掉正交约束，会得到什么？
3. 假设你有 1000 个人体网格，每个 20670 维。协方差矩阵有多大？用 SVD 需要多少内存？
4. 你能举一个“两个变量协方差为 0 但不独立”的例子吗？

第 4 章 三维世界的表示：点、网格、旋转与相机

> **本章导读**：参数化模型生活在三维空间里。要读懂 SMPL 的公式，你必须先知道“一个点如何被旋转、平移、投影到照片上”。本章讲四件事：三维点与坐标系、旋转的四种写法、刚体变换矩阵、针孔相机。最后简单介绍三角网格与可微渲染，为后面的“从图像反推模型”铺路。

4.1 三维点与坐标系

一个三维点就是三个数 (x, y, z)。但“三个数”本身没有意义，必须先约定坐标系：原点在哪、$x/y/z$ 轴指向哪、单位是米还是厘米。

人体模型通常约定：

- 原点在骨盆中心（SMPL 的根关节）；
- y 轴向上（不同实现可能是 z 轴向上，读代码时要先确认）；
- 单位是米；
- 人是面朝 $+z$ 还是 $-z$，不同模型不一样。

这些约定看起来琐碎，却是调试时的头号杀手：模型“躺着”“倒着”“镜像”，十有八九是坐标系约定不一致。

4.2 旋转的矩阵写法

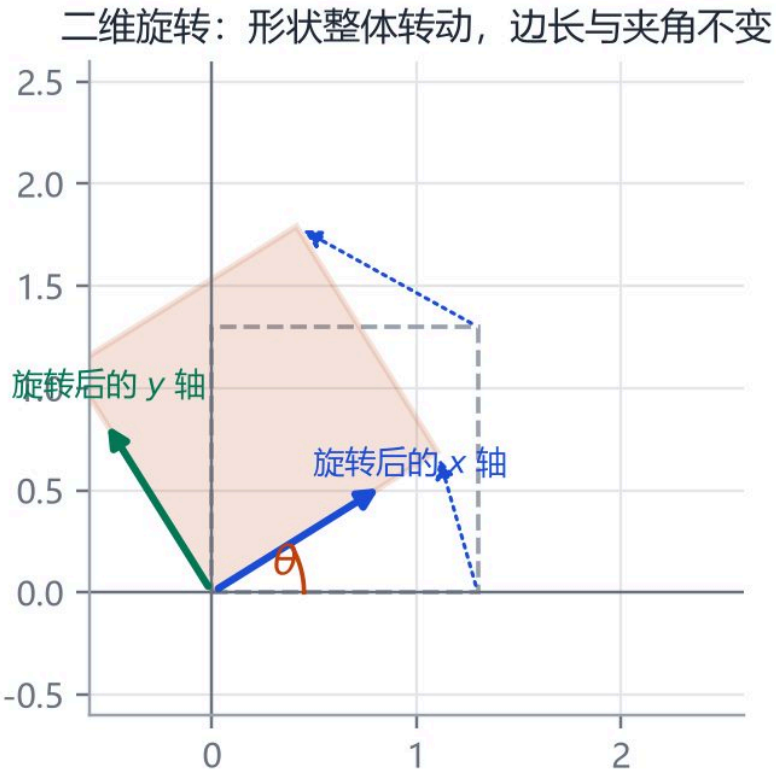


图 4-1 二维旋转。整个形状绕原点转过 θ，任意两点之间的距离、任意两条边的夹角都不变。旋转矩阵的两列，就是旋转后的 x 轴与 y 轴。

二维旋转是最容易看懂的。把点 (x, y) 绕原点逆时针转 θ，得到

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \tag{4.1}$$

这个 2×2 矩阵叫旋转矩阵 $\mathbf{R}(\theta)$。它有两个关键性质：

$$\mathbf{R}^{\top}\mathbf{R} = \mathbf{I}, \qquad \det(\mathbf{R}) = 1. \tag{4.2}$$

第一条说“转回去就是转置”，第二条说“旋转不改变面积、不镜像”。满足这两条的矩阵组成特殊正交群 $SO(2)$；三维版本叫 $SO(3)$。

三维旋转矩阵是 3×3，同样满足 $\mathbf{R}^{\top}\mathbf{R} = \mathbf{I}$、$\det \mathbf{R} = 1$。它有三列，每一列是旋转后的一个坐标轴方向。把三个轴想象成三根互相垂直的筷子，旋转就是整体转动这三根筷子。

4.2.1 绕三个坐标轴的基本旋转

$$\mathbf{R}_x(\alpha)=\begin{bmatrix}1&0&0\\0&\cos\alpha&-\sin\alpha\\0&\sin\alpha&\cos\alpha\end{bmatrix},\quad \mathbf{R}_y(\beta)=\begin{bmatrix}\cos\beta&0&\sin\beta\\0&1&0\\-\sin\beta&0&\cos\beta\end{bmatrix},\tag{4.3}$$

$$\mathbf{R}_z(\gamma)=\begin{bmatrix}\cos\gamma&-\sin\gamma&0\\\sin\gamma&\cos\gamma&0\\0&0&1\end{bmatrix}.\tag{4.4}$$

任何旋转都可以写成这三个矩阵的乘积，但顺序不同，结果不同。这就是下一节“欧拉角”的麻烦来源。

4.3 轴角：更接近直觉的写法

想象门绕合页转动：转轴是合页，转角是门打开的角度。任何三维旋转都可以这样描述：绕某个单位轴 $\mathbf{n}$ 转 θ 角。

给定轴 $\mathbf{n}=(n_x,n_y,n_z)$（长度为 1）和角 θ，旋转矩阵由 Rodrigues 公式给出：

$$\mathbf{R}(\mathbf{n},\theta)=\mathbf{I}+\sin\theta\,[\mathbf{n}]_\times+(1-\cos\theta)[\mathbf{n}]_\times^2,\tag{4.5}$$

其中

$$[\mathbf{n}]_\times=\begin{bmatrix}0&-n_z&n_y\\n_z&0&-n_x\\-n_y&n_x&0\end{bmatrix}\tag{4.6}$$

是“叉乘矩阵”。如果你不熟悉叉乘，可以先把它当成一个固定的记账方式：用它可以把“绕轴转”写成矩阵乘法。

轴角表示有三个数表示方向、一个数表示角度，共四个数，但方向向量长度必须为 1，所以实际自由度是 3，正好与旋转矩阵的自由度一致（3×3 有 9 个数，但正交约束去掉 6 个自由度）。

为什么 SMPL 用轴角？ 因为它没有冗余、容易插值、也容易加正则。SMPL 的姿态参数 $\boldsymbol{\theta}\in\mathbb{R}^{72}$ 就是 24 个关节各一个三维轴角向量（根关节另外 3 维是全局朝向）。

4.3.1 欧拉角与万向锁

欧拉角把旋转拆成“绕 x 转 α、绕 y 转 β、绕 z 转 γ”，只需三个数，非常直观。但它有两个致命问题：

1. 不唯一：(α,β,γ) 和 $(\alpha+\pi,\pi-\beta,\gamma+\pi)$ 可能表示同一个旋转；
2. 万向锁（gimbal lock）：当第二个角为 $\pm90^\circ$ 时，第一和第三个轴重合，失去一个自由度，插值会突然“翻跟头”。

所以在需要优化、插值、做动画的系统里，通常用轴角或四元数，只在界面显示时转成欧拉角。

4.3.2 四元数：四个数的旋转

四元数是形如 $q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$ 的数，单位四元数（$w^2 + x^2 + y^2 + z^2 = 1$）可以表示旋转：

$$q = \left(\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\mathbf{n}\right). \tag{4.7}$$

它没有万向锁，插值平滑（球面线性插值 SLERP），在图形学与机器人里无处不在。你不需要会推四元数乘法，只要知道：它是轴角的紧凑编码，适合做旋转的插值与组合。

4.4 齐次坐标与刚体变换

旋转之外还有平移。一个刚体变换是“先旋转再平移”：

$$\mathbf{p}' = \mathbf{R}\mathbf{p} + \mathbf{t}. \tag{4.8}$$

这个式子对矩阵乘法不友好（多了一个加法），于是我们给点补一个 1，变成四维齐次坐标 $\tilde{\mathbf{p}} = (x, y, z, 1)^\top$，就可以把旋转和平移合并成一个 4×4 矩阵：

$$\begin{bmatrix} \mathbf{p}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}. \tag{4.9}$$

这个 4×4 矩阵就是刚体变换矩阵，也是 SMPL 里每个关节的 $\mathbf{G}_k$。它有两个部分：左上 3×3 是旋转，右上 3×1 是平移。

多个变换可以连乘：先做 $\mathbf{G}_1$ 再做 $\mathbf{G}_2$，总变换是 $\mathbf{G}_2\mathbf{G}_1$。骨架的父子关系就是这样一串矩阵乘法，第 5 章会详细讲。

4.5 相机：三维如何变成二维

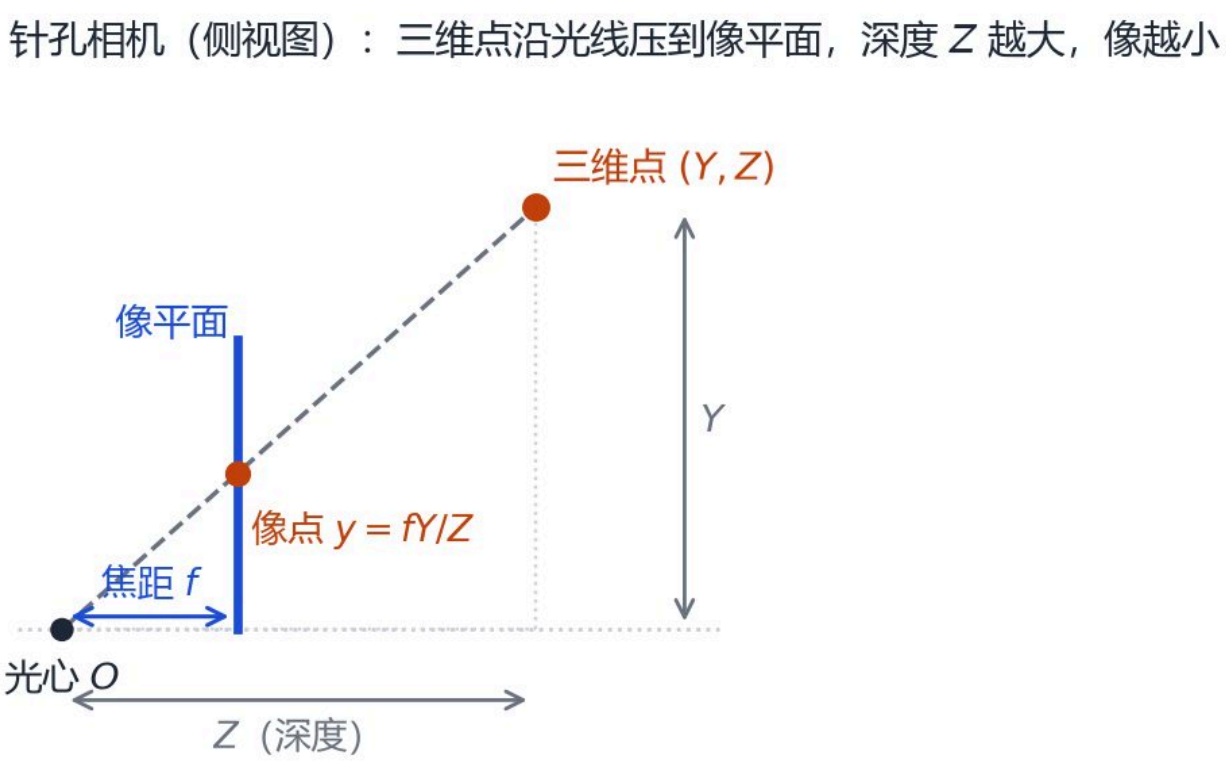


图 4-2 针孔相机模型（侧视图）。三维点与光心的连线交像平面于一点；由相似三角形可得 $y = fY/Z$。深度 Z 越大，成像越小，这就是“近大远小”。

照片是三维世界在二维平面上的投影。最简单的模型是针孔相机：所有光线都经过一个点（光心），在光心后方距离 f 处放一张像平面。

由相似三角形：

$$x = f\frac{X}{Z}, \qquad y = f\frac{Y}{Z}. \tag{4.10}$$

其中 (X, Y, Z) 是相机坐标系下的三维点，Z 是深度，f 是焦距（像素单位）。写成矩阵：

$$Z\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \mathbf{K}\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}. \tag{4.11}$$

$\mathbf{K}$ 叫内参矩阵，c_x, c_y 是主点（通常接近图像中心）。如果三维点在世界坐标系里，还要先乘一个外参矩阵 $[\mathbf{R}|\mathbf{t}]$ 把它变到相机坐标系：

$$Z\tilde{\mathbf{x}} = \mathbf{K}\,[\mathbf{R}|\mathbf{t}]\,\tilde{\mathbf{X}}. \tag{4.12}$$

这就是完整的投影公式。人体重建里的“重投影误差”，就是比较式（4.12）算出的二维点与图像上检测到的二维点：

$$E_{2D} = \sum_j \|\pi(\mathbf{J}_j) - \mathbf{j}_j^{\text{obs}}\|^2, \tag{4.13}$$

其中 π 是投影函数，$\mathbf{J}_j$ 是模型的三维关节，$\mathbf{j}_j^{\text{obs}}$ 是图像上检测到的二维关节。第 10 章的 SMPLify 就是围绕这个误差做优化。

> **注意**：式 (4.10) 假设相机没有畸变、像素是正方形。真实手机镜头有径向畸变，广角更明显。做高精度重建时要先标定并去畸变，否则边缘的人体会被拉歪。

4.6 三角网格：顶点的连接关系

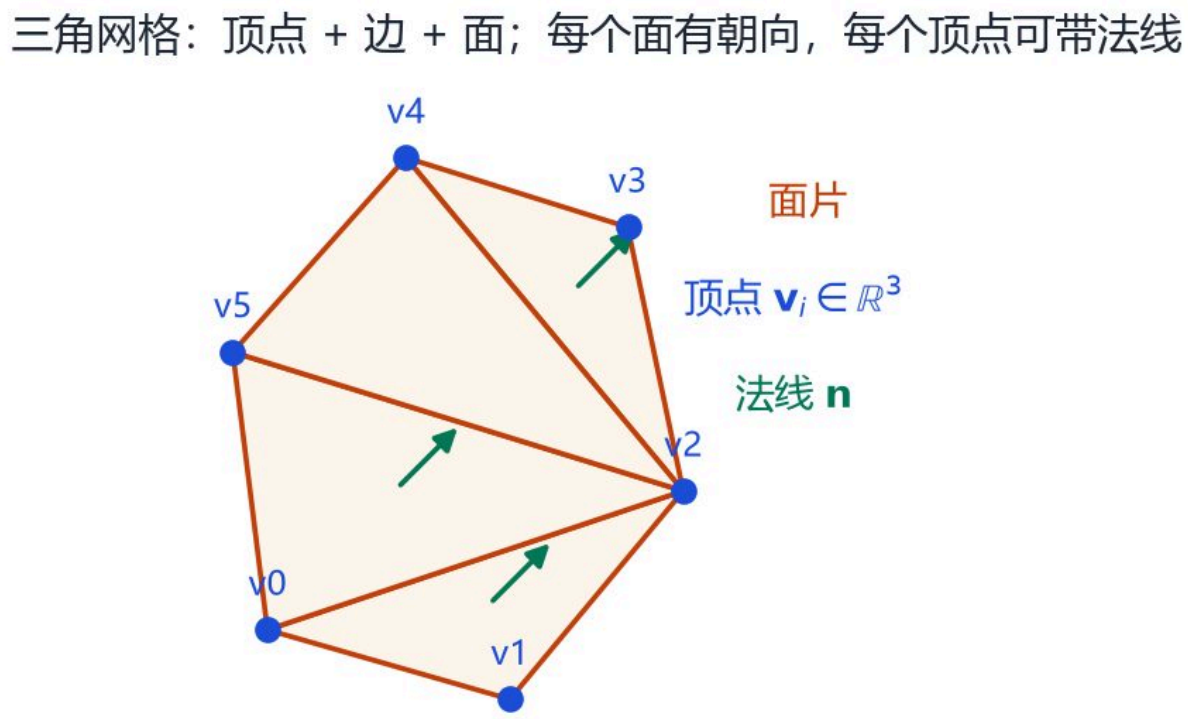


图 4-3 三角网格的组成：顶点、边、面。每个面有一个朝向（法线），用于光照与内外判断；顶点法线通常是相邻面法线的加权平均，用于平滑着色。

参数化模型的输出通常是三角网格。一个网格由两部分组成：

- 几何：N 个顶点的坐标 $\mathbf{V} \in \mathbb{R}^{N\times 3}$；
- 拓扑：F 个三角形，每个三角形记录三个顶点的编号。

拓扑是“连接关系”，它在模型变形时保持不变。这一点非常重要：SMPL 的所有人共享同一套拓扑，所以第 100 号顶点在任何人身上都对应同一个解剖位置，才能做 PCA、做对应、做蒙皮。

法线分两种：

- 面法线：三角形两条边的叉乘，归一化；垂直于面。
- 顶点法线：相邻面法线的平均；用于让曲面看起来光滑。

渲染时，光照计算需要法线；做碰撞、判断内外时，也需要法线朝向。法线方向反了，模型会看起来“里外翻转”。

4.7 可微渲染：把“渲染”变成可求导的函数

传统渲染器是一段黑箱程序：输入模型与相机，输出图片，但你没法问它“顶点往右移一点，像素会怎么变”。可微渲染（differentiable rendering）把渲染写成可导函数，于是可以把图像误差反传回模型参数。

最简单的情况是只渲染关节点：关节三维坐标经过式 (4.12) 得到二维坐标，这一步对三维坐标可导，误差可以直接反传。SMPLify 用的就是这种“稀疏关键点”渲染。

渲染完整网格要复杂得多：需要处理遮挡、光栅化、纹理采样。OpenDR、PyTorch3D、nvdiffrast、Kaolin 等库提供了可微实现。它们让式 (1.2) 的端到端优化成为可能。

> **一句话总结**：参数化模型负责“生成几何”，可微渲染负责“把几何变成图像”，两者接起来，才能从图像反推参数。

本章小结

- 三维点必须附带坐标系约定；人体模型通常以骨盆为原点、米为单位。
- 旋转可以用矩阵、欧拉角、轴角、四元数表示；优化与动画常用轴角/四元数，欧拉角只适合显示。
- 齐次坐标把旋转和平移合并成 4×4 矩阵，多个变换连乘就是骨架的父子链。
- 针孔相机的投影是 $Z\tilde{\mathbf{x}}=\mathbf{K}[\mathbf{R}|\mathbf{t}]\tilde{\mathbf{X}}$，重投影误差是人体拟合的核心监督。
- 网格 = 顶点几何 + 固定拓扑；法线决定光照与内外。
- 可微渲染把“渲染”变成可反传的算子，是模型拟合的桥梁。

思考题

1. 一个旋转矩阵的行列式为什么必须是 $+1$？如果等于 -1 会发生什么？
2. 为什么欧拉角会有万向锁，而四元数没有？试着用“三个轴是否独立”来解释。
3. 同一个人离相机远一倍，成像高度会变成多少？如果焦距也增大一倍呢？
4. 如果两个网格顶点数相同但拓扑不同，还能直接做 PCA 吗？

第 5 章 骨架、蒙皮与线性混合蒙皮（LBS）

本章导读：有了形状，还要让它动起来。这一章讲清楚“骨架—蒙皮”这套沿用三十年的机制：关节如何组成一棵树、变换如何沿树传递、一个顶点如何同时受多根骨头影响。核心公式只有一个，但每个符号都值得拆开看。读完后，你会明白 SMPL 的 $W(\cdot)$ 到底在算什么，也会知道它为什么会在腋下和膝盖处出问题。

5.1 骨架是一棵树

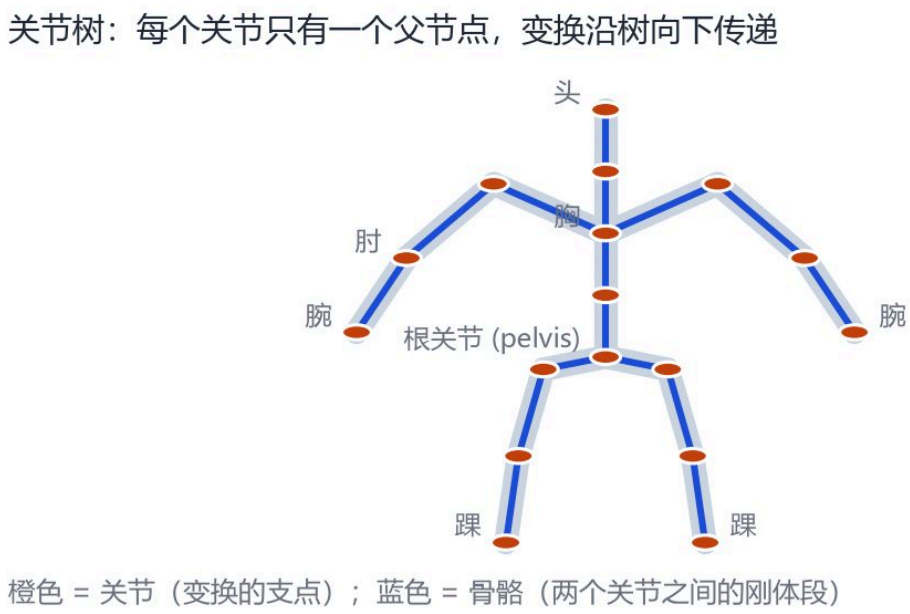


图 5-1 人体骨架的树结构。根节点在骨盆，躯干向上分出头和双臂，向下分出双腿。每个关节只有一个父节点；子关节的最终位置，由父关节的变换一路乘下来。

骨架（skeleton）由关节（joint）和骨骼（bone）组成。关节是旋转的支点，骨骼是两个关节之间的刚体段。人体骨架天然是一棵树：

- 根关节：骨盆（pelvis）；
- 骨盆 → 脊柱 → 胸 → 颈 → 头；
- 胸 → 左肩 → 左肘 → 左腕；右侧同理；

- 骨盆 → 左髋 → 左膝 → 左踝；右侧同理。

树结构的好处是：只要知道每个关节相对父关节的变换，就能算出它在世界坐标里的位置。这是一串矩阵乘法：

$$\mathbf{G}_k = \mathbf{G}_{\text{parent}(k)}\,\mathbf{L}_k, \tag{5.1}$$

其中 $\mathbf{G}_k$ 是关节 k 的全局变换，$\mathbf{L}_k$ 是它相对父关节的局部变换。根关节没有父节点，$\mathbf{G}_{\text{root}} = \mathbf{L}_{\text{root}}$。

$\mathbf{L}_k$ 通常由三部分组成：从父关节到自己的固定偏移、绕自己的旋转 $\mathbf{R}(\boldsymbol{\theta}_k)$、以及缩放（人体模型一般不含缩放）。写成 4×4 矩阵就是

$$\mathbf{L}_k = \begin{bmatrix} \mathbf{R}(\boldsymbol{\theta}_k) & \mathbf{J}_k - \mathbf{J}_{\text{parent}(k)} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}. \tag{5.2}$$

注意偏移量是在静止姿势下测量的。人摆出姿势后，关节位置会变，但“父子之间的相对偏移”在局部坐标系里保持不变——这正是树结构能高效计算的原因。

5.2 蒙皮：让顶点跟着骨头走

骨架本身只是几根线段，真正要变形的是皮肤。蒙皮（skinning）要回答：给定每个关节的变换，每个顶点应该移动到哪里？

最朴素的做法是“刚性绑定”：把每个顶点分配给一根骨头，顶点跟着那根骨头做刚体变换。但这样关节处会裂开——上臂的顶点转 90°，前臂的顶点不动，肘部就出现一条缝。

真实皮肤是连续的：肘部内侧会被挤压，外侧会被拉伸，中间的顶点同时受两根骨头影响。于是有了线性混合蒙皮（Linear Blend Skinning，LBS）[10]：

$$\mathbf{t}_i' = \sum_{k=1}^{K} w_{k,i}\,\mathbf{G}_k(\boldsymbol{\theta}, \mathbf{J})\,\mathbf{G}_k(\mathbf{0}, \mathbf{J})^{-1}\,\mathbf{t}_i. \tag{5.3}$$

这个公式信息量很大，我们逐项拆开。

5.2.1 每一项在做什么

- $\mathbf{t}_i$：静止姿势下第 i 个顶点的位置（齐次坐标，4×1）。
- $\mathbf{G}_k(\mathbf{0}, \mathbf{J})^{-1}$：把顶点从世界坐标“搬回”关节 k 的局部坐标系。可以理解为“这个顶点在骨头 k 看来在哪”。
- $\mathbf{G}_k(\boldsymbol{\theta}, \mathbf{J})$：关节 k 在当前姿势下的全局变换，把局部坐标再搬回世界。
- 两者相乘 $\mathbf{G}_k(\boldsymbol{\theta})\mathbf{G}_k(\mathbf{0})^{-1}$，就是一个“只属于骨头 k 的位移”，记作 $\mathbf{G}_k'$。
- $w_{k,i}$：顶点 i 受骨头 k 影响的权重，非负，且对每个顶点满足 $\sum_k w_{k,i} = 1$。
- 求和：把所有骨头的“建议位置”按权重加权平均。

用一句话概括：每个骨头都给出一个“如果只由我控制，这个顶点会在哪”的答案，LBS 把这些答案按权重平均。

为什么是两个矩阵相乘?

这正是最容易卡住的地方：既然“知道顶点的初始位置、也知道每根骨头的变换”，直接乘一下不就行了？

问题在于：骨架给你的 $\mathbf{G}_k$ 不是“作用在顶点上的变换”，而是“关节 k 的局部坐标系 → 世界坐标系”的变换。 它作用的对象是“在关节 k 的局部坐标系里表达的坐标”，不是“在静止姿势的世界坐标系里表达的顶点”。

所以要先换坐标系。三个坐标系要分清：

坐标系	顶点在这里长什么样	用哪个矩阵进出
静止姿势的世界系（rest space）	就是 $\mathbf{t}_i$，模型文件里存的那个	$\mathbf{G}_k(\mathbf{0},\mathbf{J})^{-1}$ 把它送进骨骼 k
骨骼 k 的局部系（joint space）	顶点“相对骨头 k 在哪”，跟着骨头一起动	$\mathbf{G}_k(\boldsymbol{\theta},\mathbf{J})$ 把它送回当前的世界系
当前姿势的世界系（posed space）	变形后的位置 $\mathbf{t}_i'$	——

于是

$$\underbrace{\mathbf{G}_k(\boldsymbol{\theta})\,\mathbf{G}_k(\mathbf{0})^{-1}}_{\text{记作 }\mathbf{S}_k\text{（蒙皮矩阵）}}\,\mathbf{t}_i$$

的意思是：先用 $\mathbf{G}_k(\mathbf{0})^{-1}$ 把顶点搬到骨骼 k 的局部系，再用 $\mathbf{G}_k(\boldsymbol{\theta})$ 把它带回当前姿势的世界。两个矩阵不是“对顶点做了两次不相关的变换”，而是拼成一个变换 $\mathbf{S}_k$——它就是你说的“这根骨头对这个顶点造成的形变”。

用 5.2.2 的数值核一遍（肘在 $(0.55, 0.30)$，顶点 $(0.91, 0.37)$）：

- 只乘 $\mathbf{G}_k(\mathbf{0})^{-1}$（相当于把顶点当成“关节局部坐标”）：$(0.91, 0.37) - (0.55, 0.30) = (0.36, 0.07)$ —— 这就是顶点在肘的坐标系里的位置；
- 再乘 $\mathbf{G}_k(\boldsymbol{\theta})$（绕肘转 $90°$ 并搬回世界）：$(-0.07, 0.36) + (0.55, 0.30) = (0.48, 0.66)$。

如果省掉左边那个逆矩阵、直接乘 $\mathbf{G}_k(\boldsymbol{\theta})$，相当于假设顶点原本就住在关节的局部系里（也就是把关节位置当成了原点）：$(0.91, 0.37)$ 会被当成“肘坐标系里的点”，转 $90°$ 变成 $(-0.37, 0.91)$，再加上关节位置 → $(0.18, 1.21)$。完全错了：顶点没有被“绕肘转”，而是被“绕原点转”了。

> 三个名字记一下，读代码时会反复遇到：- $\mathbf{B}_k = \mathbf{G}_k(\mathbf{0},\mathbf{J})$ 叫**绑定矩阵**（bind matrix）；- $\mathbf{B}_k^{-1}$ 叫**逆绑定矩阵**，glTF 里的 `inverseBindMatrices` 就是它；- $\mathbf{S}_k = \mathbf{G}_k(\boldsymbol{\theta})\mathbf{B}_k^{-1}$ 叫**蒙皮矩阵**（skin matrix）。

工程上有一个很重要的细节：$\mathbf{S}_k$ 每帧只算一次，每根骨头一个矩阵（24 根骨头就是 24 个 4×4），而不是每个顶点算一次。每个顶点只做“4 个矩阵乘向量 + 加权求和”，这也是 LBS 能在 GPU 上跑几十万顶点的原因。

5.2.2 一个数值例子

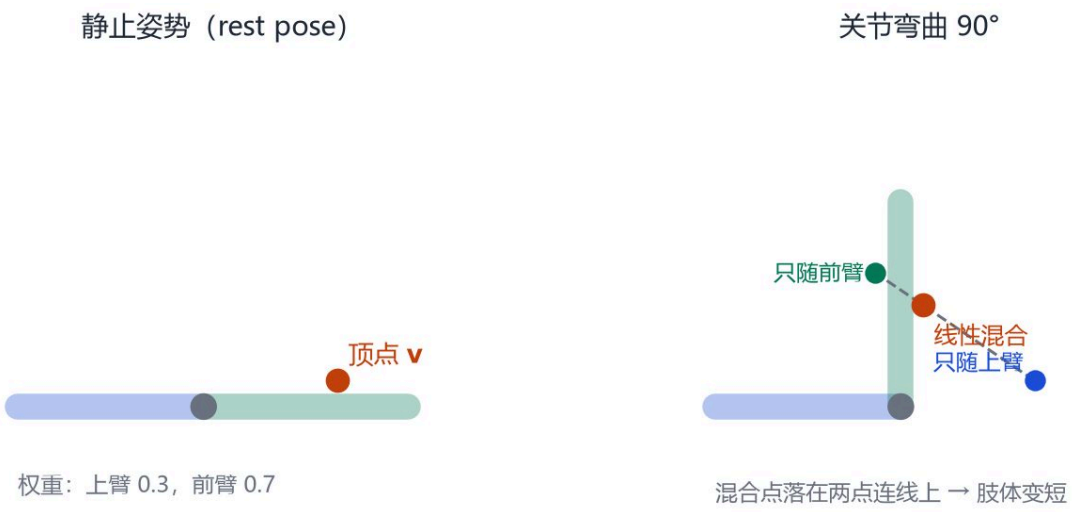


图 5-2 LBS 的数值直觉。静止时前臂水平；肘关节弯曲 90° 后，只随上臂的顶点留在原处，只随前臂的顶点转到上方，而权重 0.3/0.7 的混合点落在两点连线上。混合点离关节更近，这就是“体积损失”。

设肘关节在 $(0.55, 0.30)$，前臂沿 x 轴伸展，某顶点在 $(0.91, 0.37)$，到肘的距离约 0.367。它受上臂权重 0.3、前臂权重 0.7。

肘弯曲 90° 后：

- 若只随上臂：顶点不动，仍在 $(0.91, 0.37)$；
- 若只随前臂：绕肘转 90°，到 $(0.48, 0.66)$；
- LBS 混合：$0.3\times(0.91,0.37)+0.7\times(0.48,0.66)=(0.61,0.57)$。

混合点到肘的距离是 $\sqrt{(0.61-0.55)^2+(0.57-0.30)^2}\approx0.28$，比原来的 0.367 短了约 24%。两个刚体位置的线性平均，落在它们的连线上，而连线穿过关节内侧，所以肢体在弯曲处会变细。 这就是 LBS 最著名的“捏细”（pinching）现象。

5.2.3 权重从哪里来

权重矩阵 $\mathbf{W}\in\mathbb{R}^{N\times K}$ 有两种来源：

1. 手工绘制：动画师用笔刷在模型上涂权重，效果可控但费时。
2. 自动计算：根据顶点到骨骼的距离、测地距离或热扩散方程自动生成。热扩散方法（如 bounded biharmonic weights）能得到很平滑的过渡。

SMPL 的权重是从数据里学出来的：作者用大量不同姿势的扫描，优化出一套权重，使 LBS 的结果最接近真实扫描。每个顶点最多受 4 根骨头影响，这样既保证质量，又能用 GPU 高效计算。

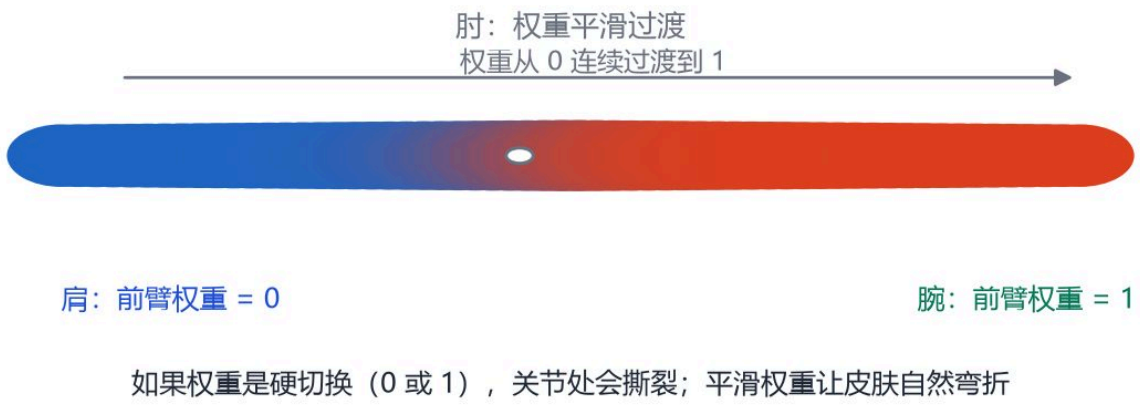


图 5-3 权重沿肢体的平滑过渡。颜色表示“前臂权重”：肩部为 0，腕部为 1，肘部附近连续变化。如果权重是 0/1 硬切换，关节处会撕裂；平滑过渡让皮肤自然弯折。

5.3 为什么 LBS 会出问题

LBS 的优点是“线性”：计算只是矩阵加权和。缺点也全来自这三个字——两个刚体变换的线性平均，一般不再是刚体变换。

具体会坏在三个地方：弯曲时变细、扭转时塌陷、权重不合适时起折痕。前两个可以精确算出来，不是“感觉上有点怪”。

5.3.1 捏细：弯曲时截面变小

上一节的数值例子已经看到了：肘弯 90°，一个权重 0.3/0.7 的顶点从离关节 0.367 缩到 0.28，短了 24%。

原因一句话：两个刚体位置的线性平均落在它们的连线上，而这条连线穿过关节内侧。弯曲角越大，弧越“抄近路”，缩得越狠。

5.3.2 糖纸效应：扭转时截面被“拧细”

“捏细”是弯曲造成的。更严重的一种叫糖纸效应（candy wrapper effect）：当前臂绕自己的长轴扭转时（手心翻到手背，专业上叫旋前/旋后），中间那段会被拧成一个细脖子——就像拧糖果的包装纸。

它不是“看起来像”，而是可以精确算出来的。考虑两根骨头：一段不转（$\mathbf{R}_1 = \mathbf{I}$），另一段绕长轴转 θ（$\mathbf{R}_2 = \mathbf{R}_x(\theta)$）：

$$\mathbf{R}_x(\theta)=\begin{bmatrix}1 & 0 & 0\\ 0 & \cos\theta & -\sin\theta\\ 0 & \sin\theta & \cos\theta\end{bmatrix}.$$

权重各一半的截面，LBS 算出来的是矩阵平均

$$\mathbf{M}=\tfrac{1}{2}\mathbf{I}+\tfrac{1}{2}\mathbf{R}_x(\theta)=\begin{bmatrix}1 & 0 & 0\\ 0 & \frac{1+\cos\theta}{2} & -\frac{\sin\theta}{2}\\ 0 & \frac{\sin\theta}{2} & \frac{1+\cos\theta}{2}\end{bmatrix}.$$

看右下角那个 2×2 块：它等于“缩放 × 旋转”，缩放因子是

$$s=\sqrt{\left(\tfrac{1+\cos\theta}{2}\right)^2+\left(\tfrac{\sin\theta}{2}\right)^2}=\sqrt{\tfrac{1+\cos\theta}{2}}=\left|\cos\tfrac{\theta}{2}\right|. \tag{5.4}$$

也就是说：扭转 θ 时，正中间那一圈的半径会变成原来的 $|\cos(\theta/2)|$ 倍。

扭转角 θ	半径比 $\cos(\theta/2)$	观感
0°	1.00	正常
90°	0.71	明显变细
120°	0.50	只剩一半
180°	**0**	**整圈塌到轴线上**

180° 时半径正好为零：整圈顶点被压到轴线上，截面退化成一个点，网格在那里自交、法线翻转——这就是渲染里那些“拧断”的破面。

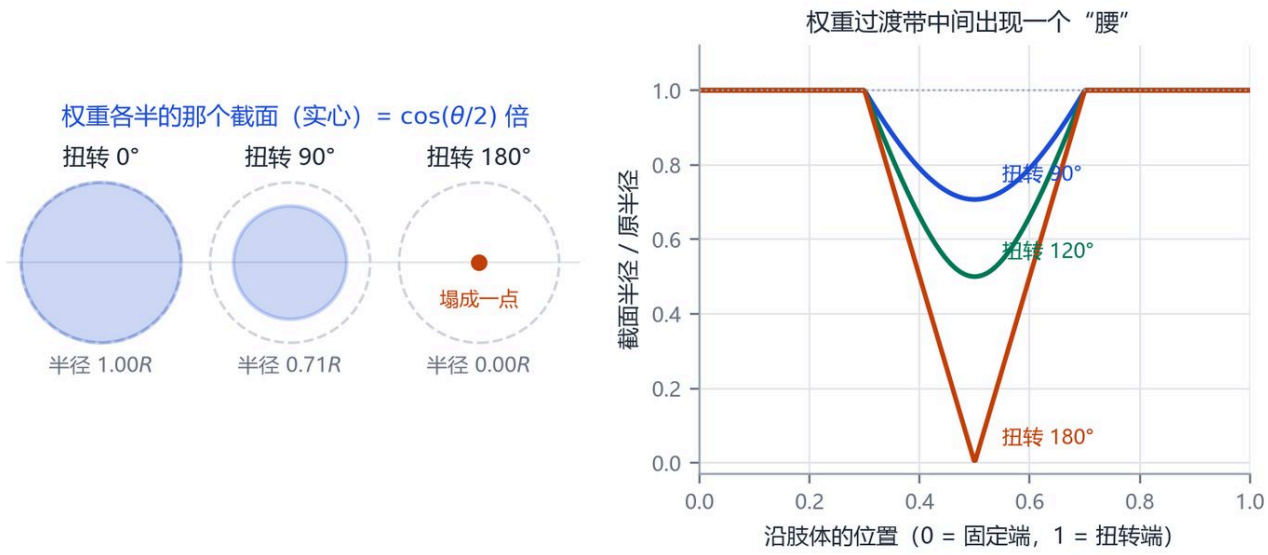


图 5-4 糖纸效应的定量图像。左：扭转 0°、90°、180° 时，权重各半的那个截面是同心圆逐步缩小，180° 时整圈塌到轴线上（半径比 $|\cos(\theta/2)|$）；右：沿肢体方向的半径曲线，权重过渡带中间出现一个“腰”，扭转越大腰越细。

> **为什么叫“糖纸”**：拧糖果包装纸时，两端反着转，中间最细的那一段会被拧成一个收腰的“颈”。LBS 的扭转伪影在形状上与它一模一样，于是这个名字就叫开了。

5.3.3 折痕与连带：权重的问题

第三类伪影来自权重本身，和角度无关：

- 过渡太窄：权重在几个顶点内从 0 跳到 1，弯曲时表面会出现一道硬折痕（像纸板折的）。
- 过渡太宽：不该动的部位被带动，比如抬手臂时胸部被拉起来、转头时肩膀跟着歪。
- 左右串扰：如果权重不是按解剖距离而是按空间距离算，抬左腿可能让右腿内侧动一点。

这三件事在工业界靠“刷权重”解决：动画师在 3D 软件里手动涂，把过渡带放在关节附近、宽度约等于肢体半径。这很费人力，也是自动蒙皮算法（热扩散、有界双调和权重）要解决的问题。

5.4 对偶四元数蒙皮：把“混合”从矩阵换成刚体变换

既然病根是“混合矩阵”，那最直接的药方就是：别混合矩阵，改混合一种天生只表示刚体变换的东西。

5.4.1 关键直觉：四元数混合不会缩放

单位四元数 $q = w + xi + yj + zk$（$|q| = 1$）表示一个三维旋转。所有单位四元数构成四维空间里的一个单位球面，每个旋转对应球面上一个点。

现在要混合两个旋转：$q' = w_1 q_1 + w_2 q_2$。这个结果一般不再是单位四元数（跑到球面里侧去了），但只要归一化

$$q' = \frac{w_1 q_1 + w_2 q_2}{\|w_1 q_1 + w_2 q_2\|},$$

它又回到球面上——又变回一个纯旋转，绝不会带缩放。这就是四元数能治糖纸效应的全部原因。

对比一下矩阵：$\frac{1}{2}\mathbf{I} + \frac{1}{2}\mathbf{R}_x(180^\circ)$ 算出来是 $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$，它把 y, z 方向压扁成 0——不再正交，也就不是旋转了。

5.4.2 为什么“只旋转”也会产生平移

这个疑问很自然：骨头不就是转一下吗，哪来的平移？

关键在于：旋转是绕关节中心转的，不是绕世界原点转的。

设关节在静止姿势下的位置是 $\mathbf{c}$，它只做一次旋转 $\mathbf{R}$。那么“绕 $\mathbf{c}$ 旋转”对任意点 $\mathbf{p}$ 的作用是

$$\mathbf{p} \longmapsto \mathbf{c} + \mathbf{R}(\mathbf{p} - \mathbf{c}) = \underbrace{\mathbf{R}\mathbf{p}}_{\text{绕原点旋转}} + \underbrace{(\mathbf{I} - \mathbf{R})\,\mathbf{c}}_{\text{平移}}. \tag{5.5}$$

也就是说：“绕关节转” = “绕原点转” + 一个由关节位置决定的平移。只要 $\mathbf{c} \neq \mathbf{0}$，平移项 $(\mathbf{I}-\mathbf{R})\mathbf{c}$ 一般不为零。

用 5.2.2 的例子算一遍。肘在 $\mathbf{c}=(0.55,0.30)$，顶点 $(0.91,0.37)$，绕肘转 90°：

1. 先挪到以肘为原点的坐标：$(0.36,0.07)$；
2. 转 90°：$(-0.07,0.36)$；
3. 再挪回去：$(0.48,0.66)$ —— 与前面算的一致。

补偿的平移量是

$$(\mathbf{I}-\mathbf{R})\mathbf{c}=(0.55,0.30)-(-0.30,0.55)=(0.85,-0.25).$$

一次“纯粹绕关节转 90°”，在数字上确实带了 $(0.85,-0.25)$ 的平移。你之所以“感觉不到平移”，是因为你站在关节上看：在关节自己的坐标系里，顶点确实只转了 90°。

而蒙皮用的矩阵正是这种“绕静止关节位置的旋转”：

$$\mathbf{S}_k=\mathbf{G}_k(\boldsymbol{\theta},\mathbf{J})\,\mathbf{G}_k(\mathbf{0},\mathbf{J})^{-1}.$$

右乘 $\mathbf{G}_k(\mathbf{0})^{-1}$ 把参照系换成关节 k 的静止局部系，所以 $\mathbf{S}_k$ 的旋转中心就是静止姿势下的关节 k，平移分量随之出现——这正是 5.2.1 说的“只属于骨头 k 的位移”。

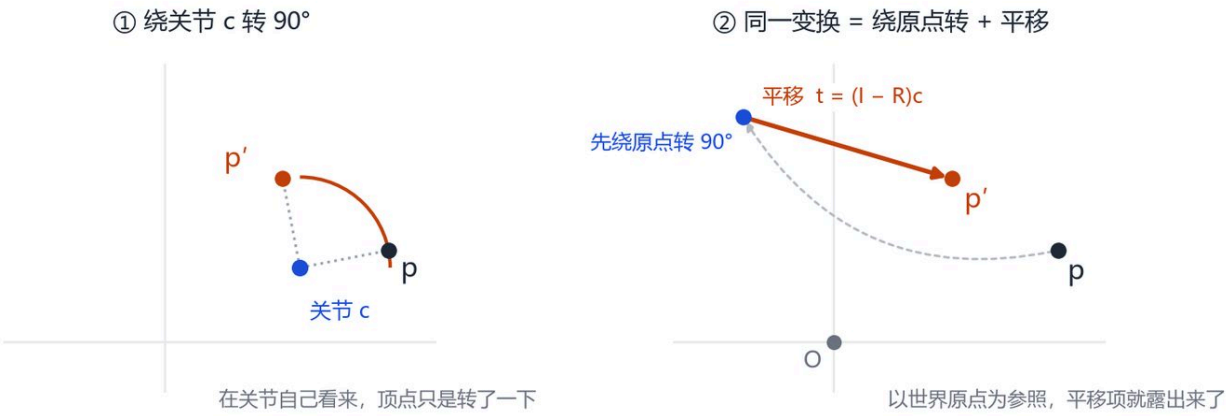


图 5-5 “绕关节转”为什么带着平移。左：在关节 $\mathbf{c}$ 自己的参照系里，顶点 $\mathbf{p}$ 只是沿圆弧转了 90°；右：同一个变换，如果以世界原点 O 为参照，就等于“绕原点转 90°”再平移 $\mathbf{t}=(\mathbf{I}-\mathbf{R})\mathbf{c}$。平移不是额外加的，而是换参照系时冒出来的。

> 极端情况：如果所有关节都恰好在原点（$\mathbf{c}_k=\mathbf{0}$），或者模型只有绕世界原点的整体旋转，那确实没有平移，普通四元数就够了。但真实骨架是树状的——子关节的位置由父链一路乘出来，不可能都落在原点。

那能不能“旋转用四元数、平移单独插值”？

很多人的第一反应是：旋转用 Nlerp，平移用线性插值，不就绕开对偶四元数了？两个原因不行：

1. 平移和旋转是耦合的。刚体变换的 6 个自由度不是独立可分的：绕一根远处的轴转一点，末端点的位移会很大。分开插值会让中间帧“脱轨”——手腕跑不到它该在的位置，误差随“旋转半径”放

大。

2. 对偶四元数插值走的是螺旋线。单位对偶四元数描述的是“绕某轴转 + 沿该轴平移”的螺旋运动（screw motion）；两个螺旋做加权混合再归一化，近似的是 SE(3) 上最短的螺旋路径（这也是 Kavan 论文里 DLB 的含义）。而“分别混合 R 和 t”相当于在 $\mathbf{SO(3)} \times \mathbb{R}^3$ 的直积上插值，两者不等价。

一句话：对偶四元数不是“为了平移打的补丁”，而是把 6 个自由度打包成一个整体——只有整体插值，才能保证中间状态仍然是一个合法的刚体变换。

对偶四元数本身长什么样

$$\hat{q} = q_r + \varepsilon\, q_d, \qquad \varepsilon^2 = 0. \tag{5.6}$$

- q_r：单位四元数，管旋转；
- $q_d = \frac{1}{2}\, t \otimes q_r$：管平移（$t$ 是把平移写成四元数 $(0, t_x, t_y, t_z)$，$\otimes$ 是四元数乘法）；
- ε 是一个“平方为零”的形式符号，作用只是把两部分分开存，不参与实际计算。

约束是两条：$\|q_r\| = 1$ 且 $q_r \cdot q_d = 0$（后者保证 q_d 真的是“沿平移方向”的那部分）。

几何上，一个单位对偶四元数正好描述一个螺旋运动：绕某根轴转一个角，同时沿这根轴平移一段。任何刚体变换都能写成螺旋运动，这正是它比“矩阵 + 四元数拼起来”更内在的原因。

它作用在点上仍然很简单：取 q_r 与平移 $\mathbf{t}$，

$$\mathbf{p}' = \mathbf{R}(q_r)\, \mathbf{p} + \mathbf{t}, \tag{5.7}$$

和矩阵形式等价；区别只在混合那一步。

5.4.3 DQS 的五个步骤

对每个顶点，算法是：

1. 算每根骨头的蒙皮变换：$\mathbf{S}_k = \mathbf{G}_k(\boldsymbol{\theta}, \mathbf{J})\, \mathbf{G}_k(\mathbf{0}, \mathbf{J})^{-1}$（和 LBS 完全一样）；
2. 转成对偶四元数：从 $\mathbf{S}_k$ 的旋转部分取出 $q_{r,k}$，从平移部分取 $\mathbf{t}_k$，算 $q_{d,k} = \frac{1}{2}\mathbf{t}_k \otimes q_{r,k}$；
3. 加权混合：$\hat{q} = \sum_k w_{k,i}\, \hat{q}_k$。注意符号：如果 $\hat{q}_k \cdot \hat{q}_1 < 0$，先把它整体取反（否则会走“远路”，插值绕一大圈）；
4. 归一化：$\hat{q} \leftarrow \hat{q} / \|\hat{q}\|$（对偶四元数的范数由 q_r 决定）；
5. 变换顶点：把 $\hat{q}$ 转回“旋转 + 平移”，作用到 $\mathbf{t}_i$ 上。

和式 (5.3) 的 LBS 对比：求和从“矩阵”搬到了“对偶四元数”上，最后才转回矩阵。就这一处改动，让混合结果始终保持刚体性。

5.4.4 效果与代价

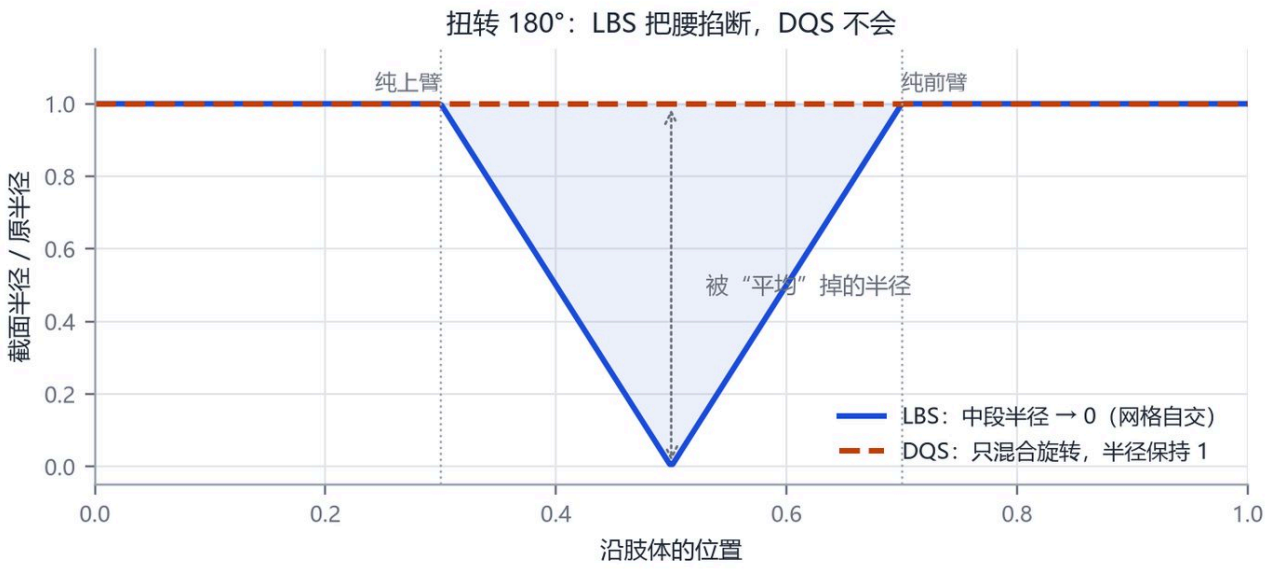


图 5-6 同一条扭转 180° 的前臂，两种蒙皮的截面半径。LBS（蓝）在中段被压到 0，网格自交；DQS（橙）只把权重过渡带的两端“卷”过去一点，中段基本保持原半径，所以不会出现细颈。

DQS 的收益：扭转伪影基本消失，弯曲时的体积损失也明显减轻（因为它不做“直线抄近路”）。

代价与坑：

- 贵一点：每顶点多做若干次四元数乘法和一次归一化，移动端要算预算；
- 会“膨胀”（bulging），但原因不是“体积不变”：这句话很容易被误读，值得说清楚。单看一个顶点，DQS 给它的确实是一个刚体变换，没有缩放；问题出在相邻顶点的变换不一样——它们各自被不同的混合结果搬走，于是相邻两点之间的边会被拉长或压短，网格被“撑开”。LBS 在同一处倾向于把网格往轴心收（捏细），DQS 倾向于保持长度，所以同样一个弯曲，DQS 的关节区域看起来更“胖”。再加上真实皮肤在关节内侧本来会被压扁——LBS 的收缩反而“歪打正着”，DQS 不收缩，观感就更鼓；
- 不能表示缩放/剪切：单位对偶四元数只能表达“旋转 + 平移”。如果模型的骨骼本身带缩放动画，DQS 没法直接表达（见下面）；
- 管线支持不齐：GPU 蒙皮、游戏引擎里 LBS 是标配，DQS 往往要自己写着色器，导出格式（FBX/glTF）也不带 DQS 数据。

为什么“骨骼会缩放”？

这不是排版错误，也不是模型坏了。骨骼的缩放通常不是“骨头真的变长”，而是动画师或引擎在借它做别的事：

用途	例子
挤压与拉伸（squash & stretch）	卡通角色落地时压扁、起跳时拉长，用缩放做弹性
模拟软组织	屈肘时用骨骼缩放让肱二头肌“鼓”起来，比再加一排肌肉骨骼便宜
动作重定向	同一段动画套到高矮不同的角色上，靠缩放骨骼长度来适配比例
局部变形器	面部、手指的细微形变；绑定里用“缩放骨骼”驱动形变（scale-driven deformer）、负缩放做左右镜像
继承来的单位缩放	父级骨骼带了非 1 的 scale，子骨骼跟着继承——这类往往是导出/导入的副产物，并非有意设计

对 DQS 的含义很直接：缩放必须先被拆出去。常见工程做法是把每个骨头的变换分解成两部分——

$$\mathbf{S}_k = \underbrace{\mathbf{T}_k\mathbf{R}_k}_{\text{刚体部分，用 DQ 混合}} \cdot \underbrace{\mathbf{K}_k}_{\text{缩放/剪切，线性混合或线性插值}}$$

刚体部分走 DQS（保住旋转的刚体性），缩放部分走 LBS 式的线性混合（保住尺寸），最后复合。也有的管线干脆把缩放烘焙进骨骼长度或形态键里，让蒙皮只面对纯刚体变换。

> 这也是为什么很多引擎的“DQS 蒙皮”选项默认只在**没有缩放动画**的骨架上生效；一旦骨骼带缩放，要么自动退回 LBS，要么把缩放单独处理。

原始论文是 Kavan 等的 Geometric Skinning with Approximate Dual Quaternion Blending [11]，工程实现可参考其中的“DLB（dual quaternion linear blending）”近似。

5.4.5 其他四条路

除了 DQS，工业界还有四种常见做法，各自针对不同毛病：

1. 姿势校正 blendshape / PSD：不改蒙皮，而是“在 LBS 结果上再叠一个随姿势变化的位移”。SMPL 的 207 个 pose blendshape 就是它的线性版；Pose Space Deformation 是这一类方法的源头 [12]。
2. 辅助关节（helper joints）：在腋下、肩部插几个虚拟关节，把一个大角度拆成几个小角度，每个关节的扭转都不大，伪影自然小。
3. 体积保持蒙皮：在 LBS 的基础上加“保持局部体积”的约束，或者用更贵的非线性蒙皮（对数矩阵混合、球面混合）。
4. 学习式蒙皮：用网络直接预测每个顶点的位移，输入是姿势和邻域信息。它绕开了“线性混合”这个假设，但需要大量数据，也不保证可解释。

> **选型经验**：实时管线优先 LBS + 校正 blendshape（SMPL 就是这么干的）；对手部、肩部这类扭转大的部位，DQS 或辅助关节更省事；离线高质量渲染可以上学习式方法。

5.5 SMPL 里的 LBS

SMPL 把上面所有东西打包成一个函数：

$$M(\boldsymbol{\beta},\boldsymbol{\theta}) = W\big(T_P(\boldsymbol{\beta},\boldsymbol{\theta}),\, J(\boldsymbol{\beta}),\, \boldsymbol{\theta},\, \mathcal{W}\big). \tag{5.8}$$

右边可以分三步读：

1. $T_P(\boldsymbol{\beta},\boldsymbol{\theta})$：先根据形状参数和姿势，把模板顶点变形到一个“中间形状”；
2. $J(\boldsymbol{\beta})$：从形状回归出关节位置（不同体型的人，关节位置也不同）；
3. $W(\cdot)$：用 LBS 把中间形状蒙皮到最终姿势，$\mathcal{W}$ 是学好的权重。

第 8 章会把这三步展开成完整公式。这里你先记住：SMPL 不是一个黑箱，它是“形状 blendshape + 关节回归 + LBS + 姿势校正”的组合。

5.6 从 LBS 到动作捕捉

有了骨架和蒙皮，动作捕捉就变成了“求每个关节的旋转”：

$$\min_{\boldsymbol{\theta}} \sum_{j} \big\| \pi\big(\mathrm{FK}_j(\boldsymbol{\theta})\big) - \mathbf{j}_j^{\mathrm{obs}} \big\|^2, \tag{5.9}$$

其中 FK 是前向运动学（把局部旋转沿树乘成全局位置），π 是相机投影。这就是“从 2D 关键点反推 3D 骨架”的基本式。第 10 章会看到，人体重建还要同时优化形状 $\boldsymbol{\beta}$，并加上姿态先验，防止手肘反折。

> **一个容易忽略的点**：LBS 的输入是“每个关节相对父关节的旋转”，而不是“每个关节的全局位置”。前者天然满足骨骼长度不变，后者需要额外约束。这也是为什么动作数据通常存成轴角或四元数序列，而不是关节坐标。

本章小结

- 骨架是一棵树，全局变换 = 父变换 × 局部变换。
- 刚体绑定会在关节处撕裂；LBS 用权重把多个刚体变换混合起来。
- LBS 公式：$\mathbf{t}_i' = \sum_k w_{k,i}\mathbf{G}_k(\boldsymbol{\theta})\mathbf{G}_k(\mathbf{0})^{-1}\mathbf{t}_i$。
- 线性混合不是旋转，弯曲处会体积损失；扭转时会出现糖纸效应。
- 校正 blendshape、对偶四元数、辅助关节、学习式蒙皮是常见补救。
- SMPL = 形状 blendshape + 关节回归 + LBS + 姿势校正。

思考题

1. 如果某个顶点的权重是 $(0.5, 0.5)$，两根骨头相差 $180°$，这个顶点会到哪里？这说明了什么？
2. 为什么权重必须归一化？如果不归一化，模型会怎样？
3. 关节位置 $J(\boldsymbol{\beta})$ 为什么也依赖形状参数？可以用一个胖子和一个瘦子的肘部位置来想。
4. 对偶四元数蒙皮能完全消除体积损失吗？为什么？

第 6 章 统计形状模型：从 ASM 到 3DMM

本章导读：前面三章分别讲了“怎么表示”“怎么降维”“怎么变换”。这一章把它们合起来，回答一个更具体的问题：**如何从一堆扫描数据里，自动学出一个可调节的形状模型？** 这条线索从 1995 年的主动形状模型（ASM）开始，经过主动外观模型（AAM），到 1999 年的三维形变模型（3DMM），构成了今天所有人脸、人体、人手、动物参数化模型的共同祖先。

6.1 一个想法：形状是可以“统计”的

1990 年代初，医学图像分析面临一个难题：在 CT 或 MRI 里自动找到器官的轮廓。不同病人的器官形状不一样，但又有共同规律。Cootes、Taylor 等人想到：如果先把很多样本的轮廓对应起来，再用 PCA 描述它们的变化，就能得到一个“形状的先验”[1]。 搜索轮廓时，只在这个先验允许的范围内找，既快又不容易跑偏。

这就是统计形状模型（Statistical Shape Model，SSM）的核心。它包含三个要素：

1. 一组已经对应好的样本；
2. 一个均值形状；
3. 一组变化模式（主成分）及其允许范围。

6.2 对应：一切的起点

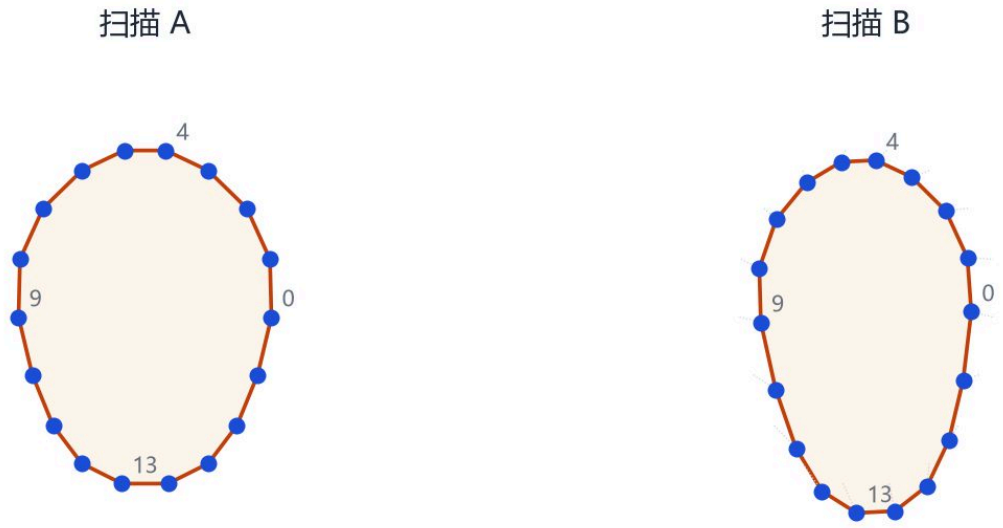


图 6-1 对应（correspondence）的含义。两张脸的轮廓都取了 18 个编号一致的点：0 号都在右侧脸颊，4 号都在额头中央，13 号都在下巴。只有编号含义一致，形状向量逐维相减才有意义。

假设你有两张人脸网格，一张 5000 个顶点，另一张 5200 个顶点。它们不能直接相减——维度都不一样。即使顶点数相同，如果第 100 号顶点在第一张脸上是鼻尖、在第二张脸上是嘴角，相减得到的数也没有任何解剖意义。

所以统计形状建模的第一步永远是对应（correspondence）：把所有样本变形到同一个模板上，使每个顶点在所有样本里都对应同一个解剖位置。常见做法有三类：

- 人工/半自动关键点：在每个人脸上标 68 个点，再用薄板样条（TPS）把模板 warp 过去。简单，但关键点之外靠插值，细节会糊。
- 非刚性配准（non-rigid ICP）：以模板为参考，迭代地找最近点、估计形变、最小化能量。Amberg 等人的“最优步长非刚性 ICP”是经典实现。
- 光流/多视角立体：对纹理清晰的扫描，用光流建立像素级对应，再反投影到三维。

对应质量直接决定模型质量。如果对应有系统性偏差（比如嘴角总是对到偏上 2 毫米），PCA 会把这种偏差当成“形状变化”学进去，模型就会莫名其妙地“歪嘴”。

> **一句话**：统计形状模型的上限，由对应质量决定；后面的 PCA、拟合都是在这个上限内做文章。

6.3 ASM：在允许的形状里搜索

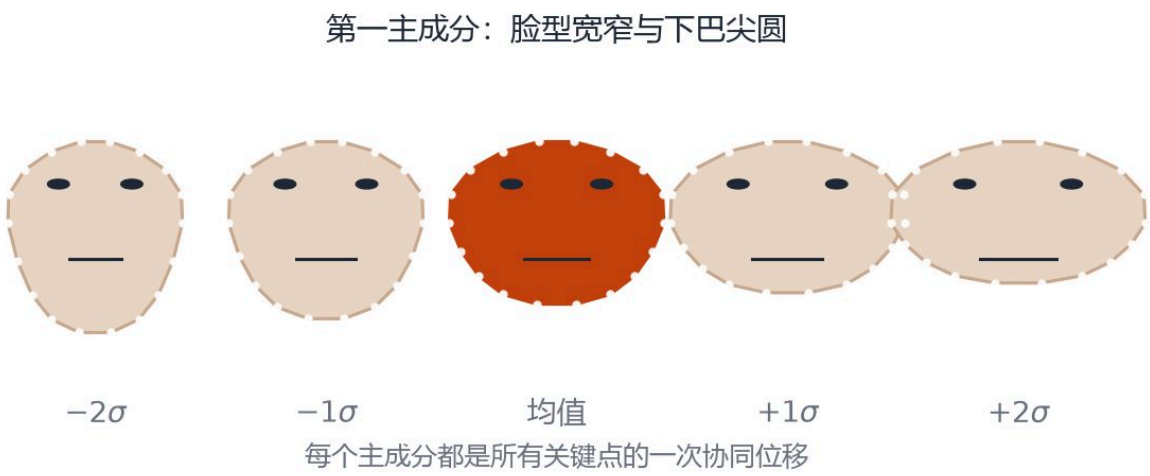


图 6-2 ASM 的形状空间。把所有样本的关键点展平成向量做 PCA，得到均值（中间）和若干主成分。沿第一主成分从 -2σ 走到 $+2\sigma$，脸型从窄长逐渐变宽圆。每个主成分都是所有关键点的一次协同位移。

ASM 把形状表示成 n 个关键点的坐标串：

$$\mathbf{x} = (x_1, y_1, x_2, y_2, \ldots, x_n, y_n)^\top \in \mathbb{R}^{2n}. \tag{6.1}$$

对所有训练样本做 Procrustes 对齐（去掉平移、旋转、缩放），再跑 PCA：

$$\mathbf{x} \approx \bar{\mathbf{x}} + \mathbf{P}\mathbf{b}, \qquad \mathbf{P} = [\mathbf{p}_1, \ldots, \mathbf{p}_t]. \tag{6.2}$$

$\mathbf{b}$ 是形状参数。为了避免生成不合理的形状，通常限制每个参数在 $\pm 3\sqrt{\lambda_i}$ 以内（λ_i 是对应的特征值），这相当于“只走 3 个标准差”。

搜索时，ASM 做两件事交替进行：

1. 局部搜索：沿每个关键点的法线方向采样灰度，找边缘最强的位置；
2. 模型约束：把找到的点拟合回式 (6.2) 的形状空间，去掉不符合先验的位移。

这两句话信息密度很高，下面拆成五小节讲：为什么非要“搜索”、两步各在算什么、为什么必须交替、代码长什么样、什么时候会失败。

6.3.1 为什么需要“搜索”

先明确目标：给一张没见过的图像 $\mathbf{I}$，输出 n 个关键点的位置 $\mathbf{x}$。我们手上只有两样东西：

- 形状空间（式 6.2）：告诉我们“什么样的形状是合法的”，但它不知道这张图里的人在哪、多大、什么姿势；
- 局部外观模型：对每个关键点，训练时学到了“沿法线方向的灰度长什么样”（比如眼角处灰度先降后升），但它只描述一个点附近，不知道全局。

任何一样单独都不够：只用形状空间，你只能画出平均脸，位置完全对不上；只用局部外观，每个点都会跑向离它最近的强边缘，整组点会散架（下巴的点跑到脖子上）。

所以 ASM 选择迭代：从初始猜测出发，让图像证据把点“拉”向边缘，再让形状先验把它们“拽”回合法形状，重复到不再移动。这个过程就是“搜索”——在形状空间允许的集合里，找让图像证据最好的那组参数。

> 一句话区分两种思路：**搜索/拟合**是“先有模型，再让模型去贴合图像”；**检测/回归**是“直接从图像预测参数”。ASM 属于前者（慢、可控），第 10 章的 HMR 属于后者（快、吃数据）。两者不是替代关系，而是互补：后来的 SPIN 就是把优化器塞进了回归网络。

6.3.2 第 1 步（局部）：沿法线找边缘

对每个关键点 i：

1. 算法线：用相邻两个关键点估计切线，再取它的垂线，得到单位法线 $\mathbf{n}_i$（朝向轮廓外侧）。
2. 采样 profile：在 $\mathbf{x}_i + \delta\,\mathbf{n}_i$ 处取 $2k+1$ 个灰度值（典型 $k = 2 \sim 4$，即 5–9 个像素），得到一条一维灰度曲线 $\mathbf{g}_i(\delta)$。
3. 和训练时学到的 profile 比：训练集里每个点都有一条平均 profile $\bar{\mathbf{g}}_i$ 和它的协方差 $\mathbf{S}_i$。取马氏距离最小的偏移：

$$\delta_i^{*} = \arg\min_{\delta} \; \big(\mathbf{g}_i(\delta) - \bar{\mathbf{g}}_i\big)^{\top} \mathbf{S}_i^{-1} \big(\mathbf{g}_i(\delta) - \bar{\mathbf{g}}_i\big). \tag{6.3}$$

4. 得到建议位置：$\mathbf{y}_i = \mathbf{x}_i + \delta_i^{*}\mathbf{n}_i$。

三个“为什么”：

- 为什么只沿法线走：轮廓的移动主要是法向的；沿切线方向滑动几乎不改变轮廓形状，却会让点“滑走”（比如从鼻翼滑到脸颊）。
- 为什么用马氏距离而不是灰度差：不同部位、不同纹理的灰度对比度差很多。$\mathbf{S}_i^{-1}$ 按训练时的方差归一化——某个点天生对比度低，就自动放宽容差；天生锐利，就严格。这正是“按数据自己的尺度衡量”的意思（第 3 章 3.3.2 讲过）。
- 为什么这条一维曲线够用：对轮廓来说，最有判别力的信息就是“沿法线跨过边界时灰度怎么变”。这就是 ASM 的名字来源：Active Shape Model，只有形状，没有纹理。

6.3.3 第 2 步（全局）：把结果投影回形状空间

现在我们有了一组“证据位置” $\mathbf{y}$。它们可能根本不成形状：有的点被强边缘拉远了，有的点没动。

于是做一次最小二乘拟合，把它们拉回形状空间。因为 $\mathbf{P}$ 的列正交（$\mathbf{P}^{\top}\mathbf{P} = \mathbf{I}$），解是闭式的：

$$\mathbf{b} = \mathbf{P}^{\top}\big(\mathbf{y} - \bar{\mathbf{x}}\big), \qquad b_i \leftarrow \mathrm{clip}\left(b_i,\, -3\sqrt{\lambda_i},\, +3\sqrt{\lambda_i}\right), \qquad \mathbf{x} = \bar{\mathbf{x}} + \mathbf{P}\mathbf{b}. \tag{6.4}$$

- 第一式：把“证据位移”投影到前 t 个主成分上——只保留形状空间能解释的部分；

- 第二式：把每个系数裁剪到 $\pm 3\sigma$，防止外插出不像人脸的形状；
- 第三式：用裁剪后的参数重建一组新点。

还没完：$\mathbf{y}$ 在图像坐标里，而形状空间是在对齐后的坐标系里。所以每轮还要更新相似变换（缩放 s、旋转 $\mathbf{R}$、平移 $\mathbf{t}$），用 Procrustes 对齐 $\mathbf{x}$ 与 $\mathbf{y}$：

$$(s, \mathbf{R}, \mathbf{t}) = \arg\min \sum_i \left\| s\mathbf{R}\mathbf{x}_i + \mathbf{t} - \mathbf{y}_i \right\|^2.$$

然后 $\mathbf{x} \leftarrow s\mathbf{R}\mathbf{x} + \mathbf{t}$。这一步让模型能“变大变小、转一点、挪过去”，否则它永远只能贴合初始位置附近的轮廓。

6.3.4 为什么必须交替：证据与先验的拔河

单独做任何一步都会坏：

只做局部搜索	只做形状约束
每个点独立跑向最近的强边缘	形状始终停在均值附近
遮挡处、眼镜框、发出的强边缘会把点拉飞	永远无法贴合个体差异（脸宽、下巴长短）
整组点散架，不再是“脸”	图像证据完全没用上

交替的结果是两者折中：局部步负责“贴合这张图”，全局步负责“像一张脸”。权重关系由形状空间的维度 t 和 $\pm 3\sigma$ 的裁剪共同决定——主成分越少，先验越强，结果越“标准”也越僵。

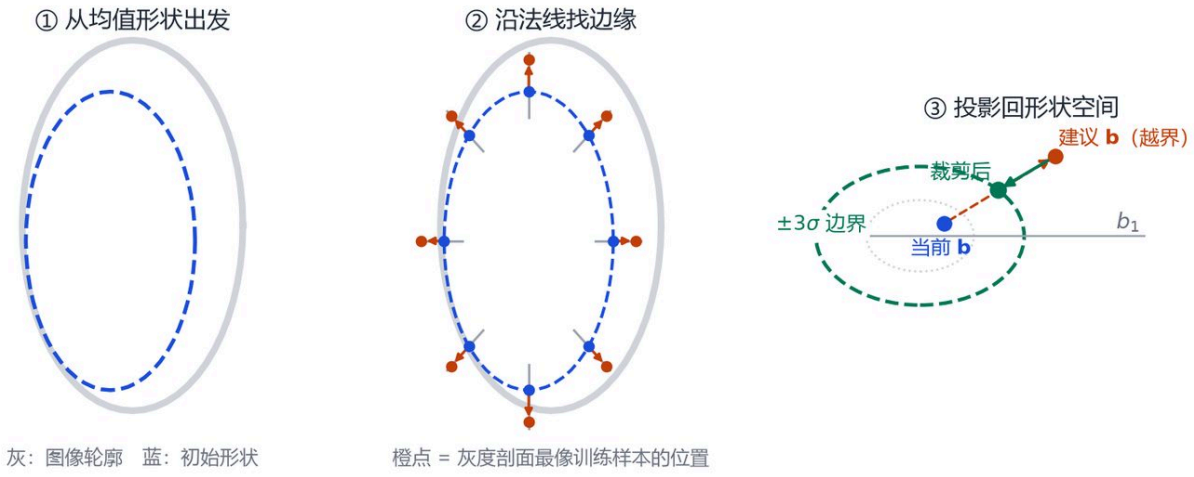


图 6-4 ASM 一轮迭代的三步。① 从均值形状出发；② 每个关键点沿法线采样灰度，找到边缘最强的偏移（箭头），得到一张“建议形状”；③ 把建议形状投影回形状空间，超出 $\pm 3\sigma$ 的部分被裁掉，得到合法形状，再进入下一轮。

6.3.5 伪代码与工程细节

```
x = init_from_detector(I)          # 人脸框 + 均值形状缩放
for it in range(max_iter):         # 典型 10-30 次
    for i in range(n):
        n_i = normal(x[i-1], x[i+1])          # 由相邻点估法线
        g = sample_profile(I, x[i], n_i, k)   # 2k+1 个灰度
        d = best_offset(g, mean_profile[i], cov[i])   # 式 (6.3)
        y[i] = x[i] + d * n_i                 # 建议位置
    b = P.T @ (y - xbar)                      # 式 (6.4)
    b = clip(b, -3*sd, 3*sd)
    x = xbar + P @ b
    s, R, t = procrustes(x, y)                # 更新位姿
    x = s * R @ x + t
    if max_abs(x - x_prev) < eps: break
```

工程上的几个经验：

- 多分辨率：先在半分辨率图上跑几轮（profile 更平滑、更稳），再到全分辨率细化。否则细节纹理容易把点带偏。
- 初始化很关键：用现成的人脸检测器给框，再把均值形状缩放塞进去；初始化偏差太大时，局部搜索会收敛到错误的边缘。
- 每轮都更新相似变换：只拟合 $\mathbf{b}$ 不更新位姿，模型无法跟随人脸的位移与尺度。
- 收敛判据：所有点位移的最大值小于阈值（比如 0.5 像素），或达到最大迭代次数。

6.3.6 什么时候会失败

- 初始化太远：大角度侧脸、极端表情，局部搜索一开始就抓错边缘；
- 强边缘干扰：眼镜框、刘海、胡须、口罩，都会给出比真实轮廓更强的梯度；
- 遮挡：被挡住的关键点没有证据，只能靠先验“猜”，一猜就可能全错；
- 光照与纹理：只用一维灰度，光照一变、或者部位本身很平（脸颊），profile 就失效；
- 形状空间覆盖不足：训练集里没有这种脸型/表情，3σ 之外的真值被裁掉，永远拟合不上。

后面几条正是 AAM 的动机：与其只看一条法线上的灰度，不如把整块纹理都统计进来。

6.3.7 AAM：把纹理也统计进来

ASM 只看轮廓，不看五官内部。AAM 把形状和纹理一起建模 [13]：先把每张脸 warp 到平均形状，得到“形状归一化后的纹理”，再对纹理做 PCA。最终模型同时能生成形状和外观，可以用“分析—合成”的方式拟合：

$$\min_{\mathbf{b}_s,\mathbf{b}_a} \left\| \mathbf{A}(\mathbf{x}(\mathbf{b}_s);\mathbf{b}_a) - \mathbf{I} \right\|^2. \tag{6.5}$$

AAM 在约束环境下（正面、光照稳定）非常有效，但一到自然场景就容易崩：纹理受光照、遮挡、表情影响太大。

6.4 3DMM：把统计形状搬到三维

1999 年，Blanz 与 Vetter 在 SIGGRAPH 上发表 A Morphable Model for the Synthesis of 3D Faces，把 ASM 的思想从二维轮廓搬到三维人脸。这就是 3D 形变模型（3D Morphable Model，3DMM）。

它的形状表示是：

$$\mathbf{S} = \bar{\mathbf{S}} + \sum_{i=1}^{m} \alpha_i \mathbf{S}_i, \tag{6.6}$$

纹理表示是：

$$\mathbf{T} = \bar{\mathbf{T}} + \sum_{i=1}^{m} \beta_i \mathbf{T}_i, \tag{6.7}$$

其中 $\bar{\mathbf{S}}$ 是平均脸，$\mathbf{S}_i$ 是形状主成分，α_i 是形状系数；纹理同理。后来为了处理表情，又加了表情基 $\mathbf{B}_j$：

$$\mathbf{S} = \bar{\mathbf{S}} + \sum_{i} \alpha_i \mathbf{S}_i + \sum_{j} \gamma_j \mathbf{B}_j. \tag{6.8}$$

训练流程和 ASM 一样，只是数据换成了三维扫描：



图 6-3 统计形状模型的训练流程。扫描 → 配准到统一模板 → 展平成向量 → PCA 得到基 → 用少量系数生成新形状。每一步都可能引入误差，其中“对应”是最关键的一步。

Blanz 与 Vetter 当年只用了约 200 个扫描，就得到了能生成逼真人脸、做表情迁移、做姿态校正的模型。这个数字今天看起来很小，但它证明了一件事：人脸的变化高度集中，少量主成分就能覆盖大部分差异。

6.4.1 3DMM 怎么拟合一张照片

给定一张人脸照片，3DMM 的拟合是一个“分析—合成”循环：

1. 猜一组形状、纹理、姿态、光照参数；
2. 用这些参数渲染出一张图；
3. 比较渲染图与输入图的像素差；
4. 根据差值调整参数，重复。

这就是式 (1.2) 在人脸上的具体化。1999 年的实现用随机梯度下降，速度慢；今天的实现用可微渲染 + 神经网络，可以实时。

拟合的难点在于身份、表情、姿态、光照纠缠在一起：同一张脸，换个光照可能被解释成“鼻子变高”。所以后来的模型会显式分离这些因素（第 15、16 章会讲 FLAME 和 DECA 怎么处理）。

6.5 从线性到非线性

线性模型有一个根本限制：它只能表示“沿主成分方向的直线变化”。如果真实数据分布是弯曲的，线性模型要么欠拟合，要么在参数空间里产生不存在的“中间形状”。

三类扩展值得知道：

- 核 PCA：先用核函数把数据映射到高维，再做 PCA。能捕捉非线性，但需要选核、计算量大。
- 自编码器（autoencoder）：用神经网络学习编码器 $\mathbf{z} = f(\mathbf{x})$ 与解码器 $\hat{\mathbf{x}} = g(\mathbf{z})$。本质上是“学出来的非线性 PCA”。第 25 章会讲它在人体生成里的应用。
- 扩散模型：直接学习数据分布，用去噪过程生成样本。它不再有“主成分”的概念，但可以以 PCA 模型为条件（例如在 SMPL 参数空间上做扩散）。

即便有了这些非线性工具，线性 PCA 仍然是工业界的主力：它可解释、可求逆、计算快，而且在“形状差异”这种相对平滑的分布上，线性已经够好。

6.6 怎么评价一个统计形状模型

评价通常看三个方面：

1. 紧凑性（compactness）：用多少个主成分能达到给定的重建精度。越少越好。
2. 泛化性（generalization）：用留出集测试，模型能否重建没见过的形状。常用“留一法”：每次去掉一个样本训练，看对它重建的误差。
3. 特异性（specificity）：从模型随机采样生成的形状，是否都像“合理的人”。如果生成了外星人，说明参数范围太宽或数据有偏。

这三个指标互相制约：参数越多，紧凑性越差但泛化可能更好；约束越强，特异性好但可能欠拟合。

本章小结

- 统计形状模型 = 对应好的样本 + 均值 + 主成分 + 参数范围。

- 对应是前提；没有顶点级对应，就没有可解释的形状向量。
- ASM 用 PCA 约束关键点搜索；AAM 把纹理也纳入统计。
- 3DMM 把同样思想搬到三维人脸：$\mathbf{S} = \bar{\mathbf{S}} + \sum \alpha_i \mathbf{S}_i + \sum \gamma_j \mathbf{B}_j$。
- 线性模型的局限催生了核方法、自编码器和扩散模型，但线性 PCA 仍是工业主力。
- 评价看紧凑性、泛化性、特异性。

思考题

1. 如果训练样本没有对齐（有人脸大、有人脸小），PCA 的第一主成分会是什么？
2. 为什么 ASM 要把搜索到的点“投影回”形状空间，而不是直接采用？
3. 3DMM 的形状和纹理分别建模，会丢失什么关联？后来的人脸模型如何弥补？
4. “特异性”差会带来什么实际问题？举一个拟合时的例子。

第二篇　人体

从 SCAPE 到 SMPL，再到可驱动的神经化身

第 7 章　SCAPE：把「形状」和「姿态」拆开
第 8 章　SMPL：一个公式统治人体
第 9 章　SMPL 家族：从身体到手，再到脸
第 10 章　从单张图片反推人体：优化法与回归法
第 11 章　视频、多人、全身与度量尺度
第 12 章　数据集、评估指标与常见坑
第 13 章　着装人体：衣服、头发与神经渲染
第 14 章　人体动作与生成模型

第 7 章 SCAPE：把「形状」和「姿态」拆开

> **本章导读**：SMPL 经常被当作人体参数化的起点，但在它之前，SCAPE 才是第一个真正把“体型”和“姿势”分离、并能生成任意体型摆任意姿势的模型。这一章讲清楚 SCAPE 为什么重要、它用了什么技巧、又为什么最终被更简单的 SMPL 取代。读完你会明白：**好的模型不是参数越多越好，而是把不该纠缠的因素分开。**

7.1 在 SCAPE 之前：只有“形状”，没有“姿势”

2003 年，Allen、Curless 与 Popović 在 SIGGRAPH 上发表 The Space of Human Body Shapes[3]。他们用全身扫描仪扫描了大量志愿者，把所有网格配准到同一拓扑，然后对顶点坐标做 PCA，得到一个人体形状空间。

这个模型能做什么？给定几个滑块，生成不同身高、胖瘦、肩宽的人。听起来已经很接近今天的 SMPL 了。

但它有一个致命限制：所有人必须站在同一个姿势。因为 PCA 是对顶点坐标做的，一旦有人抬手，手臂的顶点就会跑到完全不同的位置，PCA 会把这部分差异误当成“体型差异”学进去。结果就是：你滑动“手臂粗细”滑块，模型可能会同时把手臂抬起来。

要解决这个问题，必须把“姿势”从“形状”里剥离出来。这正是 SCAPE 的贡献。

7.2 SCAPE 的核心思想

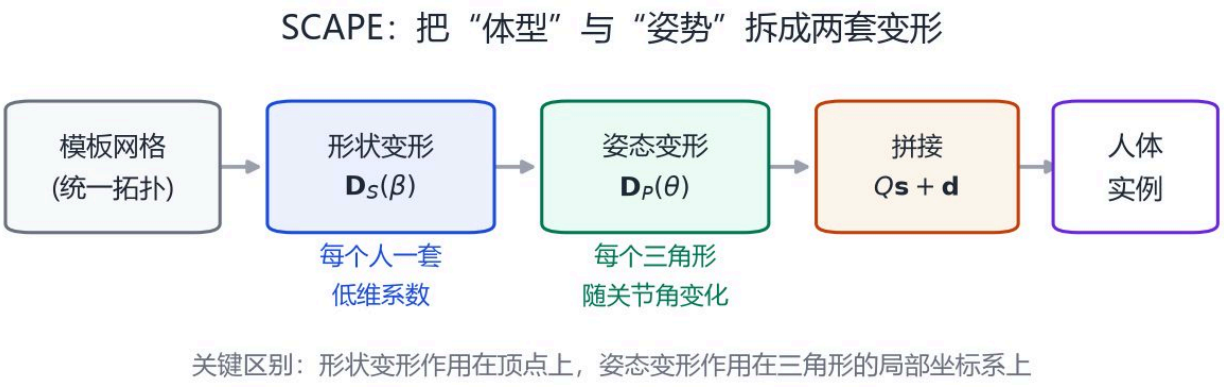


图 7-1 SCAPE 的分解。模板网格先经过形状变形得到某个人的体型，再经过姿态变形摆出姿势。形状变形是顶点级的、低维的；姿态变形是三角形级的、随关节角变化的。

2005 年，Anguelov、Srinivasan、Koller、Thrun、Rodgers 与 Davis 提出 SCAPE（Shape Completion and Animation of PEople）[4]。它的出发点是：

> **一个人的网格 = 一个模板 + 体型差异 + 姿势引起的变形。**

用公式写就是

$$\mathbf{V} = F_{\text{shape}}(\boldsymbol{\beta}) + F_{\text{pose}}(\boldsymbol{\theta}), \tag{7.1}$$

但注意，这里的加号不是简单的逐顶点相加，而是通过三角形的局部变形来实现的。

7.2.1 为什么作用在三角形上

如果像 LBS 那样直接对顶点做线性混合，弯曲处会出现第 5 章讲的“捏细”。SCAPE 换了一个思路：不直接移动顶点，而是规定每个三角形如何变形。

具体来说，对模板上的每个三角形 t，SCAPE 学习一个 3×3 的变形矩阵 $\mathbf{Q}_t(\boldsymbol{\theta})$，它描述“这个三角形在当前姿势下被旋转、拉伸、剪切成了什么样”。然后，寻找一组顶点位置，使得每个三角形的实际形状尽量接近 $\mathbf{Q}_t$ 规定的形状。

这是一个最小二乘问题：

$$\min_{\mathbf{V}} \sum_{t} \sum_{(i,j)\in t} \left\| \mathbf{Q}_t(\mathbf{v}_i - \mathbf{v}_j) - \mathbf{D}_t(\mathbf{v}_i^0 - \mathbf{v}_j^0) \right\|^2. \tag{7.2}$$

其中 $\mathbf{v}_i^0$ 是模板顶点，$\mathbf{D}_t$ 是形状变形。这个式子的意思是：每个三角形的每条边，都应该按照 $\mathbf{Q}_t\mathbf{D}_t$ 的方式被变换；所有三角形不可能同时完美满足，所以求一个全局最优的顶点位置。

这类似泊松重建：给定每个面片的形变，反解出全局一致的顶点。好处是三角形不会像 LBS 那样被“平均”到塌陷，弯曲和扭转都更自然。

7.2.2 姿态变形从哪里来

$\mathbf{Q}_t(\boldsymbol{\theta})$ 不是手写的，而是学出来的。SCAPE 的做法是：

1. 用动作捕捉数据驱动一个骨架，得到大量“模板 + 已知关节角”的配对；
2. 对每个三角形，计算它在每个姿势下相对静止姿势的变形；
3. 把这些变形表示成关节角的函数（论文用线性回归 + 局部坐标）；
4. 对新姿势，插值/回归出每个三角形的 $\mathbf{Q}_t$。

这样，模型就学到了“肘弯 90° 时，肘部外侧的三角形应该拉伸多少、内侧应该压缩多少”。它比 LBS 的刚体假设更贴近真实皮肤。

7.2.3 形状变形

形状部分相对简单：把所有扫描配准到模板姿势，对顶点偏移做 PCA，得到低维形状空间。因为姿势已经被“摆正”，PCA 学到的就只剩体型差异。

最终，SCAPE 可以：

- 输入一组形状系数，生成不同体型；
- 输入一组关节角，摆出不同姿势；
- 两者自由组合，生成“这个体型做这个动作”的网格。

7.2.4 式 (7.2) 到底怎么解

式 (7.2) 看起来很唬人，但它对未知量 $\mathbf{V}$ 是二次的（每一项都是顶点坐标的线性组合再平方），所以解出来就是一个线性方程组。把每个三角形的三组边叠在一起写成矩阵形式：

$$\min_{\mathbf{V}} \left\| \mathbf{A}\,\mathbf{V} - \mathbf{B} \right\|^2 \quad \Longrightarrow \quad \mathbf{A}^{\top}\mathbf{A}\,\mathbf{V} = \mathbf{A}^{\top}\mathbf{B}. \tag{7.3}$$

- $\mathbf{A}$ 是一个很大的稀疏矩阵：行对应“三角形的边”，列对应“顶点坐标”；每一行只有两个非零元素（因为一条边只涉及两个顶点）；
- 右边 $\mathbf{B}$ 由 $\mathbf{Q}_t\mathbf{D}_t(\mathbf{v}_i^0 - \mathbf{v}_j^0)$ 构成，也就是“每个三角形的边应该变成什么”；
- 求解 $\mathbf{A}^{\top}\mathbf{A}\mathbf{V} = \mathbf{A}^{\top}\mathbf{B}$ 用共轭梯度或 Cholesky 分解，顶点越多越慢。

> **为什么不能直接对每个三角形独立地把顶点算出来？** 因为同一个顶点被好几个三角形共用。每个三角形都希望这个顶点在自己那边，但它只能有一个位置——所以必须全局折中，这就是“稀疏线性系统”的来源，也正是 SCAPE “贵”的原因。

把这件事用一句更普通的话说：SCAPE 把“怎么变形”和“变形后顶点在哪里”分开了。前者（$\mathbf{Q}_t$）是从数据学的、与姿势有关；后者（$\mathbf{V}$）是一次线性求解。这个拆分思路后来被反复借用：今天的神经形变模型也常常是“先预测每个局部的变换，再解一个全局一致的网格”。

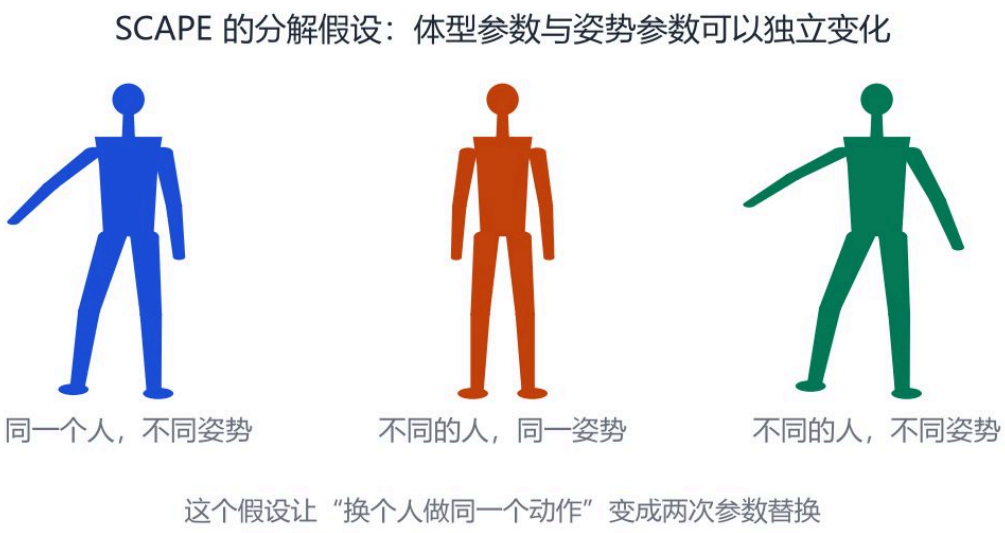


图 7-2 分解假设的直观检验。同一体型可以摆不同姿势，同一姿势可以由不同体型完成；如果模型学得好，“换体型”和“换姿势”应该互不干扰。

7.3 SCAPE 的贡献与代价

SCAPE 在 2005 年是里程碑式的，它确立了三件事：

1. 形状与姿态可以分离，而且分离后可以自由组合；
2. 形变可以作用在局部（三角形）而不是全局（顶点），从而避免线性混合的伪影；
3. 统计模型可以从扫描 + 动作捕捉中联合学习，不需要手工建模。

但它的代价也很明显：

- 计算贵：每次生成都要解一个大型最小二乘，难以实时；
- 不可微：整个流程包含配准、回归、求解，端到端训练困难；
- 依赖骨架和动作数据：姿态变形的质量取决于动作捕捉数据的覆盖范围；
- 实现复杂：三角形级变形、局部坐标、形状/姿态耦合，工程门槛高。

这些代价在 2015 年被 SMPL 用一种“更简单但足够好”的方案化解了。

7.4 中间十年：Hasler 与 FAUST

在 SCAPE 与 SMPL 之间，还有两项工作值得记住。

Hasler 等（2009） 提出 A Statistical Model of Human Pose and Body Shape[14]：他们把体型参数和姿势参数都做成线性模型，并显式地建模两者的耦合。这篇工作让“用统计模型同时描述形状和姿态”变得更清晰。

FAUST（2014） 则提供了第一个被广泛使用的评测基准 [15]：一批高精度人体扫描，每个扫描都配准到同一拓扑，并带有真实的三维对应。FAUST 让不同方法可以在同一个标准上比较“重建得准不准”，推动了整个领域从“看起来像”走向“量得准”。



图 7-3 从人体形状空间到 SMPL 的关键节点。SCAPE 提出了形状/姿态分解，FAUST 提供了评测基准，SMPL 用更简洁的公式把两者结合并做到实时。

7.5 为什么 SCAPE 最终输给了 SMPL

SCAPE 更“物理”，SMPL 更“工程”。SMPL 赢在三点：

1. 简单：LBS + 线性 blendshape，一个公式写完，GPU 上实时；
2. 可微：所有操作都是矩阵乘法和加法，可以直接接神经网络；
3. 够好：虽然 LBS 有体积损失，但加上姿势校正 blendshape 后，视觉上已经足够逼真，而且更容易大规模训练。

这不是说 SCAPE 的思路错了。恰恰相反，SMPL 的姿势校正 blendshape 本质上就是“用数据学一个简化的 SCAPE 姿态变形”。SCAPE 提出了问题，SMPL 给出了可规模化的答案。

本章小结

- Allen 2003 建立了人体形状空间，但要求所有人同一姿势。
- SCAPE 把形状与姿态分解：形状作用在顶点上，姿态作用在三角形的局部坐标系上。
- 姿态变形从动作捕捉数据中学习，能表现弯曲、扭转等真实皮肤形变。
- 式 (7.2) 对顶点是二次的，最终化成一次稀疏线性求解；顶点被多个三角形共用，所以必须全局折中。

- SCAPE 计算贵、不可微、实现复杂；SMPL 用 LBS + 校正 blendshape 换取实时与可微。
- FAUST 提供了第一个被广泛采用的对应与重建评测基准。

思考题

1. 为什么对顶点坐标直接做 PCA 会把姿势混进形状？
2. SCAPE 的三角形变形与 LBS 的顶点混合，分别在什么情况下更自然？
3. 如果让你用今天的可微渲染工具重新实现 SCAPE，哪些部分可以端到端训练？
4. SMPL 的“姿势校正 blendshape”与 SCAPE 的“姿态变形”有什么本质联系？

第 8 章 SMPL：一个公式统治人体

> **本章导读**：SMPL 是过去十年被引用最多的人体模型，也是几乎所有人体重建、动作生成、数字人系统的底座。这一章把 SMPL 的公式拆成四步，逐项解释每个矩阵的维度和含义，并说明它是怎么被训练出来的。读完你应该能回答：为什么是 6890 个顶点、10 个形状参数、207 个姿势 blendshape？这些数字不是魔法，都是可以解释的选择。

8.1 为什么还需要一个新模型

SCAPE 已经能分离形状和姿态，为什么 2015 年还要做 SMPL[5]？

因为 SCAPE 太“重”了：每次生成要解三角形级的最小二乘，不能实时；整个流程不可微，没法接神经网络；实现复杂，研究者和工程师都难以复现。

SMPL 的目标很明确：做一个足够简单、足够快、可微、又能被大规模数据训练的人体模型。 它的答案是把 SCAPE 的“三角形变形”换成“顶点级的线性 blendshape + 标准 LBS”，牺牲一点物理精度，换取工程上的巨大便利。

今天你看到的绝大多数人体重建论文，输出层都是 SMPL 的 10 个 β 和 72 个 θ。这不是因为它们只会用 SMPL，而是因为 SMPL 已经成了这个领域的“文件格式”。

8.2 输入与输出

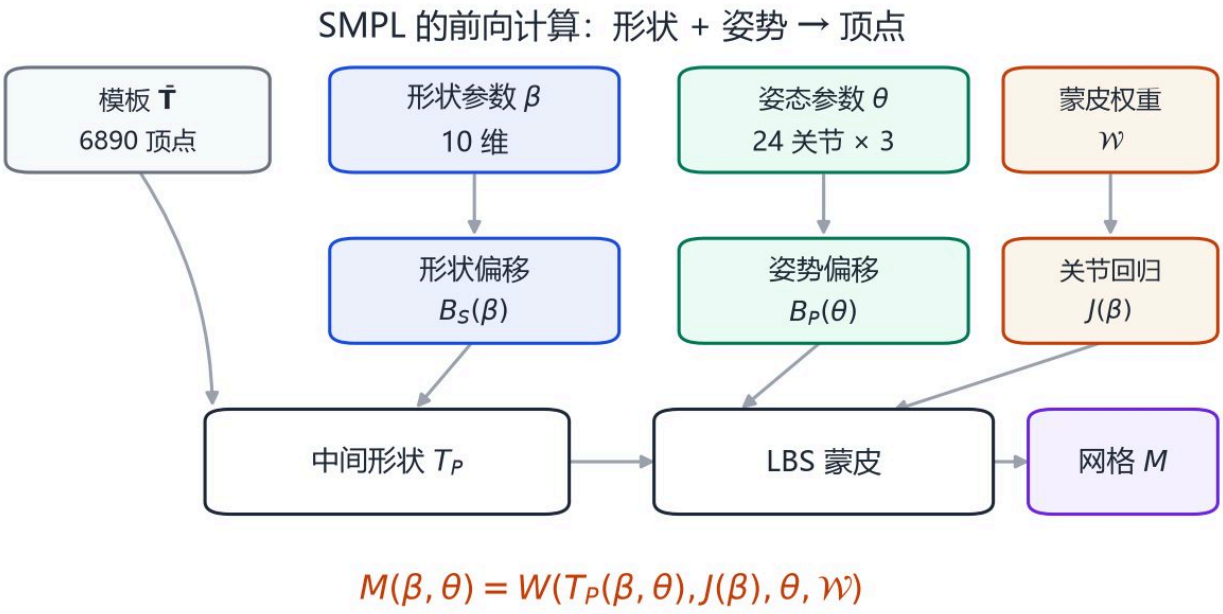


图 8-1 SMPL 的前向计算流程。模板、形状参数、姿态参数、蒙皮权重、关节回归器共同作用，最终输出一张 6890 个顶点的网格。注意形状偏移与姿势偏移是相加的，LBS 在最后一步。

SMPL 的输入只有两组参数：

- 形状参数 $\boldsymbol{\beta} \in \mathbb{R}^{10}$：控制高矮胖瘦；
- 姿态参数 $\boldsymbol{\theta} \in \mathbb{R}^{72}$：24 个关节，每个 3 个轴角。

输出是一张网格：6890 个顶点，每个顶点 3 个坐标，共 20670 个数。此外还有 24 个关节的三维位置，以及每个顶点的蒙皮权重。

整个映射记作

$$M(\boldsymbol{\beta},\boldsymbol{\theta}) : \mathbb{R}^{10} \times \mathbb{R}^{72} \to \mathbb{R}^{6890\times 3}. \tag{8.1}$$

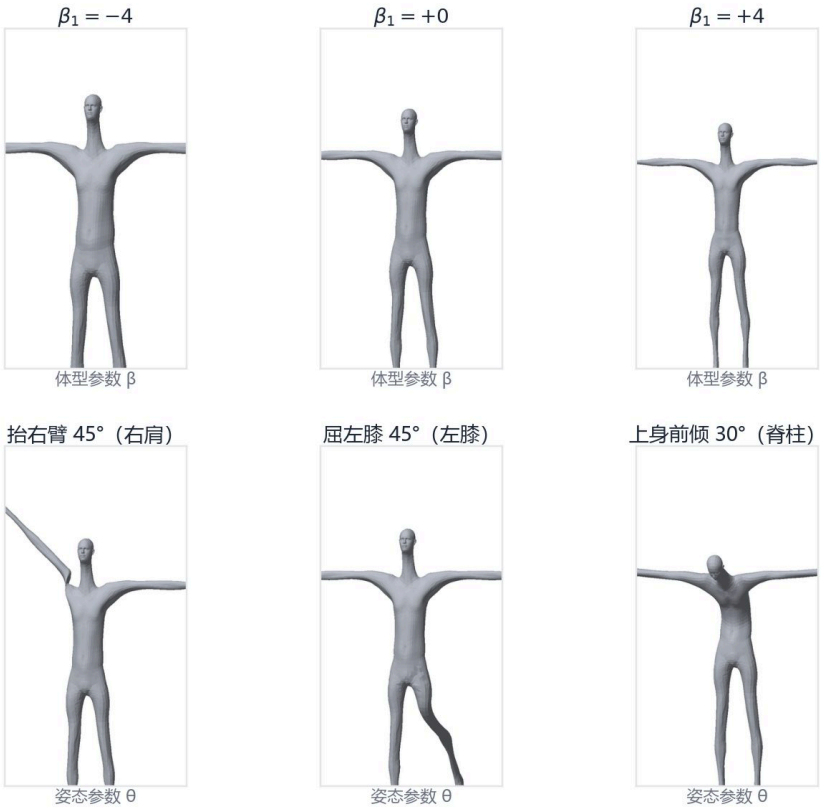


图 8-4 β 与 θ 到底长什么样。上排：同一个 T-pose，只把第一个形状参数 β_1 分别取 -4、0、$+4$（相机比例固定，所以体型差异能直接比出来）；下排：同一个体型，只改一个关节的旋转（抬右臂、屈左膝、上身前倾）。图中网格由官方 SMPL 模型渲染，仅用于非商业的学习与研究。

一眼就能看出两件事：β 改的是“这个人是谁”，θ 改的是“他在做什么”。上排三个人的姿势完全一样，但肩宽、胸围、腿粗都在变；下排是同一个人，只是某几个关节转了 30–45°。这两组参数互不干扰，正是第 7 章讲的“形状与姿态解耦”在工程上的样子。

8.3 第一步：形状 blendshape

SMPL 有一个模板 $\bar{\mathbf{T}}$，是一张“平均体型、静止姿势”的网格。形状参数先对模板做一次线性偏移：

$$\mathbf{T}_S = \bar{\mathbf{T}} + \mathbf{B}_S(\boldsymbol{\beta}), \qquad \mathbf{B}_S(\boldsymbol{\beta}) = \sum_{n=1}^{10} \beta_n \mathbf{S}_n. \tag{8.2}$$

其中每个 $\mathbf{S}_n$ 都是一个 6890×3 的矩阵，表示“第 n 个形状主成分对应的全身位移”。β_n 是系数。

这一步和你在第 3 章学的 PCA 完全一样：$\mathbf{S}_n$ 就是主成分，β_n 就是投影坐标。区别只在于，SMPL 的主成分是从大量已配准的人体扫描上学的，而不是从二维点云。

> **为什么是 10 维？** 这是精度与鲁棒性的折中。维度太少，无法区分不同体型；维度太多，从单张图反推时容易过拟合，而且参数空间会出现不合理的形状。10 维在实践中已经能覆盖大部分成年人的体型差异。

8.4 第二步：从形状回归关节

不同体型的人，关节位置不一样。胖子的髋关节可能更靠外，高个子的膝关节更长。所以 SMPL 不把关节位置写死，而是从形状回归：

$$\mathbf{J}(\boldsymbol{\beta}) = \mathcal{J}\big(\bar{\mathbf{T}} + \mathbf{B}_S(\boldsymbol{\beta})\big). \tag{8.3}$$

$\mathcal{J}$ 是一个线性回归器：它取若干个顶点（比如骨盆附近、膝盖附近），用学好的权重加权，得到关节位置。这样，体型一变，关节也跟着动，蒙皮才不会错位。

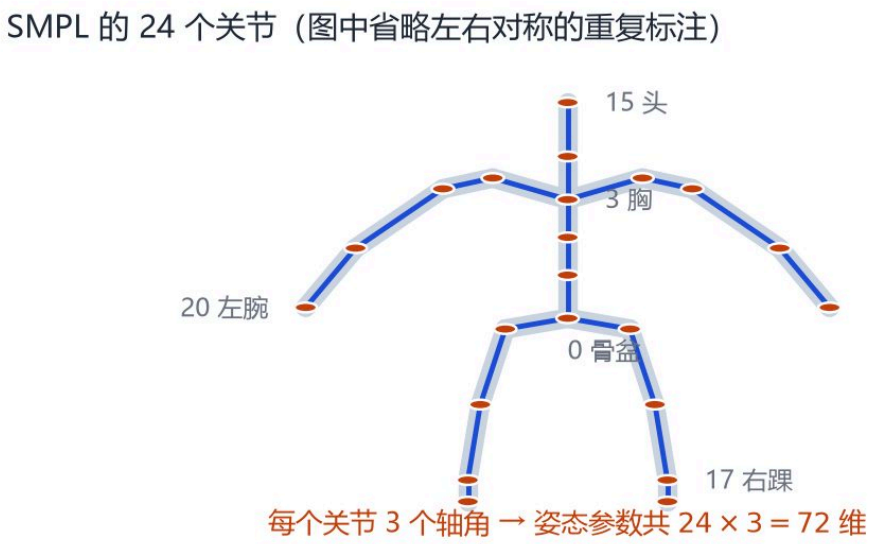


图 8-2 SMPL 的 24 个关节。根节点是骨盆，向上是脊柱、胸、颈、头，向两侧是锁骨、肩、肘、腕，向下是髋、膝、踝、脚。每个关节贡献 3 个轴角，共 72 维。

8.5 第三步：姿势 blendshape

如果只有 LBS，弯曲肘部时会出现第 5 章讲的体积损失。SMPL 的解决办法是：在 LBS 之前，先根据姿势加一个修正位移。

$$\mathbf{T}_P = \mathbf{T}_S + \mathbf{B}_P(\boldsymbol{\theta}), \qquad \mathbf{B}_P(\boldsymbol{\theta}) = \sum_{n=1}^{207} \big(R_n(\boldsymbol{\theta}) - R_n(\boldsymbol{\theta}^*)\big)\mathbf{P}_n. \tag{8.4}$$

这里有三个关键点。

第一，207 是怎么来的。 SMPL 有 23 个非根关节，每个关节的旋转矩阵是 3×3，展开成 9 个数，$23 \times 9 = 207$。所以姿势 blendshape 的“参数”不是角度，而是旋转矩阵的 9 个元素。

第二，为什么用旋转矩阵而不是轴角。 因为旋转矩阵的元素是角度的正弦/余弦，是光滑有界的；轴角在 0 附近虽然也光滑，但在大角度时会有周期性跳变。用矩阵元素做线性基，能让“姿势 → 位移”的关系更接近线性。

第三，$\boldsymbol{\theta}^*$ 是静止姿势。减去 $R_n(\boldsymbol{\theta}^*)$ 是为了保证“静止姿势下修正量为零”。这样模板本身就代表静止姿势，不会在没摆姿势时被莫名修正。

$\mathbf{P}_n$ 是学出来的 6890×3 位移基。每个基对应“某个关节的某个旋转分量从 0 变到 1 时，全身应该怎么补偿”。207 个基听起来很多，但它们都是稀疏的：大多数基只影响关节附近的顶点。

8.6 第四步：LBS

现在有了中间形状 $\mathbf{T}_P$、关节位置 $\mathbf{J}(\boldsymbol{\beta})$、蒙皮权重 $\mathcal{W}$，就可以做标准的线性混合蒙皮：

$$\mathbf{t}'_i = \sum_{k=1}^{K} w_{k,i}\, \mathbf{G}_k(\boldsymbol{\theta}, \mathbf{J})\, \mathbf{G}_k(\mathbf{0}, \mathbf{J})^{-1}\, \mathbf{t}_{P,i}. \tag{8.5}$$

$\mathbf{G}_k(\boldsymbol{\theta}, \mathbf{J})$ 是关节 k 的全局变换，由式（5.1）的父子链算出；$\mathbf{G}_k(\mathbf{0}, \mathbf{J})^{-1}$ 把它变回静止姿势的局部坐标。$w_{k,i}$ 是顶点 i 对关节 k 的权重，每个顶点最多 4 个非零权重，且和为 1。

把四步合起来，就是 SMPL 的完整公式：

$$M(\boldsymbol{\beta}, \boldsymbol{\theta}) = W\Big(\bar{\mathbf{T}} + \mathbf{B}_S(\boldsymbol{\beta}) + \mathbf{B}_P(\boldsymbol{\theta}),\, \mathbf{J}(\boldsymbol{\beta}),\, \boldsymbol{\theta},\, \mathcal{W}\Big). \tag{8.6}$$

如果你在论文里看到 $W(T_P(\boldsymbol{\beta}, \boldsymbol{\theta}), J(\boldsymbol{\beta}), \boldsymbol{\theta}, \mathcal{W})$，指的就是这个式子。



图 8-3 SMPL 的参数网格。横向换 $\boldsymbol{\theta}$ 改变姿势，纵向换 $\boldsymbol{\beta}$ 改变体型，两者可以自由组合。中间橙色是均值体型 + 静止姿势。

8.7 SMPL 是怎么训练出来的

SMPL 的训练分三步，每一步只优化一部分参数。

第一步：形状空间。用 CAESAR 数据集（数千名受试者的全身扫描），把所有扫描配准到同一拓扑，再对顶点偏移做 PCA，得到 10 个形状主成分 $\mathbf{S}_n$。

第二步：关节回归器与蒙皮权重。 用一批“同一个人摆多个姿势”的扫描，优化关节回归矩阵 $\mathcal{J}$ 和蒙皮权重 $\mathcal{W}$，使得 LBS 的输出尽量接近扫描。

第三步：姿势 blendshape。 固定形状和蒙皮，优化 207 个姿势基 $\mathbf{P}_n$，专门补偿弯曲、扭转时的偏差。

训练的目标函数大致是

$$\min \sum_{\text{scan}} \Big(\|\mathbf{V}_{\text{model}} - \mathbf{V}_{\text{scan}}\|^2 + \lambda_{\text{landmark}} \|\mathbf{J}_{\text{model}} - \mathbf{J}_{\text{scan}}\|^2 + \lambda_{\text{reg}} \|\boldsymbol{\beta}\|^2 \Big), \tag{8.7}$$

其中关键点项保证关节对齐，正则项防止形状参数爆炸。

> **一个值得注意的细节**：SMPL 的蒙皮权重不是手工画的，而是和姿势基一起从数据里学出来的。这也是它比“手工绑定 + LBS”更自然的原因之一。

8.8 使用 SMPL 时最容易踩的坑

坑一：$\boldsymbol{\beta}$ 的范围。 β 的单位是“标准差”，取 ± 1 已经是很明显的体型差异，± 3 以外基本不是正常人。从单张图回归时，网络经常输出 $\beta = 5$ 甚至更大，必须加正则或截断。

坑二：$\boldsymbol{\theta}$ 的根旋转。 前 3 维是全局朝向，不是“骨盆关节的弯曲”。如果你的模型“站着但面朝侧面”，先检查这 3 维。

坑三：坐标系。 官方 SMPL 是 y 轴向上、面朝 $+z$；但很多代码库会转成 z 轴向上、面朝 $-z$。混用会导致模型躺倒或镜像。加载后先渲染一张图确认。

坑四：形状与姿态的耦合。 虽然 SMPL 假设二者可分离，但真实数据里“胖子抬手臂”和“瘦子抬手臂”的皮肤形变并不完全一样。极端体型 + 极端姿势时，SMPL 会出现不自然的褶皱。

坑五：没有脸和手。 原始 SMPL 的手是“拳头”状的粗糙网格，脸也没有表情。需要手部细节要用 MANO / SMPL-H，需要表情要用 SMPL-X（第 9 章）。

8.9 SMPL 的局限与影响

SMPL 的局限可以列成一串：线性形状空间、LBS 伪影、固定拓扑、没有衣服、没有表情、没有手指、训练数据以欧美成年人为主。但它的影响远远超过这些局限：

- 它统一了人体重建的输出格式，让不同论文可以比较；
- 它可微、可批量、可 GPU 加速，成为神经网络的理想输出层；
- 它的参数空间成为生成模型（VPoser、动作扩散）的坐标系；
- 它的变体覆盖了手（SMPL-H）、脸（SMPL-X）、婴儿、动物（SMAL 借鉴了它的思路）。

可以说，SMPL 之于人体，就像 JPEG 之于图片：不是最完美的表示，但胜在通用、简单、生态完整。

本章小结

- SMPL 用 10 个形状参数 + 72 个姿态参数生成 6890 个顶点。
- 四步：形状 blendshape → 关节回归 → 姿势 blendshape → LBS。
- 207 个姿势基 = 23 个关节 × 9 个旋转矩阵元素；它补偿 LBS 的体积损失。
- 参数从 CAESAR 扫描与多姿势数据中分阶段训练得到。
- 使用时注意 β 范围、根旋转、坐标系、形状/姿态耦合，以及它没有脸和手。

思考题

1. 为什么 SMPL 的姿势参数是 72 维，而不是 69 维？
2. 如果去掉姿势 blendshape，只保留 LBS，弯曲手臂时会发生什么？能用第 5 章的公式解释吗？
3. 关节回归器为什么需要“从形状回归”，而不能直接用模板的关节位置？
4. 如果让你把 SMPL 扩展到“带表情”，你会把表情参数加在哪一步？为什么？

第 9 章 SMPL 家族：从身体到手，再到脸

本章导读：SMPL 只解决了“身体”。真实的人还要动手、做表情、说话。这一章按时间顺序讲 SMPL 家族的扩展：SMPL+H 加手、SMPL-X 加脸和手、STAR 换一种更稀疏的关节表示、GHUM 走向连续尺度，最后是 2025 年的 ATLAS 与 MHR——它们把“骨架”和“形状”彻底拆开。最后给出一张选型表，告诉你在不同任务里该用哪个模型。

9.1 一条家族谱系



图 9-1 SMPL 家族的时间线。2015 年的 SMPL 只覆盖身体；2017 年的 SMPL+H 接入 MANO 的手；2019 年的 SMPL-X 把脸和手都并进来；2020 年的 STAR 与 GHUM 分别从“关节表示”和“连续尺度”改进；2025 年的 ATLAS / MHR 则把骨架与形状解耦。

SMPL 的成功带来一个自然的问题：能不能把身体、手、脸放进同一个模型？ 如果分开建模，手和身体会在手腕处对不上，脸和脖子会在下颌处撕裂，动作捕捉时要分别拟合再拼接，误差会累积。

SMPL 家族的扩展思路是一致的：保持同一套 LBS 框架，往模板里加顶点、往骨架里加关节、往参数里加维度。

模型	顶点数	覆盖范围	关键新增参数
SMPL	6890	身体	$\boldsymbol{\beta} \in \mathbb{R}^{10}$，$\boldsymbol{\theta} \in \mathbb{R}^{72}$
SMPL+H	6890	身体 + 手	每只手 15 个关节（MANO 定义）

模型	顶点数	覆盖范围	关键新增参数
SMPL-X	10475	身体 + 手 + 脸	10 维表情 ψ、下颌/眼球关节，共 54 个关节
STAR	6890	身体	稀疏关节 + 姿势校正
GHUM	—	身体	连续尺度、层级关节
ATLAS	115k	身体	骨架与形状解耦、77 关节
MHR	73639–595（多 LOD）	身体 + 手	127 关节、骨架/形状解耦

> 表中“—”表示不同实现的顶点数不一致，本书不写没有核对过的数字。

9.2 SMPL+H：把手接进来

2017 年，Romero 等人在 Embodied Hands[16] 中提出 MANO（第 19 章详述），并把它与 SMPL 拼成 SMPL+H。

做法很直接：

1. 保持 SMPL 的身体网格与 24 个关节；
2. 在每只手腕处接上 MANO 的手部网格；
3. 把 MANO 的 15 个手部关节（不含全局朝向）加入骨架；
4. 重新学习手腕附近的蒙皮权重，让手和手臂平滑过渡。

结果是一个能同时表达身体姿态和精细手部动作的模型。它的意义在于：手不再是“拳头”，而是能单独弯曲每根手指的 15 个关节。 这对抓取、手势识别、VR 手柄追踪都是必需的。

但 SMPL+H 仍然没有脸。脸的表情、眨眼、说话，需要另一套参数。

9.3 SMPL-X：身体、手、脸的统一

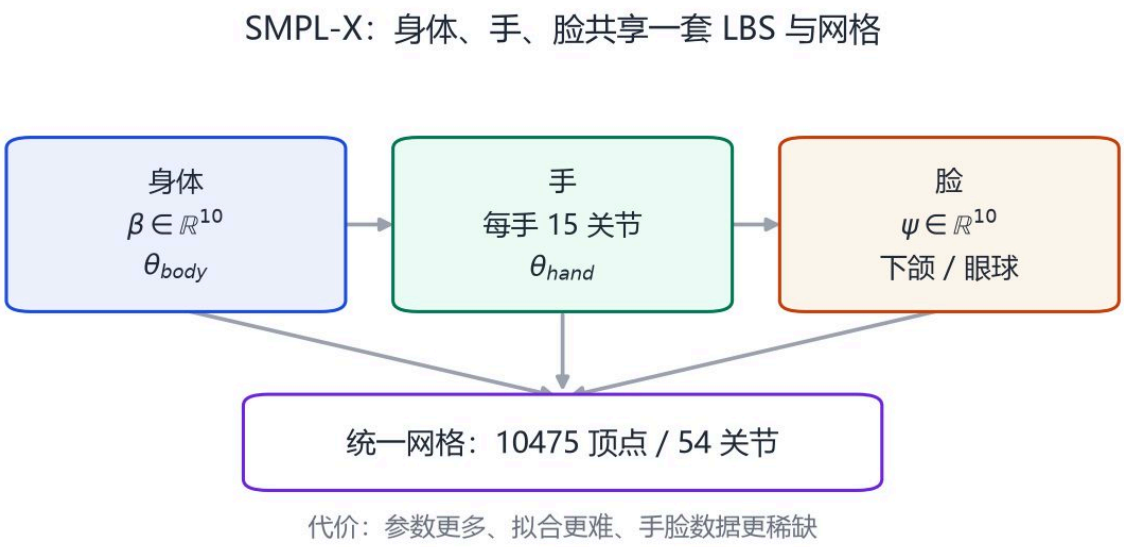


图 9-2 SMPL-X 的参数划分。身体形状、身体姿态、双手姿态、面部表情各自有参数，但共享同一张网格与同一套 LBS。输出是 10475 个顶点、54 个关节。

2019 年，Pavlakos 等人在 Expressive Body Capture 中提出 SMPL-X（SMPL eXpressive）。它把三件事合并：

- 身体：沿用 SMPL 的形状与姿态参数；
- 手：接入 MANO 的双手，每手 15 个关节；
- 脸：加入 10 维表情参数 ψ，以及下颌、眼球、眼睑等关节。

SMPL-X 的输出是 10475 个顶点、54 个关节。相比 SMPL，它多了 3585 个顶点（主要在脸和手），也多了表情与手指的自由度。

代价也很明显：

1. 参数更多：从单张图回归 100 多个参数，比回归 82 个难得多；
2. 数据更稀缺：同时有身体、手、脸的高质量扫描很少，训练要靠多来源数据拼接；
3. 耦合更强：手靠近脸时，两者的网格会互相穿插，需要额外的碰撞处理。

SMPL-X 之后，几乎所有“全身重建”论文都把输出定义为 SMPL-X 参数。你在第 10、11 章看到的 SMPLify-X、FrankMocap、PyMAF-X，都是它的下游。

9.4 STAR：更稀疏、更会“校正”的身体模型

2020 年，Osman 等人提出 STAR（Sparse Trained Articulated Human Body Regressor）[17]。它针对 SMPL 的两个痛点：

- 关节太少：24 个关节无法精细描述肩胛骨、脊柱、脚掌的变形；

- 姿势校正太全局：SMPL 的 207 个姿势基是全局的，某个关节的修正可能影响到无关部位。

STAR 的做法是：

1. 用更多、更稀疏的关节，每个关节只影响局部区域；
2. 为每个关节学习局部的姿势校正，而不是全局基；
3. 用稀疏约束让参数数量不爆炸。

结果是：在极端姿势（高举手臂、深蹲）下，STAR 的形变比 SMPL 更自然，而且参数量并没有显著增加。它常被用作“更准的 SMPL 替代品”，但生态不如 SMPL 广。

9.5 GHUM：连续尺度与层级结构

同一年，Choutas 等人提出 GHUM（Generative Human Shape and Articulated Pose Model）[18]。它关注两个被 SMPL 忽略的问题：

- 尺度是连续的：SMPL 的形状参数能改变高矮，但“身高”不是一个显式、可解释的维度；GHUM 把尺度显式建模，能生成从儿童到成人的连续变化；
- 关节是层级的：脊柱不是一根刚体，而是多段；GHUM 用层级结构描述这种多段弯曲。

GHUM 还提供了轻量版 GHUML，牺牲一点精度换取速度。它的影响主要体现在“度量准确”的重建任务上：当你需要知道“这个人到底多高”，而不只是“看起来像”，显式的尺度参数就很重要。

9.6 2025 之后：ATLAS 与 MHR——把骨架和形状拆开

SMPL 一系的共同假设是：先有表面，再从表面顶点回归出关节。这句话很方便，但留下了一个副作用——骨骼与软组织被绑在一起。

举两个真实会遇到的例子：

- 你只想把肩膀变窄。在 SMPL-X 里调小肩宽，手臂、胸腔、甚至体重也跟着变；
- 你只想增加体重。结果肩膀又弹回去了一部分（ATLAS 论文的图 1 用的就是这个例子）。

根因在关节回归器：关节位置是顶点的加权和，顶点一动，骨架就跟着动，两者没法分开调。

2025 年 Meta 的两项工作就是冲这件事去的。

9.6.1 ATLAS：先有骨架，再长肉

ATLAS（Park 等，2025，Meta + CMU）[19] 的核心改动是把形状基与骨架基显式解耦：网格不再"先有表面再猜骨架"，而是以人体骨架为锚来组织表面。

- 数据规模：60 万次高分辨率扫描，由 240 台 4K 相机同步拍摄（数据集体量叫 Goliath）；
- 分辨率：原生网格 11.5 万顶点，骨架 77 个关节，并且解剖上左右对称（SMPL-X 的平均网格在肘、脊柱、脚上能看出明显的不对称，就是因为关节是从顶点回归出来的）；

- 姿势校正：稀疏 + 非线性，只作用在被驱动关节附近，避免"弯左肘却让右肘动"这类伪相关；
- 能干什么：肩宽、身高、四肢长度、体重可以分别调；关键点拟合也不再依赖外表面。

用一句话概括：SMPL 是"用表面近似骨架"，ATLAS 是"先有骨架再长肉"。

9.6.2 MHR：把解耦做成工业级 rig

MHR（Momentum Human Rig，Meta，2025 年 11 月）[20] 把 ATLAS 的"骨架/形状解耦"和 Momentum 库的绑定系统、姿势校正系统合成一个模型，目标不是发论文，而是能直接进 AR/VR 与图形管线。

规模与结构：

- 127 个关节、204 个姿态参数，其中 68 个是骨架变换参数（不只是轴角，还包含骨长/缩放这类骨架自由度）；
- 形状被拆成几组：表面形变基由 $\boldsymbol{\beta}_s$、另一组形变基由 $\boldsymbol{\beta}_f$ 控制，骨架由 $\boldsymbol{\beta}_k$ 控制。写成式子是

$$\mathbf{X}(\boldsymbol{\beta},\boldsymbol{\theta}) = \mathcal{M}\Big(\tilde{\mathbf{X}}(\boldsymbol{\beta}_s,\boldsymbol{\beta}_f,\boldsymbol{\theta}),\ \mathbf{B}_k(\boldsymbol{\beta}_k),\ \boldsymbol{\theta},\ \omega\Big),$$

其中 $\tilde{\mathbf{X}} = \bar{\mathbf{X}} + \mathbf{B}_s(\boldsymbol{\beta}_s) + \mathbf{B}_f(\boldsymbol{\beta}_f) + \mathbf{B}_p(\boldsymbol{\theta})$：模板 + 两组线性形状项 + 姿势校正项，再交给带骨架项的蒙皮函数 $\mathcal{M}$。

- 多级 LOD：同一模型提供 73639 / 18439 / 10661 / 4899 / 2461 / 971 / 595 顶点等多档分辨率，渲染远处的人不需要全分辨率；
- 非线性稀疏姿势校正 + 手指关节，能表达"SMPL-X 线性基表达不了"的极端姿势；
- 许可：论文强调是 "industry-friendly" 的许可，代码与模型在 `github.com/facebookresearch/MHR`。

> 为什么它俩要放在一起讲：ATLAS 解决"骨架和形状能不能分开"，MHR 解决"分开之后能不能当生产工具用"。前者是方法论，后者是工程化。

9.6.3 六代身体模型对照

模型	年份	网格顶点	关节	训练扫描	形状/骨架	姿势校正
SMPL	2015	6890	24	约 1.8k	共享线性基	线性（207）
SMPL-X	2019	10475	54	约 1.8k	共享线性基	线性（9K）
STAR	2020	6890	24	约 1.8k	稀疏线性	稀疏线性
GHUM	2020	—	—	约 60k	层级	非线性网络
ATLAS	2025	115k	77	**600k**	**解耦**	稀疏 + 非线性
MHR	2025	73639–595（多 LOD）	127	—	**解耦**	稀疏 + 非线性

（“—”表示论文未给出可直接对比的数字，本书不写没有核对过的值。）

9.6.4 那为什么这本书还在讲 SMPL?

因为生态。数据集（AMASS、3DPW、Human3.6M 的 SMPL 参数版）、代码（smplx、SMPLify、HMR 系列）、评价指标（MPJPE/PVE 的对齐方式）都以 SMPL 为公共接口。ATLAS/MHR 更准、更可控，但网格与关节规模大一个量级，训练与部署成本也高。

现实中的做法通常是：用 SMPL 做"公共语言"（重建、动作、数据集），需要精细编辑（改骨长、改肩宽、做工业绑定）时，再换到 ATLAS/MHR 这类解耦模型上。

9.7 怎么选：一张实用表

你的任务	推荐模型	理由
单图/视频人体姿态与体型	SMPL	生态最全、预训练模型最多
需要手指动作	SMPL+H / SMPL-X	手部关节与网格
全身 + 表情 + 手势	SMPL-X	统一网格，避免拼接
极端姿势、更高精度	STAR	稀疏关节 + 局部校正
需要真实身高/儿童	GHUM	显式尺度
要分开调骨架（骨长/肩宽）与体型	ATLAS / MHR	骨架与形状解耦
动物	SMAL	第 22 章
只做脸	FLAME	第 16 章
只做手	MANO	第 19 章

选型不是“越新越好”。SMPL-X 参数多、拟合难；如果你的任务只需要身体，用 SMPL 往往更稳。模型复杂度应该匹配任务需求，而不是追新。

9.8 家族成员的共同点

不管加了多少顶点和关节，它们都遵循同一个模板：

$$M = W\big(\bar{\mathbf{T}} + \mathbf{B}_S(\boldsymbol{\beta}) + \mathbf{B}_P(\boldsymbol{\theta}) + \mathbf{B}_E(\boldsymbol{\psi}),\ J(\boldsymbol{\beta}),\ \boldsymbol{\theta},\ \mathcal{W}\big). \tag{9.1}$$

区别只在于：

- 模板 $\bar{\mathbf{T}}$ 的顶点数不同；
- 关节数 K 不同；
- 是否多一个表情基 $\mathbf{B}_E$；

- 权重 $\mathcal{W}$ 的稀疏模式不同。

理解了这个统一形式，你再看任何新模型（包括 2024 年之后的工业模型），都能快速判断它“新在哪”：是换了模板、加了关节、改了基，还是换了蒙皮方式。

9.9 参数是怎么拼起来的：SMPL-X 的一次完整前向

式 (9.1) 的每一项都可以写成一条显式公式。对 SMPL-X，四项相加就是：

$$\mathbf{T}_P = \bar{\mathbf{T}} + \sum_{n=1}^{10} \beta_n \mathbf{S}_n + \sum_{n=1}^{10} \psi_n \mathbf{E}_n + \sum_{n=1}^{9K} \big(R_n(\boldsymbol{\theta}) - R_n(\boldsymbol{\theta}^*)\big)\mathbf{P}_n. \tag{9.2}$$

逐项看，每个符号都对应一块具体的数组。

- 模板 $\bar{\mathbf{T}} \in \mathbb{R}^{10475\times 3}$：零姿态下的平均人体，10475 行是 10475 个顶点，每行 3 个数是 x, y, z 坐标（米）。它的零姿态是 A-pose，不是 SMPL 的 T-pose，两个模型连模板都不能混用。
- 形状项：$\boldsymbol{\beta} \in \mathbb{R}^{10}$，每个 $\mathbf{S}_n \in \mathbb{R}^{10475\times 3}$ 是一张和模板同样大的“位移图”，β_n 决定加多少倍。$\boldsymbol{\beta} = \mathbf{0}$ 就是平均人——这正是第 10 章形状正则的由来。
- 表情项：形式与形状项一模一样，只是 $\boldsymbol{\psi} \in \mathbb{R}^{10}$ 的主成分借自 FLAME，非零部分集中在脸上。两项在实现上是同一段代码，区别只在训练数据：一个来自“不同的人”，一个来自“同一个人的不同表情”。
- 姿态校正项：关节转动时皮肤不会像刚体那样跟着转，要用 $9K$ 个基补偿。$R(\cdot)$ 把轴角变成 3×3 矩阵再拉平成 9 个数，所以每个关节贡献 9 个基；减去零姿态的 $R(\boldsymbol{\theta}^*)$ 是为了让静止时这项为 0。

四项加完只得到“摆好姿势但还没蒙皮”的网格，关节位置还要从只有形状的网格回归：$\mathbf{J}(\boldsymbol{\beta}) = \mathcal{J}(\bar{\mathbf{T}} + \mathbf{B}_S(\boldsymbol{\beta}))$，$\mathcal{J} \in \mathbb{R}^{54\times 10475}$ 的每行只有少数非零——一个关节由附近一小撮顶点加权平均得到。这里没有表情项：做表情不改变骨架。

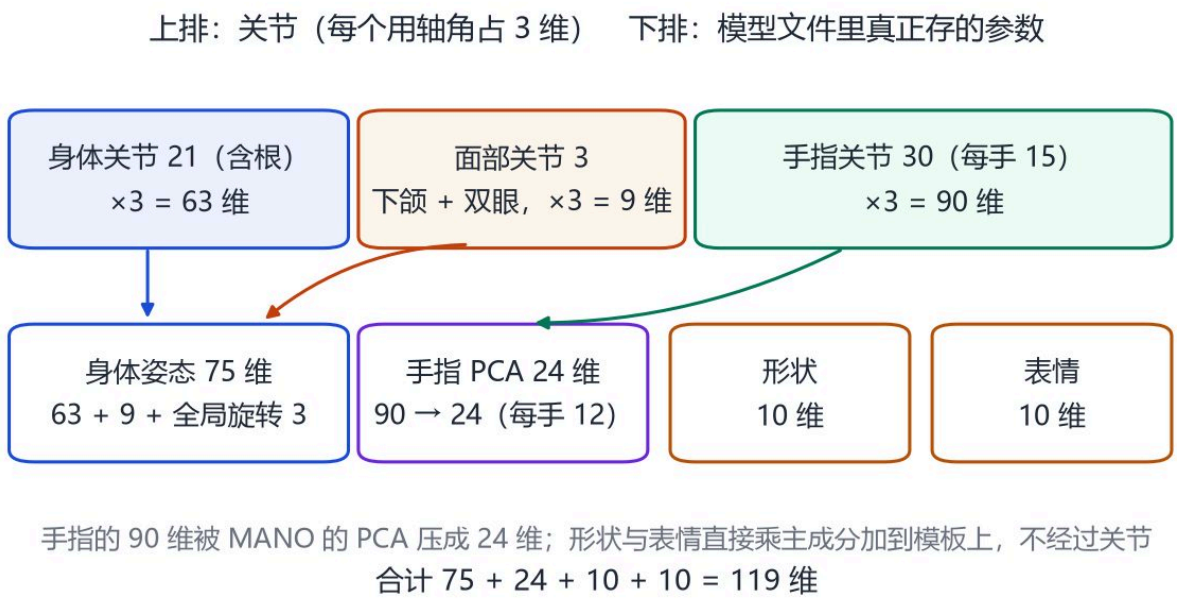


图 9-3 SMPL-X 的 54 个关节与 119 个参数怎样对上账。上排是关节数，每个关节用轴角占 3 维；下排是真正存进模型文件的参数：身体 63 + 面部 9 + 全局旋转 3 凑成 75，手指的 90 维压成 24 维，形状与表情各 10 维。箭头表示“合并”与“压缩”。

这张图回答一个常被算错的问题：SMPL-X 一共 119 个参数 = 75（全局旋转与 24 个身体/眼球/下颌关节）+ 24（双手手指 PCA）+ 10（形状）+ 10（表情）[21]。手指 30 个关节若全用轴角是 90 维，压成 24 维后参数才没有爆炸。关节数 × 3 不等于姿态维度：全用轴角时是 165 维，走 PCA 后只剩 99 个数。

9.10 换模型时要改哪些代码

从 SMPL 换到 SMPL-X 或 MANO 的改动比想象中小：公式一行都不变，变的是数组形状与参数名字。

数组	SMPL	SMPL-X	MANO
顶点数 N	6890	10475	778
模板	$(6890, 3)$	$(10475, 3)$	$(778, 3)$
形状主成分	$(6890, 3, 10)$	$(10475, 3, 10)$	$(778, 3, 10)$
表情主成分	无	$(10475, 3, 10)$	无
姿态偏移基	$(6890, 3, 207)$	$(10475, 3, 486)$	$(778, 3, 135)$
蒙皮权重	$(6890, 24)$	$(10475, 54)$	$(778, 16)$
关节回归器 $\mathcal{J}$	$(24, 6890)$	$(54, 10475)$	$(16, 778)$

数组	SMPL	SMPL-X	MANO
姿态维度	72	165（常拆成 75 + 24）	48（可压到 6）

表 9-1 三个模型的数组形状对照。

变化只有三类。一是尺寸：硬编码 `6890` 的代码一定出错，应写成 `N = v_template.shape[0]`。二是参数：SMPL-X 多了 ψ 与表情主成分，姿态也从 72 维向量变成“身体 75 + 双手 PCA 24”，必须拼成完整 θ 再送 LBS。三是关节顺序：手指按 MANO 顺序排、面部关节夹在中间，“检测器编号 → 模型关节编号”的对应表必须重写。这是最容易静默出错的地方——程序不报错，只是把手腕的关键点对到了肘部。

不变的部分比变的多：蒙皮、投影、优化器、损失函数一行都不用改——生态能复用，不是因为大家抄了同一份代码，而是因为遵守了同一个接口。

但有一件事改数组解决不了：参数含义不能跨模型通用。 两个模型的 β 都是 10 维，第一维指向的体型方向却不同。把 SMPL 的 β 喂给 SMPL-X，得到的不是“同一个人”，而是形状错乱的人——参数是相对于模型定义的，不是绝对的物理量。

本章小结

- SMPL+H 在身体上接入手部，SMPL-X 进一步接入脸与表情，输出 10475 顶点 / 54 关节。
- STAR 用稀疏关节与局部姿势校正提升极端姿势精度；GHUM 显式建模连续尺度与层级关节。
- 所有家族成员共享“模板 + blendshape + 关节回归 + LBS”的结构。
- 一次前向可以拆成六步：模板 → 加形状 → 加表情 → 加姿态校正 → 回归关节 → 蒙皮；形状项与表情项在实现上是同一段代码，只差训练数据。
- SMPL-X 的 119 个参数 = 75（全局旋转与 24 个身体/眼球/下颌关节）+ 24（双手手指 PCA）+ 10（形状）+ 10（表情）；54 个关节不等于 162 个姿态数。
- 换模型时变的是数组形状、参数个数与关节顺序，不变的是 LBS 与优化器；但参数含义不能跨模型搬运。
- 选型要看任务：身体用 SMPL，手用 SMPL+H/MANO，全身表情用 SMPL-X。

思考题

1. 为什么 SMPL-X 不直接把手和脸做成两个独立模型，再在推理时拼接？
2. STAR 的“稀疏关节”与 SMPL 的“全局姿势基”各有什么优劣？
3. 如果让你设计一个“婴儿人体模型”，你会改 SMPL 的哪几部分？
4. 为什么模型越大，从单张图回归反而可能越不准？

第 10 章 从单张图片反推人体：优化法与回归法

本章导读：模型有了，参数也有了，剩下的问题是：给我一张照片，怎么求出 β 和 θ？这一章讲两条路线——优化法与回归法——以及它们如何结合。你会看到 SMPLify、HMR、SPIN、GraphCMR 这些名字背后的具体机制，也会明白为什么单图重建至今仍然容易在遮挡、尺度、奇怪姿势上翻车。

10.1 问题定义

给定一张 RGB 图像 $\mathbf{I}$，求：

- 形状 $\boldsymbol{\beta} \in \mathbb{R}^{10}$；
- 姿态 $\boldsymbol{\theta} \in \mathbb{R}^{72}$；
- 相机参数 $\boldsymbol{\pi}$（焦距、旋转、平移，或简化为弱透视的缩放与平移）；
- 可选：衣服、纹理、光照。

使得渲染出来的模型与图像“最一致”。这就是式（1.2）的具体化。困难在于：

1. 深度歧义：单张图无法区分“人很小且很近”和“人很大且很远”；
2. 遮挡：手挡住身体、别人挡住他，关键点会缺失；
3. 姿态歧义：正面看，手肘向内还是向外，投影几乎一样；
4. 域差异：训练集是实验室，测试集是网络图片。

10.2 路线 A：优化法

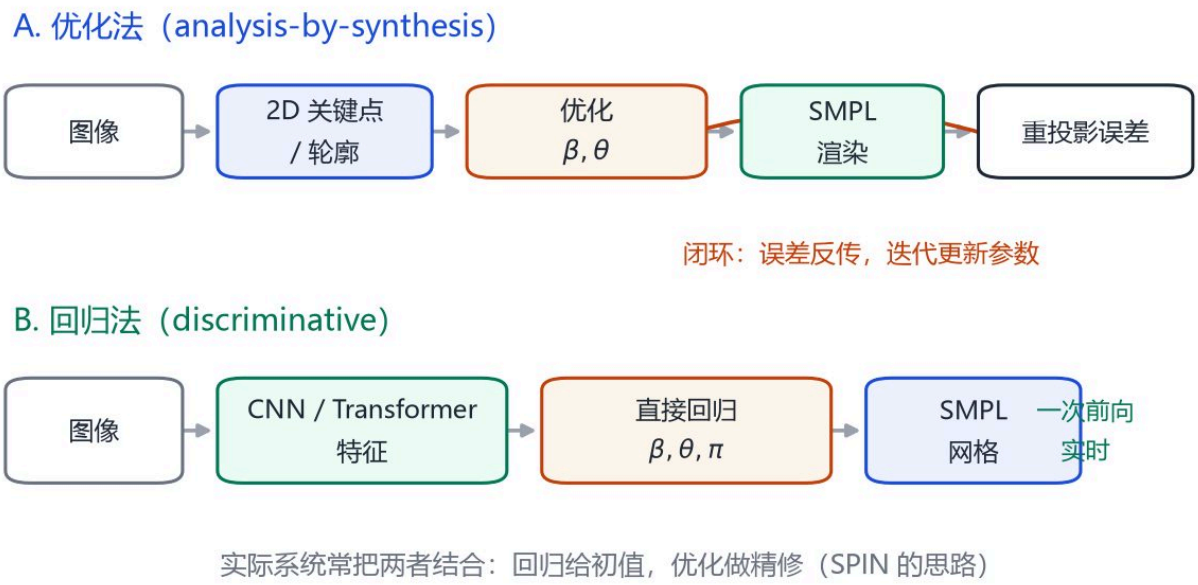


图 10-1 两条路线。优化法先用 2D 关键点/轮廓构造误差，再反复“渲染—比较—更新”；回归法用网络一次前向输出参数。实际系统常把两者结合。

优化法的思路是“分析—合成”：先猜一组参数，渲染出模型，比较渲染结果与图像的差异，再根据差异更新参数。

10.2.1 SMPLify

2016 年，Bogo 等人的 SMPLify 是第一个把 SMPL 用到单图上的系统 [6]。它的输入是 2D 关节点（由 DeepCut 等检测器给出），目标函数是四项之和：

$$E(\boldsymbol{\beta},\boldsymbol{\theta}) = \lambda_J E_J + \lambda_\beta E_\beta + \lambda_\theta E_\theta + \lambda_S E_S. \tag{10.1}$$

- E_J：3D 关节投影到 2D 后，与检测到的 2D 关节的距离平方；
- E_β：形状参数的正则，防止体型跑到不合理区域；
- E_θ：姿态先验，用高斯混合模型（GMM）在大量动作捕捉数据上学习，惩罚“反关节”的姿势；
- E_S：自相交惩罚，防止手穿进身体、腿穿进腿。

SMPLify 还做了一个很实用的设计：按性别选模型。男性和女性的体型分布不同，用对应的模型可以显著减少歧义。

优化用 OpenDR 提供的可微渲染，配合拟牛顿法（如 L-BFGS）。整个流程在 GPU 上要几十秒到几分钟，不能实时，但精度不错。

10.2.2 优化法的优缺点

优点：

- 可解释：每一项误差都有明确含义；
- 不需要训练数据：只要有模型和检测器就能跑；
- 容易加约束：物理、接触、关节限位都能写成惩罚项。

缺点：

- 慢，且对初值敏感；
- 权重 λ 难调，换一个数据集就要重调；
- 2D 关键点错了，优化会“将错就错”；
- 单图深度歧义仍然存在，优化只是在找一个“看起来合理”的解。

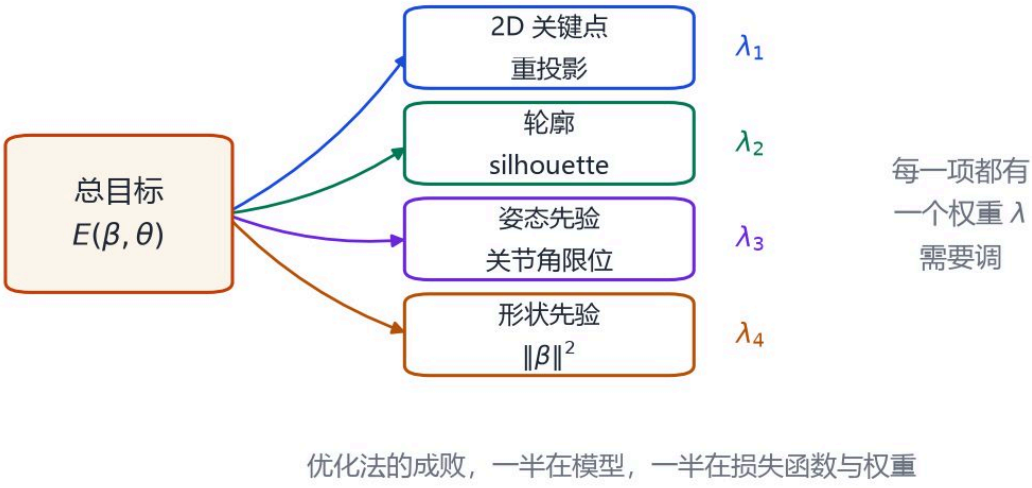


图 10-2 SMPLify 类方法的损失构成。关键点、轮廓、姿态先验、形状先验各自带一个权重；调参的本质，是在“相信图像”与“相信先验”之间找平衡。

10.3 路线 B：回归法

2018 年，Kanazawa 等人的 HMR（End-to-end Recovery of Human Shape and Pose）[7] 换了一个思路：用一个神经网络直接从图像回归 SMPL 参数。

网络结构很简洁：

1. 用 ResNet-50 提取图像特征；
2. 接一个迭代回归器，输出 $\boldsymbol{\beta}$、$\boldsymbol{\theta}$、相机 $\boldsymbol{\pi}$；
3. 用可微渲染把模型投影回图像，计算 2D 关键点损失。

训练时没有 3D 真值怎么办？HMR 用了两个技巧：

- 弱监督：只有 2D 关键点标注时，用重投影损失；

- 对抗先验：训练一个判别器，区分“真实的 SMPL 参数”和“网络预测的参数”，迫使预测落在合理的姿态分布内。

HMR 的意义在于：它把人体重建变成了一个标准的“图像 → 参数”回归问题，可以端到端训练，速度达到实时。之后的绝大多数方法都沿用了这个范式。

10.3.1 SPIN：把优化器放进训练循环

HMR 的弱点是对罕见姿势不敏感。SPIN（Kolotouros 等，2019）[8] 的想法很巧妙：既然优化法在单张图上更准，那就把优化器放进训练循环。

具体做法：

1. 网络先预测一组参数；
2. 用 SMPLify 以这组参数为初值，针对当前图像做优化；
3. 把优化后的参数当作“伪真值”，监督网络；
4. 同时用一个判别器保证参数合理性。

这样，网络在训练时就“见过”优化器如何修正自己，推理时即使不做优化，也能接近优化后的结果。SPIN 是“回归 + 优化”结合的典范。

10.3.2 GraphCMR：不回归参数，直接回归顶点

GraphCMR（Kolotouros 等，2019）[22] 走了另一条路：不预测 $\boldsymbol{\beta},\boldsymbol{\theta}$，而是用图卷积直接预测 6890 个顶点的坐标。

好处是：不受参数空间的线性限制，能表达更细的形变。坏处是：输出维度高（20670），需要强的网格先验，而且很难保证骨骼长度一致。后来的方法大多又回到“参数化输出”，因为参数空间更稳、更容易加约束。

10.4 两条路线的融合

今天的主流系统通常是混合的：

- 用回归网络给出初值（快、鲁棒）；
- 用优化做精修（准、可加约束）；
- 用时序模型平滑（视频场景）。

例如 FrankMocap 先分别回归身体和手，再用优化对齐；PyMAF 用“金字塔对齐”反复把预测投影回图像特征，实质是一种可学习的迭代优化。

10.5 为什么单图重建仍然难

深度与尺度：单目图像缺少绝对尺度。模型可以输出“看起来 1.7 米”的人，但如果没有相机内参或场景参照，真实身高无法确定。CameraHMR、NLF 等工作专门解决这个问题（第 11 章）。

遮挡与截断：下半身被桌子挡住时，网络只能猜。猜出来的腿可能长度不对、朝向不对。

罕见姿势：瑜伽、攀岩、舞蹈，训练集里少，先验会把这些姿势“拉回”常见分布。

衣服与物体：SMPL 是裸体模型，穿裙子的人会被拟合成“紧身裤”。手拿包、拄拐杖，也会干扰关键点。

评估偏差：很多论文在 3DPW 上报告 MPJPE，但 3DPW 的 3D 真值本身来自优化拟合，存在“用同类方法评估同类方法”的循环。

10.6 优化法复盘：把式 (10.1) 拆到原子级

式（10.1）把四项压成一行；写进代码后，每项都是一个循环加一个函数。

（1）关键点项。 它衡量“模型关节投影到图像上，离检测到的 2D 点多远”：

$$E_J = \sum_i w_i \, \rho\Big(\big\| \Pi_{\mathbf{K}}\big(R_{\boldsymbol{\theta}}(\mathbf{J}(\boldsymbol{\beta}))_i\big) - \mathbf{j}_i \big\| \Big). \tag{10.2}$$

i 遍历 2D 关键点，$\mathbf{j}_i \in \mathbb{R}^2$ 是像素坐标，$\mathbf{J}(\boldsymbol{\beta})$ 是 3D 关节，$R_{\boldsymbol{\theta}}$ 摆好姿态，$\Pi_{\mathbf{K}}$ 投影；w_i 是置信度，被遮挡的关节它就该少听。

ρ 是鲁棒函数（Geman‑McClure，$\rho(e) = e^2/(e^2+\sigma^2) \in [0,1)$）：一个错了的关键点会有几百像素误差，平方会把它放大成天文数字，ρ 让大误差饱和，等于放弃这个点，σ 是“多大偏差算离谱”的分界。

（2）形状项。 $E_\beta = \|\boldsymbol{\beta}\|^2$ 不是“让体型变小”，而是“让体型靠近 $\mathbf{0}$”——零向量正是平均人；各维已按主成分方差缩放过，平方和就近似“标准高斯下的负对数概率”。证据不足时，是它在决定腿有多长。

（3）姿态项。 一块是数据先验：8 个高斯拟合约 100 万个姿态（100 个被试的动作捕捉，经 MoSh 拟合成 SMPL），取负对数：

$$E_\theta = -\log \sum_{j=1}^{8} g_j \, \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \approx \min_j \big(-\log c\, g_j \, \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \big). \tag{10.3}$$

用最小值近似“求和”是为了可微、算得快，代价是交界处梯度不连续。另一块是解剖约束 $E_a = \sum_i \exp(\theta_i)$，只对膝、肘的反向弯曲求和：反关节物理上不可能，要用指数函数压死（式（10.1）把它并进了 E_θ，原论文其实是五项）。

（4）自相交项。 SMPLify 用 20 个胶囊（不含手指脚趾）近似身体，再退化成沿轴的一串球，对“不该相交”的球对用高斯乘积的积分近似重叠体积。它只是惩罚不是禁止：权重小了照样穿模；优化体型时必须关掉，否则优化器会把人体压瘦来躲开。

分工也就清楚了：证据强时 E_J 主导；关节被遮挡时 E_θ 与 E_a 接管，输出“合理但可能不对”；只拍到半身时 E_β 决定腿脚，腿被猜成平均值；左右手搞反时两者互相拉扯。量级也要匹配：ρ 有界，E_J 最多贡献“关键点个数”量级，而 $\|\boldsymbol{\beta}\|^2$ 无界增长——λ 要按残差量级调。

10.6.1 一次迭代到底在算什么

```
输入：图像 I，2D 关键点 {(j_i, w_i)}，初值 (β, θ)，性别对应的模型
重复：
  1. 前向：T_P ← 模板 + 形状 + 表情 + 姿态校正；J ← J(β)；V ← LBS
  2. 投影：p_i ← Pi_K( R_θ(J)_i )        # 3D 关节 → 像素
  3. 能量：E ← λ_J·E_J + λ_β·E_β + λ_θ·E_θ + λ_S·E_S
  4. 反传：g ← dE/d(β, θ)                # 自动微分
  5. 更新：(β, θ) ← (β, θ) − α·inv(H)·g   # 拟牛顿（L-BFGS）一步
  6. 判断：|ΔE|/E < 1e-4 或 ||g|| < 1e-3 或达最大迭代数 → 停
输出：(β, θ) 与相机参数
```

第 1、2 步算“模型长什么样”，第 3 步打分，第 4—6 步改参数并判停。收敛不能只看能量：min 近似与自相交项会造出能量平台，参数还在漂，所以同时看参数变化量与梯度范数。迭代次数取决于初值：回归给初值时几十次够用，从零姿态起步要多得多；单次优化还会掉进局部极小，常见做法是几组初值各跑一次——这就是“优化法慢”的原因。

10.7 回归法的病根：歧义与“平均人”

优化法把歧义写在明面上，回归法把歧义藏进网络权重里。

（1）尺度与深度可以互相抵消。 弱透视相机下，3D 点 (X,Y,Z) 投到像素是

$$\hat{\mathbf{x}} = s\,(X,Y) + \mathbf{t}, \qquad s = \frac{f}{Z}, \tag{10.4}$$

f 是焦距，s 是缩放（越远越小），$\mathbf{t}$ 是平移。把体型放大 k 倍、同时把人推远 k 倍：s 变成 s/k，而 $s\cdot kX = sX$——投影一个像素都没变。所以“体型 × 深度”有一整族解；真实世界靠地面接触与透视收敛打破它，裁好的一张图里这些线索往往都不在。

（2）平方误差的解是“平均”，不是“某个人”。 设两个解 $\boldsymbol{\beta}_1,\boldsymbol{\beta}_2$ 都能解释同一张图，用平方误差训练时网络的最优输出不是其中任何一个，而是条件均值 $\mathbb{E}[\boldsymbol{\beta}\mid\mathbf{I}]$；两解等概率时就是两者中点。体型参数不是一条直线：两个“都像人”的体型取平均，可能得到“谁都不像”的体型。训练集里越罕见的人（很胖、很瘦、儿童）越容易被拉向均值——这就是“塌成平均人”。

（3）判别器补的正是这块。 HMR 训练了一组判别器判断“这组参数是不是真人的参数”：形状一个（10 维），每个关节的旋转各一个（3×3 矩阵拉平），再加一个看整体姿态。分成许多小判别器不是炫技：输入维度低，网络就能很小、训练也稳；而“每个关节一个判别器”在语义上等价于从数据里学出每个关节的角度限位。

它与重投影损失互补：前者管“像不像人”，后者管“投影对不对”。HMR 的消融很说明问题——既不用 3D 监督、又去掉判别器时，网络输出“怪物”：2D 误差很小，体型姿势却都不像人。两种失败是镜

像的：先验太强 → 塌成平均人；先验太弱 → 怪物。SMPLify-X 后来把高斯混合换成变分自编码器（VPoser），因为它更会抓高维姿态流形——思路没变：给歧义配一个从数据里学来的偏好。

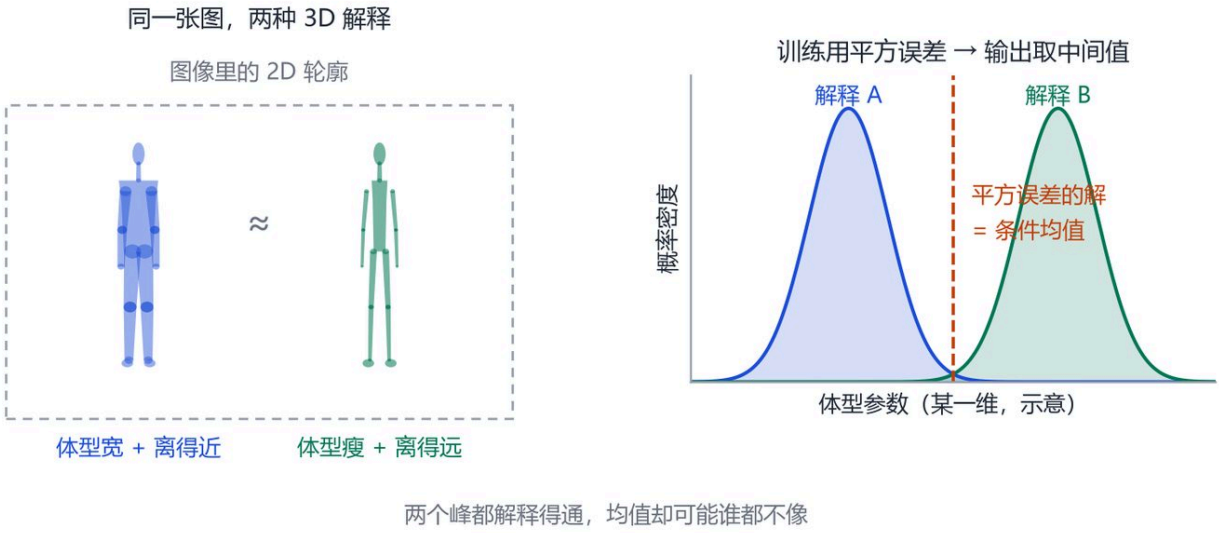


图 10-3 歧义如何变成“平均人”。左：两个 3D 体型投影几乎相同，E_J 分不开；右：平方误差训练的输出落在两峰之间（虚线），那里可能谁都不像。

本章小结

- 单图重建的输出是 $\boldsymbol{\beta},\boldsymbol{\theta}$ 与相机参数。
- 优化法（SMPLify）用 2D 关键点 + 先验 + 自相交惩罚，准但慢、难调参。
- 回归法（HMR）用网络直接预测参数，快但依赖训练分布；SPIN 把优化器放进训练循环，结合两者优点。
- GraphCMR 直接回归顶点，绕开参数空间，但更难约束。
- 深度歧义、遮挡、罕见姿势、衣服与评估偏差仍是主要难点。
- 把式（10.1）拆开：E_J 用有界的鲁棒函数抗错误关键点，E_β 把体型拉向平均人，E_θ 是 8 个高斯的姿态先验加反关节惩罚，E_S 用胶囊球惩罚自相交——谁的证据强，谁就说话。
- 一次拟合是“前向 → 投影 → 打分 → 反传 → 更新 → 判停”的循环，收敛要同时看能量、参数变化与梯度，单次容易掉进局部极小。
- 回归法的根病是尺度-深度歧义与平方误差的“求平均”，判别器/先验网络把输出拉回人体参数流形：先验太强塌成平均人，太弱则成怪物。

思考题

1. 为什么 SMPLify 需要“自相交惩罚”？不加会发生什么？
2. HMR 的对抗先验在训练中扮演什么角色？如果去掉，预测会变成什么样？
3. SPIN 用优化结果监督网络，会不会把优化器的偏差也学进去？

4. 如果给两张同一场景、不同角度的照片，深度歧义能消除吗？为什么？

第 11 章 视频、多人、全身与度量尺度

本章导读：单张图能猜出一个人，但视频里有时间、有多个人、有相机运动、“人到底走了多远”这些问题。这一章按“从帧到序列、从相机到世界、从单人到多人、从相对到度量”四条线，讲清楚 2019–2024 年的人体重建如何从“摆姿势”走向“理解场景”。

11.1 逐帧重建的三个毛病

把第 10 章的方法逐帧跑在视频上，会看到三种明显的问题：

1. 抖动：相邻帧的参数独立预测，体型、关节角会小幅跳动；
2. 身份漂移：同一个人在不同帧被预测成不同的 $\boldsymbol{\beta}$，看起来像“每帧换一个人”；
3. 深度跳变：单帧的深度歧义导致人忽大忽小、忽远忽近。

解决办法是利用时间上的约束：同一个人的形状应该不变，动作应该连续，相机运动应该平滑。



图 11-1 视频人体重建的典型流程。逐帧提特征，再做时序融合，输出 SMPL 序列，最后变换到世界坐标并估计脚与地面的接触。

11.2 时序模型：VIBE 与 4D-Humans

VIBE（Kocabas 等，2020）[23] 是最早把视频时序引入 SMPL 回归的工作之一。它用 CNN 提取每帧特征，再用 GRU 做时序融合，输出每帧的 $\boldsymbol{\beta},\boldsymbol{\theta}$。训练时用一个运动判别器，判断“这段动作序列像不像真

实动作捕捉”，从而让预测更平滑、更符合人体动力学。

4D-Humans（Goel 等，2023）[24] 把更强的单帧模型（HMR 2.0）[25] 与跟踪结合：先逐帧检测、再跨帧关联同一个人、最后做平滑。它的工程价值大于理论创新——证明了“强单帧模型 + 简单跟踪 + 平滑”就能得到可用的结果。

这两类方法的共同点是：它们仍然在相机坐标系里工作。人往前走，模型看到的是“人变大”，而不是“人走近”。

11.3 从相机坐标到世界坐标

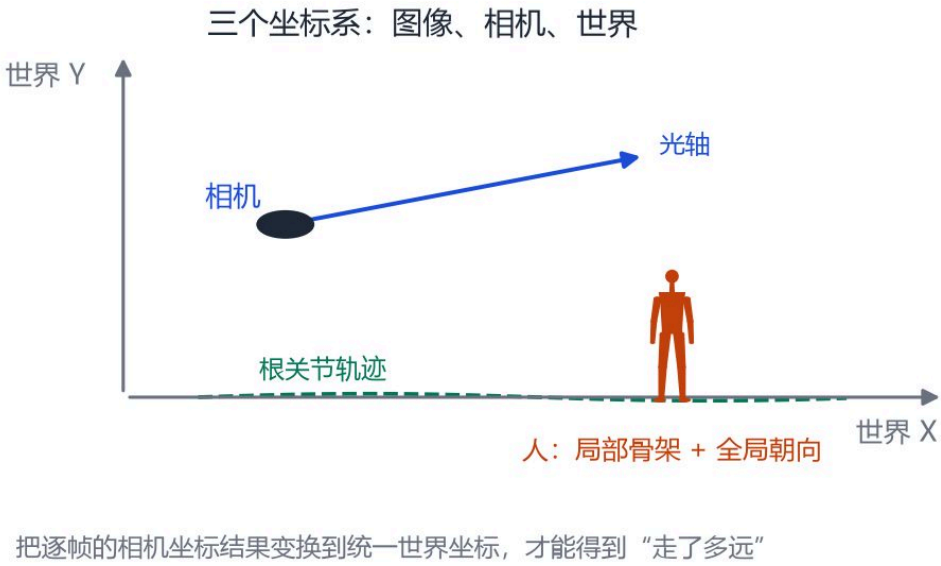


图 11-2 三个坐标系。图像坐标是像素，相机坐标以光心为原点，世界坐标以重力方向为 y 轴。只有把每帧结果变换到世界坐标，才能得到人的运动轨迹。

要把“每帧的人”拼成“一个人在场景里运动”，需要估计相机运动，并把人体变换到统一的世界坐标。这条线在 2023–2024 年成为热点：

- WHAM（Shin 等，2024）[26]：用脚与地面的接触作为约束，估计根节点的世界位移；用重力方向对齐 y 轴。
- TRAM（Yuan 等，2024）[27]：在 WHAM 的基础上加入相机轨迹估计，处理移动相机。
- GVHMR（Shen 等，2024）[28]：用“重力-视角对齐”把相机坐标下的人体转换到世界坐标，对相机运动更鲁棒。

这些方法的核心物理约束很朴素：

1. 脚踩在地上时，脚的速度应该接近 0（足部接触）；
2. 重力方向应该与世界 y 轴一致；
3. 人的身高在整段视频里不变。

有了这三条，就能把逐帧的相机坐标结果“拼”成一条连贯的世界轨迹。这也是“人体重建”走向“人体行为理解”的关键一步。

11.4 多人：一次前向 vs 先检测再拟合

多人场景有两条路线：

- 自顶向下（top-down）：先检测每个人的框，再对每个框单独跑单帧模型。简单、准，但速度随人数线性增长，且框重叠时容易出错。
- 单阶段（one-stage）：一次前向输出所有人的参数。Multi-HMR（Baradel 等，2024）[29] 是代表：它用 ViT 提取图像特征，直接回归每个实例的 SMPL-X 参数与位置，能处理相互遮挡、人与人接触的场景。

多人还带来一个新问题：相对位置。两个人谁在前、谁在后、相距多远，单目图像很难判断。Multi-HMR 通过预测每个人的根节点深度来缓解，但绝对尺度仍然依赖相机内参或场景先验。

11.5 度量尺度：从“看起来”到“量得准”

单目图像的深度歧义，本质上是尺度歧义：把人和相机同时放大 2 倍，投影完全一样。要得到真实身高，需要额外的信息：

- 相机内参：如果知道焦距，且能估计人到相机的距离，就能算出绝对尺度；
- 人体先验：假设人是正常身高（比如 1.7 米），反推深度；这对儿童、侏儒不成立；
- 场景参照：地板、门框、已知大小的物体；
- 多视角：两个视角的三角测量可以消除尺度歧义。

CameraHMR（Cho 等，2024）[30] 显式估计透视相机，利用“近大远小”的透视线索恢复度量尺度；NLF（Dwivedi 等，2024）[31] 则用神经局部场表示人体，能在任意相机下重建，并输出度量尺度。这些工作让“从一张照片量身高”从不可能变成“在受控条件下可行”。

11.6 规模化：SMPLer-X

SMPLer-X（Cai 等，2023）的思路是“大力出奇迹”：用 32 个数据集、数百万张图训练一个 ViT 大模型，输出 SMPL-X 参数。它证明了数据规模与模型规模在人体重建上同样有效：单一模型可以覆盖全身、手、脸，并在多个基准上取得很好的泛化。

它的局限也很明显：训练数据来自不同来源，标注质量参差不齐；模型很大，推理成本高；对罕见姿势仍然会失败。

11.7 怎么评价视频方法

视频评估除了单帧的 MPJPE、PVE，还要看：

- 加速度误差（acceleration error）：关节加速度与真值的差，衡量抖动；
- 世界坐标误差：整段轨迹的漂移；
- 身份一致性：同一人的 $\boldsymbol{\beta}$ 在整段视频里的方差；
- 脚滑（foot sliding）：脚踩地时是否滑动。

这些指标比单帧 MPJPE 更接近实际体验。一个方法可能单帧很准，但抖得没法看；也可能单帧一般，但时序平滑、观感很好。

11.8 相机运动怎么估：从光流到世界位移

11.3 节说要“变换到世界坐标”，没讲工程上怎么落地。设视频有 T 帧，第 t 帧已用第 10 章的方法拿到 SMPL 参数，此时关节都还在相机坐标系里。

链条有四步：

1. 只让背景参与。 把每帧里检测到的人抠掉，剩下的像素才是“不动的世界”。不抠人，人一走动就会被当成“相机在后退”。
2. 逐像素找对应。 在相邻两帧之间估计光流，得到一堆像素的位移量。
3. 解相邻帧的相对位姿。 用这些对应点解出一个 3 维刚体变换：相机从 t 到 $t+1$ 转了多少、走了多少。
4. 连乘。 把 $T-1$ 个相对位姿串起来，得到整段视频的相机轨迹 $\mathbf{R}_t, \mathbf{t}_t$。

第 3 步的两个符号要说明：$\mathbf{R}_t$ 是 3×3 的旋转矩阵，表示第 t 帧相机的朝向；$\mathbf{t}_t$ 是 3 维向量，表示相机光心在世界坐标里的位置。同一个 3 维点有两套坐标：

- $\mathbf{X}_t^c$：相机坐标，以光心为原点、z 轴沿光轴向前；
- $\mathbf{X}^w$：世界坐标，y 轴朝上（与重力反向）。

它们只差一步变换：

$$\mathbf{X}^w = \mathbf{R}_t^\top \mathbf{X}_t^c + \mathbf{t}_t \tag{11.1}$$

为什么用转置？因为 $\mathbf{R}_t$ 的方向是“世界 → 相机”，反过来用就得取逆，而旋转矩阵的逆恰好等于转置。式 (11.1) 还有个好处：只要变换根节点，其余关节会自动跟着走——根节点平移到位、全局朝向与 $\mathbf{R}_t^\top$ 复合之后，整副骨架就落进世界坐标了。

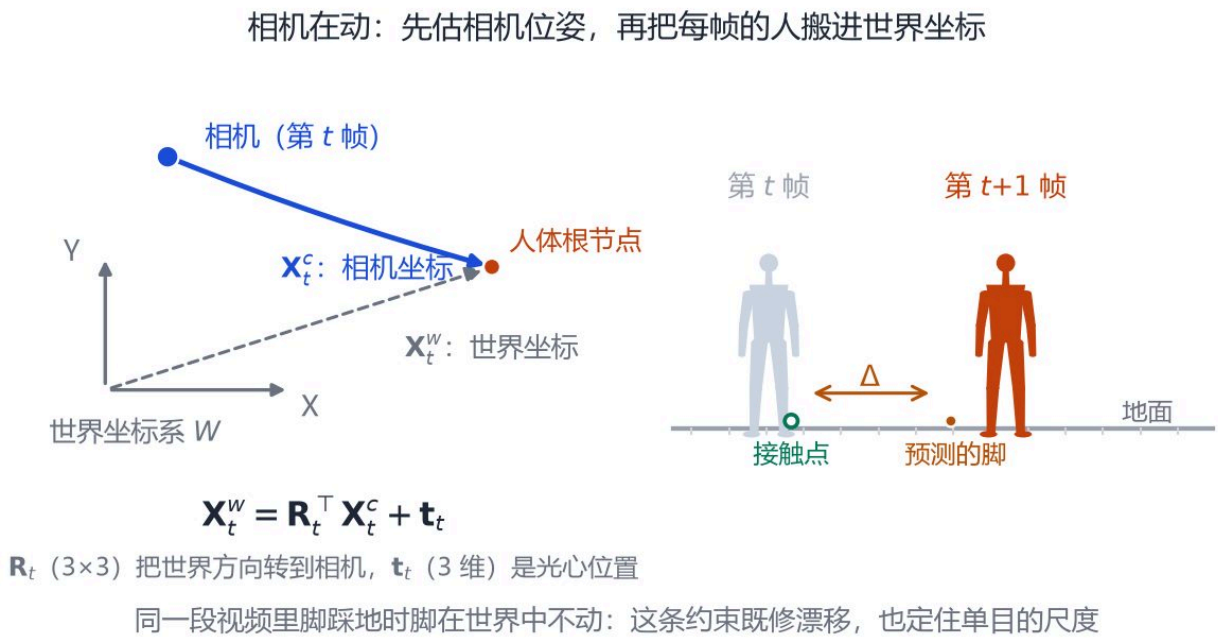


图 11-3 左：相机坐标与世界坐标的关系，两条向量指向同一个根节点。右：脚接触把误差量出来——第 t 帧的脚踩在接触点上，第 $t+1$ 帧预测的脚却偏了 Δ，把 Δ 扣回根节点即可修正。

尺度问题就藏在这里。 式（11.1）里的 $\mathbf{t}_t$ 如果来自单目 SLAM，只能恢复到一个未知倍数：$\mathbf{t}_t = s\tilde{\mathbf{t}}_t$，其中 $\tilde{\mathbf{t}}_t$ 是归一化后的轨迹，s 是一个我们还不知道的正数。人、相机、视频同时放大两倍，投影不变——11.5 节的尺度歧义跑到相机轨迹上了。

脚接触能把 s 解出来。设第 t 帧左脚踩在地上、第 $t+1$ 帧还没抬起，这只脚在两帧的世界坐标应相同：

$$s\tilde{\mathbf{t}}_{t+1} + \mathbf{R}_{t+1}^\top \mathbf{x}_{t+1}^c = s\tilde{\mathbf{t}}_t + \mathbf{R}_t^\top \mathbf{x}_t^c \tag{11.2}$$

其中 $\mathbf{x}_t^c$ 是脚的相机坐标，模型输出直接给。把含 s 的项挪到左边：

$$s\left(\tilde{\mathbf{t}}_{t+1} - \tilde{\mathbf{t}}_t\right) = \mathbf{R}_t^\top \mathbf{x}_t^c - \mathbf{R}_{t+1}^\top \mathbf{x}_{t+1}^c$$

读法很直白：左边是相机走了多远，右边是脚在画面里退了多少。 脚踩地不动，画面里的位移就全部来自相机；量出它即可反推相机步长，解出 s。式（11.2）有 3 个方程、1 个未知数，把多个接触帧叠起来做最小二乘即可。

失败模式也要记住：接触检测错了（跑、跳、被推着滑），s 会跳变、轨迹跟着抽；人一直坐着吊着，就没有接触帧，尺度无从恢复。所以 WHAM、TRAM、GVHMR 都会顺便输出一个接触概率，让下游知道哪几帧可信。

11.9 时序平滑：平滑了什么，又抹掉了什么

11.2 节说 VIBE、4D-Humans 会“做平滑”，这话太含糊：平滑对象不同，副作用完全不同。

（一）参数空间平滑。 直接对模型输出的 $\boldsymbol{\beta}$、$\boldsymbol{\theta}$ 滤波，最简单的是指数滑动平均（EMA）：

$$\hat{\boldsymbol{\theta}}_t = \alpha\,\boldsymbol{\theta}_t + (1-\alpha)\,\hat{\boldsymbol{\theta}}_{t-1}, \qquad 0 < \alpha \le 1 \tag{11.3}$$

α 是式（11.3）里的平滑系数：$\alpha = 1$ 等于不平滑，越小越平滑。它有两个副作用。第一是滞后：输出慢半拍，挥手、踢腿的峰值被削平，动作看着“糊”。第二是旋转不能直接线性平均：$\boldsymbol{\theta}$ 是轴角，两帧差别大时直接相加再除以 2，得到的朝向和“中间朝向”毫无关系。正确做法是在旋转群上插值（四元数的球面插值），或者干脆让网络自己学平滑。

（二）输出空间平滑。 对关节的 3 维轨迹或 2 维重投影滤波。好处是“看到什么就平滑什么”；坏处是它不再保证骨骼长度不变——几个关节各被拉一点，骨头就会被拉长压扁，网格像橡皮人。平滑 2D 重投影的坏处更直接：滑步。

滑步的机制值得单独说。脚踩地时，脚在世界里不动，但根节点在动（人往前走）。如果平滑把根节点的位移削掉一部分，脚的位置也跟着被削，于是“脚在地上平移”，视觉上像踩在冰面上。所以正确的顺序是：先平滑，再做接触修正。反过来做，平滑会把刚修好的接触又抹掉。

更根本的困难是：抖动和真实的高频动作长得一样。 走路时脚落地那一下本来就是高频，固定低通滤波分不清“噪声的高频”与“动作的高频”，切掉噪声的同时也切掉了力度感。VIBE 用运动判别器、4D-Humans 用真实动作训练，本质都是在学“真实动作的高频该长什么样”——这才是时序模型相对“逐帧 + 滤波”的优势。

本章小结

- 逐帧重建会抖动、漂移、深度跳变；时序融合与运动先验可以缓解。
- VIBE 用 GRU + 运动判别器；4D-Humans 用强单帧模型 + 跟踪。
- 世界坐标重建依靠足部接触、重力对齐、身高不变三条约束（WHAM、TRAM、GVHMR）。
- 多人有自顶向下与单阶段两条路线；Multi-HMR 是单阶段的代表。
- 度量尺度需要相机内参、人体先验、场景参照或多视角。
- SMPLer-X 证明了大规模数据与模型的价值。
- 相机运动可拆成四步：抠掉人 → 估光流 → 解相邻帧相对位姿 → 连乘成轨迹；式（11.1）只需变换根节点，其余关节跟着走。
- 单目相机轨迹有尺度歧义，脚接触用“脚在世界中不动”解出比例 s（式 11.2）。
- 平滑分两派：参数空间平滑会带来滞后，且旋转不能直接线性平均；输出空间平滑会破坏骨骼长度、容易滑步。正确顺序是“先平滑、再接触修正”。

思考题

1. 为什么“脚踩地”能帮助恢复世界位移？如果人一直坐着呢？
2. 单目图像中，人和相机同时放大两倍，投影不变。还有哪些变换不会改变投影？
3. 多人场景里，如何判断两个人是在握手还是前后站立？
4. 如果相机在移动，重力方向还能直接用吗？需要估计什么？

第 12 章 数据集、评估指标与常见坑

本章导读：这一章不讲新模型，讲“怎么判断一个模型好不好”。你会看到人体领域常用的数据集、五个核心指标，以及论文里经常被忽略的评估陷阱。读完这一章，你再看到“我们的方法在 3DPW 上降低了 10% 的 MPJPE”时，会先问三个问题：和谁比？怎么对齐？测试集干净吗？

12.1 数据集：从实验室到野外

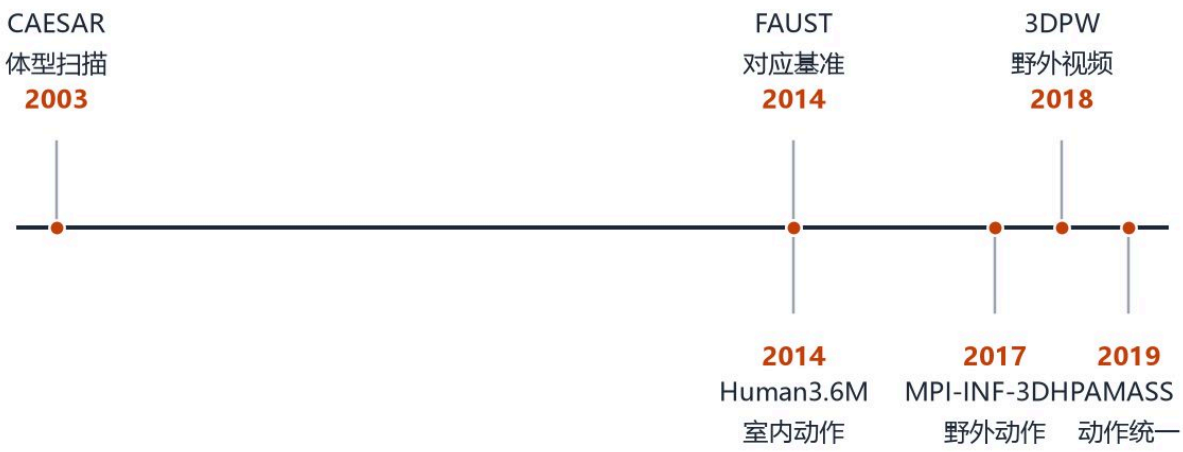


图 12-1 人体数据集的关键节点。早期以实验室扫描和动作捕捉为主，2017 年之后逐渐转向野外图像与视频，评估也从“姿态对不对”扩展到“尺度、轨迹、接触”。

人体领域的数据集可以按“有没有 3D 真值”和“在室内还是野外”分成四类。

表 12-1 常用人体数据集。

数据集	类型	特点	主要用途
CAESAR	全身扫描	静态、多体型	形状空间训练
FAUST	扫描 + 对应	高精度、同拓扑	配准与重建评估
Human3.6M	室内视频 + Mocap	360 万帧、11 人、15 类动作	3D 姿态基准
MPI-INF-3DHP	室内多视角	有标记/无标记	跨域评估
3DPW	野外视频	手持相机、自然场景	单图/视频泛化

数据集	类型	特点	主要用途
AMASS	动作捕捉统一	统一到 SMPL 参数	动作先验、生成
AGORA	多人 + 场景	全身、遮挡、接触	多人重建
BEDLAM	合成	大量渲染、精确真值	大规模预训练

CAESAR 是形状模型的“粮食”：数千名受试者的全身扫描，覆盖不同体型。SMPL 的形状空间就是在它上面训练的。

Human3.6M 是 3D 姿态的“老基准”：11 名受试者、15 类动作、360 万帧，配有动作捕捉真值。它的缺点是室内、受控、只有 15 类动作，训练出来的模型到野外容易崩。

3DPW 是“野外视频”的代表：手持相机拍摄，人体在自然场景里走动、互动。它的 3D 真值不是直接捕捉的，而是用优化方法拟合出来的，所以存在“用同类方法评估同类方法”的偏差。

AMASS 把多个动作捕捉数据集统一成 SMPL 参数，是动作先验、动作生成、物理仿真的基础 [32]。

BEDLAM 是合成的：用图形学渲染大量人体，真值精确、成本低，但存在“合成到真实”的域差异。

> **选数据集的建议**：训练形状用扫描，训练姿态用 Mocap/合成，评估泛化用野外视频。不要只用 Human3.6M 训练然后宣称“野外可用”。

12.2 五个核心指标

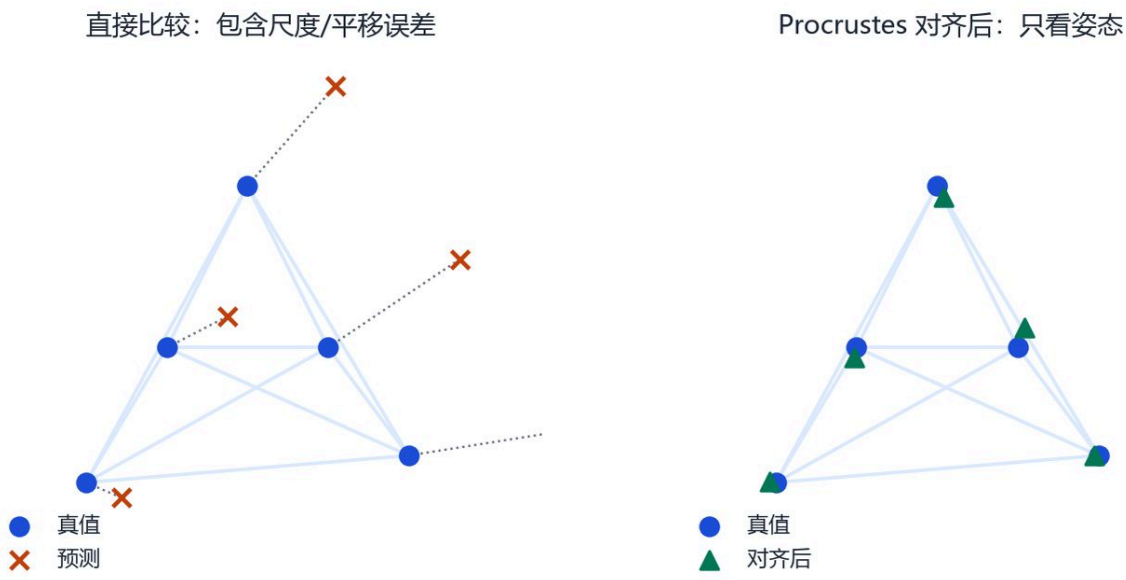


图 12-2 MPJPE 与 PA-MPJPE 的区别。左图直接比较预测与真值，包含尺度、平移、旋转误差；右图先做 Procrustes 对齐（去掉相似变换），只看姿态形状。PA 指标更低，但它会掩盖“人太大/太小/位置不对”的问题。

12.2.1 MPJPE

平均关节位置误差（Mean Per-Joint Position Error）：

$$\mathrm{MPJPE} = \frac{1}{J}\sum_{j=1}^{J} \left\| \mathbf{p}_j - \mathbf{g}_j \right\|_2, \tag{12.1}$$

单位通常是毫米。$\mathbf{p}_j$ 是预测的关节，$\mathbf{g}_j$ 是真值。通常先把根节点平移对齐，再算误差。

12.2.2 PA-MPJPE

Procrustes 对齐后的 MPJPE：先找一个最优的相似变换（旋转 + 缩放 + 平移），把预测对齐到真值，再算误差。

$$\min_{s,\mathbf{R},\mathbf{t}} \sum_{j} \left\| s\mathbf{R}\mathbf{p}_j + \mathbf{t} - \mathbf{g}_j \right\|^2. \tag{12.2}$$

PA 指标衡量“姿态形状”而不是“绝对位置与大小”。它更宽容，也更容易被滥用：一个把所有人体型都预测成平均值的方法，PA-MPJPE 可能不难看，但绝对尺度完全错了。

12.2.3 PVE / MPVE

每顶点误差（Per-Vertex Error）：把预测网格与真值网格的对应顶点相减，取平均。它比关节误差更细，能反映体型、肌肉、衣服的差异。

$$\mathrm{PVE} = \frac{1}{N}\sum_{i=1}^{N} \left\| \mathbf{v}_i - \hat{\mathbf{v}}_i \right\|_2. \tag{12.3}$$

12.2.4 PCK

正确关键点比例（Percentage of Correct Keypoints）：误差小于阈值的关节占比。阈值可以是绝对像素，也可以是“头部长度的比例”。PCK 对阈值很敏感，跨数据集比较时要看清定义。

12.2.5 加速度误差

视频方法常用：

$$\mathrm{Accel} = \frac{1}{T}\sum_{t} \left\| (\mathbf{p}_{t+1} - \mathbf{p}_t) - (\mathbf{p}_t - \mathbf{p}_{t-1}) \right\|_2, \tag{12.4}$$

衡量抖动。它不关心“准不准”，只关心“稳不稳”。

12.3 论文里常见的评估陷阱

陷阱一：对齐方式不说明。 MPJPE 前有没有做根节点对齐？有没有做 Procrustes？不同选择能差出 30%。看论文时先找“alignment”这个词。

陷阱二：在训练集上评估。 有些数据集（如 Human3.6M）的训练/测试划分很复杂，稍不注意就会把测试帧混进训练。

陷阱三：用拟合结果当真值。 3DPW 的 3D 真值来自优化拟合，如果你的方法也是优化拟合，等于“和自己比”。真正干净的真值是 FAUST 这种扫描 + 手动对应。

陷阱四：只报平均，不报分布。 平均误差低，可能只是因为大多数简单样本很好，而困难样本（遮挡、罕见姿势）完全崩。要看分位数、失败案例。

陷阱五：混淆“姿态”与“形状”。 PA-MPJPE 低不代表体型对；PVE 低不代表关节对。两个指标要一起看。

陷阱六：忽略相机。 单图方法的尺度误差常常被“弱透视假设”掩盖。真实部署时，相机内参未知，误差会放大。

12.4 一个健康的评估习惯

如果你要做一个自己的方法，建议至少报告：

1. 三个数据集：一个室内（Human3.6M）、一个野外（3DPW）、一个扫描（FAUST）；
2. 两个指标：PA-MPJPE（姿态）+ PVE（形状）；
3. 一个效率指标：FPS 或参数量；
4. 一组失败案例：遮挡、多人、极端姿势；
5. 消融实验：去掉某个模块后掉多少。

这比“在单个数据集上刷 0.5 毫米”有意义得多。

12.5 指标是怎么算出来的：五个关节走一遍

所有指标都是同一个三步：对齐 → 相减 → 平均。差别只在两处：对齐允许用什么变换、相减的是关节还是顶点。下面把数字算一遍。

表 12-2 例子里用的五个关节真值（单位：米，y 轴朝上，z 轴朝向镜头）。

编号	关节	x	y	z
0	骨盆（根节点）	0.00	1.00	0.00
1	头	0.00	1.60	0.00
2	左手	0.45	1.35	0.00
3	右手	-0.45	1.35	0.00
4	左脚	0.10	0.00	0.00

这五个点都落在 $z=0$ 平面上，好比一个人正对镜头站着。现在假设某个方法的预测，是把整个人等比缩小 10%，再平移一点点——典型的错误：模型把“标准身材”套在了偏矮的人身上。

第一步，根节点对齐。 把预测整体平移，让骨盆与真值骨盆重合：只消平移，不消旋转与尺度。

第二步，逐关节量距离。 缩小到 0.9 之后，头离骨盆 0.6 米变成 0.54 米，差 60 毫米；双手各差 57 毫米；左脚离骨盆最远（1 米），差 100.5 毫米；骨盆自己是 0。

第三步，平均。 $(0+60+57+57+100.5)/5=54.9$ 毫米，这就是 MPJPE。注意误差随“离根节点的距离”线性变大，这正是“整体缩小”的指纹。

换 PA-MPJPE：允许“缩放 + 旋转 + 平移”一起去对齐。这个预测本来就是等比缩小，Procrustes 解出 $s=1/0.9=1.111$，把预测放大 1.111 倍，恰好抵消，PA-MPJPE $=0.0$ 毫米。图 12-3 是这两步的对比。

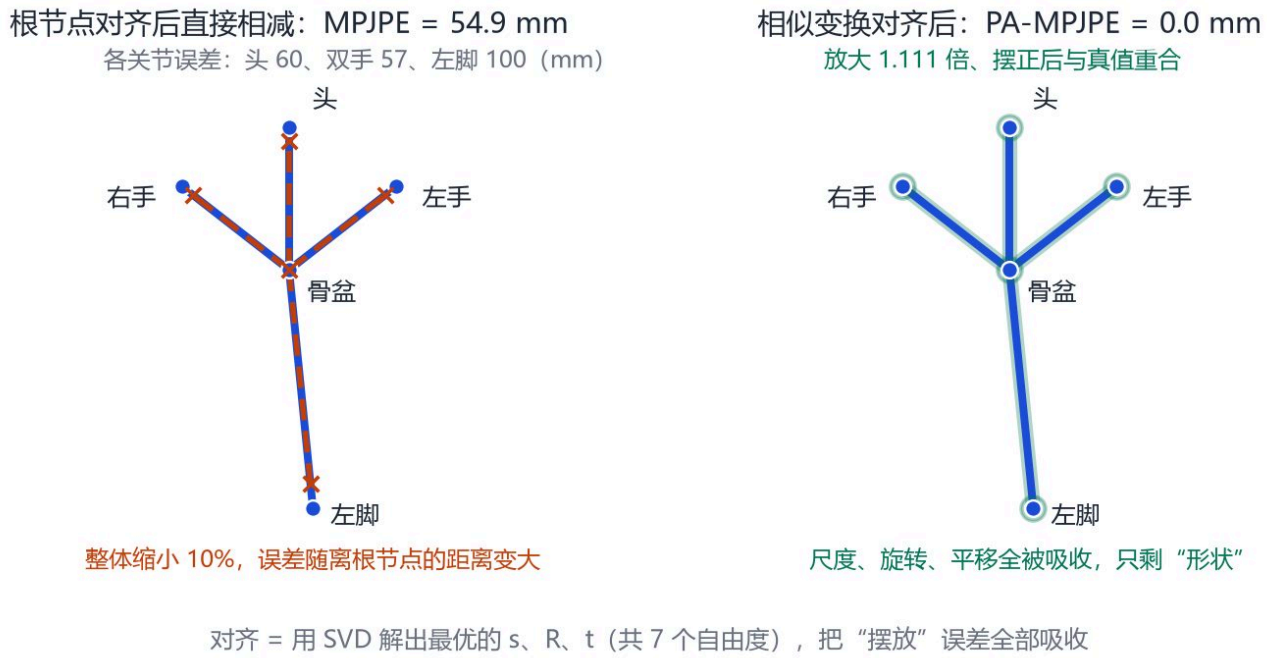


图 12-3 同一组预测的两种算法。左：只做根节点平移，误差随离根节点的距离变大，MPJPE 为 54.9 毫米。右：允许相似变换对齐后预测与真值完全重合，PA-MPJPE 为 0——“缩小 10%”这类错误被对齐彻底吸收了。

所以：PA-MPJPE 对尺度完全不敏感。 把所有人都预测成“标准身材缩小 10%”的方法，PA-MPJPE 可以是 0，MPJPE 却是 54.9 毫米，拿去“量身高”会系统性偏矮。

再加一点料：若在上面的预测上再加 10° 旋转、每个坐标再叠 8 毫米随机噪声，同一组点得到 MPJPE $=110.7$、PA-MPJPE $=17.8$ 毫米；去掉噪声、只留缩小与旋转，PA 又回到 0。可见这 17.8 毫米几乎全部来自“关节之间的相对位置不对”。

四个指标的分工：

表 12-3 四个指标各在比什么。

指标	对齐方式	比的对象	对什么敏感	掩盖什么
MPJPE	只平移（常以根节点对齐）	关节	尺度、朝向、位置	网格细节
PA-MPJPE	相似变换（7 自由度）	关节	关节之间的相对位置	尺度、朝向、位置
PVE	与 MPJPE 相同	全部顶点	体型、软组织、衣服	单个关节是否对得上（被平均掉）
PVVE	与 PVE 相同	顶点速度	动作对不对	绝对位置

PVVE 是“逐顶点速度误差”：把 PVE 里的位置换成相邻帧的速度差，用于评价视频结果。与式 (12.4) 的加速度误差是一对。

PVE 与 MPJPE 的差别也值得一个例子。假设关节全部落在正确位置，但预测把腰围做粗了 2 厘米：腹部顶点整体外移约 2 厘米，PVE 在腹部多出 20 毫米量级的误差，而 MPJPE 几乎不动——关节是从骨骼内部回归的，软组织变化影响不到它们。“关节准”和“网格准”是两件事。

12.6 一个可以直接用的评估脚本

真正算数的只有三行，其余全是防坑。输入是两个 $J \times 3$ 的数组，单位一致，按米写。

```
import numpy as np

def mpjpe(pred, gt):
    """pred, gt: (J,3), 第 0 号是根节点; 单位一致。"""
    p, g = pred - pred[0], gt - gt[0]              # 根节点对齐: 只去平移
    return np.linalg.norm(p - g, axis=1).mean()

def pa_mpjpe(pred, gt):
    """相似变换对齐后算 MPJPE, 不允许镜像。"""
    p, g = pred - pred.mean(0), gt - gt.mean(0)     # 两边都先减掉重心
    U, S, Vt = np.linalg.svd(p.T @ g)               # 3×3 的互协方差
    d = np.sign(np.linalg.det(U @ Vt))              # ±1: 防止把镜像当成对齐
    R = (U @ np.diag([1, 1, d]) @ Vt).T             # 最优旋转
    s = (S[0] + S[1] + d * S[2]) / (p ** 2).sum()   # 最优缩放
    return np.linalg.norm(s * p @ R.T - g, axis=1).mean()
```

三个要点逐行说。

为什么两边都要减重心。 Procrustes 找的是“绕重心”的最优旋转与缩放。不先减重心，平移就会混进 SVD 里，解出来的旋转是错的——这是一个不会报错、只会让指标莫名变差的 bug。

`det` 那一行在防什么。 SVD 的最优解可能是一个镜像（行列式为 -1）：当两组点近似互为镜像时（左右手搞混、或用了镜像增广），不加修正就会得到一个物理上不存在的对齐，误差虚低，而且很隐蔽。乘一个 $\mathrm{diag}(1,1,d)$ 就是把镜像翻回来。

缩放的分子为什么是 $S_0 + S_1 + dS_2$。 S 是三个奇异值，衡量预测与真值在三个方向上的相关性；三者相加再除以预测的平方和，恰好是最小二乘意义下的最优缩放。推导略长，但有一个两秒钟的自检：

把真值乘 1.7 倍、随便转一下、挪一下，再喂给 `pa_mpjpe`，输出必须是 0，而解出的 s 必须是 1.7。这个自检一定要写进你的测试。（本书的这段代码就是这样验证的：误差 0.000000，$s = 1.7000$。）

五个常见实现错误：

1. 忘了去根节点。 症：数值大得离谱，同一个动作换个站位就变，几百毫米很常见。
2. 毫米和米混用。 症：结果是别人的 1000 倍或 1/1000。算出 4.7 而论文报 47，先查单位。
3. 关节顺序不一致。 症：MPJPE 还行、PA 异常差，或者左右手误差对调。跨库（SMPL、SMPL-X、OpenPose）时必查。
4. 没做镜像修正。 症：PA-MPJPE 好得不真实，明显低于同行的报告值。
5. 把没检测到的关节填成 0。 症：个别样本误差上千毫米，平均值被拉爆。应该用可见性掩码，只对有效关节求平均。

本章小结

- 数据集分扫描、室内 Mocap、野外视频、合成四类，各有用途与偏差。
- MPJPE 看绝对误差，PA-MPJPE 看姿态形状，PVE 看网格，PCK 看阈值内比例，Accel 看抖动。
- 对齐方式、真值来源、训练/测试划分、平均 vs 分布，是评估的四大陷阱。
- 健康评估应跨数据集、多指标、带失败案例与消融。
- 所有指标都是“对齐 → 相减 → 平均”：差别只在用什么变换对齐、比关节还是比顶点。
- 同一个“整体缩小 10%”的预测，MPJPE 是 54.9 毫米，PA-MPJPE 是 0.0 毫米——PA 对尺度完全不敏感，它盖不住体型错误。
- PVE 管网格、PVVE 管速度；自己写评估要盯三件事：两边都去重心、防止镜像解、单位一致。

思考题

1. 一个方法 PA-MPJPE 很低但 PVE 很高，可能是什么原因？
2. 为什么 3DPW 的真值不能算“绝对干净”？
3. 如果只能选一个指标向非专业用户汇报，你会选哪个？为什么？
4. 合成数据训练、真实数据评估，最大的风险是什么？

第 13 章 着装人体：衣服、头发与神经渲染

本章导读：SMPL 是一个“紧身衣”模型。现实中的人穿裙子、西装、羽绒服，还有头发、胡须、配饰。这一章讲三类补细节的方法：给网格加位移、用隐式场表示表面、用神经渲染表示外观。它们分别对应“可控”“精细”“真实”三个目标，今天最好的系统往往三者混用。

13.1 裸体模型的三个不够

SMPL 能描述体型和姿态，但直接拿来用会有三个明显问题：

1. 衣服被当成皮肤：穿裙子的人会被拟合成“紧身裤”，裙摆的褶皱完全丢失；
2. 头发和配饰缺失：SMPL 的头顶是光滑的，没有头发；
3. 纹理与光照不分：模型没有颜色，更没有“这件衣服在阳光下是什么质感”。

要补上这些，必须离开“6890 个顶点 + 固定拓扑”的舒适区。



图 13-1 人体表示的光谱。越往左，越可控、越容易驱动；越往右，越真实、细节越多。实际系统通常用左边的参数化模型做“骨架”，用右边的神经表示补“皮肤”。



图 13-2 着装人体系统的通用管线。输入多视角图像或视频，先用 SMPL 拟合出“人在哪里、摆什么姿势”，再学一层残差/位移表示衣服与头发，然后把纹理与光照分离，最后得到一个可以换装、换姿势、换视角的可驱动 Avatar。整条管线的关键是把“人”与“衣服 / 头发 / 光照”分开建模。

13.2 路线一：给 SMPL 加位移

最直接的办法是：保持 SMPL 的拓扑，给每个顶点学一个位移，表示“衣服让这个顶点往外鼓了多少”。

CAPE（Ma 等，2020）[33] 用 4D 扫描训练一个“衣服位移场”：输入是 SMPL 的体型、姿态、以及一个衣服类别，输出是每个顶点的位移。它能为不同姿势生成合理的褶皱，但只能处理训练过的衣服类型。

Tex2Shape（Alldieck 等，2019）[34] 走的是另一条路：先把 SMPL 渲染成一张 UV 图，再用图像到图像的翻译网络，从单张照片预测“法线图/位移图”，最后反投影回网格。它不需要 4D 扫描，但细节受限于 UV 分辨率。

ARCH（Huang 等，2021）进一步引入“姿态校正”和“图像特征”，能同时生成衣服几何与纹理，并支持换装。

这条路线的优点是：输出仍是网格，能直接进游戏引擎、能实时驱动。缺点是：拓扑固定，无法表示裙摆、长发这种“拓扑会变”的结构。

13.3 路线二：隐式表面

隐式表示把表面定义成一个函数：给定空间点 $\mathbf{x}$，输出“这个点在表面内还是外”（有符号距离场 SDF），或者“这个点有多大概率在表面附近”（occupancy）。

PIFu（Saito 等，2019）[35] 用像素对齐的隐式函数：对每个三维点，投影到图像上取特征，再判断它是否在表面上。它能从单张图重建高细节的人体，包括衣服褶皱和头发，但输出不是参数化网格，难以驱动。

SNARF（Chen 等，2021）[36] 解决“驱动”问题：它学习一个“前向蒙皮场”，把隐式表面上的任意点绑定到 SMPL 骨骼上。这样，隐式表面也能跟着姿势动。

SCANimate（Shao 等，2021）用循环一致性，从没有配准的扫描里学习可驱动的隐式人体。

隐式表示的优点是：不受拓扑限制，能表示头发、裙摆、配饰。缺点是：渲染慢、难以实时、难以编辑。

用式子写出来，隐式表面就是函数 f 的零水平集：

$$\mathcal{S} = \{\, \mathbf{x} \in \mathbb{R}^3 \;\big|\; f_\omega(\mathbf{x};\, \mathbf{z}_{\text{image}},\, \boldsymbol{\beta},\, \boldsymbol{\theta}) = 0 \,\}, \tag{13.1}$$

其中 $\mathbf{z}_{\text{image}}$ 是图像特征（PIFu 的做法是把 $\mathbf{x}$ 投影到图像上取对应像素的特征），ω 是网络参数。如果 f 输出的是“在内部的概率”，就把它叫 occupancy；如果输出有符号距离，就是 SDF。两者只差一个单调映射，训练目标不同。

要把“零水平集”变成一张能看的图，必须用体渲染把颜色沿射线积起来：

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\, \sigma(\mathbf{r}(t))\, \mathbf{c}(\mathbf{r}(t))\, dt, \qquad T(t) = \exp\Big(- \int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds \Big), \tag{13.2}$$

$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ 是一条从相机出发的射线，σ 是密度，$\mathbf{c}$ 是颜色，$T(t)$ 是“光线到达 t 时还没被遮住的比例”（透射率）。式 (13.2) 是成百万条射线上的积分，所以慢——这就是隐式/神经渲染“质量高但慢”的根源，与它用不用 NeRF 无关。

> **一个容易搞混的点**：PIFu 类方法**不一定要体渲染**。它可以直接把隐式表面用 marching cubes 抽成网格，然后在网格上渲染。抽成网格的好处是能用常规图形管线、能进引擎；坏处是每次改姿势都要重新抽（拓扑会变）。

13.4 路线三：神经渲染

NeRF 把场景表示成“每个空间点的颜色与密度”，用体渲染合成新视角。HumanNeRF（Weng 等，2022）[37] 把人体拆成“标准姿势的 NeRF + 运动场”，用 SMPL 驱动，能从单目视频重建可自由视角观看的人。

InstantAvatar（Jiang 等，2023）[38] 把训练时间从几小时压到几十秒，方法是把 NeRF 放在一个规范空间里，用快速编码 + 少量迭代。

HumanRF（Gong 等，2023）[39] 用多视角视频重建 4D 人体，能处理快速动作与复杂衣服。

神经渲染的优点是：照片级真实感，连毛孔、布料纹理都能学到。缺点是：训练慢、渲染慢、难以编辑、难以直接进引擎。

13.5 3D Gaussian Splatting：新的折中

2023 年，3D Gaussian Splatting（3DGS）用一堆带方向的高斯椭球表示场景，渲染速度比 NeRF 快两个数量级。它很快被用到人体上：

SplattingAvatar（Moreau 等，2024）[40] 把高斯绑定到 SMPL 的三角面上，用重心坐标驱动。人一动，高斯跟着面片动，既保留了 3DGS 的实时渲染，又能被骨架驱动。

GaussianAvatars（Qian 等，2024）[41] 把高斯绑定到头部的 FLAME 模型上，做照片级人头头像。

这条路线目前最被看好：实时、可驱动、真实感介于网格与 NeRF 之间。它的挑战是：高斯的数量、内存、以及“换衣服/换表情”时的泛化。

3DGS 快在哪里？关键是把“射线积分”换成了“把椭球投影到屏幕、排序、逐个混合”这三步。一个三维高斯由中心 $\boldsymbol{\mu}$ 与协方差 $\boldsymbol{\Sigma}$ 描述：

$$G(\mathbf{x}) = \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\top}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big). \tag{13.3}$$

投影到图像平面后，二维协方差可以写成

$$\boldsymbol{\Sigma}' = \mathbf{J}\,\mathbf{W}\,\boldsymbol{\Sigma}\,\mathbf{W}^{\top}\mathbf{J}^{\top}, \tag{13.4}$$

$\mathbf{W}$ 是世界到相机的旋转，$\mathbf{J}$ 是透视投影在该点处的雅可比（近似）。屏幕上每个像素的颜色，是把沿视线方向排序后的高斯按 α 混合：

$$C = \sum_{i \in \mathcal{N}} \mathbf{c}_i \alpha_i \prod_{j<i} (1-\alpha_j), \tag{13.5}$$

$\mathcal{N}$ 是按深度排序后与当前像素重叠的高斯集合。这一步是纯光栅化，不需要网络前向、也不需要沿射线采样，所以能到实时。代价是：顺序错了就会“穿透”（前后遮挡错误），所以每帧都要排序；人一动，排序和协方差都要重算——这就是“高斯数量与内存”成为瓶颈的原因。

绑定到人体时（SplattingAvatar 一类），每个高斯记在某个三角面的重心坐标上：

$$\boldsymbol{\mu} = \sum_{k=1}^{3} b_k\, \mathbf{v}_{f(k)}, \qquad b_1 + b_2 + b_3 = 1, \tag{13.6}$$

$\mathbf{v}_{f(k)}$ 是该三角面的三个顶点，b_k 是重心坐标。顶点一动，高斯中心跟着线性变化，连导数都是现成的，所以能直接套 LBS 驱动。

13.6 把“人”和“衣服/光照”分开

无论用哪种表示，一个核心原则是解耦：

- 人体几何（SMPL）与衣服（位移/隐式残差）分开；
- 反照率（albedo）与光照分开；
- 身份（形状）与表情/姿势分开。

只有解耦，才能做到“换一件衣服”“换一个光照”“换一个姿势”而不用重新扫描。这也是数字人工业管线的核心：先做一个“裸体 + 骨架”的基础资产，再叠加衣服、头发、材质。

13.7 选型：什么时候用哪种表示

三种表示不是“新的一定比旧的好”，而是各自适配不同的约束。把选择标准列成表：

需求	位移网格	隐式/神经场	3DGS
可驱动（进动画管线）	最好：直接接 LBS	需额外学蒙皮场（SNARF 类）	好：绑到面片上
实时渲染	最好	差（体渲染）	好（光栅化）
头发/裙摆/薄片	差（拓扑固定）	好	好
可编辑（改衣服/改脸）	好（改顶点）	差（隐式场不可直接操作）	中（增删高斯）
内存	小	中	大（百万级高斯）
训练成本	低	高	中
换装泛化	只能训练过的衣服类型	可外插但不稳	需重训

实操上的三个常见坑：

1. 把“能重建”当成“能驱动”。很多方法在论文里渲染得很漂亮，但输出是一个不可驱动的静态网格；要驱动，必须额外有“前向蒙皮场”或“绑到骨架上的高斯”。
2. 忽略衣服与身体的分离。如果位移场把衣服和皮肤一起学，换姿势时裙摆会跟着大腿走——因为蒙皮权重是从身体继承的。正确做法是给衣服单独一套权重，或者用“身体 + 衣服”两层表示。
3. 在错误的指标上比较。只看渲染图的 PSNR，会系统性偏好“模糊但平均正确”的方法；要看轮廓、要看侧面视角、要看动作序列的时序稳定性。

13.8 评估与难点

着装人体的评估比裸体更难，因为：

- 没有干净的真值：衣服会动、会皱，扫描本身有噪声；
- 拓扑会变：裙摆、长发没有固定顶点对应，PVE 用不了；
- 视角依赖：布料的高光、绒毛，换个角度就不一样。

常用指标包括：几何的 Chamfer 距离、渲染的 PSNR/SSIM/LPIPS、以及人类主观评分。工业界还会看“是否穿模”“是否跟手”“是否能实时”。

本章小结

- SMPL 是“紧身衣”模型，补细节有三条路：加位移、用隐式场、用神经渲染（含 3DGS）。
- 隐式表面是函数的零水平集，渲染靠体渲染积分，所以慢；抽成网格能进引擎，但每次改姿势要重抽。

- 3DGS 把体渲染换成“投影—排序—混合”，因此能实时；代价是内存与排序开销，绑到三角面上才能被骨架驱动。
- 核心工程原则是解耦：人 / 衣服 / 光照 / 身份分开建模，才能换装、换光、换姿势。
- 选型看四件事：能不能驱动、要不要实时、有没有薄片结构、要不要可编辑。
- 加位移（CAPE/Tex2Shape/ARCH）保持网格与可驱动性，但拓扑固定；隐式表面（PIFu/SNARF/SCANimate）能表示任意拓扑，但渲染慢、难编辑；神经渲染（HumanNeRF/InstantAvatar/HumanRF）真实感最好，但训练与渲染贵；3DGS（SplattingAvatar/GaussianAvatars）在实时与真实感之间取得新平衡。

思考题

1. 为什么“给 SMPL 加位移”无法表示裙摆？隐式表示为什么可以？
2. 3DGS 与 NeRF 的本质区别是什么？为什么 3DGS 更快？
3. 如果要把一个神经化身放进游戏引擎，你会选哪种表示？为什么？
4. “解耦”会带来什么代价？哪些信息在解耦时容易丢失？

第 14 章 人体动作与生成模型

本章导读：前面讲的是“一个人长什么样、摆什么姿势”，这一章讲“人怎么动”。动作是姿态随时间变化的轨迹，比单帧多了速度、加速度、接触、平衡等约束。我们按“先验 → 生成 → 控制”三层来讲，最后讨论为什么“生成动作”比“生成图片”更难评估。

14.1 动作的表示

一个动作序列可以写成

$$\Theta = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots, \boldsymbol{\theta}_T), \tag{14.1}$$

每帧 $\boldsymbol{\theta}_t \in \mathbb{R}^{72}$ 是 SMPL 的姿态，外加根节点的位移 $\mathbf{r}_t \in \mathbb{R}^3$。所以一个 T 帧的动作大约是 $T \times 75$ 个数。

这个表示很紧凑，但直接在上面做生成会遇到两个问题：

1. 关节角有周期性：0 和 2π 是同一个角度，直接用欧拉/轴角做回归会在边界跳变；
2. 物理约束：脚不能穿地、重心不能悬空、关节不能反折。

所以动作模型通常不是“直接预测角度”，而是先学一个低维隐空间，再在隐空间里生成。

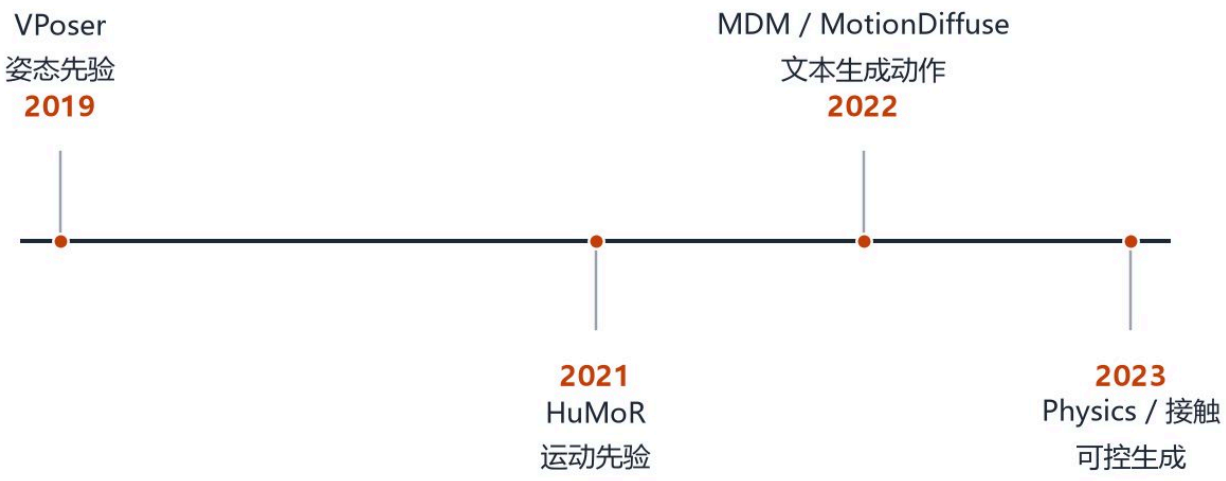


图 14-1 动作建模的四步。2019 年的 VPoser 把单帧姿态压进隐空间（姿态先验）；2021 年的 HuMoR 把先验从“单帧”扩展到“时序”（运动先验）；2022 年 MDM / MotionDiffuse 把文本条件接进来；2023 年之后开始把接触与物理约束显式加进生成过程。

14.2 姿态先验：VPoser

VPoser（Pavlakos 等，2019，随 SMPL-X 发布）[21] 是一个变分自编码器（VAE），把 72 维姿态压到一个 32 维的正态分布隐空间。

它的作用有两个：

- 生成：从隐空间采样，得到合理的姿态；
- 正则：在拟合图像时，把预测的姿态拉回“像人”的区域，防止反关节。

VPoser 的训练数据是 AMASS 等动作捕捉数据。它学到的隐空间不是均匀的，而是集中在人经常做的动作附近——所以从它采样，不会得到“手肘向后折 180° ”的姿势。

把 VAE 的目标写出来，就明白“正则”是怎么起作用的：

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\boldsymbol{\theta})} \left\| \boldsymbol{\theta} - D_\phi(\mathbf{z}) \right\|^2}_{\text{能复原：别丢信息}} + \beta \underbrace{D_{\text{KL}}\left(q_\phi(\mathbf{z}|\boldsymbol{\theta}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\right)}_{\text{隐空间靠拢正态}}, \tag{14.2}$$

- 第一项逼着编码器 q_ϕ 把姿态压成 $\mathbf{z}$、解码器 D_ϕ 再从 $\mathbf{z}$ 复原姿态；只留这一项，隐空间会“记住”每个训练样本，没有泛化；
- 第二项（KL 散度）逼着隐空间分布接近标准正态，所以“在隐空间里随便取一个点”也能解码出合理姿态——这就是采样能用的原因。

β 是权衡系数：太大，隐空间丢掉姿态细节（生成的人“千篇一律”）；太小，隐空间变成一张“样本清单”，插值与采样都会失控。这个取舍在下面的动作生成里会再次出现。

14.3 运动先验：HuMoR

单帧姿态先验不知道“动作是否连贯”。HuMoR（Rempe 等，2021）[42] 用条件 VAE 建模“从上一帧到下一帧的变化”，并引入脚与地面的接触状态。

它的生成过程是马尔可夫的：给定当前状态（姿态、速度、接触），预测下一帧的状态分布。这样既能去噪，又能补全被遮挡的帧。

HuMoR 的意义在于：它把“动作”当成一个动态系统，而不是一堆独立的姿态。这为后来的视频重建、动作补全提供了基础。

14.4 文本生成动作：MDM 与 MotionDiffuse

2022 年之后，扩散模型被用到动作生成上。

MDM（Tevet 等，2022）[43] 把一段动作看成一个序列，用 Transformer 做去噪：训练时把真实动作加噪，网络学习“从噪声中恢复动作”；推理时从纯噪声开始，逐步去噪，并用文本编码作为条件。

MotionDiffuse（Zhang 等，2022）[44] 用类似的思路，但强调“细粒度控制”：不仅能根据文本生成“一个人走路”，还能控制“左手抬高”“速度变慢”。

这类方法的共同点是：生成的是参数序列，不是像素。所以生成结果天然带骨架，可以直接驱动 SMPL、可以重定向到不同体型、可以后期编辑。

把扩散模型的两条式子写出来，就抓住了它的全部要点。前向过程是逐次加噪声：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{14.3}$$

$\mathbf{x}_0$ 是一段真实动作（把 T 帧的 $\boldsymbol{\theta}, \mathbf{r}$ 拼成一个大向量），$\bar{\alpha}_t$ 是预设的“保留多少信号”的表，t 越大噪声越多。训练目标是让网络学会猜出被加进去的噪声：

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\mathbf{x}_t,\, t,\, \mathbf{c}\big) \right\|^2, \tag{14.4}$$

$\mathbf{c}$ 是条件（文本编码、音频特征、目标轨迹）。推理则是从纯噪声一步步去噪，每一步都做“减去当前预测的噪声”：

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c})\right) + \sigma_t \mathbf{z}, \tag{14.5}$$

最后一项 $\sigma_t \mathbf{z}$ （$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$）是模棱两可的噪声——它保证了“同一个文本能生成不同动作”（多样性）。$\sigma_t = 0$ 时就是确定性的 DDIM 采样，生成稳定但多样性降低。

> **为什么动作生成特别适合扩散，而不是直接用 GAN？** 动作是长序列（上百帧），每帧又有几十维；GAN 的判别器很难对“整段动作是否合理”给出稳定信号，容易训练崩溃。扩散的目标函数是一道回归题（预测噪声），训练反而更稳；代价是推理需要几十步。



图 14-2 文本/音频驱动动作生成的典型管线。条件编码后，扩散或自回归模型生成姿态序列与根轨迹，再经过接触/物理后处理，最后渲染或驱动虚拟人。

14.5 可控性与物理约束

生成模型最容易被诟病的是“看起来像，但物理不对”：脚在地上滑、人穿进墙、重心飘。

解决办法有三类：

- 接触约束：显式预测脚与地面的接触，接触时脚的速度为 0；
- 物理仿真：把生成的动作送进物理引擎，用 PD 控制器跟踪，得到符合动力学的轨迹；
- 后处理：用逆运动学修正脚的位置，或用优化平滑轨迹。

这些方法能显著提升“可信度”，但也会牺牲多样性：约束越强，生成的动作越保守。

14.6 为什么动作生成难评估

图像生成可以用 FID、CLIP Score；动作生成没有这么幸运。先把几个常用指标到底在算什么说清：

- FID（在动作特征空间）：把生成动作与真实动作各送进一个预训的动作编码器，得到两组特征；假设两组特征都近似高斯，FID 就是这两个高斯分布的距离：

$$\mathrm{FID} = \left\|\boldsymbol{\mu}_g - \boldsymbol{\mu}_r\right\|^2 + \mathrm{Tr}\left(\boldsymbol{\Sigma}_g + \boldsymbol{\Sigma}_r - 2(\boldsymbol{\Sigma}_g\boldsymbol{\Sigma}_r)^{1/2}\right). \tag{14.6}$$

它只比较“分布像不像”，完全不管“某一句文本对不对”。

- R-Precision：从一批真实动作里取出与文本配对的那一个，再让模型给“生成动作与各文本的相似度”排序；配对的文本排进前 k 的比例就是 R-Precision@k。它考察文本—动作对应，但依赖另一个预训编码器。
- 多样性 / 多模态距离：同一文本生成多条，算两两距离（多样性）；再与真实数据集里同一文本的多条比较（多模态距离）。
- 物理指标：脚滑（着地期位移）、穿地深度、关节超限比例、能量消耗。

问题在于：这些指标没有一个能完全对应“人看起来自不自然”。一个 FID 很低的方法，可能生成的动作僵硬、重复；一个多样性很高的方法，可能生成根本做不到的动作。

所以阅读动作生成论文时，一定要看视频，不要只看表格。

14.7 从动作到行为

动作生成的下一步是“行为”：不仅要知道“手抬起来了”，还要知道“他在招手”“他在拒绝”。

这需要把动作与场景、物体、语言、意图结合：

- 手物交互（第 21 章）：抓杯子、开门；
- 社交信号：眼神、手势、身体朝向；
- 长时序任务：做饭、组装家具。

今天的大模型可以理解“描述”，但很难生成“物理上可行、社交上合理”的长动作。这是未来几年的核心问题之一。

本章小结

- 动作是姿态序列，紧凑但有周期性与物理约束，所以生成通常在低维隐空间里做。
- VPoser 用 VAE 学单帧姿态先验：重建项保信息，KL 项把隐空间拉成正态以便采样。
- HuMoR 把先验从“单帧”扩展到“动态系统”，引入速度与接触状态。
- 文本生成动作的主流是扩散：前向加噪、训练预测噪声、反向逐步去噪；噪声项决定多样性。
- 可控性靠接触约束、物理仿真、后处理三类手段，越强越可信、也越保守。
- 评估指标（FID、R-Precision、多样性、物理量）各有缺口，看论文一定要看视频。
- 直接生成关节角会遇到周期性与物理约束问题，所以主流做法是“参数序列 + 隐空间/扩散”；输出参数而不是顶点，才能重定向与后期编辑。

思考题

1. 为什么动作生成通常不直接输出顶点，而输出 SMPL 参数？
2. VPoser 的隐空间为什么能“防止反关节”？它的训练数据起了什么作用？
3. 扩散模型生成动作时，如何保证“脚不滑”？
4. 如果让你评估一段“打招呼”的动作，你会设计哪些指标？

第三篇　人脸

3DMM、FLAME 与高保真数字人

第 15 章　3DMM 深入与 Basel Face Model
第 16 章　FLAME：脸、颈、表情的统一参数化
第 17 章　高保真人脸：细节、纹理与身份
第 18 章　人脸驱动与神经头像

第 15 章 3DMM 深入与 Basel Face Model

> **本章导读**：第 3 章讲了 PCA 怎么算，第 8 章讲了 SMPL 怎么把“形状 + 姿态”塞进一个公式。这一章把同样的思路搬回人脸，但重点不在公式，而在三个问题：为什么 1999 年的形变模型能当地基用二十多年？为什么后来大家要去折腾“双线性”？为什么一个只有几百个参数的线性模型，至今仍是几乎所有单图人脸重建系统的骨架？读完这一章，你应该能说清楚 3DMM 的能力边界在哪里，以及这条边界是被什么划出来的。

15.1 一张照片，两个世界

1999 年以前，计算机处理人脸主要有两条路。

第一条是二维统计模型。1991 年提出的“特征脸”（Eigenface）把一批对齐好的人脸照片拉成向量做 PCA，得到“平均脸 + 若干张明暗图案”，在人脸识别上非常有效。但它有个前提：所有照片得在差不多的姿态和光照下拍。头稍微侧一点、灯换一盏，识别率就往下掉。

第二条是几何建模。图形学那边早就能用多边形网格画人脸，电影里的数字替身就是这么做的。可那是艺术家一个顶点一个顶点雕出来的，雕一张脸要几周，而且雕出来的是“某一张脸”，不是“人脸这一类东西”。

布兰兹（Volker Blanz）和维特（Thomas Vetter）在 SIGGRAPH ’99 上的论文 [2] 做的是把两条路接起来。他们的判断很朴素：

> 姿态和光照引起的变化，在三维里几乎是“免费”的——转一下模型、换一个光源就能算出来；而在二维里，每一种姿态和光照都得单独学一个模型。

既然如此，就把“人与人之间的差异”放到三维里学，把“姿态与光照”交给渲染。分工一定，剩下的事就顺理成章：

1. 用激光扫描仪扫一批人脸，得到三维点集；
2. 把所有扫描对齐到同一个模板上，让第 1000 号顶点在每张脸上都对应“鼻尖偏左 2 毫米”这个位置；
3. 对对齐好的网格做 PCA，得到“平均脸 + 若干位移场”；
4. 想生成新脸，就调几个系数，渲染出来；
5. 想从照片反推，就调系数，直到渲染结果和照片最像。

第 4 步叫合成（synthesis），第 5 步叫分析（analysis），合起来就是分析-合成（analysis-by-synthesis）。这个框架今天听起来理所当然，但在 1999 年，它同时回答了“怎么表示人脸”和“怎么从图像得到人脸”两个问题，所以影响极深。

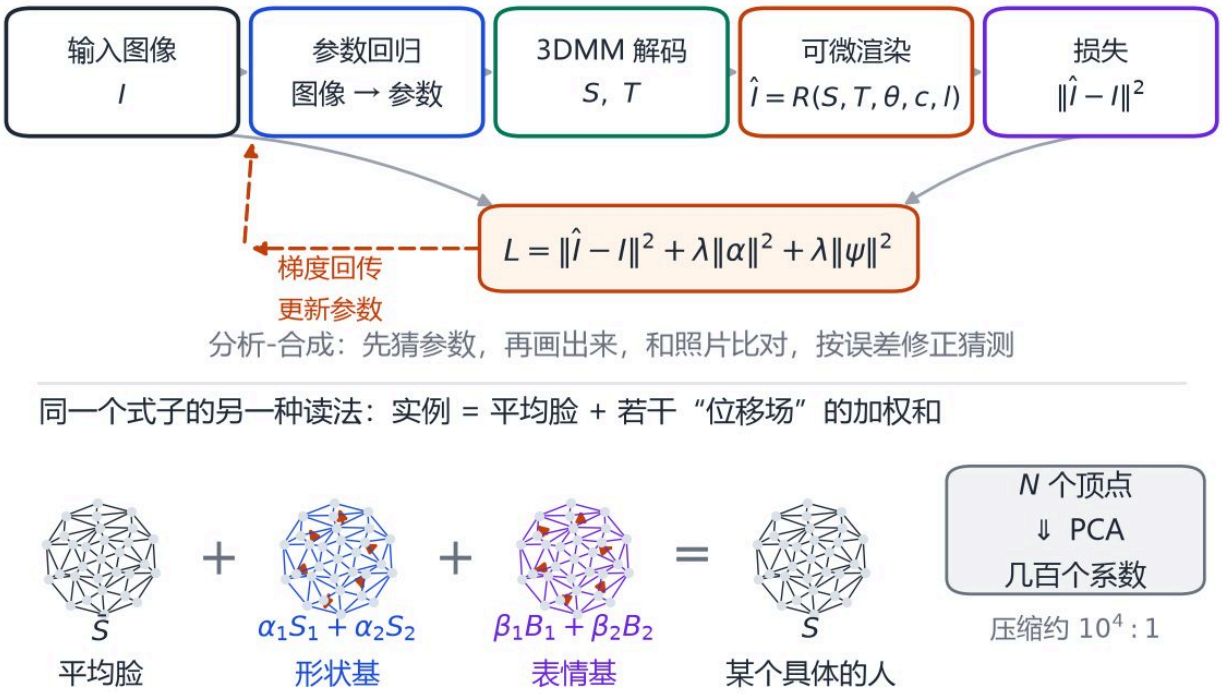


图 15-1 人脸参数化二十五年的主线。注意 2005 与 2009 两个节点：它们不是“更准的模型”，而是补上了 1999 年方案缺失的两块——表情与可复现性。

15.2 公式：三个子空间，一套加法

先把符号定清楚。设每张人脸网格有 N 个顶点，每个顶点三个坐标。把第 i 个人的网格按“先所有顶点的 x，再所有 y，再所有 z”的顺序拉直，得到一个长度为 $3N$ 的列向量 $\mathbf{s}_i$。这一步很关键：只有把网格变成向量，才能对它做第 3 章里的 PCA。

对 m 张脸做 PCA，得到均值 $\bar{\mathbf{s}}$（“平均脸”）和前 k 个主成分 $\mathbf{s}_1, \dots, \mathbf{s}_k$。于是任意一张脸可以写成

$$\mathbf{s} = \bar{\mathbf{s}} + \sum_{i=1}^{k} \alpha_i \, \mathbf{s}_i. \tag{15.1}$$

逐项解释：

- $\bar{\mathbf{s}}$ 是 $3N$ 维向量，就是“平均脸”这个网格拉直的结果。
- $\mathbf{s}_i$ 也是 $3N$ 维向量，但它不是一张脸，而是一张位移图：它告诉你“如果第 i 个系数加 1，每个顶点该往哪儿挪多少”。

- α_i 是一个标量，就是第 1 章里说的“滑块”。

纹理同理。把每张脸的彩色纹理展开到一张公共的二维图上（叫 UV 空间），再拉直成向量，做 PCA：

$$\mathbf{t} = \bar{\mathbf{t}} + \sum_{j=1}^{l} \beta_j \, \mathbf{t}_j. \tag{15.2}$$

注意形状和纹理是两个独立的 PCA，各有一套系数。布兰兹和维特从一开始就这么设计，理由是形状和颜色可以分开调，拟合时先解形状再解颜色，省事。后来也有人证明联合建模能利用“形状与纹理相关”这条先验，但那是后话 [45]。

第三块是表情。1999 年的模型只扫了中性表情的脸，所以它本身不会笑。论文里给了一个非常朴素的补丁：如果同一个人有中性脸 $\mathbf{c}_{\text{ne}}$ 和表情脸 $\mathbf{c}_{\text{exp}}$，那么差值

$$\Delta = \mathbf{c}_{\text{exp}} - \mathbf{c}_{\text{ne}} \tag{15.3}$$

就是“做这个表情需要每个顶点挪多少”。把这个差值加到另一个人的中性脸上，就能让那个人也做出类似的表情。这在当时叫表情迁移（expression transfer），是“加法”思想最直接的体现。

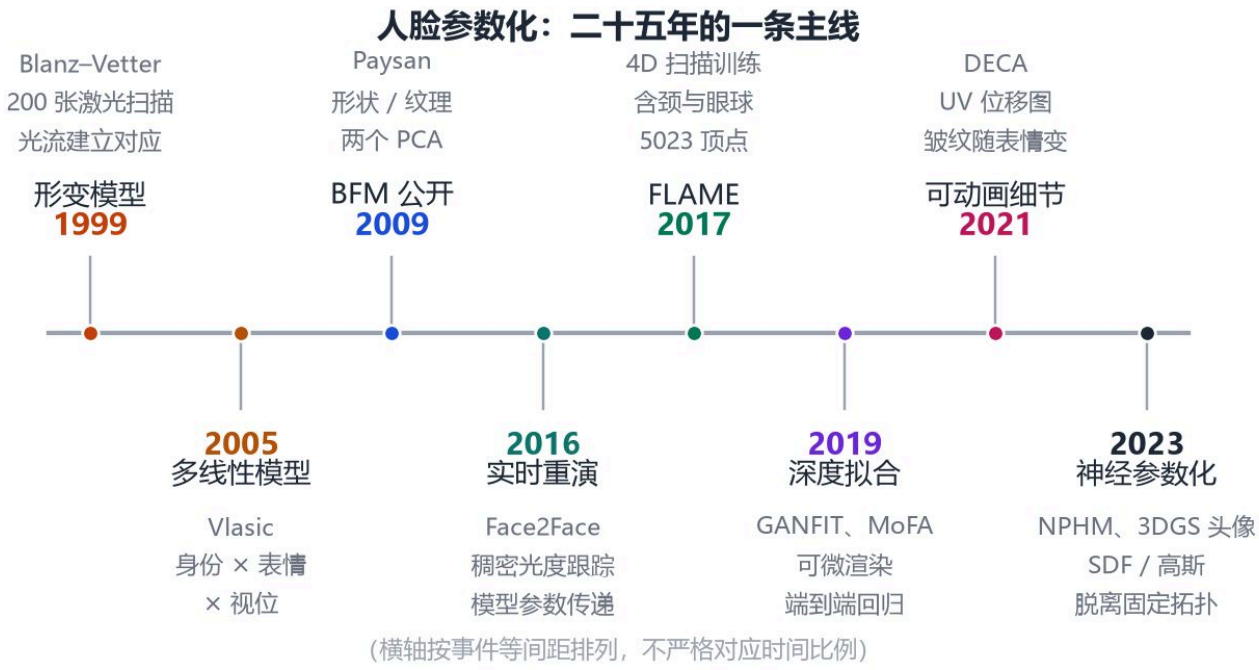


图 15-2 3DMM 的两半。上半：分析-合成的闭环——先猜参数、渲染出来、和照片比对、按误差修正。下半：式 (15.1) 的几何含义——平均脸加上若干张位移场的加权和。

图 15-2 下半部分画的正是式 (15.1)：平均脸，加上几张形状位移场的加权和，再加几张表情位移场的加权和，等于某个具体的人。

这里有个容易被忽略的细节：加法成立的前提是“位移可以线性叠加”。小幅度下这没问题——嘴角往斜上方挪 2 毫米，和同时往右 1 毫米、往上 1 毫米，效果几乎一样。但幅度一大就坏了：真实的脸

在大笑时，脸颊鼓起、法令纹加深、眼角挤在一起，这些是肌肉一起收缩的耦合效应，不是若干独立位移的简单相加。这个隐患我们留到 15.7 节再说。

15.2.1 参数到底有多少个

一个常见的误解是“3DMM 有几十万个参数”。恰恰相反：N 可能是几万，但 α 只有几十到几百个。

以 FLAME 的作者做对比实验时的设置为例：他们用 198 个形状主成分去对齐 BFM（比 BFM 少一个，因为要匹配性别专用模型）[46]。也就是说，一张几万顶点的脸，被压缩成了不到两百个数字，压缩比达到四个数量级。

这件事的意义不只是省空间。更重要的是：参数的维度决定了这个模型能表示什么、不能表示什么。低维意味着强先验——你几乎不可能生成一张“不像人”的脸，因为所有主成分都是从真人身上学来的；低维也意味着表达力有限——任何不在这 k 个方向上的变化，模型永远画不出来。

15.3 最难的一步其实是对应

如果你自己实现一遍 3DMM，会发现 PCA 那五行代码十分钟就写完了，真正花时间的是“把 200 张扫描对齐”。

原因很简单：PCA 的前提是“每一列是同一个量”。如果第一张脸的 1000 号顶点在鼻尖、第二张脸的 1000 号顶点在嘴角，那么把它们相减得到的数字毫无意义，PCA 学到的只会是噪声。

布兰兹和维特的办法是光流（optical flow）。他们的扫描仪输出的不是散乱点云，而是一张柱面展开的深度图——把脸“卷”到一个圆柱面上，于是三维配准问题变成了二维图像配准，可以直接套用当时已经成熟的二维光流算法。

这一步技术含量不低，却常被一句“建立稠密对应”带过。[45] 在回顾这段历史时特别指出：原始 3DMM 之所以能在参数化曲面上用光流、多分辨率、插值这些二维算法，是因为扫描仪本身就给了柱面参数化；不少后来者以为这些操作是“三维的”，其实是误会。

有了对应，PCA 才有意义；有了 PCA，才有“滑块”。这条因果链值得记住：在人脸参数化里，数据与对应的重要性，长期高于模型形式本身。二十多年里，模型从线性到双线性再到神经网络换了好几代，但“怎么扫、怎么对齐”始终是决定上限的那一环。

15.4 Basel Face Model：把论文变成基础设施

1999 年的模型没有公开。真正的转折点是 2009 年，帕伊桑（Pascal Paysan）等人发布了 Basel Face Model（BFM）[47]。

BFM 的数据基础是 200 位受试者的三维扫描，全部为中性表情 [45]。它提供形状与纹理两套 PCA 基，分男、女等性别版本，以及配套的拟合代码。此后十几年，它成了人脸研究里的公共插座：做人脸识别的拿它做数据增广，做重建的拿它当几何先验，做渲染的拿它当基准模型。

BFM 的意义不在技术，而在可比性。在它之前，每篇论文用自己的私有数据、自己的对应方法，别人无法复现，也无法判断“到底谁更好”。在它之后，至少有一件事统一了：大家用的是同一张平均脸、同一套顶点编号。

但公共设施也有代价。

第一是人口学偏差。 受试者以年轻欧洲人为主。用它去拟合东亚人、老人、儿童，会系统性地偏离。这不是算法的错，是数据的错，而且很难靠调参补救——模型里根本没有那个方向。

第二是没有表情。 全是中性脸，意味着 BFM 天生不会笑、不会皱眉。要加表情，只能另找数据、另建模型，这就是下一节的故事。

第三是“中性”本身很模糊。 让人“保持中性表情”其实很难，很多人会不自觉微微抿嘴。这些微小的动作会被当成身份差异学进形状基里，造成身份与表情的轻微纠缠。

BFM 后来有 2017 与 2019 两个版本：前者补了 160 个表情扫描，后者扩展到完整头部与多分辨率[45]。但影响最大的仍是 2009 版——因为它最简单，也最稳定。

15.5 表情：从加法到双线性

加法补丁有个明显的毛病。把 A 的表情差值加到 B 脸上，结果往往像“B 戴了一张 A 的面具”：轮廓是 B 的，表情的走向却是 A 的。原因不复杂——两个人的脸不一样大、肌肉不一样厚，同一句“笑”，需要的位移场本来就不同。

要修这个问题，最直接的办法是：给每个人都拍同一套表情。于是数据从“一堆网格”变成了“一张表格”——行是人，列是表情，每个格子里是一个网格。三个维度（顶点、身份、表情）叠起来，就是一个三阶张量。

2005 年，弗拉西奇（Vlasic）等人的 Face Transfer with Multilinear Models [48] 正是这么做的。他们把人脸看成一个张量，沿“身份”“表情”“视位”三个模式做多线性分解。分解之后，新脸可以写成

$$\mathcal{V} = \mathcal{C}_r \times_{\text{id}} \mathbf{w}_{\text{id}} \times_{\text{exp}} \mathbf{w}_{\text{exp}}, \tag{15.4}$$

其中 $\mathcal{C}_r$ 是核心张量（一个很小的三维数组，存“身份与表情如何组合”的规律），$\mathbf{w}_{\text{id}}$ 与 $\mathbf{w}_{\text{exp}}$ 分别是身份和表情的系数向量，$\times_n$ 表示“沿第 n 个模式相乘”。

不必纠结符号。它的实际含义可以用一句话说清：先按身份挑一组表情基，再按表情在这组基里插值。所以它天然支持两件线性模型做不到的事：只换表情不换人，或者只换人不换表情，而且两个方向可以各自连续调节。

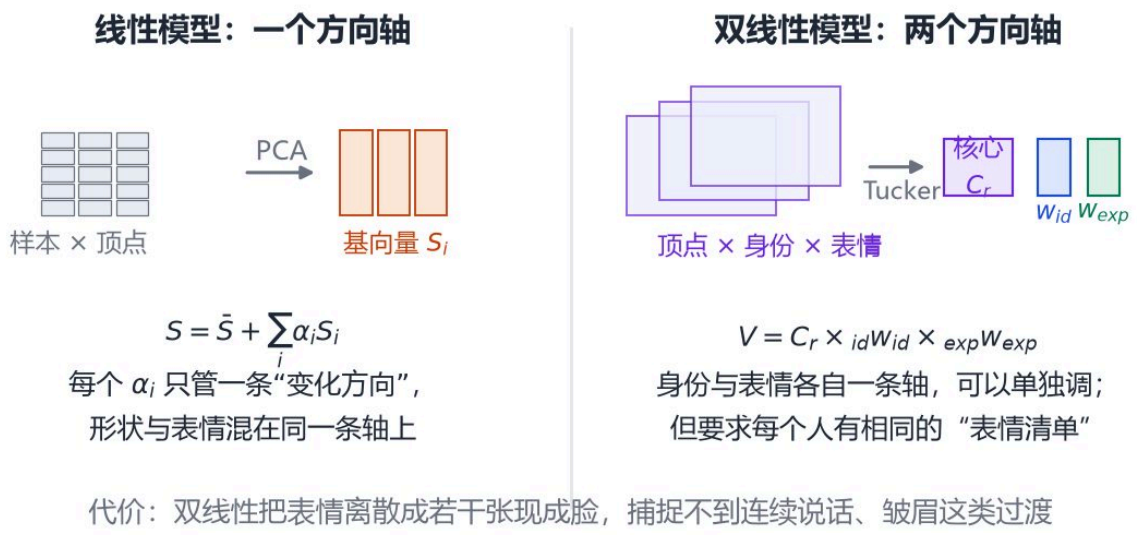


图 15-3 线性模型与双线性模型的区别。左边只有一个变化方向，身份与表情混在同一条轴上；右边拆成两条轴，于是“这个人做那个表情”可以直接由两组系数组合出来。

后来的数据集把这个思路推到更大规模。FaceWarehouse 采集了 150 位受试者、每人 20 个表情[45]；FaceScape 更彻底，938 位受试者、每人 20 个表情、共 18760 个毛孔级网格，先拟合成每人 52 个 blendshape，再在“26317 顶点 × 52 表情 × 938 身份”的张量上做 Tucker 分解[49]。

但双线性也有它自己的墙。

第一，它要求“表情清单”对齐。每个人都得做同样的 20 个表情，而且这 20 个表情在语义上要能对上。闭嘴的笑和露齿的笑算不算同一个？很难说。数据采集时的一点偏差，会被张量分解放大成“身份差异”。

第二，它把表情离散化了。真实的表情是连续流动的，说话时嘴唇每秒变形好几次；而双线性模型的输入是“第 7 号表情占 0.6、第 12 号表情占 0.4”。中间过渡靠线性插值，容易出现“两张脸叠影”的塑料感。

第三，它假设身份与表情可分离。现实中两者是耦合的：胖的人笑起来法令纹更深，老人皱眉时纹路走向也不同。张量分解强行把它们拆开，就等于假设“表情的位移场与脸型无关”，这个假设只在中等幅度下近似成立。

也正因为如此，2017 年的 FLAME 又退回到“线性形状基 + 线性表情基 + 4D 序列训练”的路线上（见第 16 章）：它放弃了“每个人一套对齐好的表情”这个要求，改用连续的表情空间，用 4D 扫描序列去学“表情如何随时间变化”。这是螺旋上升，不是简单回头。

15.6 反解：怎么从一张图得到参数

模型建好之后，剩下的核心问题就是：给定一张照片 I，怎么找到那组参数？

答案是最小化“渲染结果与照片的差”：

$$\min_{\alpha,\beta,\theta,c,l} \left\| I - R\big(S(\alpha), T(\beta), \theta, c, l\big) \right\|^2 + \lambda\|\alpha\|^2 + \lambda\|\beta\|^2. \tag{15.5}$$

这里 $S(\alpha)$ 是由式（15.1）生成的网格，$T(\beta)$ 是纹理，θ 是头姿态，c 是相机，l 是光照，R 是渲染器。后面两项是正则项，防止系数跑到离谱的地方（第 3 章讲过，外插太远就不是人了）。

1999 年的实现有几个细节很能说明当年的工程处境：

- 不逐像素算。 全图逐像素比对太慢，他们每次随机挑一小批三角形，按面积决定被选中的概率——面积大的三角形影响大，就多被采样。这在数学上就是随机梯度下降（SGD）。
- 要手动初始化。 因为式（15.5）高度非凸，随便给个初值会掉进局部极小。原始系统需要人工把脸的大致位置、姿态标出来，再开始优化。
- 相机距离固定。 透视投影里，脸的“大小”同时受真实尺寸和离相机远近影响，两者无法从单张图区分。论文里干脆把距离写死。

这些困难不是当年算力不足造成的，而是问题本身病态（ill-posed）。[45] 把这类歧义总结成三对：形状与尺度、相机与焦距、光照与反照率。最后一对最麻烦：一张脸偏暗，可能是因为皮肤黑，也可能是因为背光；模型无法分辨，只能猜。

深度学习进来之后，形式变了，本质没变。有三条典型做法：

一是把它变成自编码器。 MoFA [50] 用一个卷积网络把图像编码成 3DMM 参数，再用一个可微渲染器解码回图像，整个网络用“重建图像是否像输入”来训练，不需要三维真值。这等于把式（15.5）的优化从“测试时”搬到了“训练时”。

二是用判别器代替像素误差。 GANFIT [51] 不再逐像素比，而是训练一个判别器来判断“这张渲染图看起来真不真”，用它当损失。论文里提到，它的重建结果可以用一个 700 维的向量表示——这 700 个数就是形状、纹理、光照、相机参数的总和。

三是换一个更容易拟合的纹理模型。 Booth 等人的“野外 3DMM” [52] 指出：形状模型可以用受控条件下扫的数据学，但纹理不行——受控数据里的纹理太“干净”，拟合野外照片时会崩。他们的做法是从野外图片里学一个纹理模型，于是拟合时不必再显式估计光照，少了一大块不确定性。

回头看，这三条路都没有推翻分析-合成，只是把“渲染-比较-修正”这个循环挪到了别处：挪进训练集，挪进判别器，挪进数据先验。

15.7 3DMM 的四面墙

讲到这里，可以系统地列出线性 3DMM 的局限了。这四条局限，正好对应第三篇后面三章要解决的问题。

第一，它是线性的。 式（15.1）只能生成“若干位移场的线性组合”。真实人脸的形状变化是弯的：从瘦到胖不是沿一条直线走，从婴儿到老人更不是。更严重的是细节——皱纹、毛孔这些高频起伏，根本不在前几百个主成分里。第 17 章会专门讲怎么把它们补上。

第二，表情要么没有，要么很弱。 BFM 只有中性脸；双线性模型有表情但被离散化；线性表情基对极端表情（大笑、惊恐）会失真，因为它假设“表情 = 若干基的加权和”，而真实的极端表情伴随皮肤拉

伸、体积变化，不是简单叠加。

第三，分辨率不够。 网格顶点数是固定的（BFM 是几万，FLAME 只有 5023），再细的起伏只能靠贴图或位移图“画”上去，而不是真的几何。这在近景渲染时很明显：侧光一打，脸是光滑的。

第四，身份与表情纠缠。 这是最隐蔽的一条。同一个人笑与不笑，PCA 把差异摊到了形状基和表情基里，两边都沾一点。结果就是：你想“保持这个人不变，只换表情”，换完之后脸会微微变形。2021 年的 DECA 用一个“细节一致性损失”专门治这个病（见第 17 章），说明它到深度学习时代仍然是个真问题。

还有一条不算“墙”但很要命的缺失：BFM 没有脖子。它到下颌就截断了，不能转头，也不能和身体模型对接。这件事听上去像工程细节，实际上直接催生了下一章的主角。

本章小结

- 3DMM 的核心分工：差异放三维学，姿态与光照交给渲染；合成靠参数，分析靠优化。
- 主公式 $\mathbf{s} = \bar{\mathbf{s}} + \sum_i \alpha_i \mathbf{s}_i$：平均脸加若干位移场的加权和，几百个系数就能描述几万个顶点。
- 真正难的不是 PCA，是稠密对应；数据与配准的质量决定了模型的上限。
- BFM 2009 的价值在“可比性”：同一张平均脸、同一套顶点编号，让整个领域有了共同基准，代价是人口学偏差与表情缺失。
- 表情的两次尝试：先做“加法迁移”，再做“双线性张量”。后者能分离身份与表情，但要求表情清单对齐，且把连续动作离散化。
- 从图像反解参数是病态问题，三大歧义（形状-尺度、相机-焦距、光照-反照率）至今没有彻底解决；深度学习只是把优化挪进了训练。
- 线性、表情弱、分辨率低、身份-表情纠缠，这四条局限划出了 3DMM 的能力边界。

思考题

1. 如果训练集里所有人的鼻子都比平均脸大，那么第一主成分大概会是什么？这会带来什么后果？
2. 式 (15.3) 的“加法表情迁移”在什么情况下会明显失败？请举一个具体例子，并说明失败时的视觉表现。
3. 双线性模型要求每个人都做同样的 20 个表情。如果某个人只做了 18 个，你会怎么处理这个缺失的格子？
4. 为什么“光照-反照率”歧义在单张图上无法消除？如果给你同一个人的两张不同光照照片，情况会好转吗？

第 16 章 FLAME：脸、颈、表情的统一参数化

本章导读：第 15 章的 3DMM 有一个致命缺口——它到下颌就截断了，既不会转头，也没有脖子。而第 8 章的 SMPL 有骨架、有蒙皮，却不认识表情。2017 年的 FLAME 把这两件事接了起来：它用 5023 个顶点、4 个关节、不到 500 个参数，同时描述“这是谁”“他什么表情”“他头朝哪边”。本章讲清它的公式、它的数据，以及它为什么能成为今天几乎所有三维人脸系统的默认底座。

16.1 BFM 缺的三样东西

如果你拿 2009 年的 BFM 去做一个“数字人直播”，很快会撞上三堵墙。

第一堵墙：没有脖子。 BFM 是一张“面具”，顶点到下颌线就结束了。这意味着两件事做不到：一是转头——转的不是头，是整个面具，看起来像有人在屏幕后面举着一张照片平移；二是接到身体上——脖子是头与躯干的过渡区，没有它，头只能“浮”在肩膀上方。

第二堵墙：表情是外挂的。 第 15 章讲过，BFM 本身只有中性脸，表情要靠外部模型“加”上去。加法的问题在于，它只改几何，不改关节——下颌不会真的“张开”，只是下巴附近的顶点被推下去。张开嘴时，上下牙之间的距离、口腔内部的空间，全都没有。

第三堵墙：不统一。 形状是一套 PCA，表情是另一套（可能是双线性的、可能是 blendshape），姿态则是旋转矩阵，三套东西各说各话。你想“只改表情不动身份”，得自己保证两套参数不串味；换一个模型，参数的含义全变了，代码要重写。

所以 FLAME 的目标非常明确：把 SMPL 的那套“形状 + 关节 + 蒙皮 + 姿态矫正”的配方，原封不动地搬到头上，再加一个表情空间。

这个类比不是修辞。FLAME 的论文里直接说“我们采用与 SMPL 一致的记号”，并且明确它做的事就是把 SMPL 推广到头部 [46]。

16.2 公式：模板 + 三种偏移 + 蒙皮

FLAME 的完整定义是这样的：

$$M(\boldsymbol{\beta},\boldsymbol{\theta},\boldsymbol{\psi}) = W\Big(T_P(\boldsymbol{\beta},\boldsymbol{\theta},\boldsymbol{\psi}),\, J(\boldsymbol{\beta}),\, \boldsymbol{\theta},\, \mathcal{W}\Big), \tag{16.1}$$

其中

$$T_P = \bar{\mathbf{T}} + B_S(\boldsymbol{\beta}) + B_E(\boldsymbol{\psi}) + B_P(\boldsymbol{\theta}). \tag{16.2}$$

别被 W 和 J 吓到，这两个符号第 8 章已经讲过：W 是线性混合蒙皮（把每个顶点按权重分配给若干关节，再让关节带着它走），J 是关节位置（由形状参数回归出来）。真正属于 FLAME 的新东西是式（16.2）里的四个加号项：

- $\bar{\mathbf{T}}$：模板网格，5023 个顶点，处在“零姿态”下。它是所有变化的起点。
- $B_S(\boldsymbol{\beta})$：形状偏移。$\boldsymbol{\beta}$ 是身份参数，B_S 是一个线性基，负责“这是谁”——脸的长短、眉骨高低、下巴宽窄。
- $B_E(\boldsymbol{\psi})$：表情偏移。$\boldsymbol{\psi}$ 是表情参数。这是 FLAME 相对 SMPL 新增的那一项。
- $B_P(\boldsymbol{\theta})$：姿态矫正。$\boldsymbol{\theta}$ 是姿态参数。它修正“纯靠蒙皮转关节”带来的失真，比如脖子扭转时皮肤被拉薄。



图 16-1 FLAME 的计算流程。三个参数各自只作用在自己那一项上，三项相加之后才交给蒙皮。眼球顶点是刚体，脖子顶点的权重在颈与身体之间过渡——后者正是它能被“缝”进全身模型的原因。

这个结构有个很实际的好处：可分解。想换人，只改 $\boldsymbol{\beta}$；想换表情，只改 $\boldsymbol{\psi}$；想转头，只改 $\boldsymbol{\theta}$。三者互不干扰，因为它们在相加之前各占一项。这比“一个大网络吃进所有参数吐出网格”要透明得多，也更容易在游戏引擎里做动画混合。

16.2.1 为什么表情也用线性基

这里有个设计选择值得停一下：FLAME 的表情空间是线性的，也就是“表情 = 若干表情基的加权和”，而不是一堆独立的 blendshape 滑块。

论文里专门解释了为什么不走 FACS 那条路 [46]。FACS（面部动作编码系统）把表情拆成几十个“动作单元”（抬眉、皱鼻、嘴角上扬……），每个单元一个滑块。这套体系在动画里很好用，因为动画师能

精确控制某一束肌肉；但它有个数学上的毛病：过完备。多个不同的滑块组合可以产生同一张脸，于是“给定一张照片，求滑块”这个问题就有无穷多解，很难优化。

FLAME 换成一组正交的表情基（就是 PCA 出来的那组），好处是：给一张脸，系数基本唯一；坏处是：系数没有语义，“第 7 个系数是干什么的”说不清。这是一个典型的取舍——为了“能从数据反解”，牺牲了“人能以语义理解”。后来的 DECA、EMOCA 都建在这套正交基上，说明这个取舍是对的。

16.3 5023、4、15：三个数字怎么来的

刚接触 FLAME 的人常常被三个数字绕晕，我们把它们一次说清。

5023 是顶点数 N。这是一个相当低的分辨率——BFM 是几万，FaceScape 的基网格是 26317。FLAME 选这么低，是刻意的：它要能塞进游戏引擎、要能在手机上行用、要能几毫秒算完一次前向。代价是，中频细节（法令纹、眼袋）它画不出来，只能靠后处理加（见第 17 章）。

4 是关节数 K。分别是：颈、下颌、左眼、右眼 [46]。注意这里没有“头”这个关节——头的整体旋转被当作全局旋转单独处理，不算在 K 里。也没有肩、脊柱，因为 FLAME 只管到头为止。

15 是姿态向量的维度。论文给出的定义是 $\boldsymbol{\theta} \in \mathbb{R}^{3K+3}$：$K$ 个关节各一个三维旋转向量（轴角表示，第 2 章讲过），再加一个全局旋转的三维。代入 $K=4$，$3\times 4+3=15$。



图 16-2 四个关节与它们的父子关系。颈是头与身体的唯一接口：颈一转，它下面所有节点（下颌、双眼）都跟着走。这正是 BFM 完全没有的能力。

关节的父子关系（运动链）是：全局 → 颈 → {下颌，左眼，右眼}。这个结构决定了运动如何传递：你转脖子，下巴和眼球会一起被带过去；但你转眼睛，脖子不动。第 8 章讲过的树状变换在这里原样适用。

为什么要把眼球单独建模？ 论文里给的理由很实在：以前的配准流程完全忽略眼睛，于是眼睑被迫去“解释”眼球的噪声几何，眼周的重建质量很差；把眼球作为两个刚体加进模板之后，眼睑有了正确的参照物，配准精度明显提升 [46]。这也顺带解决了“眼神”这个动画里极重要的信号——眼睛能转，才有“看”的感觉。

眼球顶点的一个细节：它们的蒙皮权重全部给自己那个眼球关节，因此是刚体，不参与表情变形。所以无论模型笑得多厉害，眼球形状都不变。图 16-1 左下角标的正是这一点。

16.3.1 参数量总账

把三类参数加起来：

参数	含义	维数	说明
β	身份	300	90 维已解释 98% 的变化
ψ	表情	100	100 维解释 98% 的变化
θ	姿态	15	4 关节 × 3 + 全局 3
合计		415	描述一个人 + 一个表情 + 一个姿势

300 与 100 这两个数字不是拍脑袋定的。论文里的做法是：画出“紧凑性、泛化性、特异性”三条曲线，看加到多少维之后收益饱和。结论是 90 个身份主成分已经能解释 98% 的数据变化，300 个基本到 100%；表情则取 100 个，同样对应 98% [46]。

注意这里有个容易被忽略的点：“解释 98% 的变化”不等于“看起来像”。方差大的方向通常是“脸的大小、胖瘦”，这些容易解释；而“眼角的一点点下垂”方差极小，可能排在第 200 个主成分之后。所以实际系统里，追求“像不像本人”时往往要用几百维，追求“跑得快”时才砍到几十维。

16.4 数据：3800 张头、400 段 4D、33000 次扫描

FLAME 的训练数据由三块拼起来 [46]：

第一块，身份。 从 CAESAR 人体数据库里取出约 4000 个人的头部扫描。CAESAR 本来是给全身建模用的，但它的扫描包含头部，于是被“顺手”拿来当人脸数据。这块数据的优点是人口学覆盖广（年龄、性别、族裔都有），缺点是分辨率低——全身扫描里的头只有几百个像素大小。

第二块，表情。 D3DFACS 数据集里 400 多段 4D 序列（4D = 随时间变化的三维序列，也就是“会动的扫描”），加上作者自己用多相机主动立体系统补拍的一批。他们的采集系统由三对立体相机、三台彩色相机、三个散斑投影器和三块白光 LED 面板组成，输出平均 4.5 万顶点的网格，帧率 60 帧/秒 [46]。

第三块，规模。 论文说模型总共从超过 33000 次扫描中学出来。这个数字值得记住：它比 BFM 的 200 张高两个数量级，也是 FLAME 比 BFM“更能表示各种脸”的直接原因。

数据多不等于能用。这里有个技术难点：CAESAR 的头像很低分辨率，D3DFACS 的扫描质量高但没有配套的对齐网格。要让它们共同训练一个模型，必须先解决“对应”。

作者用的办法叫联合配准（co-registration）：不是先配准再建模型，而是一边建模型、一边用模型去配准。具体说，先拿一个粗糙的初始模型去套每一帧扫描，得到一个初步的对应；用这些对应更新模型；再用更新后的模型重新套扫描；如此迭代。收缩到后面，模型越来越准，配准也越来越准。

这个方法在人体建模里早有人用，但用在 4D 人脸序列上是第一次。它解决了一个鸡生蛋的问题：没有好模型就配不准，配不准就建不出好模型。

配准还需要“锚点”来防止漂移。他们用 CMU Intraface 自动检测 49 个二维关键点，并在模板上手工标出对应的 49 个顶点，作为配准时的软约束 [46]。49 这个数量不多，但足够把“大致位置”钉住，剩下的交给几何。

16.5 FLAME 与 SMPL-X：缝合点在哪里

FLAME 设计脖子不是为了好看，而是为了能被“缝”到身体上。这个用途在 2019 年的 SMPL-X 里变成了现实 [21]。

SMPL-X 的目标是“一个模型同时描述身体、手、脸”。它的起点是 SMPL+H（带手的身体模型）加上 FLAME（头部），但作者强调：他们不是把两个模型直接拼在一起。

这一点值得展开。早先有一个叫 Frank 的模型，做法是“简化版 SMPL + 艺术家设计的手部骨骼 + FaceWarehouse 人脸模型”直接拼接，结果是各部件各自为政，接缝处不自然，而且参数体系不统一 [21]。SMPL-X 的做法是把拼好的整体再去对齐 5586 个三维扫描，重新学形状基和姿态矫正，让脸、手、身体在数据层面真正“长在一起”。

最终的数字是：10475 个顶点、54 个关节。姿态向量被拆成三块——下颌关节 $\boldsymbol{\theta}$_f、手指关节 $\boldsymbol{\theta}$_h（30 个手指关节，对应 90 维）、其余身体关节 $\boldsymbol{\theta}$_b；表情则直接沿用 FLAME 的表情空间 $\boldsymbol{\psi}$ [21]。

对本章来说，这个例子说明了两件事：

1. 好的接口设计比好的算法更难得。 FLAME 把“脖子顶点权重平滑过渡”这个细节做对了，才使后来的缝合成为可能。如果它当初像 BFM 一样到下颌就截断，SMPL-X 就得另想办法。
2. 参数化的价值在于可组合。 两个模型能缝，是因为它们用的是同一套语言：模板 + 线性基 + 关节 + 蒙皮。换成一个黑箱网络，这种组合几乎不可能。

16.6 从 FLAME 到 DECA、EMOCA

FLAME 发布后，很快成为三维人脸重建的默认底座。原因不难理解：它有明确参数、有拓扑、有开源代码，而且“身份 / 表情 / 姿态”已经分好了。但直接用它有两个明显短板：只有 5023 个顶点（细节不够），以及它只关心几何（不关心“这张脸表达了什么情绪”）。接下来的两项工作分别补了这两块。

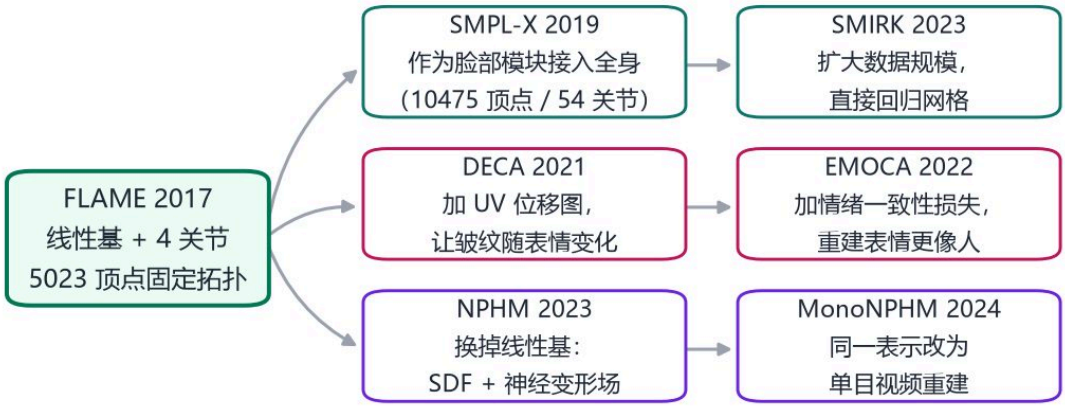


图 16-3 FLAME 之后的两条路线。上、中两条保留了 FLAME 的参数与拓扑，只把细节、数据或训练信号做得更足；下面一条则把表示本身换成了神经场。

DECA：给 FLAME 加一层“可动画的皮”。[53] 的出发点很直接：FLAME 的分辨率决定了它只能表示低频形状，而人脸上最有说服力的信息恰恰在中频——法令纹、抬头纹、眼袋。他们的办法是在 FLAME 网格上叠一张 UV 位移图：把网格展开到二维平面，每个像素存一个沿法线的位移量（论文里的范围是 −0.01 到 0.01 米），渲染时再贴回三维。

关键在于怎么让这层皮“会动”。如果位移图是静态的，那么人一笑，皱纹位置不变，看起来就像戴了一张面具。DECA 的做法是把位移图写成两个东西的函数：一个 128 维的身份细节码（记录这个人的痣、毛孔、永久性纹路）和 FLAME 的表情与下颌参数（决定现在该在哪长皱纹）。为了让这两者真的分开，他们设计了一个细节一致性损失：同一个人两张不同表情的照片，把细节码互换后重建，结果应该仍然像原来那两张。这个损失逼着网络把“静态特征”和“动态皱纹”分开。

粗糙部分用的参数是：前 100 个 FLAME 形状参数、50 个表情参数、50 个反照率参数，加上相机与光照，总共 236 维 [53]。这个数字很像第 15 章那个“700 维”——都是“一张脸可以被几百个数说清”的又一个例证。

EMOCA：让几何“带情绪”。[54] 发现问题出在损失函数上。当时主流方法用三种损失训练：关键点重投影误差、光度误差、人脸识别损失。这三种都只问“位置对不对”“像不像这个人”，从不问“表情对不对”。结果就是：重建出来的脸几何误差很小，但一看就知道“不是那个情绪”。

他们的解法是加一项情绪一致性损失：拿一个在大规模情绪数据集 AffectNet 上训练好的 ResNet-50 情绪网络，把输入照片和重建结果的渲染图分别送进去，要求两者提取出的情绪特征相近。同时，网络还能直接回归 valence（愉快—不愉快）与 arousal（平静—激动）两个连续值，以及基本表情类别。

工程上有个细节很有意思：EMOCA 只训练表情分支，把 DECA 的身份分支冻住 [54]。原因是身份那部分的训练数据与目标不同，动它会把已经很好的身份重建能力弄坏。这种“只改该改的那一块”的做法，在大型系统里非常常见。

除了这两条线，还有一条更简单粗暴的：SMIRK [55] 把重点放在规模上，用更大的数据直接回归 FLAME 网格。它没有发明新的表示，而是验证了一个经验：在参数化模型已经足够好的前提下，数据量往往比结构创新更管用。

16.7 四面墙与两个突围方向

FLAME 很强，但它不是终点。它的局限可以归为四类。

一是拓扑固定。 5023 个顶点是写死的，意味着它永远不能表示头发、牙齿、口腔内部，也不能表示比顶点间距更细的起伏。你想让它“张嘴露齿”，只能另外贴一张牙齿模型上去。

二是线性基 + 蒙皮的双重限制。 线性基假设“表情是若干基的加权和”，蒙皮假设“顶点按权重跟随关节”。前者对极端表情失真（真实的狂笑伴随皮肤拉伸与体积变化），后者对软组织滑动失真（鼓腮、吐舌）。

三是单目拟合的歧义。 给定一张正脸照，“鼻子高一点 + 光从上方来”与“鼻子矮一点 + 光从正面来”可以产生几乎一样的图像。这个问题第 15 章已经讲过，FLAME 并没有解决它，只是换了一个参数空间。

四是它只到脖子。 严格说这是设计选择，不是缺陷，但它意味着“头肩协调”“说话时耸肩”这类动作，FLAME 自己管不了。

针对前三类，2023 年之后的突围方向是把“线性基”整个换掉。NPHM [56] 的做法是：用一个神经隐式场（signed distance field，有向距离场）表示“某个人的静态头部”，再用一个神经变形场表示“表情如何把这个静态头变成当前形状”，另外还加了一组围绕面部锚点分布的局部场来捕捉细节。它的训练数据是自己扫的 5200 多帧高精度头部扫描，来自 255 个不同身份，平均每帧约 350 万个面 [56]。

这套表示的优点是：不再受固定拓扑束缚，理论上可以表达任意细节；缺点是：它不再是一个“网格”，要得到网格得额外做等值面提取，而且计算量大得多。

MonoNPHM [57] 则把同一套表示用到单目视频上：给一段普通 RGB 视频，通过可微体渲染把几何、表情、外观一起解出来。论文自己拍了 20 段 Kinect 序列做评测，并强调这是“让神经参数化模型变得易得”的一步 [57]。

这里有一条容易被误读的线：NPHM 看起来“推翻了 FLAME”，其实并没有。它保留的恰恰是 FLAME 最重要的思想——把“身份”和“表情”拆成两个不交的潜空间。变的只是每个空间用什么表示（线性基 vs 神经场），不变的是“可分解”这个结构。这也是本书反复强调的一点：参数化的本质不是 PCA，而是“用少数几个有明确含义的量去控制一个高维形状”。

本章小结

- FLAME 解决的问题：BFM 没脖子、没关节、表情靠外挂，三套参数各说各话。
- 公式结构：模板 + 形状偏移 + 表情偏移 + 姿态矫正，再交给线性混合蒙皮。四项相加之前互不干扰，这是“可分解”的来源。

- 三个关键数字：5023 个顶点（低分辨率换速度）、4 个关节（颈、下颌、双眼）、15 维姿态（$3K+3$）。
- 表情基选正交 PCA 而不选 FACS，是为了让“从图像反解参数”有唯一解；代价是系数失去语义。
- 数据规模是它与 BFM 的分水岭：约 4000 张头 + 400 多段 4D 序列，总计超过 33000 次扫描；关键是联合配准，一边建模型一边对齐。
- FLAME 的脖子设计让 SMPL-X 的缝合成为可能；SMPL-X 用 5586 个扫描把脸、手、身体重新学到一起。
- DECA 用 UV 位移图 + 128 维细节码补上中频细节，并用细节一致性损失把“静态特征”与“表情皱纹”分开；EMOCA 用情绪一致性损失让几何带上情绪。
- 后续方向（NPHM、MonoNPHM）换的是表示，不换的是“身份与表情分离”这个结构。

思考题

1. 如果把 FLAME 的顶点数从 5023 提高到 20000，会带来什么好处和什么代价？为什么作者没有这么做？
2. 眼球被建成两个刚体关节，蒙皮权重全给自己。如果某个顶点的权重一半给眼球、一半给眼皮，会发生什么？
3. DECA 的位移图范围是 ± 1 厘米。如果一个人的法令纹深到需要 1.5 厘米的位移，会发生什么？你能想到两种补救办法吗？
4. NPHM 用神经场替掉了线性基，但仍然保留“身份潜码 + 表情潜码”。为什么这个分解如此重要？如果合并成一个潜码，会丢失什么能力？

第 17 章 高保真人脸：细节、纹理与身份

本章导读：第 15、16 章的两个模型，顶点数分别只有几万和 5023。它们能表示“这是谁”“什么表情”，却始终画不出法令纹里的一道沟、鼻翼上的一颗痣。这一章讲研究者怎么把丢掉的东西一层层补回来：先用“频率”这把尺子说清到底丢了什么，再看两条数据路线（大规模低精度 vs 小规模高精度）、一块被忽视的领域（纹理与反射率），最后讲评估——为什么论文里的“误差 1 毫米”常常不能直接比较。

17.1 线性模型到底丢了什么

先把问题量化。一张脸上的起伏，可以按“每隔多远重复一次”来分类，这叫空间频率：整体脸型的起伏很慢，跨越十几厘米才一个周期，是低频；法令纹、抬头纹的间距在毫米到厘米之间，是中频；毛孔、皮肤纹理的周期不到一毫米，是高频。

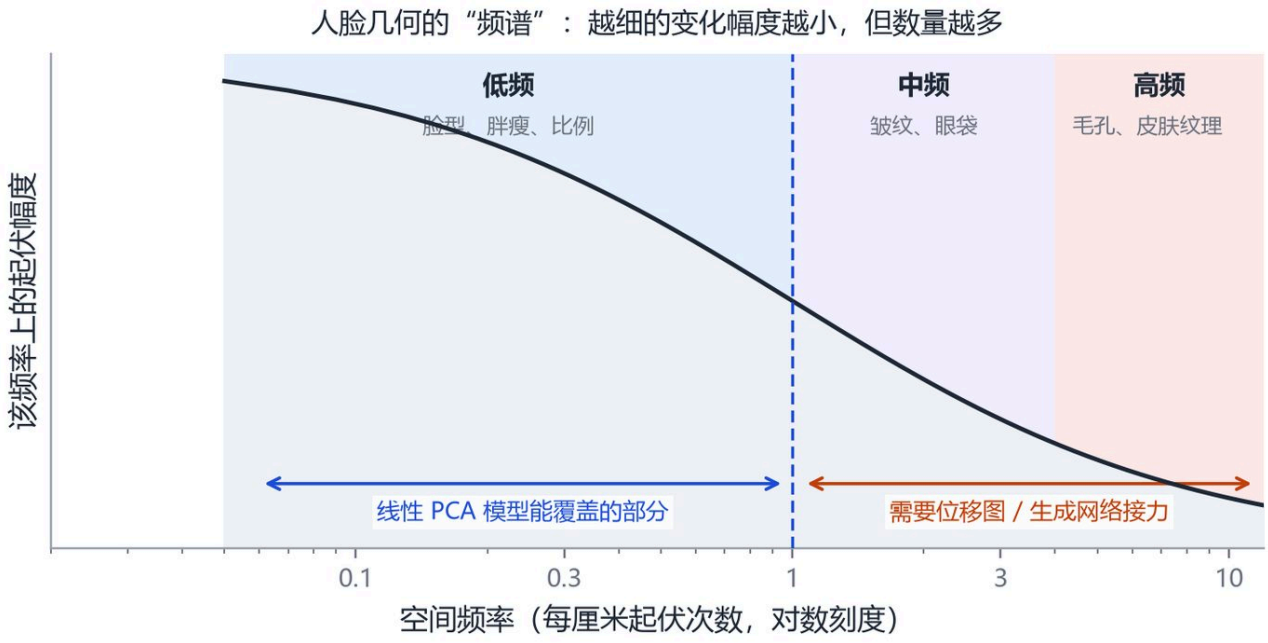


图 17-1 人脸几何的频谱示意。越往右频率越高，单个起伏的幅度越小，但数量急剧增多。线性 PCA 模型主要覆盖左侧一段；中高频必须靠位移图、生成网络或纹理来补。

图 17-1 是一条示意曲线，不是实测数据，但它反映了真实规律：幅度随频率下降，数量随频率上升。低频的起伏少而大（决定脸型），高频的起伏多而小（决定质感）。

现在回想 PCA 在做什么。它按“方差从大到小”排序，前几个主成分对应方差最大的方向——也就是低频的大幅度变化。等你取到第 100 个主成分时，剩下的方差已经很小了，而这些“小方差方向”里，混着大量高频细节。

这就解释了一个反直觉的现象：用 300 个主成分拟合一张脸，几何误差可以低到零点几毫米，但看起来仍然“不像真人”。因为误差的平均值被低频主导，而人眼对中频的皱纹极其敏感。误差小 $\neq$ 观感对，这是高保真人脸的第一个坑。

更麻烦的是，线性基连“皱纹该怎么长”这件事都学不到。皱纹不是随机的凹凸，它跟表情强相关：笑的时候法令纹从鼻翼延伸到嘴角，皱眉的时候是眉间的竖纹。这种“随动作出现、随动作消失”的结构，用一组固定的基向量加权重是表达不出来的——除非你为每个表情单独学一套基，那就回到了第 15 章双线性模型的老路。

所以补细节这件事，本质上要回答两个问题：细节放在哪里存（表示），以及细节怎么随表情变（驱动）。

17.2 两条数据路线：FaceVerse 与 FaceScape

补细节需要数据。而高精度人脸数据有个天然的矛盾：扫得细的扫不快，扫得快的扫不细。

多相机阵列加结构光可以扫出毛孔级几何，但一套设备几十万，一个人要站进去几秒到几十秒不能动，一天扫不了多少人。消费级深度相机（比如结构光模组）便宜、快、五到十秒一个人，但噪声大、分辨率低，扫出来的脸像蒙了一层砂纸。

2019 年之后，两个工作给出了几乎相反但都有效的答案。

FaceScape：把“细”做到极致。[49] 用多相机系统采集了 938 位受试者、每人 20 个指定表情，共 18760 个带纹理的三维人脸，几何细到毛孔级。他们做了一件关键的工程：把所有扫描拓扑统一——用一个公共的基网格去非刚性配准每一张扫描，于是每张脸都有相同的 26317 个顶点和相同的连接关系。统一拓扑之后，粗糙形状可以用双线性模型表示，而基网格表达不了的中高频，则存成 UV 空间里的位移图（每个像素一个沿法线的偏移量）。

这套“基网格 + 位移图”的两层表示非常高效：论文报告，它只占原始网格数据量的约 2%，而平均绝对误差小于 0.3 毫米 [49]。换句话说，用五十分之一的数据，换回几乎一样的几何。

写成公式更清楚。设基网格的第 i 个顶点是 $\bar{\mathbf{v}}_i$，它在 UV 展开图上的坐标是 (u_i, v_i)，法线是 $\mathbf{n}_i$，则最终顶点为

$$\mathbf{v}_i = \bar{\mathbf{v}}_i + \sum_{k} w_k \, \mathbf{b}_{k,i} + \Delta(u_i, v_i)\, \mathbf{n}_i, \tag{17.1}$$

前两项是“低维骨架”：基网格加上双线性模型的加权形状。第三项是位移图在对应像素上的取值，沿法线方向推出去。注意这一步的妙处：位移图是在二维 UV 空间里学的，所以它天然带着“图像”的性质，可以用卷积网络、生成对抗网络去建模；而它作用到三维时，又通过法线 $\mathbf{n}_i$ 自动适配局部朝向。

FaceVerse：把“广”做到极致。[58] 走的是另一条路。他们用消费级深度相机采集了约 6 万帧融合后的 RGB-D 人脸点云，覆盖东亚人群的不同性别与年龄；另外用多相机系统补采了 2000 个高保真头模。前者保证“泛化”——模型见过各种各样的人；后者保证“保真”——模型知道细节长什么样。

结构上，他们做了一个由粗到细的两段式：粗模型是从 6 万帧里学的线性 PCA 基，负责给出一张大致正确的脸；细模型是一个条件 StyleGAN，输入是粗模型展开的 UV 图，输出是加了细节的几何与纹理图。因为是生成式模型，细节是可以“调”的——换一个噪声向量，同一个人的痣和皱纹分布会变，但基本脸型不变。这一点比“一次性预测一张位移图”更进一步。



图 17-2 由粗到细的两段式。左侧用便宜设备换覆盖度，右侧用昂贵设备换保真度，最后在参数层面合并。这也是 FaceVerse 与 FaceScape 共享的骨架。

两条路线的差别，其实是对“数据不够”这一现实的两种应对：FaceScape 选择“少而精”，靠统一的拓扑与位移图把信息压到极致；FaceVerse 选择“多而杂 + 少而精”，用大规模数据撑泛化、用小规模高精数据撑细节。

而它们的共同点是：都没有放弃参数化模型。细节再细，也是挂在一个低维的“骨架”上的。这个骨架提供了对应关系、提供了可驱动性，也提供了“这是同一个人”的一致性。

17.2.1 为什么细节必须“可驱动”

一个容易被忽略的要求：细节不只是要“有”，还要“跟着动”。

如果一个人在静态扫描上有一道很深的法令纹，你把这张静态网格贴到一个会笑的表情上，笑容一大，法令纹的位置就错了——因为真实的法令纹是随着嘴角拉开而加深、延长的。静态位移图会把皱纹“钉”在原地，结果就是脸在动、纹不动，观感立刻崩掉。

FaceScape 的做法是学“表情相关的动态细节”：用神经网络从大量“同一个人 × 多个表情”的配对里，学出“给定表情，细节图应该怎么变”。DECA（第 16 章）的做法更轻量：把细节写成“身份码 + 表情参数”的函数。两者思路一致——细节必须被表情调制。

17.3 纹理与反射率：被忽视的一半

到这里我们一直在说几何。但一张渲染图看起来真不真，一半以上取决于外观：肤色、油光、血色、毛孔的颜色。

传统的 3DMM 只有一张反照率贴图（albedo）——本质上是“把照片上的颜色扒下来贴到模型上”。这样做有两个问题：一是阴影被“烤”进了颜色里，换个光源重渲，阴影还在原地；二是没有镜面反射，皮肤看起来像塑料。

2020 年的 AvatarMe [59] 把外观拆成了四张图：漫反射反照率、漫反射法线、镜面反照率、镜面法线。这四张图合起来近似一个空间变化的 BRDF（双向反射分布函数），也就是“每个点多亮、多光滑、朝哪偏”。有了它，就能在任意环境光下重新渲染，还能做出“鼻尖一点高光、额头一片油光”这种细节。

论文里强调的成果是：从一张低分辨率野外照片，重建出 4K × 6K 分辨率、可直接进渲染器的三维人脸 [59]。这个“4K × 6K”不是网格分辨率，是纹理分辨率——换句话说，几何还是那个几何，但皮肤被画到了能看清毛孔的精度。

后续的 AvatarMe++ [60] 把这件事做得更彻底：不只是反照率和法线，而是完整的 BRDF，并且配套一个考虑了自遮挡与次表面散射近似的可微渲染器。次表面散射很重要——真实皮肤不是不透明的，光会渗进去一点再出来，所以耳朵边缘、鼻翼两侧会有一种“透”的红，这是纯漫反射模型永远做不出来的。

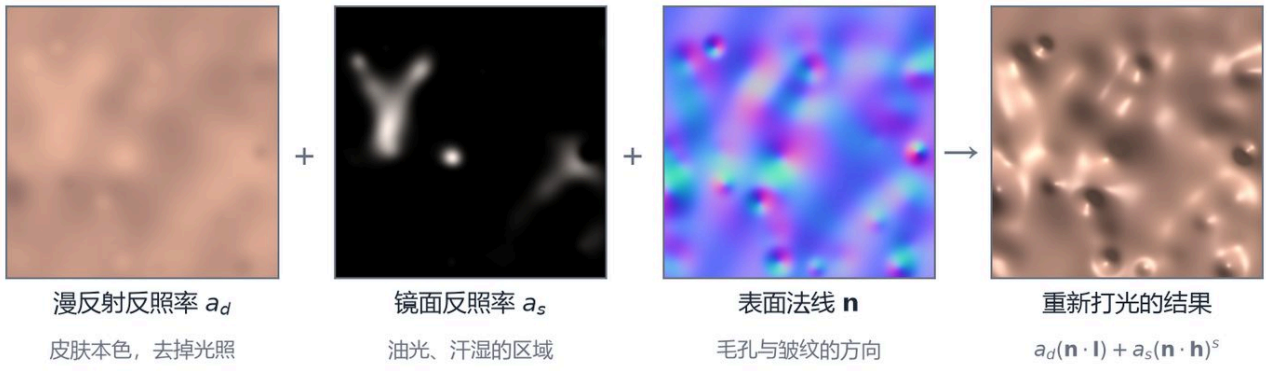


图 17-3 把外观拆成逐像素的分量（示意图）。左起：漫反射反照率、镜面反照率、表面法线；右端是按“漫反射 + 镜面高光”重新打光的结果。3DMM 只有最左那一张，所以无法重打光。

这一节想说的道理是：参数化的对象不只是形状。肤色、油光、法线都可以被参数化——AvatarMe 用 PCA 建了反照率与法线的基，FaceVerse 用 StyleGAN 建了纹理的潜空间。识别一个人，纹理的贡献可能比几何更大；这一点在第 17.4 节会再碰到。

17.4 身份与度量：不只是“看着像”

到这里，我们已经能让一张脸“又细又真”。但还有一个更基本的问题没回答：这张脸的尺寸对吗？

单张照片里，一个鼻子看起来多高，同时取决于三件事：鼻子本身多高、相机离脸多远、镜头焦距多长。这三者从一张图里无法分开——第 15 章提过，在自拍距离（大约半米）下，透视与正交投影的差别大约相当于瞳距的 6% [45]。百分之六听起来不多，但它会直接变成“鼻子高了 2 毫米还是矮了 2 毫米”的错误。

对“看着像”的应用，这无所谓。但对三类应用，它是致命的：

- 医疗与人类学：需要“下颌宽度是 132 毫米”这种绝对数，而不是“比例尺”。
- 定制穿戴：眼镜、口罩、头盔要按真实尺寸做，差 3 毫米就戴不上。
- 三维人脸识别：需要“同一个人在不同照片里重建出的形状彼此接近”，而不同人之间拉开距离。

MICA [61] 是这条线上被引用最多的代表。它把“度量准确”（metrically accurate）直接写进标题，意思是：不只要形状对，还要尺寸对。这个目标的难点在于，它要求模型同时做好两件平常被分开处理的事：一是从像素里抽出足够强的身份信息（不能只靠 68 个关键点，那些点几乎不携带“脸有多宽”的信息），二是把这个身份信息映射到一个有绝对尺度的形状空间里，而不是每次重新猜尺度。

这里有一条经常被忽略的因果链：你用什么样的损失，就会得到什么样的脸。如果只用关键点重投影误差训练，网络会倾向于输出“平均脸”——因为平均脸的关键点位置最安全，误差最小。要让它敢于输出有棱角的、有个人特征的脸，就必须加一个“识别网络说这俩是同一个人”的损失。这也是第 16 章 EMOCA 的思路（用情绪网络的特征当监督）在身份问题上的翻版：找一个已经学会了“什么重要”的网络，向它看齐。

还有一点值得提醒：身份不只存在于几何里。FaceVerse 这类工作之所以要同时建模纹理，就是因为纹理（肤色、痣、眉形、胡须）对“这是谁”的贡献，往往大于几毫米的形状差异。只用几何做识别，等于掐掉了一半信息。

17.5 神经参数化头模型：跳出拓扑

前面所有方法都建立在一个共同前提上：有一张固定拓扑的模板网格。这带来了对应、带来了可驱动性，也带来了天花板：你永远画不出比顶点间距更细的东西，也永远长不出头发和牙齿。

2023 年之后，一批工作开始拆这个前提。其中最有代表性的是 NPHM（第 16 章已介绍其结构）[56]。它的关键设计在于“局部场集合”：不是一个全局的神经场包住整个头，而是在面部的多个锚点周围各放一个小场，每个小场只负责一小块区域。这样做的好处是细节容量随锚点数量增长，而不是被一个全局网络的能力卡住。配合他们自建的 5200 多帧高精度扫描（255 个身份，平均每帧约 350 万个面），模型能重建出接近扫描精度的褶皱与皮肤起伏 [56]。

MonoNPHM [57] 把同一个表示搬到单目视频上：输入一段普通 RGB 视频，通过基于 SDF 的可微体渲染，把几何、表情、外观一起解出来。论文里自拍了 20 段 Kinect 序列做评测，并强调这一步的意义是“让神经参数化模型变得易得”——不再需要专用扫描设备。

但这不等于线性模型要退场。两者分工其实很清楚：

	线性参数化（FLAME 类）	神经参数化（NPHM 类）
输出形式	直接是网格	隐式场，要等值面提取
驱动方式	改参数，实时	改潜码，较慢
细节上限	受拓扑限制	理论上无限
工程成本	能进引擎、能上手机	需要 GPU、需要额外处理

所以现实系统常见两种搭配：用线性模型做“实时驱动层”，用神经模型做“离线高清生成层”，两者通过参数或对应关系互通。

17.6 评估：为什么“误差 1 毫米”不能直接比

高保真人脸这个领域，论文里的数字最容易让人误读。同一个模型，换一种对齐方式、换一个测试集，平均误差可以相差一倍。下面是最常见的五个坑。

坑一：对齐协议不同。 比较两张脸的顶点误差前，必须先对齐。允许平移旋转（刚性对齐）、允许缩放（相似变换）、还是完全不对齐直接算，结果完全不同。前两种都是“形状相似度”，第三种才是“绝对几何”。论文里往往只写“平均顶点误差 X 毫米”，不写对齐方式。

坑二：尺度归一化。 很多工作先把重建结果缩放到与真值相同的尺寸，再算误差。这会把“尺寸猜错”这个错误完全抹掉——而尺寸恰恰是最难的那部分（见 17.4）。如果你的应用是定制头盔，这个数字对你毫无意义。

坑三：测试集同源。 如果训练集和测试集来自同一台扫描仪、同一个配准流程、同一批受试者，那么误差会系统性偏低。这就是为什么 FLAME 的作者要特意在“不在训练集里的 BU-3DFE 中性扫描”上做身份评估 [46]。

坑四：只看关键点。 关键点误差小，只说明“眼睛鼻子嘴在大概位置”，完全不能说明中频细节对不对。一道浅一半的法令纹，关键点误差几乎不变。

坑五：用渲染图验收。 这是最隐蔽的。渲染图的真实感大部分来自纹理与光照，几何错了一点，纹理能盖住。看到一张“很好看”的渲染图，先问一句：把纹理换成灰色陶土，它还像吗？

那么该怎么看数字？比较推荐的做法是看累积误差曲线：横轴是顶点误差阈值，纵轴是“误差小于该阈值的顶点占比”。它比单一平均值信息量大很多。举个例子，FLAME 论文里的数字是：在相同参数量下，FLAME 有 74% 的顶点误差小于 0.5 毫米，而 BFM 是 69%、FaceWarehouse 是 67%；用上全部主成分后，FLAME 是 94%，BFM 是 92% [46]。

这几个数字的绝对值都不大，差距也只有几个百分点。但它说明了两件事：一是同类模型之间的差距本来就不悬殊，二是要判断“谁更好”，必须把数据、对齐、参数量都对齐了才有意义。读到“我们的误差降低了 30%”这种说法，第一反应应该是问：分母是什么？

本章小结

- 线性模型丢的是中高频：幅度小、数量多、方差小，所以 PCA 把它们排在最后，但它们恰恰决定了“像不像真人”。
- 误差小 ≠ 观感对：平均值被低频主导，而人眼对皱纹极其敏感。
- 补细节需要两样东西：一个能存细节的表示（位移图、UV 图、神经场），以及一个能把细节与表情关联起来的机制。
- 两条数据路线：FaceScape 用 938 人 × 20 表情的 18760 个扫描把“细”做到极致（两层表示只占 2% 数据量，误差 < 0.3 毫米）；FaceVerse 用 6 万帧 RGB-D + 2000 个高保真头模把“广”做到极致。
- 外观不只是颜色：AvatarMe 把皮肤拆成漫反射、镜面反照率与法线，才能换光源重渲；AvatarMe++ 进一步做成 BRDF 并考虑次表面散射。
- 度量精度是另一个维度：形状、距离、焦距三重耦合，靠关键点无法解开；MICA 这类工作把“尺寸对”当成首要目标。
- 神经参数化（NPHM、MonoNPHM）换掉了固定拓扑，但没有换掉“身份潜码 + 表情潜码”这个结构。
- 读数字前先问三句：什么数据集？怎么对齐？尺度归一化了吗？

思考题

1. 一张脸的扫描有 20 万个顶点，用 26317 顶点的基网格 + 位移图表示，误差小于 0.3 毫米。这 0.3 毫米大概相当于什么尺度的差异？（提示：一根头发大约 0.07 毫米）
2. 如果只用关键点误差训练模型，为什么会得到“平均脸”？用一句关于梯度的话解释。
3. 把重建结果缩放到与真值同尺寸再算误差，会掩盖什么错误？举一个实际会出问题的场景。
4. NPHM 的输出是隐式场而不是网格。如果要把它接到一个现成的游戏引擎里，你需要多做哪几步？每一步会损失什么？

第 18 章 人脸驱动与神经头像

本章导读：前面三章讲的是“怎么造一张脸”。这一章讲“怎么让它动起来”：让照片里的人跟你做同样的表情，让一张静止的肖像按你写的句子说话，让一段视频里的人改说另一种语言。这里的技术路线分成三条——参数化、神经渲染、以及两者的混血；而所有方法的共同难题都是一句话：**你给的驱动信号越简单，能控制的东西就越少**。读完这一章，你应该能对着一个“AI 数字人”产品，说出它大概用了哪条路线、卡在哪一步。

18.1 三条路线，两种矛盾

给定一个目标人物，要让他“动起来”，问题可以拆成两半：用什么表示这个人，以及用什么信号驱动他。

第一半有三条路线。

参数化路线是最省事的一条。既然第 16 章的 FLAME 已经把脸拆成“身份 + 表情 + 姿态”，那么驱动就等于“给出另一组表情参数”。模型负责几何，贴图负责外观，光栅化负责成像。它的优点是可控性极强——参数可以导出、可以混合、可以重定向到另一个角色；缺点是上限被网格分辨率和贴图分辨率锁死。

神经路线放弃参数，直接对“这个人”做逐场景优化：用多视角视频或单目视频，训一个 NeRF 或一组三维高斯，把外观“背”下来。写实度可以高到分不清真假，但它本身不懂“表情”是什么——想让它动，还是得外挂一个参数模型来提供变形场。

混合路线是近两年的主流：把神经表示绑在参数模型上。三角面怎么动，挂在它上面的高斯就怎么动。这样既有参数级的控制，又有像素级的写实。



图 18-1 三条技术路线。注意第二、三条的差别不在“用不用神经网络”，而在“动不动得起来”——纯神经表示要先解决控制问题，混合表示则把控制权直接交给了参数模型。

第二半是驱动信号。这就是本章真正的主题，也是整个领域最本质的矛盾。



图 18-2 驱动信号的演化谱。越往右，素材越容易获得，但你对结果的控制越弱。两条箭头方向相反，这个矛盾至今没有被真正解决。

图 18-2 这条谱值得多看两眼。左边是动画师手动拖滑块：每一帧的每个参数都由人指定，控制精度最高，但成本也最高，一分钟动画可能要几天。往右走，演员自己演（表演捕捉），信号变“自然”了，

但你要接受演员的表演；再往右，拿一段现成视频当驱动源（视频重演），连演员都不需要了，但视频里有什么你才能用什么；再往右，只给声音；最右边，只给一句话。

每往右一步，素材成本降一个数量级，可控性也降一个数量级。所有“AI 数字人”产品的差异，很大程度上就是这个谱上取的位置不同。

18.2 表情与情绪：几何对了还不够

先看一个具体的技术问题：为什么“重建得很准”的脸，看起来却“没有情绪”？

2022 年的 EMOCA 给出了一个很实在的答案 [54]。他们检查了当时主流方法的训练损失，发现只有三类：关键点重投影误差（位置对不对）、光度误差（像素像不像）、人脸识别损失（是不是同一个人）。没有一项在问“表情对不对”。于是优化器会很自然地找到一个“几何上平均、情绪上模糊”的解——因为微笑的嘴角和面无表情的嘴角，在关键点误差上可能只差几个像素。

他们的补丁是加一条情绪一致性损失：拿一个在大规模情绪数据集上训练好的情绪识别网络，把输入图和重建结果的渲染图都送进去，要求两边提取出的特征接近。这一招的本质是借一个已经学会“什么重要”的网络来做监督——和第 17 章讲的“用识别特征去约束形状”是同一个套路。

情绪这条线上还有两个值得知道的方向。

一个是 SPECTRE [62]。它注意到一个细节：人的表情不只是情绪，还携带语音信息——读唇的时候，我们其实是在读表情。所以它把“视觉语音”的感知纳入监督，让重建出来的嘴形在“能读出什么音”这个意义上也是对的。这对语音驱动特别重要：如果重建的嘴形读不出正确的音素，那么用它当训练数据，学出来的说话脸一定口型不对。

另一个是 EMOTE [63]。它的标题就点明了思路：“内容与情绪解耦”（content-emotion disentanglement）。同样一句话，可以用平静的语气说，也可以用激动的语气说；嘴形（内容）大体相同，但眉毛、眼角、嘴角的张力（情绪）差别很大。EMOTE 把这两者拆成两条支路，于是你可以“用 A 的情绪说 B 的话”。

为什么情绪这么难？三个原因叠在一起：标注主观（同一张脸，不同标注者给出不同情绪标签）、数据稀少（大规模情绪数据集多是二维图片，没有三维真值）、评价困难（几何误差指标对情绪几乎不敏感，只能靠人来做主观评测）。EMOCA 的论文里就做过这样的实验：让被试看重建结果、判断情绪类别，EMOCA 的一致率明显高于同期方法——而这种“人工打分”恰恰说明，这个任务还没有一个好用的自动指标。

18.3 语音驱动：让一张脸说话

语音驱动说话脸（talking face generation）是人脸驱动里最“产品化”的一支。它的管线非常固定：



图 18-3 语音驱动的通用管线，以及各方法主要改进的位置。可以看到，大家的差异几乎都集中在“运动生成”这一段——因为其余几段已经是成熟工程。

难点不在“把音频变成一串数字”，而在一对多：同一段音频，可以有无数种合理的口型与表情。你说“你好”的时候可以点头，也可以歪头；可以笑，也可以面无表情。模型如果只学“平均解”，出来的就是一张嘴在动、脸在飘的“僵尸脸”。

SadTalker [64] 的思路是“别让网络直接从音频画像素，让它先输出三维参数”。他们设计了一个 ExpNet，从音频回归 3DMM 的表情系数，再用一个三维感知的渲染器把参数变成图。这样做的好处是：中间的三维系数是可解释、可复用的——同一组系数可以驱动别的角色，也可以手工微调。论文里特别强调，ExpNet 是通过“蒸馏”学出来的：先有三维重建得到的系数作为老师，再让音频去预测它 [64]。

GeneFace [65] 盯的是另一个问题：泛化。早期基于 NeRF 的说话脸在训练集内的音频上很好看，换一段没见过的音频（比如另一种语言、另一种语速）就崩。原因是训练数据太少，模型把“这段音频对应这个口型”死记硬背下来了。GeneFace 的办法是两步：先在一个大规模唇读语料上训练一个变分运动生成器（唇读数据里音频与嘴形是天然配对的，量大且多样），再针对目标人物做域自适应校正；最后用一个以面部动作为条件的 NeRF 渲染。他们还专门设计了“头部感知的躯干 NeRF”，解决头和脖子接缝处的撕裂感 [65]。

Media2Face [66] 把条件从“音频”扩展到“多模态”。它先训练一个把面部几何与图像映射到统一表情潜空间的变分自编码器（论文里叫 GNPFA），再用它从大量视频里抽出高质量的表情与头姿，构成一个大规模共语数据集；最后在这个潜空间上训扩散模型，条件可以是音频、文本、情绪标签 [66]。于是“笑着说这句话”这种指令才有可能被表达——因为它把“说什么”和“怎么说”分别条件化了。

把这三家放在一起看，会发现一条清晰的技术演进：从“直接生成像素”到“生成参数”，再到“生成潜码”。越往后，中间表示越抽象，能接受的条件越多，但也越依赖大规模数据。

除了算法，语音驱动还有两个工程上的老大难：

口内与牙齿。 参数化模型的嘴里是空的。你一开口，观众看到的是一个黑洞。很多方法要额外检索一张口内图像贴上去，或者干脆用生成模型“编”一个。这也是为什么很多说话脸视频只敢拍正脸、不敢用侧光。

节奏与停顿。 语音里的停顿、呼吸、重音，在梅尔频谱上几乎是空白，但它们对“像不像真人在说话”影响巨大。这一块目前主要靠后处理与手工规则。

18.4 重演与换脸：把 A 的表情装到 B 身上

语音驱动的驱动信号是音频，而重演（reenactment）的驱动信号是另一段视频。给定源人物 A 的表演，让目标人物 B 做出同样的表情——这就是 2016 年 Face2Face 做的事 [67]。

它的技术路线值得细看，因为它几乎把所有能省的地方都省了。

第一步，跟踪。 大多数实时人脸跟踪靠关键点（68 个点），但关键点丢掉了大量信息——嘴唇的深度、鼻翼的张开、眼角的细纹。Face2Face 改用稠密光度一致性：把模型渲染出来，逐像素和目标画面对比，让“渲染得像”本身成为跟踪目标。这样得到的不只是点的位置，而是整张脸的形变。

第二步，恢复身份。 单目视频里的身份是欠定的，他们用“非刚性、基于模型的 bundle 优化”把整段视频的信息集中起来解一个人脸模型，而不是逐帧猜。

第三步，形变传递。 源与目标用同一套参数化模型，于是“把 A 的表情参数给 B”就变成了直接的参数传递。这一步之所以能成立，完全依赖于上一章的 FLAME/3DMM 类模型——两个不同的人，用的是同一套“语言”。

第四步，补口内。 模型嘴里是空的。他们的办法很实在：在目标视频里找一张嘴形最接近的帧，把它扭到当前口型上贴进去。这是一种“检索 + 形变”的土办法，但效果比当时的生成模型可靠得多。

第五步，重打光合成。 把渲染出来的脸按目标视频的光照与肤色融合回原帧，避免“贴了一张面具”的观感。

两年后的 Deep Video Portraits [68] 把能传的东西扩大了：不只是表情，还包括刚性头姿、视线方向、眨眼。技术上也有一个关键转变：它不再直接输出渲染图，而是先用参数化模型渲染出一张“合成图”，再用一个时空架构的生成网络把这张图“翻译”成目标人物的照片级画面。这一步的妙处在于，网络只需学“如何把已经对齐的合成图变真”，而不必自己学三维几何——任务变简单了，真实感却上去了。

2020 年的 StyleRig [69] 则是另一个方向。前两者要的是“一个视频”，StyleRig 要的是“一张由 StyleGAN 生成的图”。问题在于：StyleGAN 能生成极真实的人脸，但它的输入是一个 512 维的“潜码”，没有任何语义——你不知道该改哪一维才能让他笑。StyleRig 训练了一个 RigNet，把 3DMM 的姿态、表情、光照参数映射到 StyleGAN 的输入空间，而且训练是自监督的（不需要人工标注“这张图是什么表情”），StyleGAN 本身完全冻结 [69]。

这三项工作放在一起，恰好是三种“控制 2D 外观”的思路：用几何驱动图像（Face2Face）、用几何引导图像翻译（DVP）、用几何操纵生成模型的潜空间（StyleRig）。它们的共同前提都是：有一个

可解释的三维参数空间当“中间人”。

18.5 3DGS 头像：把高斯钉在脸上

2023 年，三维高斯泼溅（3D Gaussian Splatting）[70] 给整个神经渲染领域换了一次发动机。它的想法很物理：场景不是一张隐式的辐射场，而是几十万个各向异性的三维高斯，每个有自己的位置、朝向、尺度、颜色和不透明度；渲染就是把它们投影到屏幕上、排个序、混合起来。因为是显式表示，它能实时。

对人脸来说，这套表示有一个天然的优势：它能被“钉”在网格上。

GaussianAvatars [71] 就是这么做的：在 FLAME 网格的每个三角形中心初始化一个高斯，并把它绑定到这个三角形的局部坐标系。FLAME 一动，三角形怎么平移旋转缩放，高斯就跟着怎么变。这样，你既得到了参数模型的控制能力，又得到了高斯的写实能力。

但绑定会带来一个新问题：想加高斯怎么办？比如头发、牙齿这些网格上没有的东西。他们提出了“绑定继承”策略：从已有高斯“分裂”出新高斯，新高斯继承父级的绑定关系，于是即使高斯数量变化，整张脸仍然可驱动 [71]。这个细节听着很小，实际上是“可控性”能否保住的关键。

FlashAvatar [72] 换了一个更轻的思路：不把高斯放在三角面上，而是维持一个均匀嵌在表面的高斯场，另外学一组空间偏移，用来覆盖非表面区域（头发、眼镜、牙齿）。因为初始化的高斯数量可以很少（利用几何先验），渲染极快：在 512×512 分辨率、单张 RTX 3090 上达到 300 帧每秒；而重建只需要一段很短的手机视频，几分钟就能完成 [72]。

这里有个值得品味的对比。两年前做同样的事，需要多相机阵列、几小时优化、一张专业显卡实时渲染；现在只需要一部手机、几分钟、一张消费级显卡。 技术民主化的速度，往往比指标提升的速度更快。

局限也明显：头发依旧难（尤其是细发丝的动态），口腔内部依旧要额外处理，动态阴影（比如手在脸上投的影）基本没有建模。

18.6 伦理与安全：可编辑性的另一面

本章讲的每一项技术，都有正当用途：影视后期、游戏、远程会议、语言无障碍、心理治疗中的虚拟形象、已故亲人的纪念影像。也都有另一面。

需要正视的是，滥用的成本已经降到很低。五年前做一段换脸视频需要专业团队和几天时间；现在只需要一张照片、一段音频、一台带显卡的电脑。而且这类技术的产物不再是“一眼假”的抠图感——它继承了目标人物的光照、肤色、发丝、背景，统计痕迹很少。

从技术角度说，有个不太好听的事实：参数化本身就是可编辑性，而可编辑性就是可滥用性。一个能把“表情”和“身份”完全分开的模型，天生就能把任何人的表情装到任何人脸上。这不是模型设计者能通过一个开关关掉的能力。

所以防线只能在技术之外：

- 知情同意。 用谁的脸，必须得到谁的授权；演员去世后的形象使用，应该有遗嘱或家属授权。
- 内容标识。 国内外的深度合成管理规定都要求对生成内容做显著标识，并在元数据里保留来源。技术上这可以是水印，也可以是隐形指纹。
- 检测与溯源。 被动检测（看像素痕迹）永远在追赶生成模型；更可靠的是主动方案（生成时就嵌入可验证的签名）。
- 平台与法律。 发布环节的审核、举报机制、明确的法律责任。

对读者——未来的工程师——有三条建议。第一，在项目立项时就问清楚“如果这段能力被用在别人身上，会发生什么”。第二，把“可识别”当成需求的一部分，而不是上线前才补的补丁。第三，不要把“检测很难”当成“不必检测”的理由。

本章小结

- 人脸驱动可以拆成两个问题：用什么表示人（参数化 / 神经 / 混合），用什么信号驱动（手动 / 表演 / 视频 / 语音 / 文本）。
- 驱动信号的谱系上，可控性与易得性方向相反；所有产品的差异，很大程度上是这条谱上的位置差异。
- 情绪是几何之外的一维：关键点与光度损失看不见“情绪对不对”，EMOCA 用情绪网络的特征做一致性监督；SPECTRE 把视觉语音纳入监督；EMOTE 把内容与情绪拆开。
- 语音驱动的通用管线是“音频 → 特征 → 运动 → 3D 表示 → 渲染”，差异集中在运动生成；一对多、口内、域外泛化是三个长期难点。
- 重演的三级跳：Face2Face 用几何直接驱动图像，Deep Video Portraits 用几何引导图像翻译，StyleRig 用几何操纵生成模型的潜空间。
- 3DGS 头像把显式高斯的写实能力与参数模型的控制能力结合起来；GaussianAvatars 靠“绑定到三角面 + 绑定继承”，FlashAvatar 靠“均匀嵌入 + 空间偏移”。
- 可编辑性就是可滥用性。防线在同意、标识、溯源与法律，而不在某一个开关。

思考题

1. Face2Face 用“稠密光度一致性”而不是关键点来跟踪，好处是什么？在什么情况下它反而会失败？
2. 语音驱动里“同一段音频可以有多种合理表情”。如果让你设计一个模型来表达这种多样性，你会怎么改扩散模型的条件？
3. GaussianAvatars 把每个高斯绑定到一个三角面。如果高斯数量增加十倍，渲染速度会怎么变？可控性会变吗？
4. 假设你要为一个公益项目做“已故亲人语音复原”的交互系统，你会设置哪些同意与标识机制？请写出三条你认为是底线的规则。

第四篇　人手

21 个关节、MANO 与手物交互

第 19 章　手的解剖与 MANO
第 20 章　单手重建：从优化到 Transformer
第 21 章　手物交互与抓取

第 19 章 手的解剖与 MANO

本章导读：前面六章我们反复用同一个套路：先找到一个“平均形状”，再学会“形状怎么变”和“姿态怎么动”，最后用蒙皮把它们缝在一起。这套套路在人体上叫 SMPL，在脸上叫 FLAME，在手上有一个人人都绕不开的名字：MANO。本章先回答一个问题——为什么手比整个人体还难？然后从腕骨、掌骨、指骨讲到 21 个关键点，再拆开 MANO 的五个部件，看它怎样用 778 个顶点和 16 个关节描述一只手，最后说明它如何并进 SMPL+H 与 SMPL-X。需要的前置知识只有第 3 章的 PCA、第 5 章的蒙皮，以及第 8 章的 SMPL 公式。

19.1 手为什么这么难

如果把人体参数化当成一场考试，人体是必答题，人脸是加分题，手就是那道“看起来简单、实际最容易丢分”的题。

先看数字。一只手的重量不到体重的百分之一，长度不到身高的十分之一，却能做出抓、捏、拧、握、弹、指、写这些动作。它的自由度集中在 5 根手指上：常用的记法是，除拇指外每根手指 4 个自由度（掌指关节 2 个——弯曲与张开，近端指间关节 1 个，远端指间关节 1 个），拇指 5 个，合计 $4\times 4+5=21$。作为对比，整个身体的“大关节”通常只用二十几个自由度就够了。也就是说，一只手和半个人体的自由度差不多，却只占身体面积的百分之几。

这带来四个具体困难。



图 19-1 右手骨骼与 21 个关键点的编号约定。橙点是被模型显式建模的 16 个关节，蓝点是 5 个指尖点，两者合起来是常说的“21 个关键点”。编号用的是 MANO 的顺序（腕、食指、中指、小指、无名指、拇指），实际工程中各库的编号可能不同，加载数据时务必核对。

第一是尺度。 成年人的手长约 18 厘米，手掌宽约 8 厘米。在全身扫描或全身图像里，手可能只占几十个像素。分辨率不够时，两根相邻的手指在图像上会糊成一片，扫描仪也会在指缝之间生成一层“蹼”状的假面。MANO 的作者在论文里就明确指出：全身扫描仪的分辨率不足，手指分辨不出来，指间会出现这种 webbing 伪影 [16]。

第二是自遮挡与深度歧义。 手指太细，一根手指绕到另一根后面时，投影上几乎看不出差别。更麻烦的是“深度翻转”：一只手指向镜头的手，与一只背向镜头但手指弯曲方向相反的手，在某些视角下的二维投影可以非常接近。三维姿态到二维图像的映射天生是多对一的，而手的小尺寸让这个歧义更加严重。

第三是外观几乎无特征。 人脸有眼睛、鼻子、嘴这些高对比度的地标，人手没有。五根手指的颜色、纹理、粗细都差不多，网络很难靠局部外观区分“这是食指的第二关节”还是“中指的第二关节”。这一点在 2019 年被明确写进了单图手部重建的论文里 [73]。

第四是数据难采。 人体的三维真值可以用多相机扫描或动作捕捉拿到，手不行：标记点太大，贴上去会限制手指活动，还会改变手形；数据手套同样会改变形状，采到的姿态分布也不再自然。所以手部数据长期依赖“合成渲染”或“精心设计的多视角标注流程”——这两条路线会在第 20、21 章反复出现。

把这四点放在一起，就能理解为什么 SMPL 明明在 2015 年就做好了，手却要等到 2017 年才有对应的模型：不是没人想做，而是没有足够好的数据去学一个统计模型。

19.2 手部解剖：从腕到指尖

要理解 MANO 的 16 个关节从哪来，得先知道手上有多少块骨头。

一只手有 27 块骨头，分三组：

- 腕骨 8 块，排成两排，像一组小石子，负责手腕的灵活转动；
- 掌骨 5 块，从腕部向指尖方向伸出，构成手掌的骨架；
- 指骨 14 块，其中拇指 2 块（近节、远节），其余四指各 3 块（近节、中节、远节）。

27 块骨头通过关节串成一条条链。真正被参数化模型显式建模的，是这些链上的转动中心：

- 1 个腕关节（手掌相对前臂的转动）；
- 5 个掌指关节（Metacarpophalangeal，MCP），即手指根部与掌骨相连处；
- 拇指 1 个指间关节（Interphalangeal，IP），其余四指各 2 个（近端指间关节 PIP、远端指间关节 DIP）。

数一数：$1+5+1+4\times 2=15$。再加上一个控制整只手朝向的根关节，正好 16 个。这就是 MANO 的骨架，也是“16 个关节”这个数字的来源 [16]。

21 个关键点又是怎么来的？ 关键点（keypoint）是给算法做监督用的语义点，比关节更“贴近表面”。最常见的约定是：1 个手腕点 + 5 根手指各 4 个点（根部、第一关节、第二关节、指尖），共 21 个。它比 16 个关节多了 5 个指尖点——指尖是抓取和触摸时最重要的接触位置，但在 MANO 的骨架里并没有独立的转动中心。所以工程上常见的做法是：用 16 个关节做骨架，另外从网格上取 5 个指尖顶点补足 21 个关键点。Boukhayma 等人在 2019 年就是这么做的 [73]，后来的 HaMeR 干脆把输出写成 $K=21$ 个关节 [74]。

这里有一个容易混淆的地方，值得提前说清楚：“关节数”和“自由度数”不是一回事。 MANO 把每个关节都当成球关节（ball joint），一个球关节用 3 个数（轴角）描述，于是 16 个关节对应 $16\times 3=48$ 个姿态参数。但真实的手指关节大多是铰链，只能弯不能扭，一个手指关节实际只有 1 到 2 个自由度。所以 MANO 的姿态表示是过参数化的：48 个数里有很多组合在解剖上根本不可能出现。MANO 的论文自己承认了这一点，并用两个办法补救——姿态主成分分析（把 48 维压到 6 维）和更强的正则化。这两件事我们在 19.3 节展开。

19.3 MANO 的五个部件

MANO 的全名是“带关节与非刚性形变的手部模型”（hand Model with Articulated and Non-rigid defOrmations）。它的结构几乎是 SMPL 的移植：模板、形状基、姿态基、蒙皮权重、关节回归器，一个不少。

$$M(\boldsymbol{\beta},\boldsymbol{\theta})=\mathcal{W}\big(T_P(\boldsymbol{\beta},\boldsymbol{\theta}),\,J(\boldsymbol{\beta}),\,\boldsymbol{\theta},\,\mathbf{W}\big), \tag{19.1}$$

$$T_P(\boldsymbol{\beta},\boldsymbol{\theta})=\bar{\mathbf{T}}+\mathbf{B}_S(\boldsymbol{\beta})+\mathbf{B}_P(\boldsymbol{\theta}), \tag{19.2}$$

其中 $\bar{\mathbf{T}} \in \mathbb{R}^{778\times 3}$ 是模板（零姿态下的平均手），$\boldsymbol{\beta} \in \mathbb{R}^{10}$ 是形状参数，$\boldsymbol{\theta} \in \mathbb{R}^{48}$ 是姿态参数，$\mathbf{W}$ 是 778×16 的蒙皮权重，$J(\boldsymbol{\beta})$ 是由顶点线性回归出来的 16 个关节位置。$\mathbf{B}_S$ 和 $\mathbf{B}_P$ 分别是形状与姿态的偏移量，和第 8 章的 SMPL 完全同构。

所以真正值得讲的不是公式，而是这五个部件各自是怎么学出来的、和 SMPL 有什么不同。

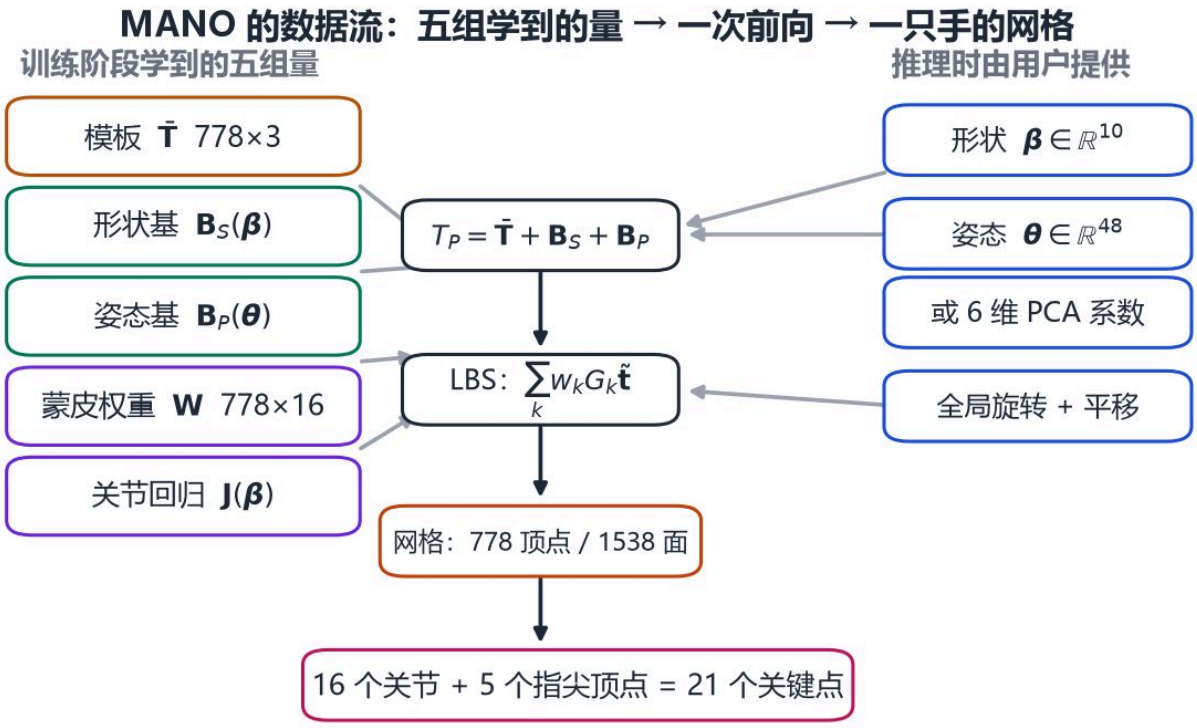


图 19-2 MANO 的输入输出与内部数据流。左边是训练阶段学到的五组量，中间是式 (19.1)(19.2) 的计算顺序，右边是推理时真正需要用户提供的三类参数：形状 10 维、姿态 48 维（或 6 维 PCA 系数）、全局朝向与平移。

（一）模板与形状基：2018 次高精度扫描。 MANO 的数据来自一台专门扫手的结构光系统（3dMDhand），它有 5 个扫描单元，被试把手腕搁在支架上，保持腕部固定、手指自由活动。作者一共采集了 31 名被试、最多 51 种姿势，累计 2018 次扫描，覆盖了抓握分类学里的 31 种典型抓法以及一些混合姿势 [16]。为了把左右手的数据合起来用，他们把左手扫描镜像成右手，训出一个统一的右手模型，再镜像回去得到左手模型——这个技巧把训练数据翻了一倍，也顺便保证了左右手在拓扑上严格对称。

形状基的做法和 SMPL 一样：把所有配准好的扫描统一到零姿态，做 PCA，取前若干个主成分。论文正文没有强调保留了多少维，工程中常用的配置是 10 维形状参数，与 HaMeR、DexYCB 等后续工作使用的 $\boldsymbol{\beta} \in \mathbb{R}^{10}$ 一致 [74][75]。

（二）姿态偏移：为什么要“局部化”。 手的问题在于关节太多、彼此太近。SMPL 训练姿态偏移 $\mathbf{B}_P$ 时对所有元素用同一个正则权重，这在人体上没问题——躯干关节隔得远，一个关节的偏移不会莫名其妙跑到脚上。但手不行：16 个关节挤在 18 厘米的范围里，用同样的权重训出来的偏移会“糊”成一片，弯曲食指时中指根部也跟着鼓起来。

MANO 的解法很直接：给每一对“输入关节—输出顶点”配一个权重，权重正比于两者在模板网格上的测地距离（沿表面走的距离，而不是直线距离）：

$$\Lambda_P(i,j) = \mathbf{J}_j \, D_{\text{geo}}(\mathbf{t}_i, \bar{\mathbf{T}}), \tag{19.3}$$

D_{geo} 是顶点 $\mathbf{t}_i$ 到其他所有顶点的测地距离向量，$\mathbf{J}_j$ 是第 j 个关节的回归权重。距离越远，惩罚越重，于是学出来的姿态偏移就自动变成“局部的”。这个改动听起来很小，效果却是肉眼可见的：论文里专门放了一组对比图，说明不做局部化时指关节会出现不自然的凸起 [16]。

（三）姿态空间：6 维解释 81% 的方差。 这是 MANO 最有意思的一部分，也是它继承自神经科学的地方。手的 21 个自由度并不是独立控制的——你很难只弯无名指的第二个关节而不动其他手指。Santello 等人在 1998 年的实验里就发现，抓握姿势的方差主要落在少数几个主成分上。MANO 把这个观察变成了模型的一部分：对训练集里所有姿态（轴角表示）做 PCA，得到一组“姿态协同”（pose synergy）基，用低维系数代替 48 维轴角。

$$\boldsymbol{\theta} = \bar{\boldsymbol{\theta}} + \sum_{n=1}^{m} \alpha_n \, \mathbf{M}_n. \tag{19.4}$$

论文报告：每只手取 6 个主成分，可以解释训练姿势约 81% 的方差，这已经足够描述大多数常见手势 [16]。这个数字很值得记住，因为它解释了一件事：为什么后来很多手部重建网络只回归很少的几个姿态数也能work——不是网络厉害，而是手的姿态本来就住在一个低维薄片上。

顺便说一句，这里的取舍是有权衡的：主成分越多越精确，但对噪声数据越敏感。论文明确指出，高质量数据可以用很多主成分，噪声数据则要少用，以换取鲁棒性。第 20 章会看到，Boukhayma 的方法取了 10 个姿态主成分 [73]，比 MANO 论文推荐的 6 个多，正是因为它的输入是噪声更大的单张图像。

（四）零姿态带来的麻烦。 SMPL 的零姿态是 T-pose，手是张开、手指平伸的；MANO 也沿用了“平手”作为零姿态，为的是和 SMPL 兼容。但数据集里的平均姿态并不是平手，而是略微弯曲的自然状态。这就造成一个技术问题：模型先优化模板、后优化姿态偏移，模板会把“从平手到平均姿态”的形变吸收进去，结果是指关节被拉出不自然的凸起。作者的补救办法是给训练样本加权：姿态离零姿态越远，权重越低，在优化模板的第一轮里基本不参与。

（五）蒙皮。 权重矩阵 $\mathbf{W}$ 是 778×16，每个顶点最多受少数几个关节影响。手部蒙皮的难点在于指缝：两根手指之间的皮肤同时受两个关节影响，权重过渡不好就会在弯曲时“粘”在一起。这也是 19.1 节说的“数据必须够好”的原因——权重是从配准好的扫描里学出来的，配准错了，权重就错了。

19.4 MANO 与 SMPL：同一套机器，不同的调参

把 MANO 和 SMPL 并排放，会发现它们的公式几乎逐字相同。这不是巧合，而是设计目标：MANO 必须能无缝插进 SMPL。

表 19-1 三个模型的规格对比。数字本身不重要，重要的是它们之间的层级关系。

项目	SMPL [5]	MANO [16]	SMPL-X [21]
顶点数 N	6890	778	10475
关节数 K	24（含根）	16（含根）	54（含手指 30 个）
形状参数	10	10	10
姿态参数	$24 \times 3 = 72$	$16 \times 3 = 48$，可压到 6	全身 75 + 双手 PCA 24
手部表达	无（手是固定的板）	16 关节 LBS + 姿态偏移	复用 MANO 的姿态空间

表 19-1 最重要的信息不是数字，而是层级关系：SMPL 没有手指，MANO 只有手，SMPL-X 两者都有。这就是 19.5 节要讲的整合故事。

在讲整合之前，先把两者的差异列清楚，因为它们决定了工程中的很多“坑”。

差异一：正则化的粒度。 SMPL 用全局常数正则，MANO 用测地距离加权的局部正则（式 19.3）。原因是关节密度完全不同。

差异二：姿态空间的维度。 SMPL 直接优化 72 维姿态，没有人会把它压到 6 维；MANO 必须压，因为 48 维在全身体扫描的低分辨率手部区域里根本优化不动。

差异三：形状空间的来源。 SMPL 的形状空间来自 CAESAR 人体测量数据库（数千人的全身扫描）。MANO 起初把 CAESAR 当作形状先验的来源，后来发现它在手上的配准存在系统性偏差（姿势、遮挡、扫描技术共同造成），最终改用自己采集的 31 人中性姿态数据来建形状空间 [16]。这是一条很实用的经验：别人的数据不一定能直接拿来用，哪怕它是“标准数据集”。

差异四：坐标与朝向。 MANO 的模板把腕部放在原点附近，手指沿一个固定方向伸展；SMPL 的坐标系则是以骨盆为参考、Y 轴向上。两者对接时必须做一次刚体变换，否则手会“长”在错误的位置、朝向错误的方向。这一点在第 26 章跑代码时会再提醒一次。

19.5 从 MANO 到 SMPL-X 的手

有了 MANO，接下来要解决的问题是：怎样让一个人既有一副好身体，又有一双会动的手？

2017 年的答案叫 SMPL+H [16]。做法非常“工程”：把 MANO 里和手指有关的部分——蒙皮权重、姿态偏移、关节回归器——原样搬进 SMPL 的手部区域，手腕及其以外的身体部分继续用 SMPL 的。手部的形状则不再用 MANO 的形状基，而是用 SMPL 的全身形状空间去决定——理由是手的胖瘦和身体的胖瘦是相关的，用同一个形状参数控制更自然。

代价立刻出现了：SMPL 原本有 66 个自由度（不含手），加上两只手各 6 维姿态 PCA，一共 78 维。论文作者自己评价说，这把姿态维度“翻了一倍还多”，而多出来的维度全都集中在身体上最小、数据最差的一块区域，反而让拟合更不稳定。所以他们坚持用 6 维 PCA，而不是 48 维原始轴角。

两年后的 SMPL-X 把这个思路推得更彻底 [21]。它用一套统一的模型同时表达身体、手、脸：

- 顶点数 $N = 10475$，关节数 $K = 54$；

- 手指有 30 个关节，对应 90 个原始姿态参数，但通过 MANO 的姿态 PCA 压缩成每只手 12 维，两只手共 24 维；
- 参数总数 119：75（全局朝向 + 身体、眼睛、下颌）+ 24（双手 PCA）+ 10（形状）+ 10（表情）。

请注意这里的“12 维”比 MANO 论文推荐的 6 维多。原因还是那个权衡：SMPL-X 的拟合场景（单图、单目视频）比 MANO 的训练场景噪声更大，但需要表达的动作也更复杂，12 维是折中。

从工程角度看，SMPL-X 的手部设计给出了一个很重要的启示：在一个全身模型里，一只手只值 12 个数。 这 12 个数就是“接口”：上游的检测器、下游的动画系统、渲染器、物理引擎，全都可以只认这 12 个数，而不必关心 778 个顶点怎么排。这也是为什么第 20 章的手部重建网络大多输出 MANO 参数而不是顶点——参数是通用的，顶点不是。

代价同样明显：12 维不足以表达精细的手部动作（比如弹钢琴时每根手指的独立屈伸）。实践中常见的做法是“两级估计”：先用全身模型拿到大致的手部位置和朝向，把手部区域裁剪放大，再用一个专门的手部网络重新估计 48 维（或更多）的姿态。FrankMocap 就属于这一类系统，我们在第 20 章详细讲[76]。

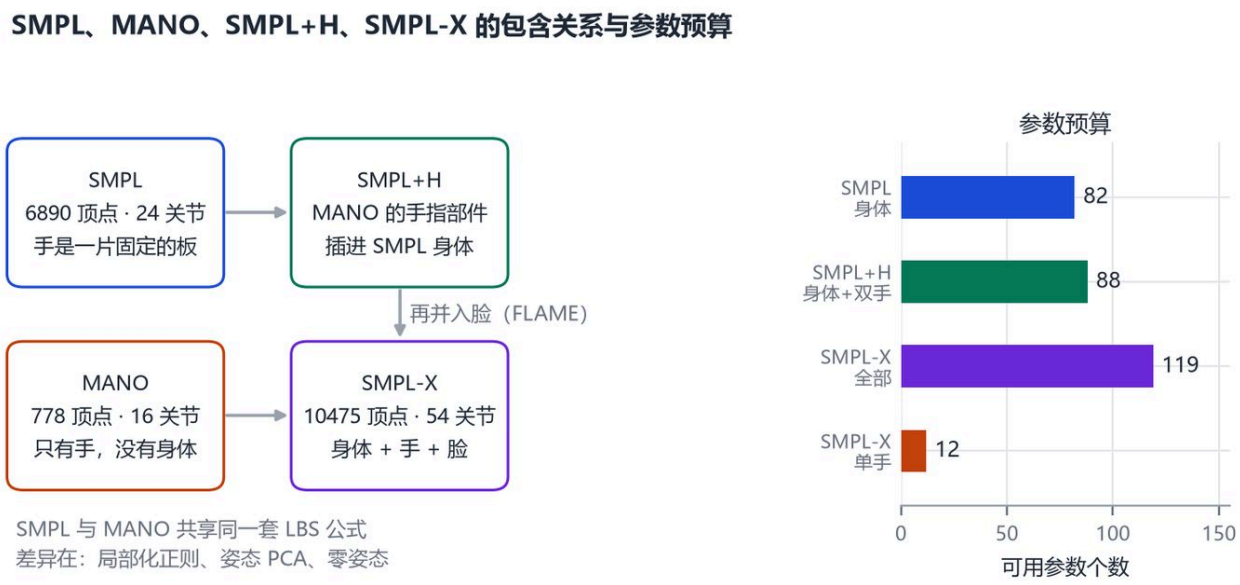


图 19-3 三个模型的包含关系与参数预算。SMPL 提供身体，MANO 提供手，SMPL+H 把 MANO 的手指部件插进 SMPL，SMPL-X 再并进 FLAME 的脸。注意右侧的参数预算：从全身到单手，可用的数字急剧减少，这正是“细节被压缩”的定量表现。

19.6 参数量、单位与坐标约定

最后把工程上最容易出错的地方集中交代一遍。这些内容在论文里通常一笔带过，但真正写代码时，八成的 bug 都出在这里。

参数清单。 一次完整的 MANO 前向需要以下几组数（表 19-2）：

表 19-2 一次 MANO 前向所需的参数。

名称	维度	含义	常见取值
`betas`	10	形状（胖瘦、掌宽、指长比例）	标准正态附近，±2 以内
`global_orient`	3	整只手的全局旋转（轴角）	弧度
`hand_pose`	48 或 6/10/12	16 个关节的相对旋转，或 PCA 系数	弧度 / 无量纲系数
`transl`	3	手腕在世界坐标中的平移	米

单位是米。 MANO 和 SMPL 一样，模板尺度以米为单位，成年男性手长约 0.18–0.19 米。如果你把模型加载进来发现手有 18 个单位长，那不是模型错了，是你把毫米当成了米。

旋转用轴角，不是欧拉角。 轴角（axis-angle）用一个三维向量表示旋转：向量方向是转轴，长度是转角（弧度）。它和四元数可以互相转换，和第 4 章讲的是同一套东西。注意轴角的长度超过 π 时会出现“绕远路”的等价表示，做插值和优化时要注意把角度归一到 $(-\pi, \pi]$。

左右手是镜像出来的。 这一点最容易被忽略。镜像一个网格时，顶点坐标的某个轴取反，但三角形的顶点顺序（绕向）也跟着反了，法线方向会朝里。正确做法是：坐标取反 + 翻转面的索引顺序。如果只做前者，渲染出来的手会是“内外翻转”的，光照完全不对。

零姿态是平手，不是自然手。 加载模型后直接渲染，你会得到一只五指笔直伸开的手。想让它看起来自然，必须给一点姿态参数（或者直接用数据集里估计出来的参数）。

不同库的关节顺序不一样。 MANO 官方模型、PyTorch3D 的封装、SMPL-X 里的手部关节，编号顺序都可能不同；有的库还把指尖点拼在 16 个关节后面凑成 21 个。跨库对比结果之前，先打印一遍关节坐标，确认“第 5 号关节”在两边指的是同一根手指的同一个位置。这不是小题大做——第 20 章的所有评估指标都建立在“关键点一一对应”的前提上。

本章小结

- 手难在四点：自由度多（约 21 个）、尺度小、自遮挡严重、外观缺乏特征；四者叠加导致高质量三维数据极难采集。
- 一只手有 27 块骨头；被显式建模的转动中心是 15 个（腕 1 + 掌指 5 + 指间 9），加上根关节共 16 个；21 个关键点 = 16 个关节 + 5 个指尖。
- MANO 沿用 SMPL 的公式，但做了三处关键修改：用测地距离加权让姿态偏移局部化；用 6 维 PCA 压缩姿态空间（解释约 81% 方差）；改用自己采集的 31 人、2018 次扫描来建形状空间。
- MANO 输出 778 个顶点、1538 个三角面，姿态参数 48 维（16 关节 × 3），形状参数 10 维。
- SMPL+H 把 MANO 的手指部件插进 SMPL，全身自由度从 66 升到 78；SMPL-X 进一步统一身体、手、脸，用每只手 12 维 PCA 表达手指，总参数 119 个。

- 工程上要盯住五件事：单位是米、旋转是轴角、左右手镜像要翻面、零姿态是平手、不同库的关节顺序可能不同。

思考题

1. MANO 把每个手指关节都当成球关节，48 个姿态参数里有很多物理上不可能的组合。为什么作者不直接用铰链关节，把参数减到 21 个？
2. 如果每只手只用 6 个姿态主成分就能解释 81% 的方差，那么剩下的 19% 主要是什么动作？为什么这些动作更难被低维空间捕捉？
3. SMPL-X 给一只手只留 12 维 PCA。请举一个 12 维明显不够用的手部动作，并说明如果要做这个动作的捕捉，你会怎么改系统结构。
4. 假设你拿到一份全身扫描数据，想在上面拟合出带手指的模型。你会先拟合身体再拟合手，还是同时拟合？分别会碰到什么问题？

第 20 章 单手重建：从优化到 Transformer

本章导读：第 19 章造好了 MANO 这只“可调的假手”，本章要回答：给它一张照片，怎样把参数调出来？这条路走了将近十年，从“在深度图上反复试参数”的优化，到“一次前向就出结果”的回归，再到今天的 Transformer 大模型。我们按时间顺序看四种范式各自解决了什么、又留下了什么，最后落到评估指标和失败模式——这部分比模型结构更有用，因为它告诉你什么时候不该相信结果。

20.1 先把问题说清楚

给定一张包含手的图像 $\mathbf{I}$，我们想要一只手的三维网格。用 MANO 的语言，就是求一组参数：

$$(\hat{\boldsymbol{\beta}}, \hat{\boldsymbol{\theta}}, \hat{\boldsymbol{\pi}}) = \arg\min_{\boldsymbol{\beta},\boldsymbol{\theta},\boldsymbol{\pi}} \sum_{i} \left\| \Pi\big(\mathbf{J}_i(\boldsymbol{\beta},\boldsymbol{\theta}); \boldsymbol{\pi}\big) - \mathbf{u}_i \right\|^2 + \lambda_\beta \|\boldsymbol{\beta}\|^2 + \lambda_\theta \|\boldsymbol{\theta}\|^2, \tag{20.1}$$

其中 $\mathbf{J}_i$ 是第 i 个关节的三维位置，$\Pi(\cdot;\boldsymbol{\pi})$ 是相机投影，$\mathbf{u}_i$ 是图像上的人工标注（或检测器输出的）二维关键点，λ 是正则项权重。式 (20.1) 和第 1 章的式 (1.2) 是同一件事：把“像不像”写成一个可以最小化的数。

这个式子看起来很简单，但它有三个天生的麻烦，值得在动手之前想明白。

第一，投影会丢信息。 一个三维点投影到图像上只剩两个数，深度没了。手指朝前伸还是朝后弯，在某些视角下投影几乎一样。这就是第 19 章说的深度翻转歧义，它意味着式 (20.1) 的解往往不唯一，必须靠先验（λ_θ 那一项，或者 MANO 的姿态 PCA）来挑一个“最像手”的解。

第二，手太小。 一张全身照片里，手可能只有 40×40 像素。网络从这么小的区域里要分辨出“中指第二关节弯曲了 20 度”，本质上是在超分辨率。

第三，标注很贵。 三维关键点需要多相机系统或精心设计的手工流程，规模上不去；二维关键点便宜得多，但二维监督不能唯一确定三维。

接下来近十年的方法演化，本质上都是在回答：用哪种表示、配哪种监督、花多少算力，来对付这三点。



图 20-1 四种范式的管线对比。左上：优化式拟合，在参数空间里反复试，慢但对数据要求低；右上：参数回归，一次前向出参数；左下：顶点/热图回归，绕开参数直接回归几何；右下：Transformer 路线，把图像块与网格顶点放进同一个注意力空间。

20.2 深度图时代：把拟合当成搜索

2010 年代中期的标准做法，是拿一台深度相机（Kinect、RealSense），把手放在镜头前，然后解式 (20.1) 的一个变体。

那时候还没有 MANO，手模型通常是几何基元的组合：手掌一个长方体，每根手指一串圆柱或球体。Oikonomidis 等人用粒子群优化（PSO）在参数空间里撒一把“粒子”，每个粒子是一组关节角，渲染出来和深度图比对，好的粒子留下来继续撒。这类方法能跑起来，但有两个硬伤：一是慢，一帧要几秒；二是需要好的初始化，手一动快了就跟丢。

后来出现了几种改进：用高斯混合模型代替基元（Sridhar 等），用球-网格（sphere-mesh）把手指做得更光滑（Tkach 等），或者用带蒙皮的三角网格代替基元。MANO 的论文里有一张图专门对比了这十种手部表示 [16]，那是这个阶段的一份“化石记录”。

这个阶段的核心洞察是：模型先验比图像证据更重要。 深度图在指缝处经常是空的，算法必须靠“手应该长什么样”把洞补上。这也解释了为什么 MANO 一出现就迅速取代了基元模型——它把“手应该长什么样”用统计的方式学到了极致。

但深度图路线有一个无法回避的限制：它需要专用硬件。手机、网络视频、监控画面里只有彩色图像，没有深度。于是问题变成：只用一张 RGB 图，能不能做到？

20.3 单张 RGB 的第一步：把模型塞进网络里

2019 年，Boukhayma 等人给出了第一个端到端的答案 [73]。他们的设计非常“教科书”，值得完整看一遍。

网络分成两半：

- 编码器：一个卷积网络，输入是手部裁剪图（可选再拼上一组二维关键点热图），输出一组参数：形状 $\boldsymbol{\beta}$、姿态 $\boldsymbol{\theta}$、视角 $\boldsymbol{\pi}$。
- 解码器：完全固定、不参与训练。它做两件事——用 MANO 从参数生成网格和骨架，再用弱透视相机把三维关节点投影回图像。

$$\mathbf{J}_{2D} = \underbrace{\Pi_{\text{weak}}\big(\mathbf{J}(\boldsymbol{\beta},\boldsymbol{\theta});\, s, \mathbf{t}\big)}_{\text{固定，可微}}, \qquad \mathbf{J}(\boldsymbol{\beta},\boldsymbol{\theta}) = \mathcal{J}\big(M(\boldsymbol{\beta},\boldsymbol{\theta})\big). \tag{20.2}$$

弱透视相机只有三个参数：缩放 s 和二维平移 $\mathbf{t} = (t_x, t_y)$（并假设主点在图像中心）。它比完整的透视相机少一个焦距，对尺度变化不敏感，在“物体离相机足够远”的假设下够用，也让网络更容易学。

这个设计最妙的地方在于损失函数。他们不监督参数——参数没有唯一正确答案（同一个姿态可以由不同的 PCA 系数组合出来），监督参数等于强迫网络模仿标注者的偏好。他们监督的是投影后的关节点：

$$\mathcal{L} = \lambda_{3D} \big\| \mathbf{J}_{3D} - \hat{\mathbf{J}}_{3D} \big\|^2 + \lambda_{2D} \big\| \Pi(\mathbf{J}_{3D}) - \hat{\mathbf{u}} \big\|^2. \tag{20.3}$$

第一项只在有三维标注的数据上生效，第二项在只有二维标注的数据上生效。这样一份数据可以只提供一半监督，网络照样能训——这就是“弱监督”的含义。论文明确说，正是这一点让他们能用上 MPII+NZSL 这类“在野外”的二维标注数据，从而提升泛化能力。

还有两个细节值得记住，因为它们是后续所有工作的公共常识：

1. 姿态用 PCA 系数，不用 48 维轴角。 他们取 10 个姿态主成分，形状也取 10 维。理由在第 19 章：48 维过参数化，且噪声数据下容易过拟合。
2. 指尖点要从网格上补。 MANO 的骨架里没有指尖关节，但数据集标注的 21 个点里有指尖。他们的做法是直接从 778 个顶点里挑 5 个指尖顶点拼到关节后面 [73]。你在复现时会遇到同样的问题。

这一代方法的局限也很清楚：监督信号只有关节点，没有像素级对齐。关节对了，手的轮廓可能还是偏的；手指之间的相对位置关系没有被显式约束，容易“挤”在一起。要解决这个，得让网络看见像素。

20.4 从参数到像素：热图、图网络与注意力

I2L-MeshNet（2020） 提出的问题很尖锐：把图像送进全连接层、直接吐出参数，等于把像素之间的空间关系全部打散了。一个在图像上相邻的关节点，在参数向量里可能隔得很远，网络学起来更吃力。

他们的替代方案是热图。人体姿态估计早就用热图了：为每个关节点预测一张二维概率图，取峰值位置作为坐标。这个表示保留了空间结构，也能表达“不确定”（峰值很平时说明网络没把握）。但直接把手部热图推广到三维是不行的：关节只有 20 个左右，而网格顶点有 778 个，为每个顶点预测一个三维体素热图，显存立刻爆炸。

于是他们提出了 lixel（line + pixel，线素）：把一维空间离散成格子，为每个顶点坐标预测一条一维热图。名字是类比 voxel（体素）造出来的。显存随热图分辨率线性增长，而不是立方增长，因此可以给到足够的分辨率 [77]。

网络结构是级联的：第一个子网（PoseNet）预测关节的 lixel 热图，第二个（MeshNet）在此基础上预测所有顶点的 lixel 热图。注意这条路线是 model-free 的——它不回归 MANO 参数，而是直接输出顶点坐标，关节位置靠 MANO 的关节回归矩阵乘上去。好处是不受参数空间的表达限制，坏处是输出不保证是一只有效的手（可能出现自交、指节塌陷）。

Mesh Graphormer（2021） 走了另一条路：既然网格是一个图，就用图卷积捕捉“邻居顶点之间的局部关系”；既然 Transformer 擅长长程依赖，就用自注意力捕捉“手指 A 与手指 B 之间的关系”。他们把两者塞进同一个 Transformer 块（每个块里既有图残差分支，也有自注意力分支），叠 4 层，让关节 token 与顶点 token 互相注意，同时还能注意到图像的网格特征 [78]。

这个组合的直觉是：手既需要局部光滑（图卷积），也需要全局协调（注意力）。 一根手指弯曲时，相邻手指会不自觉地跟着动一点，这是长程关系；指节处的皮肤褶皱是局部关系。两者缺一不可。

20.5 规模化：HaMeR

到 2021 年，手部重建在 FreiHAND 这类受控数据集上的误差已经降到 6 毫米左右，继续在结构上做微调收益越来越小。2023 年的 HaMeR 换了一个思路：不发明新结构，把数据和模型一起放大 [74]。

具体做了两件事：

数据。 他们把能找到的带手部标注的数据集合起来——包括有三维真值的受控数据集（FreiHAND、HO-3D、InterHand2.6M、DexYCB 等）和只有二维关键点的野外数据集（COCO-WholeBody、Halpe、MPII-NZSL 等），一共约 270 万个训练样本，是 FrankMocap 训练集的四倍。为了让模型在真实场景下站得住，他们还新标注了一个评测集 HInt：从 YouTube 视频和自我中心视频里取图，标了 40400 只手的关键点，其中 86.7% 的手正在与物体接触。

模型。 用大规模视觉 Transformer（ViT）做主干，后面接一个 Transformer 解码器，直接回归 MANO 的姿态、形状和相机参数。没有图卷积，没有热图，没有中间几何表示——就是“图块进、参数出”。

结果很有意思：在已经饱和的 FreiHAND 上，它的 PA-MPJPE 是 6.0 毫米，并不比 2020 年的 MobRecon（5.7 毫米）更好；但在 HInt 这类野外数据上，它的 PCK@0.05 比之前的方法高出两到三倍。换句话说，大模型带来的不是“更准”，而是“更不容易崩”。

这个结果值得停下来想一想。它说明在受控数据集上刷榜和在真实场景里可用，是两件不同的事。受控数据集的光照、视角、背景、手的姿势分布都被精心设计过，方法之间的差异被压缩了；而真实图像的分布极其杂乱，只有见过足够多数据、容量足够大的模型才能覆盖。

HaMeR 论文里还有一组数字很能说明问题：在 HInt 上，可见关节的 PCK@0.05 是 60.8，遮挡关节只有 27.2——遮挡让精度直接掉了一半还多。这不是某个模型的缺陷，而是这个问题本身的难度。

把前面几节提到的方法按时间排一遍，会看得更清楚：两个转折点分别落在 2019 年和 2021 年之后。



图 20-2 单手重建的时间线。橙色标注的是本章重点讨论的工作，灰色是同一时期的重要旁支。可以看到两个明显的转折：2019 年前后从“优化”转向“端到端回归”，2021 年之后从“改结构”转向“堆数据与容量”。

20.6 全身一体化：FrankMocap 与 PyMAF

单手模型假设“图像里只有一只手，而且已经裁好了”。真实应用往往要先在全身图像里找到手。这就引出另一条线：全身 + 手的一体化估计。

FrankMocap（2020/2021） 的结构可以概括成“先回归，再整合” [76]。它有几个互相独立的模块：一个估身体（SMPL-X 的身体部分），一个估手（MANO），一个估脸。各模块先用回归网络给出初值，再由一个整合模块把三者拼成一个完整的 SMPL-X，并用渲染-比对（render-and-compare）的方式做几步优化，让渲染出来的轮廓和图像对齐。

这个“分而治之”的设计有很实际的考虑：手的训练数据远比身体少，把手单独训一个网络，可以充分利用手部数据集；而整合模块负责处理“手和身体接不上”的问题——比如手腕的位置、朝向、尺度。

PyMAF（2021） 解决的是另一个问题：参数的一点点偏差，会导致网格和图像明显错位 [79]。原因是参数到顶点是非线性的，姿态参数差 0.1 弧度，指尖可能就差几厘米。他们的办法是引入一个“网格对齐反馈”回路：先把当前估计的网格投影回图像，在特征金字塔的对应位置上采样特征，再用这些“网格对齐的证据”去修正参数。反复几次，网格就慢慢贴到图像上了。

这个思路和经典的迭代误差反馈（iterative error feedback）很像，区别在于反馈的信号不是“误差图”，而是“在网格投影位置采样到的图像特征”——更直接地和几何对齐挂钩。

把 FrankMocap 和 PyMAF 放在一起看，会发现它们代表了一体化系统的两种取向：一个强调模块化（各部件独立，接口清晰），一个强调闭环（估计—渲染—比对—再估计）。工程上两者经常混用：用模块化拿到初值，用闭环做精修。

20.7 数据从哪来：多视角标注与弱监督

第 19 章说过手部数据难采。这一节把这几年真正可行的几条路列清楚，因为方法的上限基本由数据决定。

路线一：多视角 + 联合优化标注。 HOnnotate 是代表 [80]。它用一台或多台 RGB-D 相机拍手抓物体的序列，然后把“手和物体的三维位姿”当成一个整体，在所有帧上同时优化——手用 MANO，物体用已知的 CAD 模型，两者互相提供约束：手不能穿进物体，接触处要贴合。这样即使某一帧里手被物体挡住，也能靠前后帧和物体几何补出来。用这个方法，他们做出了 HO-3D：68 段序列、77558 帧、10 个人、10 个物体。这批数据后来成了手物交互研究的标准训练集之一（第 21 章详述）。

路线二：合成数据。 在仿真里摆好手和物体，渲染出图像，标注是免费的。ObMan 是这条路线的代表作，第 21 章会讲它的规模和局限。合成数据的核心问题是“域差异”：渲染图的光照、材质、噪声和真实照片不一样，网络容易学到渲染器的特征。

路线三：二维弱监督 + 大规模真实图。 这就是 Boukhayma 和 HaMeR 用的办法：只要有二维关键点，就能约束投影后的手；三维的歧义由 MANO 的先验来消除。这条路能利用互联网规模的图像，是当前最有效的规模化手段。

路线四：伪标签自训练。 先用有三维真值的数据训一个模型，拿它去预测无标注图像，把高置信度的预测当标签，再训一轮。这在实践中非常常用，但要注意它会放大模型自身的偏见。

四条路线不是互斥的。今天最强的系统通常是：合成数据预训练 + 三维数据精调 + 二维弱监督扩域 + 伪标签迭代。

20.8 评估指标与失败模式

最后讲怎么判断一个方法好不好。手部重建的指标不算多，但每一个都有陷阱。

PA-MPJPE（Procrustes-Aligned Mean Per-Joint Position Error）。 先把预测的骨架和真值做一次刚体对齐（允许旋转、平移、缩放），再算关节的平均欧氏距离，单位毫米。加“PA”是因为我们通常不关心绝对位置和朝向上的小差异，只关心姿态本身。代价是：一个把手整体旋转了 30 度但姿态完美的方法，PA-MPJPE 会很好看。

PA-MPVPE。 同样的对齐，但算的是网格顶点而不是关节。它比关节指标更严格，因为顶点还反映了形状（胖瘦、指长）。

F@5 / F@15。 F-score：在 5 毫米（或 15 毫米）阈值内，预测顶点与真值顶点的匹配比例，兼顾 precision 和 recall。它比平均误差更能反映“有多少地方明显错了”。

PCK（Percentage of Correct Keypoints）。 二维指标：预测关节点落在真值一定阈值内的比例（阈值常按手部尺度归一化，如 0.05 倍掌宽）。HInt 这类只有二维标注的评测集只能用 PCK。

图 20-3 是 FreiHAND 上的公开结果，数据取自 HaMeR 论文的对比表 [74]。请注意横轴的范围：只从 5 毫米跨到 8 毫米，从 2020 年的 7.4 到 2021 年的 5.7，两年只降了不到两毫米。

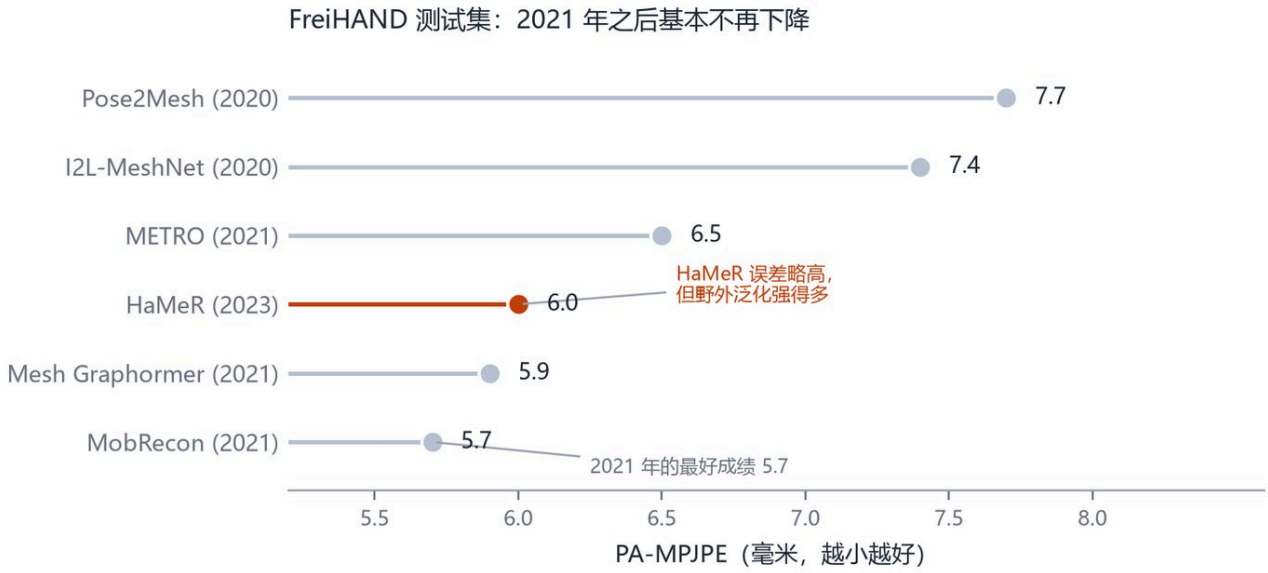


图 20-3 FreiHAND 测试集上的 PA-MPJPE（毫米，越小越好），数值取自 HaMeR 论文的对比表 [74]。注意两个现象：一是 2021 年之后指标基本不再下降，说明这个数据集已接近饱和；二是 HaMeR（6.0）并不比 2020 年的 MobRecon（5.7）更好，但它在野外数据上的泛化能力远远领先——这正是 20.5 节讨论的重点。

失败模式清单。 下面几条是实践中反复出现的，比指标更能帮你判断“这个结果能不能用”：

1. 遮挡。 手抓物体、双手交叠、手放在身后。HaMeR 在 HInt 上的数字说明，遮挡关节的精度大约是可见关节的一半。
2. 极端视角。 正对指尖（“指人”的视角）时，手指全部缩短成一堆点，深度方向几乎没有信息。自我中心视角（第一人称）也常出现这种构图。
3. 尺度歧义。 单目图像里，一只远处的大手和一只近处的小手投影可以一样大。弱透视相机把尺度吸收进参数 s，所以“手的绝对大小”基本不可信，除非有额外的深度或人体上下文。
4. 左右手混淆。 一只背对镜头的手，与另一只手的镜像，在轮廓上高度相似。网络经常给出“镜像正确、朝向错误”的结果。
5. 时间抖动。 逐帧独立估计的视频会出现高频抖动，即使每帧的误差都不大。做产品时通常要加时间平滑或时序模型（第 10 章讨论过视频人体重建的同类问题）。
6. 手部与身体的接缝。 分别估计手和身体，再拼起来时，手腕处常有错位或断裂。这是 FrankMocap 那类系统要专门做“整合”的原因。

本章小结

- 单手重建就是把第 1 章的“渲染—比较—调参”落到 MANO 上：求一组参数，使投影后的关节（或轮廓）与图像一致。
- 三条根本困难：投影丢深度导致歧义、手在图中太小、三维标注太贵。
- 演化路线：深度图时代的优化拟合（慢、需初始化、依赖硬件）→ Boukhayma 的端到端回归（固定可微解码器 + 二维/三维关节弱监督）→ 热图与图网络（I2L-MeshNet 的 lixel、Mesh Graphormer 的图卷积 + 注意力）→ 规模化 Transformer（HaMeR，270 万样本，赢在泛化而非榜单）。
- 全身一体化有两条取向：FrankMocap 的模块化 + 整合，PyMAF 的网格对齐反馈闭环。
- 数据决定上限：多视角联合优化标注（HOnnotate/HO-3D）、合成渲染、二维弱监督、伪标签自训练，今天最强的系统往往四者混用。
- 评估要看指标的含义：PA-* 允许刚体对齐，F-score 看“错得多的比例”，PCK 只能评二维。遮挡、极端视角、尺度、左右手、时间抖动、手与身体的接缝是六类典型失败。

思考题

1. Boukhayma 的方法不监督 MANO 参数，只监督投影后的关节。如果改成直接监督参数，会发生什么？请从“参数不唯一”这个角度解释。
2. I2L-MeshNet 用一维热图代替直接回归坐标。为什么一维就够了，而不需要像体素那样三维？这样做丢失了什么信息？
3. HaMeR 在 FreiHAND 上没有明显超过 2020 年的方法，但在野外数据上大幅领先。如果让你设计一个新的评测集，你会怎么设计才能区分“刷榜型”和“实用型”方法？
4. 一只手在图像里只占 40×40 像素。假设你把图像放大四倍再送进网络，精度会提升吗？会不会引入新的问题？

第 21 章 手物交互与抓取

本章导读：前面两章的手是“悬在空中”的——没有东西可抓，也没有东西挡着。可手的意义恰恰在于操作物体：握住杯子、拧开瓶盖、捏起一枚硬币。一旦把物体放进来，问题就从“估计一只手”变成了“同时估计手和物体，并且让它们看起来像是真的在接触”。本章讲三件事：为什么必须一起建模、现有数据能提供什么、以及从“模板拟合”到“隐式表达”再到“高斯泼溅”的技术演化。前置知识是第 19 章的 MANO 与第 20 章的重建范式。

21.1 为什么手和物体必须一起建模

先做一个思想实验。给你一张照片：一只手握着一个马克杯。你分别跑两个“专家系统”——一个只估手，一个只估物体——然后把结果叠在一起。会发生什么？

第一，手会被物体“吃掉”一块。 只估手的模型看不见被杯子挡住的那部分手指，它只能靠 MANO 的先验去猜。猜出来的手指很可能穿过杯壁，或者缩在杯子里面。

第二，物体会飘。 单目图像里的物体位姿有尺度和深度歧义，一个 8 厘米的杯子和一个 16 厘米的杯子在图上可以一样大。但如果这只手是标准大小（MANO 有固定的尺度先验），那么“手能握住它”这件事本身就提供了尺度约束。手是物体的一把尺子。

第三，接触关系没人管。 真实抓取里，手指的指腹贴着物体表面，拇指和其他手指形成对抗。这种“贴住但不穿透”的关系，是判断一个重建结果合不合理的最强证据。分开建模时，没有任何一项损失在管这件事。

所以从 2019 年起，这条线上的工作都建立在同一个判断上：手和物体是一个系统，必须联合估计。 而“联合”带来了一个新的问题——用什么表示物体？

这个问题比看起来难。人体有 SMPL，手有 MANO，因为人体和手的形状空间相对稳定、可以统计学习。物体呢？世界上有几万种杯子、几万种瓶子，它们的形状没有共同的拓扑，也没法用几十个 PCA 系数概括。于是研究分成了两条路：一条假设“物体模型已知”（CAD 或扫描好的网格），另一条试图“从图像里现场重建物体”。本章会看到，这条分界线决定了方法的全部结构。

21.2 数据：四种采集哲学

手物交互的数据集不多，但每一个都代表一种不同的取舍。图 21-2 把它们放在一起对比。

四个代表性数据集：采集方式决定了能拿到什么

数据集	输入模态	规模	物体	标注方式
ObMan	合成 RGB	14.1 万帧（训练）	2772 个网格（8 类）	仿真抓取 + 渲染，标注完美
GRAB	真实 MoCap	10 人 · 51 件物体	真实扫描 + 接触图	手背标记点，掌心无遮挡
HO-3D	真实 RGB-D	68 段 · 77558 帧	10 件物体	多视角联合优化标注
DexYCB	真实 RGB-D × 8	1000 段 · 58.2 万帧	20 件 YCB 物体	众包 2D 关键点 + 三角化

共同困境：真实数据的规模被采集成本锁死，合成数据的真实度被渲染器锁死。
注意 GRAB 的“掌心无标记”——只有这样才能采到真实的掌面接触，而掌面恰恰是抓取最关键的区域。

图 21-1 四个代表性数据集的逐项对比。注意“标注方式”一列：每个数据集的强项恰好是它的采集方式的直接结果——合成数据标注完美，GRAB 有真实掌面接触，HO-3D 能在遮挡下标注，DexYCB 则胜在规模与统一的任务定义。

ObMan：全部合成。 Hasson 等人从 ShapeNet 里选了 8 类日常物体（瓶子、碗、罐子、刀、手机、相机、遥控器等），共 2772 个网格，用 GraspIt 仿真生成 21000 个抓取，再把 MANO 手摆成抓取姿态、渲染成图像，训练集共 14.1 万张 [81]。它的好处是完美标注、无限规模；坏处是渲染图和真实照片的分布差异明显，网络容易学到“渲染器的味道”。

GRAB：全身 + 物体 + 接触。 Taheri 等人请了 10 位被试（5 男 5 女），抓握 51 件日常物体，每个动作都要求全身参与——不只是手，手臂、躯干、甚至另一只手都会配合 [82]。采集时被试戴着动作捕捉标记，只有手背贴标记，掌心完全空着，这样抓握时掌心能真实接触物体。有了手、身体、物体的三维网格，就可以计算接触区域：物体表面哪些点被身体碰到了，碰了多大面积。

这批数据的一个重要贡献是把“抓取”的概念拓宽了：不只是“单手稳稳举起物体”，还包括递、接、换手、使用。这些动作里，物体在手中的位置会变，接触区域也会变。

HO-3D：多视角 + 联合优化标注。 Hampali 等人用一台或多台 RGB-D 相机拍手抓物体的序列，然后用一个关键技巧做标注：把“手的三维位姿”和“物体的六自由度位姿”放到同一个优化里，在所有帧上同时求解 [80]。手不能穿进物体、接触处要贴合，这两个约束让遮挡帧也能被正确标注。最终得到 68 段序列、77558 帧、10 个人、10 个物体，是第一个无标记的真实手物数据集。

DexYCB：规模化 + 标准化任务。 Chao 等人把任务限定得很窄：从桌上抓起物体。但把规模做大了——10 位被试、20 个 YCB 物体、8 个视角、1000 段序列、58.2 万帧 RGB-D [75]。标注方式是众包人工标二维关键点（手 21 个点、物体若干点），再用多视角三角化和拟合得到三维。它的价值在于基准：任务定义清晰、评价协议统一，后来的方法几乎都会在 DexYCB 上报一个数。

把四个数据集并排看，会发现一个共同的困境：真实数据的规模被采集成本锁死，合成数据的真实度被渲染器锁死。 这个矛盾直接催生了 21.5 节的在线合成方法。

21.3 联合重建：从接触损失到类别无关

第一代：已知物体 + 接触损失。 Hasson 等人的做法是端到端的：输入一张 RGB 图，网络同时输出 MANO 的手参数和物体的形状、位姿、尺度，然后用一个接触损失把两者绑在一起 [81]。这个损失由两项组成，方向相反：

$$\mathcal{L}_{\text{contact}} = \underbrace{\sum_{\mathbf{o}\in\mathcal{O}} \min_{\mathbf{v}\in\mathcal{H}} \|\mathbf{o}-\mathbf{v}\|^2}_{\text{吸引：物体表面要靠近手}} + \underbrace{\lambda \sum_{\mathbf{v}\in\mathcal{H}} \max\big(0, -d_{\mathcal{O}}(\mathbf{v})\big)^2}_{\text{排斥：手不能陷进物体}}. \tag{21.1}$$

$d_{\mathcal{O}}(\mathbf{v})$ 是手部顶点 $\mathbf{v}$ 到物体表面的有符号距离，在物体内部为负。第一项把物体上的每个点拉向最近的手部顶点，第二项把已经陷进去的顶点推出来。

这个写法很聪明，但有一个明显的弱点：它不区分“该接触的地方”和“不该接触的地方”。 手背碰到杯子也该被“吸引”吗？论文自己也指出，两项之间的权重平衡非常关键，偏了就会得到“手贴在物体表面但姿态别扭”的结果。

第二代：类别无关（category-agnostic）。 第一代方法的隐含前提是“物体有模板”——要么是已知的 CAD 模型，要么是一个学好的物体形状空间。真实场景里，你手里的那支笔可能从来没被扫描过。

HOLD 就是冲着这个前提去的 [83]。它只需要一段单目视频，物体是什么类别完全不管，直接从视频里把手的关节和物体的表面一起重建出来。技术上的关键是用隐式表示替代模板：物体不再是“一个已知网格 + 一个位姿”，而是一个由神经网络表示的连续表面（用有符号距离场描述，给一个三维点，网络告诉你它在表面内还是外、离表面多远）。手的表面也用类似的方式表达，两者通过“手与物体不能互相穿透、接触处要贴合”的约束耦合在一起。

隐式表示的好处是拓扑自由——杯子有把手、碗是凹的、剪刀有两个环，都不需要预先规定网格结构；坏处是它需要多视角或多帧的观测才能约束住形状，单帧基本无解。

第三代：把物体也当成“可优化的高斯点”。 这是 21.6 节的内容，先按下不表。

还有一条支线是第一视角（egocentric）采集：相机戴在头上，手总在画面里、离镜头近，但视角极端、运动模糊严重、物体经常被手完全挡住。它和第三视角的问题几乎是互补的。

21.4 物理约束：穿透、接触与力闭合

上一节的接触损失是“软约束”——它只是让网络倾向于不穿透，并不保证。真正要判断一个抓取是否成立，得回到力学。

穿透。 最硬的约束：手和物体的体积不能重叠。实践中很少直接做碰撞检测（太慢且不可微），而是用有符号距离场：

$$E_{\text{pen}} = \sum_{\mathbf{v}\in\mathcal{H}} \max\left(0, -\text{SDF}_{\mathcal{O}}(\mathbf{v})\right)^2, \tag{21.2}$$

只惩罚“陷进去”的顶点，深度越大惩罚越重。SDF 可以来自网格的预计算，也可以来自隐式网络。

接触。 光不穿透还不够：手悬在物体上方一毫米，物理上完全合法，但那就不是抓取了。所以还需要一项“该接触的地方要接触”。难点在于哪些地方该接触事先不知道，于是实践中常用“双向最近点”（式 21.1）或者接触图（contact map）来近似——接触图是一张贴在物体表面的概率图，标出每个位置被碰到的可能性。GRAB 就提供了这样的接触标注：给定手和物体的网格，逐顶点算距离，小于阈值的记为接触 [82]。

摩擦锥。 接触点上的力不能任意方向。设接触法线为 $\mathbf{n}$，摩擦力系数为 μ，则接触力 $\mathbf{f}$ 必须满足

$$\|\mathbf{f} - (\mathbf{f}\cdot\mathbf{n})\mathbf{n}\| \;\le\; \mu\,(\mathbf{f}\cdot\mathbf{n}), \tag{21.3}$$

几何上，这约束 $\mathbf{f}$ 落在一个以 $\mathbf{n}$ 为轴、半角为 $\arctan\mu$ 的圆锥里——这就是摩擦锥（friction cone）。$\mu \approx 0.5$ 时半角约 27 度；$\mu = 1$ 时 45 度。手指的指腹是软的，实际能提供的摩擦比刚性接触更大，这也是人为什么能用很小的力捏住光滑的玻璃杯。

力闭合。 一个抓取成立的条件是：存在一组满足式 (21.3) 的接触力，能够抵抗任意方向的外力旋量（三个方向的力 + 三个方向的力矩）。这叫力闭合（force closure）。直观地说：不管你怎么推、怎么拧这个物体，手指都能顶住。

判断力闭合在数学上是一个凸优化问题（在所有接触点上找一组非负的法向力和切向力，使它们的合力矩为零且能张成整个六维旋量空间）。工程上常用的近似指标有 ϵ 指标（最差方向上的抵抗能力）和体积指标。这些指标不只用于评价，也被用作训练的损失——让网络生成“力闭合”的抓取，比让它生成“看起来像抓取”的抓取要难得多。

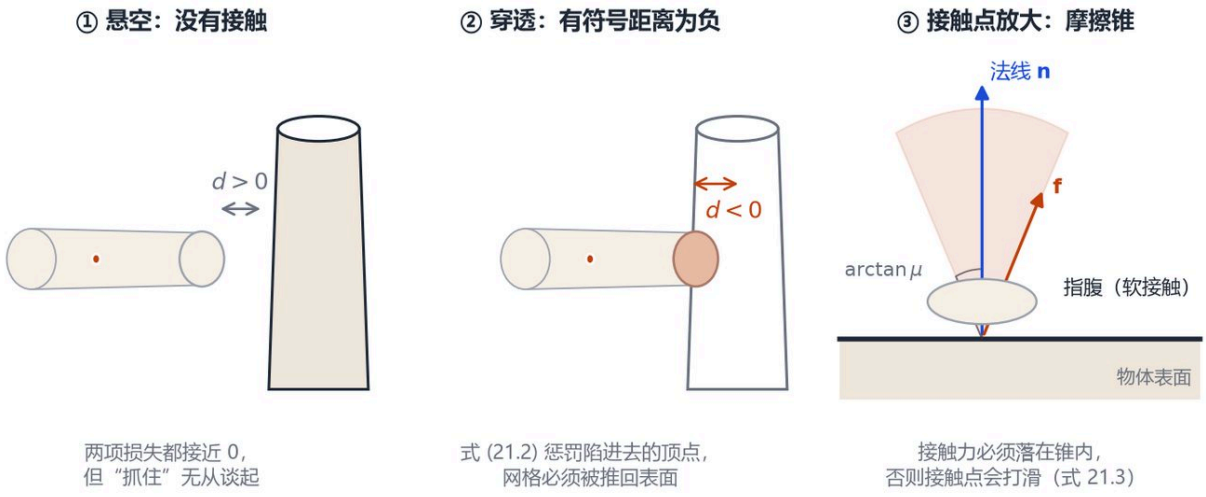


图 21-2 接触的三种状态与摩擦锥。左：手悬空，有符号距离 $d > 0$，式 (21.1) 的两项都接近 0，但并没有抓住；中：手陷进物体，$d < 0$，式 (21.2) 产生惩罚；右：接触点的放大图，接触力 $\mathbf{f}$ 必须落在以法线 $\mathbf{n}$ 为轴、半角为 $\arctan\mu$ 的锥内。

21.5 生成抓取：先造数据，再学估计

如果真实数据只有几万帧，而物体有上万种，那么“覆盖”是不可能的。生成抓取这条线就是回答：能不能用算法造出足够多、足够真实的抓取？

GANHand：把抓取当成生成问题。 Corona 等人不直接回归一个抓取，而是训练一个条件 GAN：给定场景里物体的几何（点云或体素），生成器输出一组 MANO 手参数，判别器判断“这看起来像不像真实的抓取” [84]。判别器是用真实抓取数据训出来的，所以生成器学到的不是“手能贴上去”，而是“手会这样贴上去”。

这一路线的副产品很实用：为了训练和评测，他们对 YCB 物体手工标注了大量 MANO 抓取姿态，形成了后来的 YCBAfford 数据集。这种“为了做方法而顺手做出数据”的模式在这个领域非常常见。

ArtiBoost：在线合成 + 难度自适应。 ArtiBoost 的出发点是：离线合成的数据是固定的，而训练过程中模型“已经学会了什么”在变，所以应该边训边造，缺什么补什么 [85]。

他们定义了一个三维离散空间，叫 CCV 空间（Composited hand-object Configuration and Viewpoint）：

$$\mathcal{S} = \left\{ (n_o, n_p, n_v) \,\middle|\, n_o \le N_o,\ n_p \le N_p,\ n_v \le N_v \right\}, \tag{21.4}$$

三个维度分别是物体类型、手部姿态、视角方向。以 DexYCB 的 20 个物体为例，$20 \times 100 \times 288 = 576000$ 个组合。构造方式是：对每个物体做基于接触约束的抓取拟合（每个物体试 300 个初值，人工剔除严重穿透和不自然的，最后保留最多 100 个），再对每个抓取采样视角。

训练时，每个 batch 混入一批从 CCV 空间采样、现场渲染的合成图。采样不是均匀的：每个组合有一个权重，训练损失大的组合权重高——模型在哪里错得多，就多看哪里的合成数据。论文的消融实验显示，同样数量的合成抓取，来自 CCV 空间的那批比来自 YCBAfford 的更能提升性能。

这个思路其实很朴素：数据增强的本质是“在模型薄弱的方向上加密采样”。ArtiBoost 只是把它做成了一个闭环。

21.6 无标记捕捉与高斯泼溅：MANUS

前面所有方法都在假设“输入是图像，输出是网格”。2023 年之后，三维重建的表示本身变了：3D 高斯泼溅（3D Gaussian Splatting, 3DGS）用一堆带方向、带颜色、带透明度的椭球来表示场景，渲染快、质量高，而且可以求导。

MANUS 把 3DGS 用在手物抓取捕捉上 [86]。它的核心想法是：不要一上来就把手和物体压成网格，而是先用高斯点表示整个“手 + 物体”的场景，从多视角视频里优化这些高斯点，然后再把手的高斯点绑定到 MANO 的骨架上——手弯曲时，对应的高斯点跟着刚体变换走。这叫“关节化高斯”（articulated Gaussians）。

这样一来，几何不再是固定的，而是可以随姿态变形；同时，物体也不需要事先知道模型，它只是另一组高斯点。输出的结果既包括手部姿态（MANO 参数），也包括物体的表面。

这条路线解决的问题是 21.3 节留下的：当物体没有模板、图像又有遮挡时，怎么同时拿到手和物体的几何？ 用高斯点表示的好处是它对“部分观测”很宽容——被手挡住的那部分物体，可以从其他帧、其他视角的高斯点补上来；坏处是需要多视角采集，单目视频效果会明显下降。

从更长的历史看，MANUS 代表了一个趋势：参数化模型从“表示”退到“骨架”。手仍然用 MANO 描述姿态，但表面细节交给高斯点或隐式场。这和人体那边的神经渲染（第 13、25 章）是同一个走向。

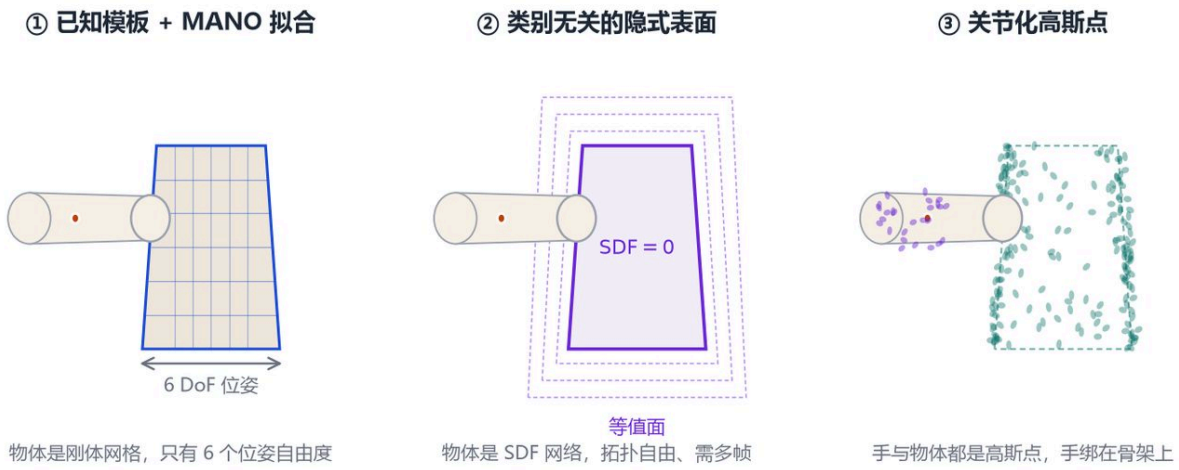


图 21-3 手物联合重建的表示演化。左：已知物体模板 + MANO 拟合，物体是刚体网格、只有 6 自由度位姿；中：类别无关的隐式表面，物体由网络表示、拓扑自由；右：关节化高斯点，手与物体都由高斯点表示，手的高斯绑定在 MANO 骨架上。三者对“物体先验”的依赖依次减弱，对“观测数量”的要求依次增强。

21.7 应用与开放问题

已经能用的地方。

- 机器人示教。 从人的示范视频里恢复手物轨迹，映射到机械手上。这里的难点不是精度而是可比性：人手有 21 个自由度、柔软的指腹，机械手可能只有二指夹爪或三指灵巧手，需要做“抓取类型”的抽象。
- AR/VR 手势与抓取。 用户徒手在空中比划，系统要判断“他在抓什么”。这里更看重实时性和稳定性，精度要求反而低。
- 动画与游戏。 让虚拟角色自然地拿起道具。工业界的做法通常是“先检索、再微调”：从抓取库里找最像的一个，再用手部 IK 贴合到具体道具上。
- 手部康复与技能分析。 精确测量抓握时的接触面积、手指协同模式，用于评估康复进展。

还没解决的问题。

1. 接触力无法从图像直接观测。 我们能重建几何，但力是另一回事。有工作尝试从手的形变反推力的大小（用手指变形的程度），但精度有限。
2. 柔软物体。 目前几乎所有方法都假设物体是刚体。抓一块布、一团面、一个塑料袋，接触会改变物体的形状，整个“手 vs 物体”的二分法就失效了。

3. 双手与工具使用。 一只手拿锤子，另一只手拿钉子，物体之间还有约束。多体交互的复杂度是指数增长的。
4. 数据规模与真实度的矛盾。 21.2 节说的困境仍然存在：合成数据不够真，真实数据不够多。目前的缓解手段（在线合成、伪标签、域适应）都只是缓解。
5. 评价标准不统一。 手部误差用毫米，物体位姿用度和厘米，接触用面积比例，力闭合用一个无量纲指标。不同论文报不同的数，很难横向比较。这也是为什么 DexYCB 这类“任务定义清晰”的基准特别有价值。

本章小结

- 手和物体必须联合建模，原因有三：遮挡让单独估计失效、手为物体提供尺度、接触关系是判断合理性的最强证据。
- 物体的表示是这条线的核心分岔：已知模板（Hasson、GRAB 类）→ 类别无关的隐式表面（HOLD）→ 关节化高斯点（MANUS）。对物体先验的依赖越来越弱，对观测数量的要求越来越强。
- 四个代表性数据集：ObMan（合成 14.1 万帧、2772 个网格）、GRAB（10 人 51 物体，全身 + 接触标注）、HO-3D（68 段序列 77558 帧，多视角联合优化标注）、DexYCB（1000 段序列 58.2 万帧，标准化抓取基准）。
- 接触损失通常是“吸引 + 排斥”两项：把物体拉向手，同时把手推出物体；权重平衡是关键，且它不区分该接触与不该接触的位置。
- 物理上判断抓取是否成立要看摩擦锥（式 21.3）与力闭合；这些指标既用于评价，也可以作为训练损失。
- 数据不够就造数据：GANHand 用条件 GAN 生成抓取，ArtiBoost 在 CCV 空间里边训边采样、按损失加权。
- 开放问题：接触力不可观测、软体、双手与工具、数据规模与真实度的矛盾、评价标准不统一。

思考题

1. 式 (21.1) 的吸引项把物体表面每个点都拉向最近的手部顶点。如果一个杯子有把手，而手只握住了杯身，这一项会产生什么错误倾向？你会怎么改？
2. 摩擦系数 $\mu = 0.2$（比如沾了油的玻璃）时，摩擦锥的半角是多少度？这对“捏住”这个动作意味着什么？
3. HOLD 用隐式表面替代物体模板，代价是需要多帧观测。如果只有一张图，你会怎么折中？
4. ArtiBoost 按训练损失给合成样本加权。这样做有没有风险？会不会导致模型在某个困难子集上过拟合？
5. 假设你要做一个 AR 应用，让用户用手抓取虚拟物体。你需要的是“几何精确的重建”还是“物理合理的抓取”？两者的技术选型会有什么不同？

第五篇　动物

SMAL 与跨物种的四足、鸟类、鱼类参数化

第 22 章　SMAL：四足动物的参数化
第 23 章　动物重建：从 SMALST 到 MagicPony
第 24 章　跨物种：鸟类、鱼类与动物生成

第 22 章 SMAL：四足动物的参数化

本章导读：把人体的那套方法搬到动物身上，第一反应是“把 SMPL 的骨架改成四条腿”。但真正的困难不在骨架，而在**数据**：动物不会站着让你扫，物种之间的形状差异又极大。SMAL 的解法很聪明——用玩具模型学习形状空间。这一章讲它怎么把“狮子、老虎、马、河马”装进同一个模型，以及为什么 33 个关节、一套拓扑就能覆盖这么多物种。

22.1 为什么不能直接套 SMPL

SMPL 的假设是：所有样本共享同一套拓扑、同一个骨架结构（一个脊柱、两条腿、两条手臂）。动物不满足这个假设：

- 腿的数量与朝向不同：四足动物的“前肢”对应人的手臂，但关节弯曲方向、脚掌结构完全不同；
- 形状差异巨大：河马和猎豹的体型差异，远大于人与人之间的差异；
- 没有关键点标注：人脸有 68 个点，人体有 17 个点，动物没有公认的关键点定义；
- 数据难采集：活的动物不配合，3D 扫描几乎不可能。

所以，动物参数化必须重新回答三个问题：用什么模板？用什么骨架？用什么数据训练？

22.2 SMAL 的构建：从玩具到模型



图 22-1 SMAL 的构建流程。先扫描一批摆好姿势的玩具动物，用分段刚体拟合 + ARAP 细化建立对应，再把所有扫描摆回同一姿势、做 PCA 得到形状空间，最后用模型反过来约束配准，迭代若干次。

2017 年，Zuffi 等人在 3D Menagerie[87] 中提出 SMAL（Skinned Multi-Animal Linear model）。它的数据来源非常“土办法”：41 个玩具动物扫描，包括狮子、老虎、斑马、马、牛、河马等。

为什么用玩具？因为：

1. 玩具可以摆成任意姿势，便于采集“同一形状、不同姿态”的配对；
2. 玩具表面纹理清晰，配准相对容易；
3. 玩具便宜、可重复购买，数据规模可控。

构建流程分五步：

1. 分段刚体拟合：把每个玩具的扫描拆成若干刚体段（头、躯干、四条腿、尾巴），用 GLoSS 等方法拟合；
2. ARAP 细化：把顶点吸到扫描表面，保持局部刚性，避免配准“糊成一团”；
3. 姿态归一化：用 LBS 把所有扫描摆回同一个标准姿势；
4. PCA 形状空间：对归一化后的顶点做 PCA，得到体型主成分；
5. 联合配准：用当前的 SMAL 模型反过来约束配准，迭代 4 次左右。

> **最关键的一步是“对应”**：把形状差十倍的动物配准到同一套顶点编号上。配不准，后面的 PCA 全部建立在错位的关系上。

22.3 33 个关节与一套拓扑

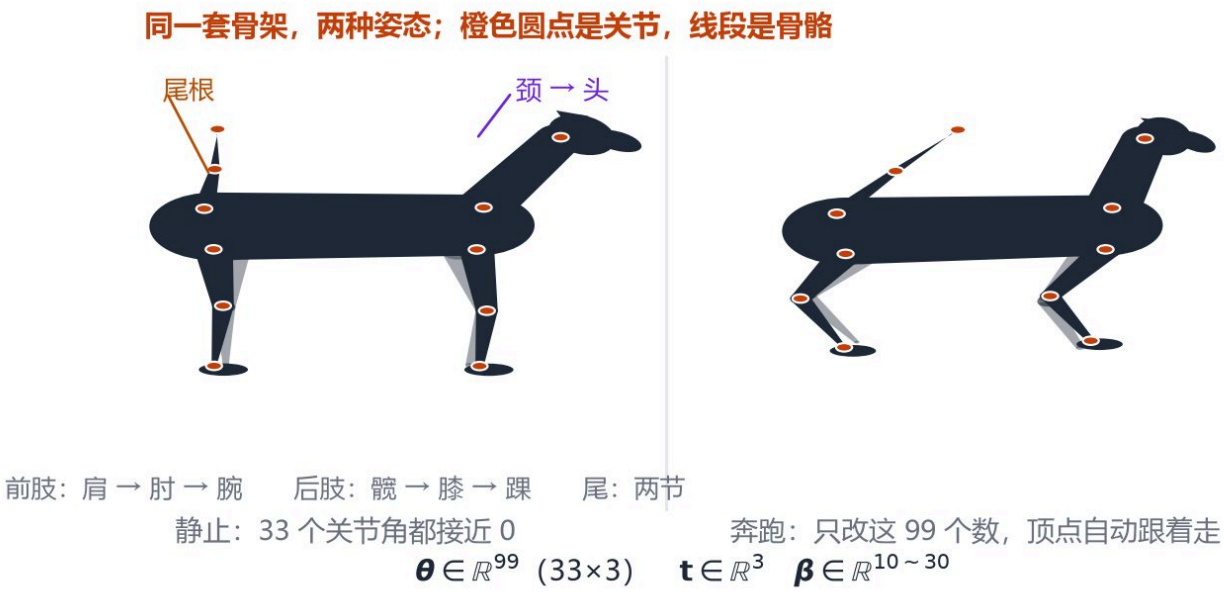


图 22-2 SMAL 的骨架与姿态。33 个关节串成一条脊柱、四条腿、一条两节尾巴；姿态参数 $\boldsymbol{\theta} \in \mathbb{R}^{99}$，形状参数 $\boldsymbol{\beta}$ 控制物种与体型。改变关节角，顶点通过 LBS 自动跟随。

SMAL 的骨架有 33 个关节，姿态参数是 $33 \times 3 = 99$ 维轴角。骨架结构大致是：

- 一条脊柱（从骨盆到颈）；
- 头、下颌；
- 两条前肢（肩 → 肘 → 腕 → 掌）；
- 两条后肢（髋 → 膝 → 踝 → 跖）；
- 一条两节的尾巴。

所有物种共享这一套关节拓扑。狮子、马、河马的区别，主要通过形状参数 $\boldsymbol{\beta}$ 表达，而不是改骨架。这就把“跨物种”变成了“在同一个形状空间里取不同的点”。

这个设计非常聪明：它让“狗”和“马”的网格顶点一一对应，于是可以做形状插值、动作迁移、跨物种的统计比较。你今天看到的“把狗的走路动作迁移到马身上”，底层就是这套对应。

22.4 形状空间：物种作为方向

SMAL 的形状空间是在玩具扫描上学的。每个玩具是一个物种、一个姿势，归一化后做 PCA，得到若干主成分。

这些主成分往往对应可解释的解剖差异：

- 第一主成分：体型大小 / 腿长比例；
- 第二主成分：躯干粗细；
- 第三主成分：头骨形状（吻部长短）；
- 后续主成分：耳朵、尾巴、角的细节。

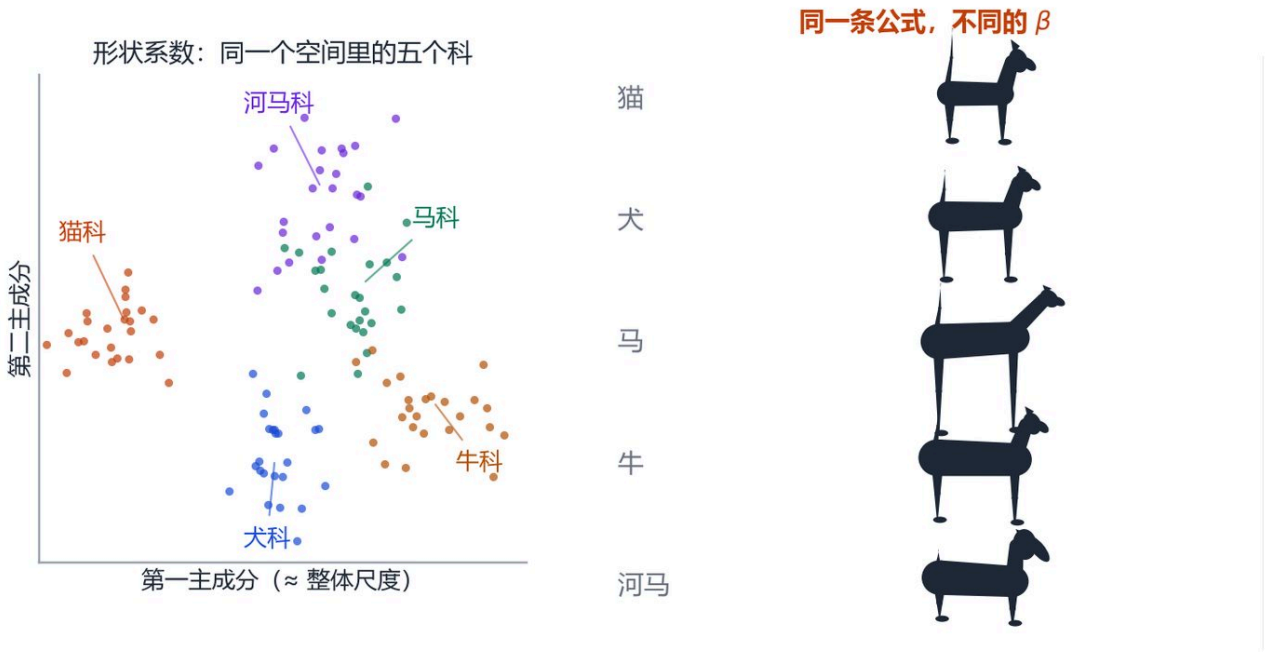


图 22-3 形状空间里的物种。左：不同科属的样本在二维主成分平面上的分布（示意）；右：同一套骨架、不同的形状系数，生成猫、犬、马、牛、河马。它们共享拓扑，差异体现在 β 上。

因为训练数据只有几十个玩具，SMAL 的形状空间维度不高，覆盖的物种也有限。遇到训练集里没有的物种（长颈鹿、大象、鲸），模型只能“外插”，结果往往不像。

22.5 配准到底在解什么

前面反复提到“把扫描配准到同一套顶点编号上”。这句话听起来简单，实际上是整条流水线里最难的一步。把它拆开看，配准要同时满足四个互相拉扯的要求：

1. 点到面的距离要小：每个顶点都应该落在扫描表面上，而不是飘在空气里；
2. 局部形状不能塌：相邻顶点之间的相对位置要保持，否则会在腿、耳朵这些细长部位“糊成一团”；

3. 顶点不能乱跑：配准是病态问题（很多种对应方式都能让距离很小），必须加正则；
4. 语义要对齐：狮子的左前腿必须配到模板的左前腿上，不能让两条腿交换。

写成优化问题，通常是这样一项一项加的：

$$E(\mathbf{V}) = \underbrace{\sum_i \left\| \mathbf{v}_i - \Pi_{\mathcal{S}}(\mathbf{v}_i) \right\|^2}_{\text{贴近表面}} + \lambda_{\text{arap}} \underbrace{\sum_{(i,j)\in\mathcal{E}} \left\| (\mathbf{v}_i - \mathbf{v}_j) - \mathbf{R}_i(\bar{\mathbf{v}}_i - \bar{\mathbf{v}}_j) \right\|^2}_{\text{局部刚性}} + \lambda_{\text{reg}} \underbrace{\left\| \mathbf{V} - \mathbf{V}_0 \right\|^2}_{\text{别跑太远}}, \tag{22.1}$$

其中 $\Pi_{\mathcal{S}}(\mathbf{v}_i)$ 是“把顶点投影到扫描表面上最近的点”，这一项只要求顶点靠近表面，不要求它落在哪个位置——这正是“对应未定”的来源。$\mathcal{E}$ 是网格的边集合，$\mathbf{R}_i$ 是第 i 个顶点附近的局部旋转，$\bar{\mathbf{v}}$ 是模板顶点。

中间那一项叫 ARAP（As-Rigid-As-Possible，尽可能刚性）。它的意思是：局部的一小片网格，允许整体旋转，但不允许被拉长、压扁。把 $\mathbf{R}_i$ 显式写出来，是为了让“旋转”不进入惩罚——如果不允许旋转，弯腿的时候膝盖内侧就会挨罚。

求解方式是交替优化（也叫局部-全局迭代）：

1. 固定顶点、解每个顶点的最优旋转 $\mathbf{R}_i$（这是一个小型的 SVD，闭式解）；
2. 固定旋转、解所有顶点位置（这是一个大型稀疏线性方程组）；
3. 重复，直到能量不再下降。



图 22-4 配准为什么难，以及怎么解。左：能量由四项加权相加——贴近表面、ARAP 局部刚性、正则、语义/初值；右：求解采用交替优化，局部步固定顶点解旋转（每顶点一个小 SVD），全局步固定旋转解顶点（一次稀疏线性求解）。

> **为什么这一步叫“对应”而不是“拟合”？** 拟合只要求形状像；对应还要求“第 3000 号顶点在狮子上和在马上是同一个解剖位置”。有了这个性质，才能把不同物种的顶点排成一列做 PCA，也才能把狗的动作迁移到马身上。

分段刚体拟合（GLoSS 一类方法）的作用是给 ARAP 一个好的初值：先把整只动物拆成头、躯干、四条腿、尾巴这些段，每段单独估一个刚体变换，再拼回全身。直接从“球”开始配准会陷入很差的局部极小值——比如把两条前腿配成一条。

22.6 SMAL 的参数与代码接口

如果你要用 SMAL，先要清楚它到底提供哪些数组。和 SMPL 一样，一个 SMAL 模型文件（通常是 `.pkl`）里有五张“表”：

字段	形状	含义
`v_template`	$N \times 3$	模板网格（四足站姿的平均形状）
`shapedirs`	$N \times 3 \times S$	形状主成分，每个主成分是一个 $N \times 3$ 的位移场
`posedirs`	$N \times 3 \times 9K$	姿态校正场（消除 LBS 的捏细）
`J_regressor`	$K \times N$	从顶点回归关节位置的稀疏矩阵
`weights`	$N \times K$	蒙皮权重，每行和为 1

前向计算与第 8 章的 SMPL 完全同构，只是把“人”换成“四足动物”：

$$M(\boldsymbol{\beta}, \boldsymbol{\theta}, \mathbf{t}) = W\Big(T_P(\boldsymbol{\beta}, \boldsymbol{\theta}),\, J(\boldsymbol{\beta}),\, \boldsymbol{\theta},\, \mathcal{W}\Big) + \mathbf{t}, \tag{22.2}$$

- $\boldsymbol{\beta}$：形状系数，控制物种与体型；
- $\boldsymbol{\theta} \in \mathbb{R}^{99}$：33 个关节各 3 维轴角；
- $\mathbf{t} \in \mathbb{R}^3$：整体平移（很多实现把全局旋转也放进 $\boldsymbol{\theta}$ 的第一个关节）。

实践上最常犯的三个错：

1. 忘记姿态是“相对父关节”的。SMAL 的 $\boldsymbol{\theta}$ 和 SMPL 一样是局部旋转，直接把世界坐标的朝向塞进去，结果会像被拧断。
2. 形状系数超出训练分布。$\boldsymbol{\beta}$ 的每一维在训练集上有方差，取到 $\pm 5\sigma$ 之外的数值，网格会自交、腿部翻面。第 9 章讲 SMPL 时说过的“参数空间先验”，在动物上更严格。
3. 单位与尺度。SMAL 的模板以米为单位，但网络图片里的动物尺度未知，单图重建时必须把“相机焦距 × 距离”与 $\mathbf{t}$ 一起估，否则会把大象估成猫。

22.7 从玩具到真实动物：域差距

用玩具训练、拿真实动物用，中间隔着三重域差距（domain gap）：

材质与反照率。 玩具是塑料的、颜色单一；真实动物有皮毛、鳞片、羽毛，表面反射性质完全不同。基于光度（渲染颜色要像原图）的损失在真实图像上会被材质骗过去。

姿态分布。 玩具摆出的姿势受关节阻尼限制，通常是“站、走、低头”这类温和姿态；真实的猫会蜷成球，真实的马会扬蹄。测试时的姿态落在训练分布之外，形状与姿态就会互相污染——这正是第 23 章“用 EM 迭代自修正”要解决的问题。

遮挡与背景。 玩具可以放在干净背景上单独拍摄；野生动物照片里，草、树枝、同伴都会挡住身体。轮廓损失在遮挡处会给出错误梯度，必须配合可见性掩码（visibility mask）使用。

> **一个有用的检查方法**：把你重建出来的网格渲染回图像，遮住原图的一半，只让人看另一半，问自己“这一半的轮廓对不对”。动物重建的错误通常集中在腿和尾巴——它们最细、最容易被遮挡、也最容易被轮廓损失忽略。

22.8 SMAL 的局限

第一，只有四足。 鸟类、鱼类、蛇类的骨架结构完全不同，SMAL 套不上。

第二，拓扑固定。 角的形状、象鼻、鲸鳍，这些“多出来”的结构没有对应的顶点。

第三，训练数据小。 41 个玩具扫描，覆盖的物种和姿势都有限；对真实动物的皮毛、肌肉，模型一无所知。

第四，形状与姿态仍有耦合。 玩具是刚性的，真实动物在奔跑时肌肉会变形，SMAL 的 LBS 无法表现。

尽管有这些局限，SMAL 仍然是动物参数化的起点。第 23 章的所有方法，几乎都以它为骨架或对照。

本章小结

- 动物参数化的难点在数据与对应，不在骨架。
- SMAL 用 41 个玩具扫描，经分段拟合、ARAP 细化、姿态归一化、PCA、联合配准构建。
- 配准要同时满足四个要求：贴近表面、局部刚性、不乱跑、语义对齐，标准做法是带 ARAP 项的交替优化。
- 模型文件里有五张表（模板、形状基、姿态校正基、关节回归器、蒙皮权重），前向与 SMPL 同构。
- 33 个关节、一套拓扑，跨物种差异由形状参数表达。
- 玩具到真实动物隔着材质、姿态分布、遮挡三重域差距；SMAL 只覆盖四足，拓扑固定，训练数据小。

思考题

1. 为什么用玩具而不是真实动物？这会带来什么偏差？
2. 33 个关节是如何分配的？如果让你加“象鼻”，会加在哪里？
3. 跨物种共享拓扑的好处与坏处各是什么？

4. 如果训练集只有狗和马，模型能生成猫吗？为什么？

第 23 章 动物重建：从 SMALST 到 MagicPony

> **本章导读**：有了 SMAL，下一步是“从一张动物照片反推它的形状与姿态”。这比人体难得多：没有关键点标注、姿态千奇百怪、皮毛还会伪装。这一章按时间讲四代方法：用轮廓和关键点弱监督（SMALST）、用 EM 闭环自我修正（Dogs Out）、用品种分类做先验（BARC）、直接从图片集合学铰接先验（MagicPony）。最后讲评估为什么比人体更棘手。

23.1 单图动物的四个难点

第一，没有关键点。 人体有 17 个关节的标注规范，动物没有。狮子的肩在哪、马的飞节在哪，不同标注者的定义都可能不同。

第二，姿态太自由。 人可以站、坐、走，动物可以跑、跳、飞、游、蜷缩、倒挂。SMAL 的姿态空间覆盖不了这么多。

第三，纹理伪装。 斑马条纹、豹纹、鹿的斑点，会让轮廓检测和光流都失效。

第四，尺度与相机耦合。 野生动物照片的焦距未知、距离未知，单张图无法确定“这只老虎多大”。

所以，动物重建的方法几乎都依赖弱监督：不要求 3D 真值，只用轮廓、2D 关键点、甚至只用“渲染出来像不像”。

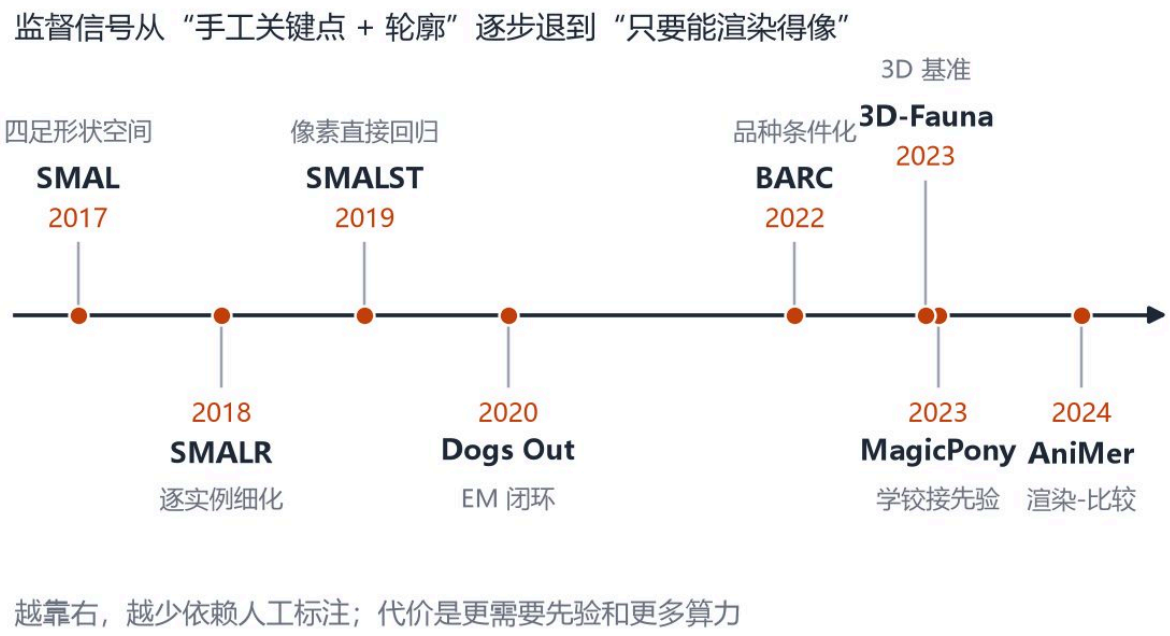


图 23-1 动物重建的监督信号演化。从 SMAL 的 3D 形状空间，到 SMALST 的轮廓/关键点，到 BARC 的品种分类，再到 MagicPony 的“只要求渲染相似”，人工标注越来越少，对先验与算力的要求越来越高。

23.2 SMALST：用轮廓和关键点做弱监督

SMALST（2019）[9] 是第一代“单图动物”方法。它的输入是一张图和一个包围框，输出 SMAL 的参数。

训练损失全部定义在 2D 上：

- 轮廓损失：渲染出的剪影与检测到的动物掩码做 IoU；
- 关键点损失：少量人工标注的 2D 关键点（如鼻尖、耳根、肩、髋）的重投影误差；
- 形状/姿态先验：限制参数不要跑到不合理的区域；
- 纹理损失：让渲染的颜色接近原图。

它不需要任何 3D 真值，因此可以用大量网络图片训练。缺点是：轮廓损失对“腿被挡住”“尾巴很细”不敏感，关键点又太少，导致深度和关节角经常估错。

23.3 Dogs Out：把拟合放进训练循环

Who Left the Dogs Out?（Biggs 等，2020）[88] 把 EM 算法用到了动物重建上：

1. E 步：用当前网络预测每张图的 SMAL 参数；
2. M 步：用这些参数与图像，更新形状/姿态先验；
3. 重复，直到收敛。

它的核心洞察是：与其手工设计先验，不如让模型从数据里“自己总结”先验。 在狗的数据上，这个方法能学到不同品种的体型分布，并把姿态估计得更稳。

23.4 BARC：品种作为先验

BARC（Biggs 等，2022）[89] 发现：狗的形状差异，很大程度上由品种决定。与其让网络从像素里猜“这是柯基还是哈士奇”，不如先做一个品种分类，再把品种信息作为条件输入形状回归。

这就是“Breed-Augmented Regression using Classification”：

- 分类分支预测品种（或品种混合）；
- 回归分支在品种条件下预测形状 β；
- 两者联合训练，分类的梯度帮助形状分支学到“品种 → 体型”的映射。

结果显示：显式的品种先验能显著改善稀有品种的重建，因为网络不再需要从少量样本里硬记体型。

23.5 MagicPony：从图片集合学“怎么动”

MagicPony（Wu 等，2023）[90] 走得更远：它不需要任何 3D 标注、不需要关键点、不需要相机参数，只用同一类动物的大量网络图片，就能学出一个可铰接的 3D 模型。

它的做法大致是：

1. 假设每张图里的动物都是同一个“规范形状”经过关节运动得到的；
2. 用可微渲染，把规范形状渲染到每张图的视角与姿态；
3. 用图像重建损失 + 关节正则，联合优化形状、关节、相机；
4. 训练好后，模型可以“动起来”，也可以迁移到新图片。

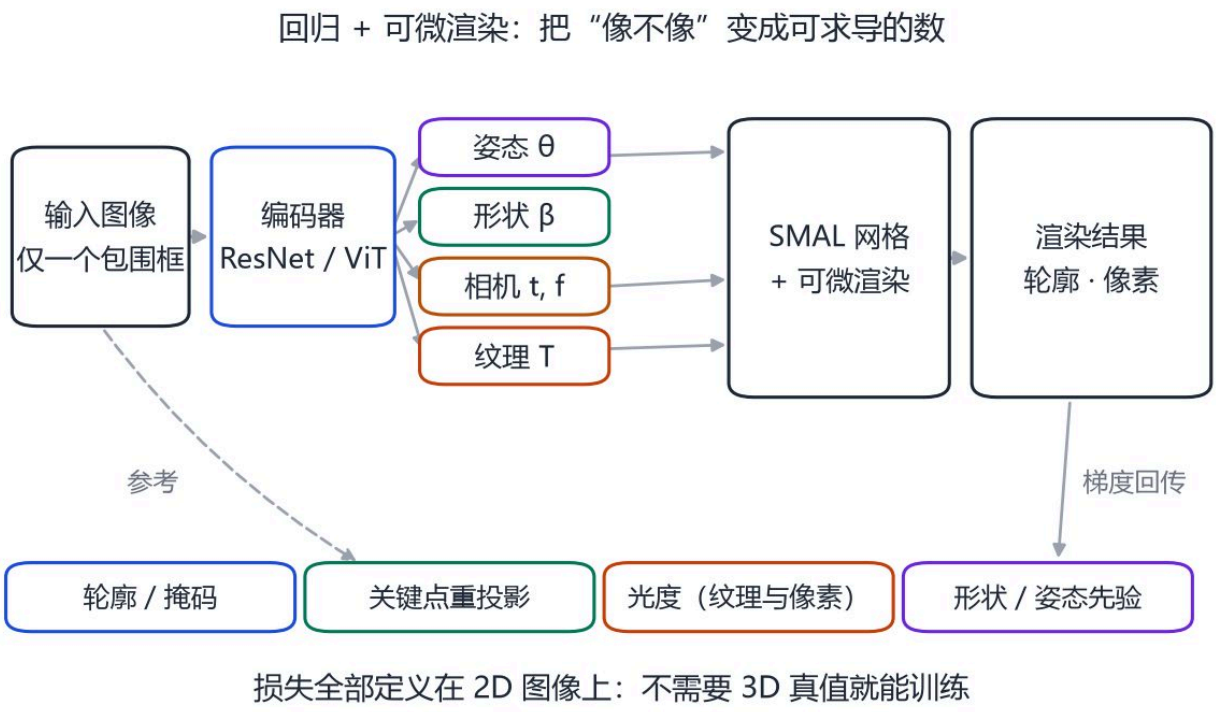


图 23-2 回归 + 可微渲染的动物重建管线。编码器输出姿态、形状、相机、纹理，渲染后与输入图比较；所有损失都在 2D 上定义，不需要 3D 真值。

MagicPony 的意义在于：它把“动物模型”的构建从“扫描 + 配准”变成了“图片集合 + 优化”。对于没有玩具、没有扫描的稀有物种，这是唯一可行的路。

23.6 数据与评估

动物 3D 数据的规模远小于人体：

- Animal Kingdom（2022）[91]：大规模动物行为视频数据集，主要用于行为识别，不是 3D。
- 3D-Fauna（2023）[92]：动物 3D 重建的数据集与基准，提供多物种的 3D 标注。
- SMAL 的玩具扫描：仍然是形状空间的主要来源。

评估比人体更棘手，因为“什么算对”没有共识。常用的三个量（图 23-3）：

1. PCK：关键点误差小于“躯干长度 × 阈值”的比例；
2. IoU：预测剪影与标注掩码的重叠；
3. 尺度误差：预测的米制尺寸与真实尺寸的差。

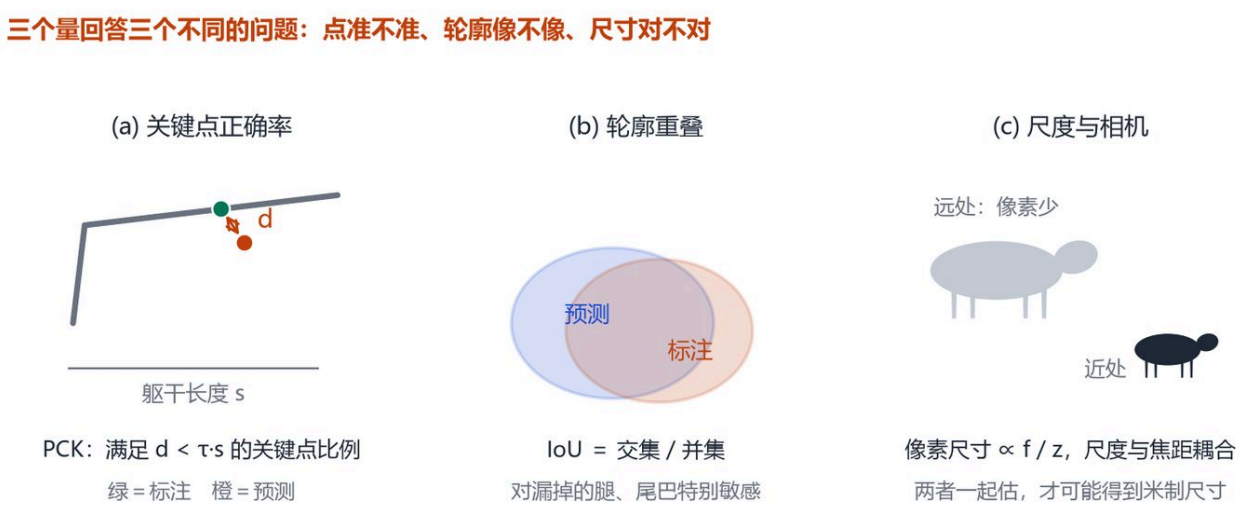


图 23-3 动物评估的三个侧面。PCK 看关键点准不准，IoU 看轮廓像不像，尺度看“尺寸对不对”。三者经常互相矛盾：轮廓很准的模型，可能把动物估得只有一半大。

23.7 把这些方法写成同一个优化式

SMALST、Dogs Out、BARC、MagicPony 看起来差别很大，但拆开看，它们解的是同一个问题：

$$\min_{\boldsymbol{\beta},\boldsymbol{\theta},\mathbf{t},\boldsymbol{\pi}} \underbrace{\lambda_{\text{sil}} E_{\text{sil}}}_{\text{轮廓}} + \underbrace{\lambda_{\text{kp}} E_{\text{kp}}}_{\text{关键点}} + \underbrace{\lambda_{\text{tex}} E_{\text{tex}}}_{\text{颜色}} + \underbrace{\lambda_{\beta}\|\boldsymbol{\beta}\|^2 + \lambda_{\theta}\|\boldsymbol{\theta}-\boldsymbol{\theta}_0\|^2}_{\text{先验}}, \tag{23.1}$$

四项的含义：

- E_{sil}：把网格渲染成剪影，与检测到的动物掩码比较，通常用 IoU 或 Dice；
- E_{kp}：少量 2D 关键点的重投影误差，$\|\pi(\mathbf{J}_k) - \mathbf{j}_k^{\text{obs}}\|^2$；
- E_{tex}：渲染出的颜色与原图的差（只在可见像素上算）；
- 先验：把 $\boldsymbol{\beta},\boldsymbol{\theta}$ 拉回“像动物”的区域。

四个方法的区别，全在这四项怎么加权、先验从哪里来：

方法	主要监督	先验从哪来	典型失败场景
SMALST	轮廓 + 稀疏关键点	手工先验（固定权重）	腿被遮挡时深度错
Dogs Out	轮廓 + 纹理	EM 迭代中从数据里学	跨物种（先验只对狗好）
BARC	轮廓 + 品种标签	品种分类条件化	混血品种、无品种标签的物种
MagicPony	图像重建（无需关键点）	同类图片集合学出的铰接先验	图片太少、姿态分布单一

一个常被忽略的细节：E_{sil} 对“腿在里面还是外面”几乎不敏感，因为剪影只关心像素在不在。这就是为什么动物重建的关节角经常“看上去对、实际上反折”——必须靠 E_{kp} 和姿态先验把角度约束回来。

23.8 尺度与相机：单图只能定到“乘积”

野生动物照片通常没有相机内参，也没有距离。这带来一个原理上的限制，值得单独说清楚。

针孔相机的投影是

$$u = f\frac{X}{Z}, \qquad v = f\frac{Y}{Z}. \tag{23.2}$$

如果把动物整体放大 k 倍（X, Y, Z 全部乘以 k），同时把 f 乘以 k，投影坐标 u, v 完全不变。这就是单目重建的尺度歧义：

- 你可以得到“k 米大的动物在 $k \cdot Z$ 米处”；
- 但无法区分“一只小猫在两米处”和“一只大猫在四米处”。

工程上的三个解法：

1. 给先验：假定物种的平均肩高（例如成年家猫约 25 cm），把尺度固定下来；这是最常用也最危险的——遇到幼崽或亚种就会系统性偏大/偏小；
2. 给参照物：图里有栅栏、车道、人体时，用参照物的已知尺寸反推；
3. 多视图或视频：相机移动时，三角化能恢复绝对尺度（与第 11 章人体度量尺度完全同一个道理）。

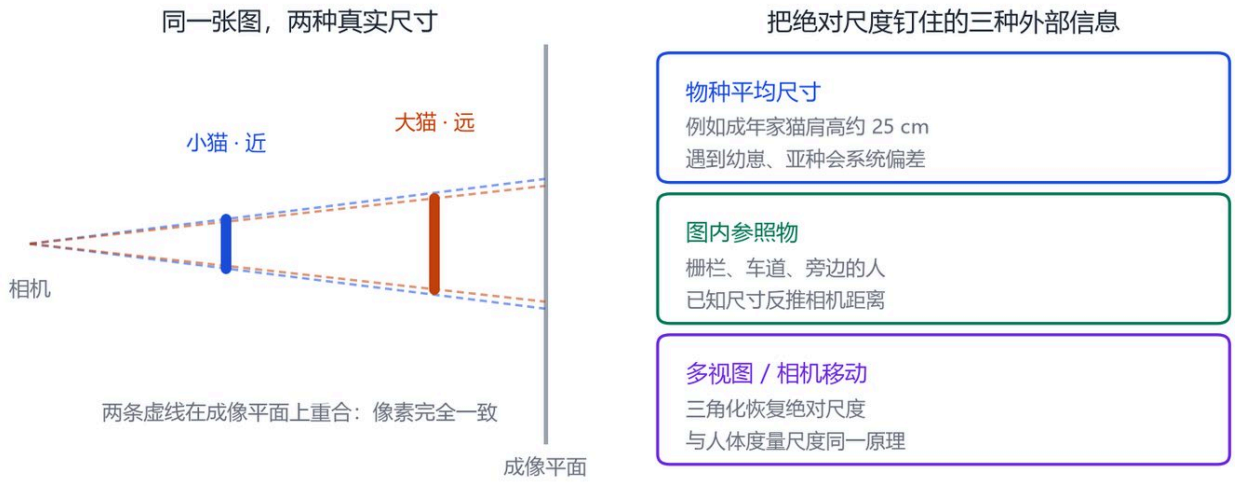


图 23-4 尺度歧义。左：小动物在近处与大动物在远处投影到同一张图，像素完全一致；右：真实应用里只能靠“物种平均尺寸、参照物、相机移动”三种外部信息之一把尺度钉住。

23.9 失败模式

遮挡：草丛里的狮子，只露出头，模型会把身体补成“最小可能体积”。

稀有姿态：倒挂的蝙蝠、飞翔的鹰，SMAL 的姿态空间里没有。

纹理伪装：斑马条纹会让轮廓检测把“条纹”当成“边界”。

物种外插：训练集只有四足，遇到长颈鹿、大象、鲸，形状空间只能外插，结果往往比例失调。

尺度歧义：没有相机内参，模型只能猜“这是一只小猫还是远处的大猫”。

本章小结

- 动物重建依赖弱监督：轮廓、关键点、渲染相似度。
- 四个方法可以写成同一个优化式，区别在权重与先验的来源：SMALST（手工先验）、Dogs Out（EM 学先验）、BARC（品种条件）、MagicPony（图片集合学铰接）。
- 剪影损失对关节角不敏感，必须配关键点与姿态先验，否则会出现“轮廓对、关节反折”。
- 单图存在尺度歧义：只能定到“尺寸 × 距离”的乘积，需要物种先验、参照物或多视图才能钉住绝对尺度。
- 评估看 PCK、IoU、尺度，三者经常矛盾。
- 遮挡、稀有姿态、伪装、物种外插、尺度歧义是主要失败模式。

思考题

1. 为什么动物重建很少用 3D 真值？如果要做，成本在哪里？
2. BARC 的“品种分类”会不会引入偏见？比如把混血狗强行归到某个品种。
3. MagicPony 不需要关键点，它靠什么避免“所有动物都塌成同一个形状”？
4. 如果只给你 100 张长颈鹿的图片，你会怎么建模？

第 24 章 跨物种：鸟类、鱼类与动物生成

本章导读：SMAL 只覆盖四足[87]。世界上还有鸟、鱼、蛇、鲸、昆虫。这一章讨论三件事：不同拓扑的动物如何参数化、跨物种模型如何面对“没见过的物种”、以及生成模型如何被用来“造”动物。最后回到应用与伦理：野生动物保护、畜牧、影视，都需要可信的动物模型。

24.1 从四足到鸟类、鱼类



图 24-1 三种拓扑。四足（SMAL）有 33 个关节、一条脊柱、四条腿、一条尾巴；鸟类的前肢变成翅膀，多出一段“腕掌”；鱼类没有四肢，身体靠脊柱连续弯曲。拓扑不同，参数化方式必须重新设计。

鸟类的挑战在于翅膀。鸟的翅膀在解剖上对应前肢，但骨骼数量、关节方向、羽毛结构都不同：

- 翅膀有一段额外的“腕掌骨”，用来折叠；
- 羽毛没有骨骼支撑，却决定了轮廓；
- 喙和尾羽没有对应的骨骼，SMAL 的顶点无处安放。

所以鸟类模型通常需要：一套专门的翅膀骨架、一层“羽毛壳”（用额外的面片/粒子表示），以及一个能处理“非刚体”的形变模型。

鱼类的挑战更根本：鱼没有四肢，身体是一整条连续的脊柱。LBS 的“刚体段 + 关节”假设在这里几乎失效：

- 鱼的弯曲是连续的，不是几个关节的旋转；
- 鳍是薄片，会随水流摆动；
- 鳞片、鳃盖的运动，需要更细的形变。

可行的做法是把鱼表示成“脊柱曲线 + 截面放样”：给定一条三维曲线和沿曲线的半径/截面，生成身体表面。这样，“姿态”就是曲线的控制点，“形状”就是截面的参数。

把它写成式子会更清楚。设脊柱曲线为 $\mathbf{c}(s)$（$s \in [0,1]$ 是沿身体的参数），在每一点取三个方向：切向 $\mathbf{t}(s)$、法向 $\mathbf{n}(s)$、副法向 $\mathbf{b}(s)$，它们构成沿曲线的局部坐标系（也叫标架）。若截面在局部坐标下是 $\mathbf{q}(\varphi) = (0,\, r(s)\cos\varphi,\, r(s)\sin\varphi)$，则身体表面为

$$\mathbf{S}(s,\varphi) = \mathbf{c}(s) + r(s)\cos\varphi\,\mathbf{n}(s) + r(s)\sin\varphi\,\mathbf{b}(s). \tag{24.1}$$

这个式子把所有东西拆得很干净：

- 形状由两个函数决定——中心线 $\mathbf{c}(s)$ 与半径 $r(s)$；
- 姿态就是改变 $\mathbf{c}(s)$ 的弯曲与扭转；
- 想加鳍、鱼鳍的薄片，就在某些 s 上额外贴一片面片，不需要改整体拓扑。

代价是：截面只能是“一圈圈套在中心线上”的形状，脸部、鳍根这种局部细节表达不了；而且曲线一变，所有顶点的编号就跟着变——这正好丢掉了 SMAL 最宝贵的东西：跨个体的顶点对应。所以“曲线参数化”通常只用于单个体重建，不适合建统计形状空间。

> **两条路线的取舍可以总结成一句话**：想要统计（PCA、跨物种插值、动作迁移），就必须固定拓扑、做顶点对应；想要任意拓扑（鱼、蛇、鸟），就只能放弃对应，改用隐式表示。目前还没有同时拿到两者的成熟方案。

24.2 跨物种：共享什么，不共享什么

跨物种模型的核心问题是：哪些东西可以共享，哪些必须分开？

- 可以共享：形状空间的统计方法（PCA）、蒙皮的计算框架、可微渲染、关键点定义的部分语义（头、躯干、四肢）。
- 不能共享：拓扑（顶点数、连接关系）、关节数量与层级、局部解剖结构（角、象鼻、翅膀、鳍）。

SMAL 的选择是“共享拓扑 + 共享骨架”，所以只能覆盖四足。要覆盖鸟类/鱼类，只有三条路：

1. 多套模型：四足用 SMAL、鸟用 BirdModel、鱼用 FishModel，互不兼容；
2. 超模型：设计一套“万能拓扑”，用额外的顶点/关节表示物种特有的结构。难度极大；

3. 隐式/神经表示：不做顶点对应，用隐式场或神经场表示形状，用骨架/曲线控制运动。这是目前最活跃的方向。

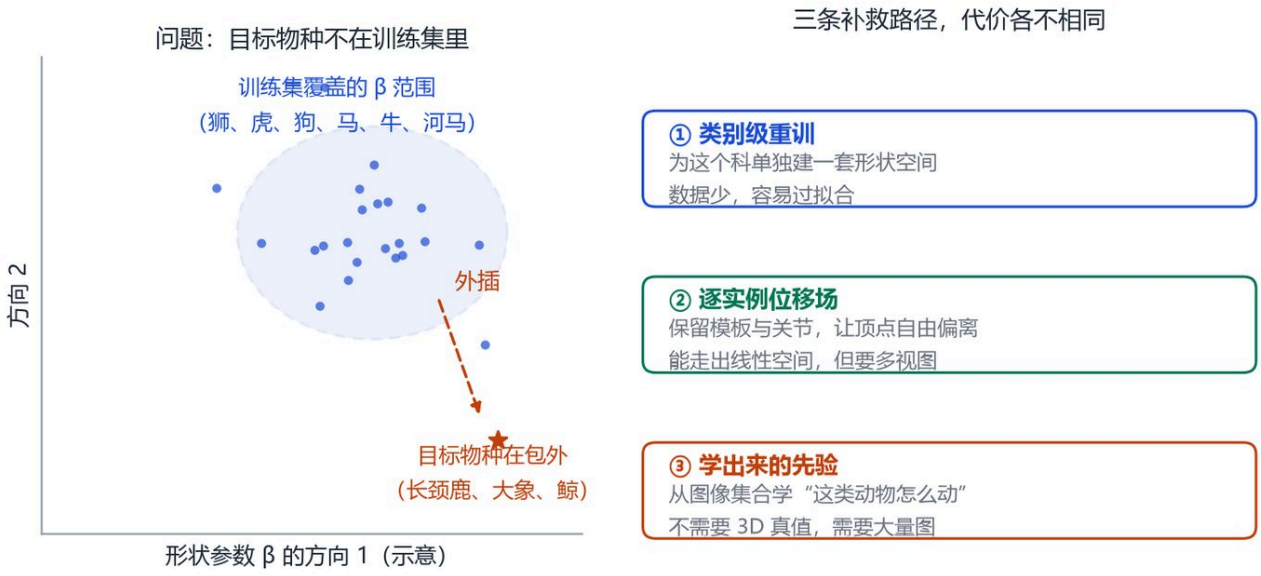


图 24-2 目标物种不在训练集里时的三条补救路径：类别级重训（数据少、易过拟合）、逐实例位移场（能走出线性空间，但需要多视图）、学出来的先验（不需要 3D 真值，但需要大量图）。

24.3 任务阶梯：从单图到生成



图 24-3 动物任务的四级阶梯。L1 单图静态网格，L2 多视图更准形状，L3 视频 4D，L4 任意物种与动作的生成。越往上，越依赖先验而不是真值。

L1 单图 → 静态网格：SMALST、BARC、AniMer 这一类[@zuffi2021smalst,@biggs2022barc,@zuffi2024animer]。输入一张图，输出 SMAL 参数。适合“拍一张照片，估个体型”。

L2 多视图 → 更准形状：同一只动物的多张图，利用光度一致性约束形状。SMALR、3D-Fauna 属于这一类[92]。

L3 视频 → 4D：从视频恢复形状 + 运动。需要时序、光流、关节轨迹。BARC、LiveStock 的流水线可以扩展到视频[@biggs2022barc,@li2024livestock]。

L4 生成 → 任意物种与动作：用大规模图文/视频先验，生成没见过的物种、没见过的动作。扩散模型 + 参数化骨架是目前的组合。

24.4 生成动物：能做什么，不能做什么

生成模型在动物上的应用有三类：

1. 文本/图像到 3D 动物：用 SDS 或 3DGS，从文字/图片生成动物模型。适合影视、游戏的概念设计。
2. 动作生成：给定“一只狗在跑”，生成符合解剖的动作。难点是四足步态、接触、平衡。
3. 物种插值：在形状空间里插值，生成“狮虎兽”式的混合体。科学上不严谨，但视觉上有用。

不能做的也很明显：

- 解剖正确性：生成模型不懂“这块骨头应该连到哪里”，容易生成反关节、多余肢体。
- 物理合理性：脚会滑、身体会穿模。
- 个体识别：生成的是“一类动物”，不是“这一只”。

24.5 应用与伦理

野生动物保护：用相机陷阱（camera trap）的图像估计种群数量、体型、健康状况。动物参数化模型可以提供“可比较的体型指标”，比人工测量更客观。

畜牧：牛、猪、羊的体重估测、跛行检测、体况评分。LiveStock 这类数据集就是为此准备的。

影视与游戏：快速生成可信的动物角色、动作重定向、群集动画。

伦理风险：生成逼真的动物视频可能被用于虚假宣传、非法野生动物贸易；相机陷阱数据涉及隐私与地点泄露。技术本身中性，使用需要规范。

24.6 四足步态：为什么“会动”比“长得像”更难

形状重建的目标是“静止时像”；而一旦要让它走路，就会碰到一套完全不同的约束。四足动物的步态（gait）本质上是四条腿的相位关系：

步态	相位关系	典型速度	能量特征
行走 walk	四条腿依次落地，任意时刻至少三足着地	慢	稳定性最好
小跑 trot	对角两足同时落地（左前 + 右后）	中	像弹簧，动能与势能交替
袭步 gallop	前腿先后落地，中间有腾空期	快	背部弯曲储能

把步态写成参数，就是给每条腿一个相位偏移 ϕ_i 与周期 T：

$$\theta_{\text{leg}\,i}(t) = \theta_{\text{leg}}\big(2\pi(t/T + \phi_i)\big), \qquad i = 1, \dots, 4. \tag{24.2}$$

小跑时对角腿同相（$\phi_1 = \phi_4$，$\phi_2 = \phi_3$，两组相差半个周期）；行走时四个相位均匀分布。只要把相位关系定对，即使关节角是从动作捕捉里搬来的，看上去也会“像在跑”而不是“在飘”。

真正难的是接触。脚不能穿地、着地时不能滑动、身体重心必须在支撑多边形内（否则就会倒）。这三条约束写进优化里，通常长这样：

$$E_{\text{contact}} = \sum_{t}\sum_{i} \mathbf{1}[\text{着地}_{i,t}] \Big(\underbrace{\|\mathbf{p}_{i,t} - \mathbf{p}_{i,t+1}\|^2}_{\text{不滑}} + \underbrace{\max(0, -z_{i,t})^2}_{\text{不穿地}} \Big). \tag{24.3}$$

这里 $\mathbf{p}_{i,t}$ 是第 i 只脚在第 t 帧的世界坐标，z 是高度（地面为 0）。$\mathbf{1}[\cdot]$ 是“只对着地的帧算”——所以还得先估一个“哪只脚着地”的开关变量，这又是一个鸡生蛋问题：接触靠动作定，动作又靠接触约束。工程上一般交替优化，或者用一个小的分类头直接从图像/姿态预测接触。

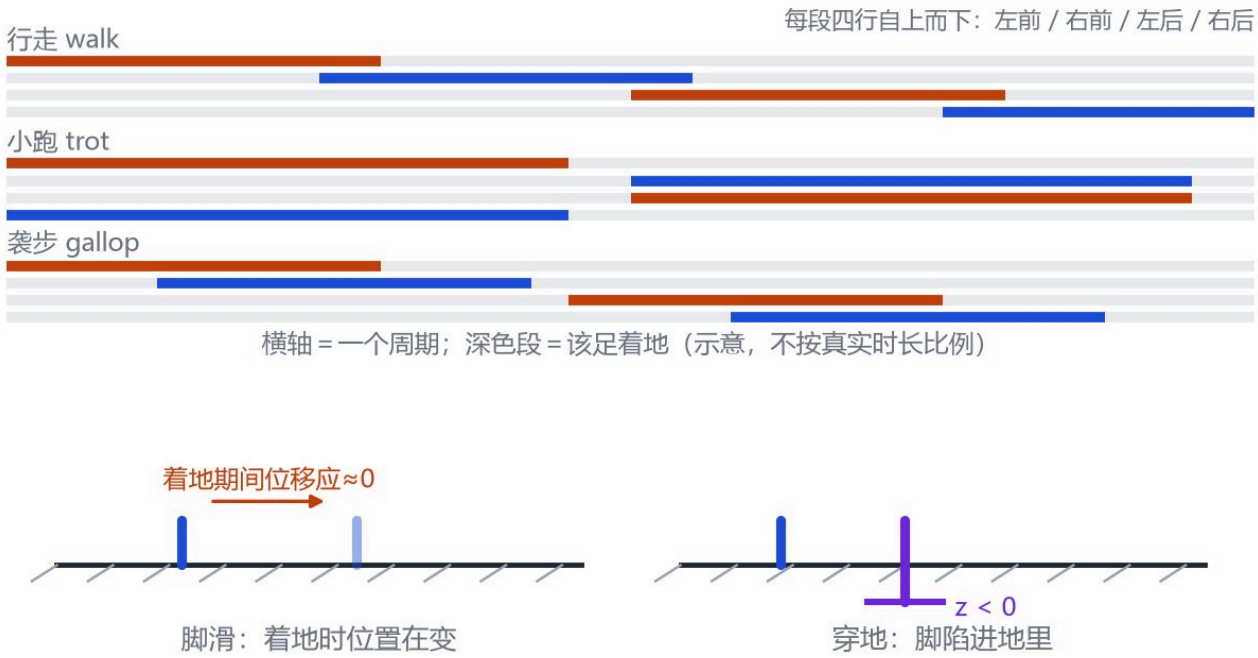


图 24-4 步态与接触。上：三种步态的四足着地时序（深色为着地）；下：脚滑与穿地是两类不同错误——前者是“着地时位置在变”，后者是“脚陷进地里”，必须分别约束。

24.7 把一个新物种接进现有流水线：检查清单

假设你手上已经有一套跑通的四足动物重建代码，现在想支持“长颈鹿”。按下面六步排查，能避开大部分坑：

1. 拓扑能不能用：长颈鹿的脖子占身体一半，SMAL 的脊柱节数表达不了。要么接受“短脖子四足”的近似，要么换带更长脊柱的模型。
2. 关节数对不对：新增的颈节意味着 $\boldsymbol{\theta}$ 维度变化，蒙皮权重也得重新学——不能只改关节数不改权重。
3. 形状空间支不支持：如果新物种在训练集的形状空间里只能“外插”，先检查外插幅度（用 $\boldsymbol{\beta}$ 的先验方差归一化后看多少 σ）。超过 4σ 的结果基本不能信。
4. 关键点语义：能复用哪些点（肩、髀、膝、蹄）？哪些需要新定义（角、颞背、尾巴分节）？标注规范不一致会让评估全乱。
5. 尺度先验：给一个物种的平均肩高与体长区间，把单目重建的尺度钉在一个合理范围。
6. 评估集：至少要 20–30 张图的人工标注（轮廓 + 少量关键点），否则无法判断“是不是变好了”。

> **一个现实建议**：如果你的目标只是“让这个物种看起来能动”，用生成模型或动画重定向会快得多；只有在需要**可比较的体型指标**（保护、畜牧）时，才值得花力气做参数化重建。

24.8 开放问题

- 统一拓扑：能否设计一套“可扩展”的拓扑，让新物种只加顶点、不改连接？
- 少样本：一个物种只有几十张图，如何学出可信的 3D？
- 运动先验：四足步态、鸟类振翅、鱼类摆尾，能否用一个统一的动力学模型描述？
- 评估：没有 3D 真值，如何公平比较不同方法？
- 跨域：玩具上训练的模型，如何迁移到真实动物？

本章小结

- 鸟类、鱼类的拓扑与四足不同，不能直接套 SMAL。
- 连续弯曲的身体可以用“脊柱曲线 + 截面放样”表示，但会丢掉顶点对应，不适合建统计形状空间。
- 跨物种可共享统计方法与渲染框架，但不能共享拓扑与关节。
- 四足“会动”的关键是相位关系与接触约束：不滑、不穿地、重心在支撑多边形内。
- 任务分四级：单图、多视图、视频、生成；越往上越依赖先验。
- 生成动物能用于影视、游戏，但解剖与物理正确性仍不足。

- 保护、畜牧、影视是主要应用，伦理风险需要重视。

思考题

1. 如果让你设计一个“鸟类参数化模型”，你会怎么表示羽毛？
2. 鱼类没有关节，LBS 还能用吗？如果不能，替代方案是什么？
3. 少样本物种建模，你会优先加数据还是加先验？为什么？
4. 生成动物的“解剖正确性”可以怎样评估？

第六篇　前沿与实战

生成模型、基础模型，以及亲手跑一遍

第 25 章　生成式人体：从扩散到基础模型
第 26 章　动手实践：从 PCA 到拟合
第 27 章　研究脉络与未来

第 25 章 生成式人体：从扩散到基础模型

> **本章导读**：2020 年之后，人体领域出现三条新线索：神经渲染让外观达到照片级，扩散模型让“用文字生成人”成为可能，视觉基础模型让“从任意图像提取人体”变得更鲁棒。这一章不追求罗列所有新论文，而是把这三条线索讲清楚，并给出一个“感知—参数—生成—应用”的四层框架。读完你会知道，今天的“人体基础模型”到底由哪些零件拼成。

25.1 三条趋势

趋势一：表示从“网格”走向“神经场/高斯”。 NeRF[93]、3DGS[70] 能表示头发、布料、皮肤细节，突破了固定拓扑的限制。

趋势二：生成从“判别”走向“扩散”。 以前是“给图预测参数”，现在是“从噪声生成参数/外观”，可以无条件生成，也可以用文本、音频、骨架控制。

趋势三：感知从“专用网络”走向“基础模型”。 SAM 分割[94]、DINOv2 特征[95]、深度大模型[96]、视觉语言模型，提供了强大的通用先验，人体系统越来越多地“站在巨人的肩膀上”。

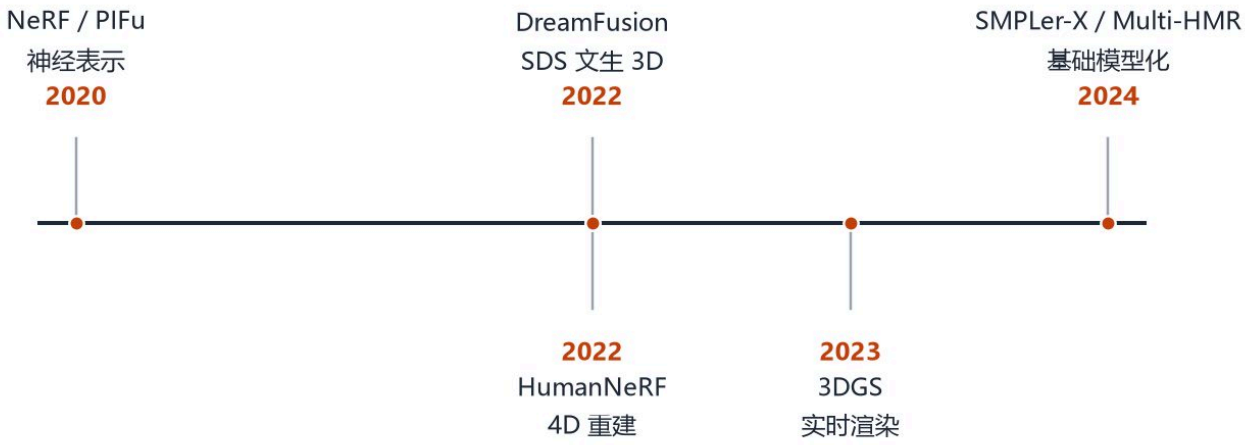


图 25-1 2020 年之后的关键节点。神经表示、扩散生成、3DGS、基础模型四条线交织，最终汇入“人体基础模型”。

25.2 扩散模型：零基础版

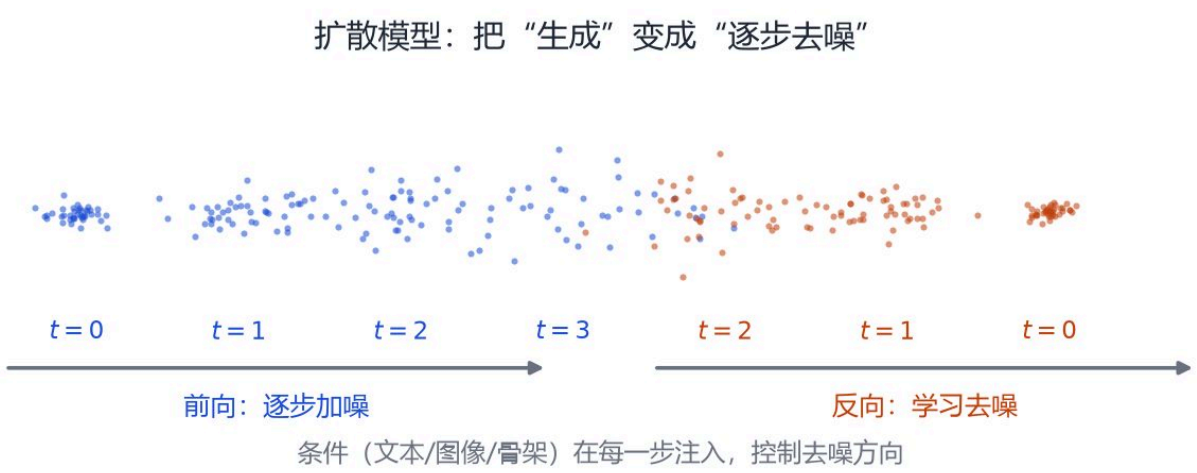


图 25-2 扩散模型的前向与反向过程。前向把数据逐步加噪，直到变成纯高斯噪声；反向学习一个去噪网络，从噪声一步步恢复数据。条件信息在每一步注入。

扩散模型的思想可以用一句话概括：先学会“加噪”，再学会“去噪”。

前向过程：给定一张图（或一组人体参数）$\mathbf{x}_0$，每一步加一点高斯噪声：

$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{25.1}$$

加 T 步之后，$\mathbf{x}_T$ 几乎就是纯噪声。这个过程没有可学参数，是固定的。

反向过程：训练一个网络 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$，输入“带噪数据 + 时间步”，输出“这一步加进去的噪声”。损失就是

$$L = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right\|^2. \tag{25.2}$$

训练好之后，从纯噪声 $\mathbf{x}_T$ 开始，反复“预测噪声 → 减去一部分 → 加一点点新噪声”，就能生成一张新图。

条件生成：如果想“按文本生成”，把文本编码也喂给去噪网络；如果想“按骨架生成”，把骨架作为额外通道。条件的作用是控制去噪方向，让最终结果符合要求。

> **为什么扩散模型适合人体？** 因为人体参数空间（SMPL 的 82 维、SMPL-X 的一百多维）比像素空间小得多，扩散模型容易训练；而且生成的是参数，天然可驱动、可重定向。

25.3 文本到 3D：SDS 与它的后代

2022 年，DreamFusion[97] 提出 Score Distillation Sampling（SDS）：不需要 3D 训练数据，只用一个预训练的 2D 扩散模型，就能优化一个 3D 表示（NeRF、网格、高斯），使它从任意视角渲染出

来都“像文本描述的东西”。

做法是：

1. 随机采一个视角，渲染当前 3D 表示；
2. 把渲染图加噪，喂给 2D 扩散模型；
3. 扩散模型告诉你“这张图应该往哪个方向改”；
4. 把这个梯度反传回 3D 表示，更新它。

SDS 的意义在于：它把 2D 生成模型的知识“蒸馏”到 3D。对人体的应用是：用文本描述“一个穿红裙子的女人在跳舞”，生成对应的 3D 人体/动作。但 SDS 容易产生“多面体”伪影，且优化慢。

后来的工作用 3DGS 替代 NeRF、用视频扩散替代图像扩散、用人体参数化模型做先验，逐步改善了质量与速度。

25.4 3DGS：实时的照片级化身

3D Gaussian Splatting 用一堆高斯椭球表示场景，每个高斯有位置、协方差、颜色、透明度。渲染时把它们投影到屏幕、排序、混合，速度极快。

在人体上，3DGS 有两种用法：

- 静态重建：多视角拍摄一个人，重建出可自由视角观看的高斯化身；
- 可驱动化身：把高斯绑定到 SMPL/FLAME 的面片上，人一动，高斯跟着动（SplattingAvatar、GaussianAvatars）。

它的挑战是：

1. 泛化：换姿势、换表情时，高斯会拉伸、露洞；
2. 存储：一个高质量化身可能有几十万到几百万个高斯；
3. 编辑：想换衣服、换发型，需要重新训练或额外解耦。

25.5 基础模型如何改变人体重建

SAM 提供“任意物体分割”，可以自动把人从背景里抠出来，减少对检测框的依赖。

DINOv2 提供强大的自监督视觉特征，替代 ImageNet 预训练的 ResNet，显著提升跨域泛化。

深度大模型（Depth Anything、Metric3D）提供单目深度/度量深度，帮助解决尺度歧义。

视觉语言模型（CLIP、LLaVA 等）提供“文本—图像”对齐，用于文本驱动的编辑、检索、生成。

这些模型不是为人体设计的，但它们提供了“通用感知”，让 人体系统可以把精力放在“人体特有的先验”上。



图 25-3 现代人体系统的四层栈。感知层提供通用特征，参数层提供人体先验，生成层负责细节与多样性，应用层面向具体场景。参数层是承上启下的“共同语言”。

25.6 一个正在形成的“人体基础模型”

把上面的线索拼起来，可以看到一个正在成形的架构：

1. 输入：图像/视频/文本/音频；
2. 感知：基础模型提取特征、分割、深度；
3. 参数化：回归 SMPL-X/FLAME/MANO 参数（身体、脸、手）；
4. 生成：扩散/3DGS 补细节、补外观、补动作；
5. 输出：可驱动、可编辑、可渲染的化身。

这个架构的关键是参数层：它把“千变万化的人”压缩成一组标准参数，让不同模块可以对接。没有参数层，每个模块都要自己定义“人”，系统会碎成一地。

25.7 开放问题

数据：高质量 4D 人体数据仍然稀缺；合成数据有域差异；网络数据有隐私与版权问题。

评估：照片级真实感、物理合理性、身份一致性、可控性，缺少统一指标。

效率：扩散模型慢、3DGS 占内存、大模型推理贵；实时数字人仍然需要大量工程优化。

伦理：Deepfake、身份盗用、虚假视频；参数化模型让“换脸/换身”变得更容易，监管与检测必须跟上。

通用性：儿童、老人、孕妇、残障人士、不同种族，现有模型的覆盖仍然不足。

本章小结

- 三条趋势：神经表示、扩散生成、基础模型。
- 扩散模型学习“加噪—去噪”，条件控制生成方向；SDS 把 2D 扩散知识蒸馏到 3D。
- 3DGS 用高斯椭球实现实时渲染，可绑定到参数化模型上做可驱动化身。
- SAM、DINOv2、深度大模型提供通用感知，人体系统越来越“站在巨人肩膀上”。
- 四层栈：感知 → 参数 → 生成 → 应用；参数层是共同语言。
- 数据、评估、效率、伦理、通用性是主要开放问题。

思考题

1. 为什么扩散模型在“参数空间”上训练比在“像素空间”上更容易？
2. SDS 的梯度来自 2D 扩散模型，它为什么能让 3D 表示变得“像文本”？
3. 3DGS 化身在换姿势时为什么会露洞？有哪些补救办法？
4. 如果基础模型已经能分割、测深度，参数化人体模型还有存在的必要吗？

第 26 章 动手实践：从 PCA 到拟合

> **本章导读**：这一章把前 25 章的核心公式变成可以运行的代码。我们刻意不依赖 SMPL 模型文件（它需要申请许可），而是用 14 个二维关键点搭一个“迷你人体”，完整走一遍 PCA、蒙皮、拟合。所有代码都在 `scripts/ch26_demo.py` 里，跑一遍只要几秒。读完你应该能自己改参数、加约束、换数据。

26.1 环境准备

只需要三样东西：

```
python -m pip install numpy matplotlib scipy
```

本书使用的 Python 是 3.12。代码不依赖 GPU，也不依赖任何深度学习框架。

26.2 实验一：用 PCA 学一个“迷你形状空间”

先定义一个人：14 个关键点（头、颈、双肩、双肘、双腕、双髋、双膝、双踝），由 7 个参数控制：身高、肩宽、髋宽、臂长、腿长、手臂张开角、腿张开角。

```
def body(params):
    h, sw, hw, al, ll, pa, pl = params
    ...
    return np.array([head, neck, sh_l, sh_r, el_l, el_r, wr_l, wr_r,
                     hip_l, hip_r, kn_l, kn_r, an_l, an_r])
```

随机采样 300 个人，把每个人展平成 28 维向量，做 PCA：

```
def pca(X, k=3):
    mu = X.mean(0)                  # 均值
    Xc = X - mu                     # 中心化
    C = Xc.T @ Xc / (len(X) - 1)    # 协方差
    w, V = np.linalg.eigh(C)        # 特征分解
    idx = np.argsort(w)[::-1]
    return mu, V[:, idx][:, :k], w[idx][:k]
```

把第一主成分的位移画出来，就是图 26-1 左边那组箭头：所有关键点一起向上，代表“整体变高”。第二主成分则让手臂和腿的末端向外，代表“四肢比例变长”。

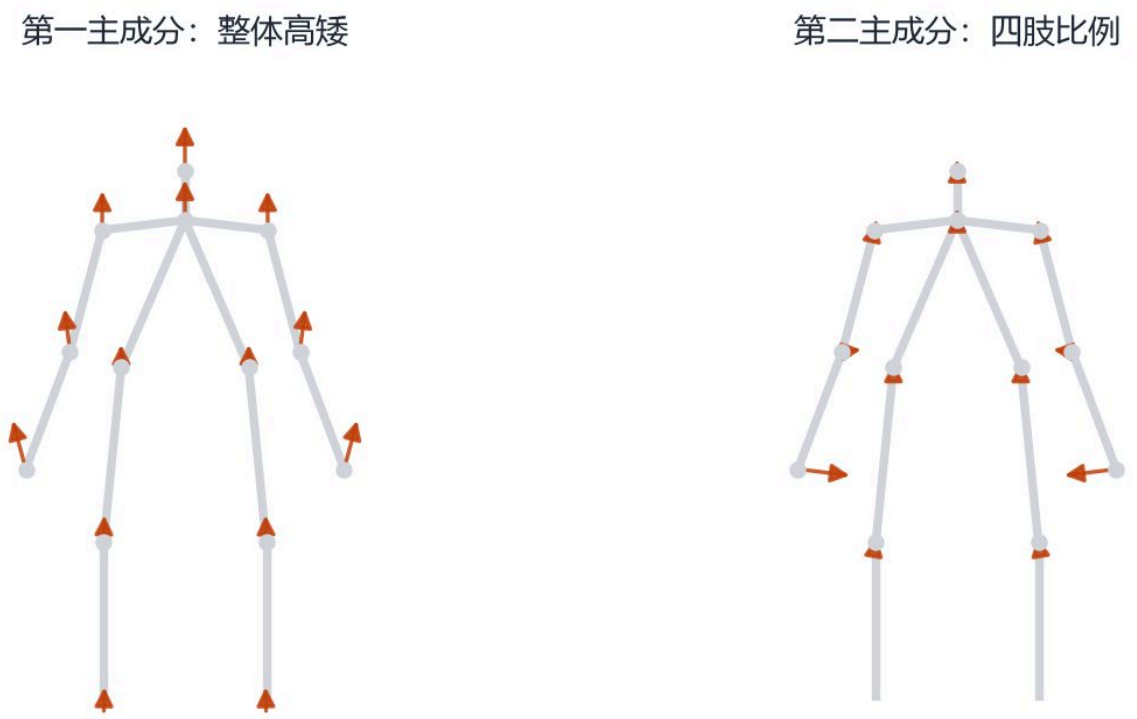


图 26-1 从 300 个随机样本里学出的主成分。灰色是均值骨架，橙色箭头是关键点在 $+2\sigma$ 时的位移。左：整体高矮；右：四肢比例。

> **你可以改的地方**：把采样标准差从 0.06 改成 0.15，看主成分是否还是“高矮”；把样本数从 300 改成 20，看基向量是否变得杂乱。

26.3 实验二：二十行实现 LBS

下面是一个二维、两根骨头的 LBS，足够看清公式的每个部分。

```
import numpy as np

def rot2(a):
    c, s = np.cos(a), np.sin(a)
    return np.array([[c, -s], [s, c]])

def lbs(points, weights, rotations, pivots):
    """
    points:  (N, 2) 静止顶点
    weights: (N, 2) 每个顶点对两根骨头的权重，行和为 1
    rotations: list of 2 个旋转角（弧度）
    pivots: list of 2 个关节位置 (2,)
    """
    out = np.zeros_like(points)
    for k in range(2):
        R = rot2(rotations[k])
        p = pivots[k]
        # 先把顶点搬到关节局部坐标，旋转，再搬回来
        moved = (points - p) @ R.T + p
        out += weights[:, k:k+1] * moved
    return out
```

注意 `(points - p) @ R.T + p` 就是 $\mathbf{G}_k\mathbf{G}_k(0)^{-1}\mathbf{t}$ 在二维、无平移时的样子：先减去关节位置（进入局部坐标），乘旋转，再加回关节位置。三维版本只是把 2×2 换成 4×4。

如果你把 `weights` 设成 0/1 硬切换，弯曲处会撕裂；设成 0.5/0.5，弯曲处会变细。这就是第 5 章讲的体积损失。

26.4 实验三：从 2D 关键点拟合形状

假设我们观测到一组“目标关键点”，想反推它属于形状空间里的哪个位置。这就是一个最小二乘问题：

```
from scipy.optimize import least_squares

def fit(target, mu, P, w):
    def resid(z):
        return (mu + P @ z - target.ravel())
    z0 = np.zeros(P.shape[1])
    # 限制在 ±3 个标准差内，防止外插出“外星人”
    bounds = (-3 * np.sqrt(w), 3 * np.sqrt(w))
    sol = least_squares(resid, z0, bounds=bounds)
    return sol.x
```

`P` 是主成分矩阵，`mu` 是均值，`z` 是要求解的系数。`least_squares` 会自动选初值、算雅可比、迭代到收敛。

运行结果见图 26-2：蓝色是拟合结果，橙色是目标，平均误差不到 1 厘米（按身高 1.7 米折算）。误差最大的点在肩膀和手肘，因为这两个位置对“臂长”和“张开角”最敏感，而这两个参数在数据里相

关性很强，存在歧义。

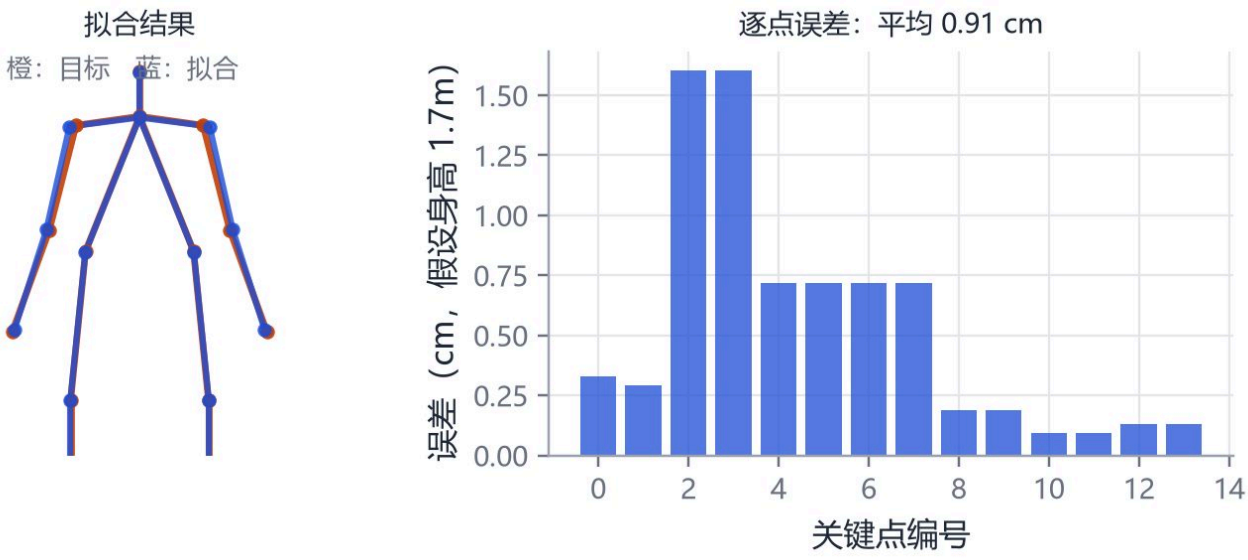


图 26-2 拟合结果。左：蓝色拟合与橙色目标几乎重合；右：逐点误差，肩、肘处最大。这不是代码写错了，而是参数之间存在歧义——不同的臂长 + 张开角组合可以产生几乎相同的关键点。

> **这就是“病态问题”**：数据只能约束某些方向，另一些方向几乎不受约束。解决办法是加先验（正则项）或加更多视角。

26.5 换成真实的 SMPL

真实 SMPL 的使用流程与上面完全一样，只是把“14 个关键点”换成“6890 个顶点”，把“7 个参数”换成“10 + 72 个参数”。

26.5.1 获取模型

SMPL、SMPL-H、SMPL-X、MANO、FLAME、SMAL 都需要在各自官网注册并同意许可协议，不能随意分发。官方入口：

- SMPL：`smpl.is.tue.mpg.de`
- SMPL-X：`smpl-x.is.tue.mpg.de`
- MANO：`mano.is.tue.mpg.de`
- FLAME：`flame.is.tue.mpg.de`
- SMAL：`smal.is.tue.mpg.de`

下载后会得到 `.pkl` 或 `.npz` 文件，里面包含模板顶点、形状基、姿势基、关节回归器、蒙皮权重。

26.5.2 用 smplx 库加载

```
import smplx, torch

model = smplx.create("models/", model_type="smplx", gender="neutral",
                     use_pca=False, batch_size=1)
out = model(betas=torch.zeros(1, 10),
            expression=torch.zeros(1, 10),
            body_pose=torch.zeros(1, 63),
            global_orient=torch.zeros(1, 3),
            left_hand_pose=torch.zeros(1, 45),
            right_hand_pose=torch.zeros(1, 45))
verts = out.vertices          # (1, 10475, 3)
joints = out.joints           # (1, 54, 3)
```

注意几个坑：

- `body_pose` 是 63 维（21 个关节 × 3），不含根关节；
- `global_orient` 单独给；
- 手部姿态默认是 PCA 空间，`use_pca=False` 时是 45 维轴角；
- 坐标系是 y 向上、面朝 $+z$，渲染前先确认。

26.5.3 一个最小的渲染

不想装 PyTorch3D 的话，可以用 matplotlib 的 3D 散点/三角面：

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

V = verts[0].detach().numpy()
F = model.faces
ax = plt.figure(figsize=(6, 8)).add_subplot(projection="3d")
ax.add_collection3d(Poly3DCollection(V[F], alpha=0.3, facecolor="tan"))
ax.set_xlim(V[:,0].min(), V[:,0].max())
ax.set_ylim(V[:,2].min(), V[:,2].max())
ax.set_zlim(V[:,1].min(), V[:,1].max())
ax.view_init(elev=10, azim=-60)
```

这段代码很慢，只适合检查模型是否加载正确。要做实时渲染，用 PyTorch3D、nvdiffrast 或 Blender。

26.6 三个可以立刻做的练习

练习一：把 LBS 改成三维。把 `rot2` 换成 `scipy.spatial.transform.Rotation`，把关节位置换成三维，观察弯曲时体积如何变化。

练习二：给拟合加正则。 在 `resid` 里加上 `0.1 * z`，看误差和参数范围如何变化。这就是第 2 章的岭回归。

练习三：加一个“关节限位”。 给每个角度加 `bounds`，比如肘部只能向一个方向弯，观察拟合结果是否更合理。

本章小结

- PCA、LBS、最小二乘都可以用几十行 NumPy 实现。
- 拟合的本质是“在形状空间里找一组系数，使重建关键点与观测最接近”。
- 参数之间存在歧义时，拟合误差会在某些点集中，这不是 bug，是问题的病态性。
- 真实 SMPL 的使用流程相同，只是维度更大、需要申请模型许可。
- 先在小数据上跑通，再换大模型，是学习任何几何库的正确顺序。

思考题

1. 把 `least_squares` 的初值从 0 改成 3σ，结果会变吗？为什么？
2. 如果在 LBS 里把权重之和改成 1.2，模型会发生什么？
3. 为什么真实 SMPL 的 `body_pose` 是 63 维而不是 72 维？
4. 你能设计一个“只看轮廓、不看关键点”的拟合损失吗？

第 27 章 研究脉络与未来

本章导读：这是全书的最后一章。我们不再介绍新模型，而是把四条主线放回同一张地图，看看它们从哪里来、现在走到哪、接下来可能往哪走。最后给准备进入这个领域的读者一些具体建议：读什么、怎么读、怎么做出自己的东西。

27.1 四次范式转移



图 27-1 人体参数化技术的四次范式转移。每一次都把前一次的成果“封装”成新系统的一层，而不是抛弃它。

第一次（1995–2005）：统计形状。 ASM、AAM 证明了“从样本里学形状先验”可行；3DMM 把它带到三维人脸；SCAPE 把形状与姿态分开。这一阶段的关键词是对应 + PCA。

第二次（2005–2015）：参数化 + 蒙皮。 SMPL 用 LBS + blendshape 把模型做得又简单又可微；FAUST 提供了评测基准。这一阶段的关键词是实时 + 可微 + 标准格式。

第三次（2016–2020）：学习回归。 SMPLify 用优化从图像反推参数；HMR 用网络直接回归；SPIN 把两者结合；DECA、FrankMocap 把同样思路带到脸和手。这一阶段的关键词是图像 → 参数。

第四次（2021 至今）：神经与生成。 NeRF、3DGS 把外观做到照片级；扩散模型让生成变得可控；SAM、DINOv2、深度大模型提供通用感知。这一阶段的关键词是细节 + 生成 + 基础模型。

值得注意的是：每一次转移都没有让前一次“过时”。今天最先进的系统，内部仍然在跑 PCA、LBS、最小二乘，只是外面包了神经网络和神经渲染。

27.2 不变的三角：表示、先验、拟合

把 27 章的内容压缩，你会发现所有方法都在处理同一个三角：

1. 表示：用什么描述人？网格、参数、隐式场、高斯。
2. 先验：什么才算“像人”？PCA 的形状空间、姿态先验、物理约束、扩散模型的分布。
3. 拟合：如何从观测推回表示？优化、回归、或两者的结合。

三者互相制约：

- 表示越灵活，先验越难写，拟合越容易过拟合；
- 先验越强，拟合越稳，但可能生成“平均脸”、丢失个性；
- 拟合越快，通常越依赖前两者的简化。

今天的研究创新，本质上都是在三角的某一条边上做改进：换一个更强的表示（3DGS）、学一个更好的先验（扩散）、设计一个更快的拟合（Transformer）。

27.3 未来可能的方向

统一模型。 身体、脸、手、动物目前还是四套模型。未来可能出现一个“全物种、全年龄、全表情”的统一参数化模型，用同一套骨架和形状空间覆盖人、猿、狗、鸟。SMAL 已经展示了跨物种的可行性，但离“统一”还很远。

数据与标注。 高质量 4D 数据仍然稀缺。合成数据、自监督、视频自标注是三条可能的路。谁能低成本地获得“带对应、带物理、带语义”的数据，谁就能训练出更好的模型。

物理与接触。 今天的人体模型大多不懂物理：脚会滑、手会穿模、衣服不会飘。把刚体、软体、布料仿真与参数化模型结合，是数字人走向“可信”的关键。

交互与行为。 从“这个人在做什么动作”到“他为什么做、接下来要做什么”，需要把动作、物体、场景、语言结合起来。手物交互是第一步，长时序任务规划是下一步。

可控与可解释。 生成模型越强，越需要“可编辑”：改一个属性，不要连带改变其他属性。解耦表示（身份/表情/姿态/光照/衣服）仍是核心难题。

伦理与安全。 参数化模型让“换脸、换身、换动作”变得廉价。检测、水印、授权、法规，会和生成技术一起演进。

27.4 给读者的建议

第一，先动手，再读论文。 把第 26 章的代码跑一遍，改几个参数，比读十篇综述更有用。

第二，沿着一条主线读。 不要从 1995 年读到 2025 年，而是选一个对象（比如人脸），把它的 5–8 篇关键论文读透，再横向扩展到其他对象。

第三，复现一个最小的实验。 不需要复现 SOTA，复现一个 loss、一个模块、一个小数据集上的结果，就足以理解论文的“手感”。

第四，关注评估，而不只是模型。 一个方法的价值，一半在它解决的问题，一半在它如何被证明解决了问题。学会看对齐方式、真值来源、失败案例。

第五，保持对“人”的理解。 参数化模型再复杂，描述的还是人的形状与运动。解剖、比例、姿态习惯、文化差异，这些“非技术”的知识，往往决定模型是否可信。

27.5 结语

从 1995 年 ASM 的几十个轮廓点，到今天能实时驱动、照片级渲染、用文字生成的人体基础模型，这个领域走了三十年。它的核心思想却异常朴素：

> **用少数参数，描述一个高维的形状；用统计先验，约束不合理的解；用可微的管线，把模型和图像连起来。**

如果你读完了这本书，并且能用自己的话解释这句话，那么你已经具备了进入这个领域的基础。剩下的，就是选一个你真正感兴趣的问题，动手做。

本章小结

- 四次范式转移：统计形状 → 参数化蒙皮 → 学习回归 → 神经与生成。
- 所有方法都在处理“表示—先验—拟合”的三角。
- 未来方向：统一模型、数据、物理、交互、可控、伦理。
- 学习建议：动手、沿主线、复现最小实验、重视评估、理解人。

思考题

1. 如果让你预测 2030 年的人体模型，它会是什么样？哪些今天的限制会被解决？
2. “统一模型”最大的技术障碍是什么？是数据、架构，还是评估？
3. 为什么“物理约束”在视频里比单帧更重要？
4. 选一个你最感兴趣的应用（游戏、医疗、影视、健身），画出它需要的技术栈。

附录 A 数学补充

> 本附录把正文用到但没展开的几个工具集中讲一遍：SVD 的几何意义、矩阵求导、概率的几条基本规则、四元数插值。它们不是“必须背下来”的公式，而是遇到问题时可以回来查的参考。

A.1 SVD 的几何意义

任意矩阵 $\mathbf{X}\in\mathbb{R}^{m\times n}$ 都可以分解为

$$\mathbf{X}=\mathbf{U}\mathbf{S}\mathbf{V}^{\top},\tag{A.1}$$

其中 $\mathbf{U}$ 的列是 m 维正交向量，$\mathbf{V}$ 的列是 n 维正交向量，$\mathbf{S}$ 是“对角”矩阵，对角线上是奇异值 $s_1\ge s_2\ge\cdots\ge 0$。

几何上，$\mathbf{V}^{\top}$ 先旋转输入空间，$\mathbf{S}$ 沿坐标轴拉伸/压缩，$\mathbf{U}$ 再旋转到输出空间。所以任何线性变换都可以看成“旋转—拉伸—旋转”。

对中心化的数据矩阵 $\tilde{\mathbf{X}}$（每行一个样本），SVD 与 PCA 的关系是：

$$\mathbf{V}=[\mathbf{p}_1,\dots,\mathbf{p}_n],\qquad \lambda_i=\frac{s_i^2}{n-1}.\tag{A.2}$$

也就是说，右奇异向量就是主成分，奇异值的平方除以 $n-1$ 就是特征值。这就是为什么 `sklearn` 的 PCA 默认用 SVD 实现。

A.2 矩阵求导的三条规则

只要记住三条，就够读懂本书的推导。

规则一：线性项。

$$\frac{\partial}{\partial\mathbf{x}}(\mathbf{a}^{\top}\mathbf{x})=\mathbf{a}.\tag{A.3}$$

规则二：二次型。

$$\frac{\partial}{\partial\mathbf{x}}(\mathbf{x}^{\top}\mathbf{A}\mathbf{x})=(\mathbf{A}+\mathbf{A}^{\top})\mathbf{x}.\tag{A.4}$$

如果 $\mathbf{A}$ 对称，就是 $2\mathbf{A}\mathbf{x}$。

规则三：最小二乘。

$$\frac{\partial}{\partial \mathbf{w}} \|\mathbf{A}\mathbf{w}-\mathbf{y}\|^2 = 2\mathbf{A}^\top(\mathbf{A}\mathbf{w}-\mathbf{y}). \tag{A.5}$$

令其为 0，就得到正规方程 $\mathbf{A}^\top\mathbf{A}\mathbf{w}=\mathbf{A}^\top\mathbf{y}$。

A.3 概率的三条规则

加法规则（互斥事件）：

$$p(A \text{ 或 } B) = p(A) + p(B). \tag{A.6}$$

乘法规则：

$$p(A, B) = p(A \mid B)\,p(B). \tag{A.7}$$

贝叶斯公式：

$$p(A \mid B) = \frac{p(B \mid A)\,p(A)}{p(B)}. \tag{A.8}$$

在拟合问题里，A 是“参数”，B 是“观测”。贝叶斯公式告诉我们：后验 $\propto$ 似然 $\times$ 先验。第 10 章的姿态先验、第 3 章的高斯模型，都是这条公式的实例。

A.4 四元数插值

单位四元数 $q = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2}\mathbf{n})$ 表示绕轴 $\mathbf{n}$ 转 θ。两个四元数之间的插值用球面线性插值（SLERP）：

$$\operatorname{slerp}(q_0, q_1, t) = \frac{\sin((1-t)\Omega)}{\sin\Omega} q_0 + \frac{\sin(t\Omega)}{\sin\Omega} q_1, \tag{A.9}$$

其中 $\Omega = \arccos(q_0 \cdot q_1)$ 是两个四元数的夹角。它保证插值过程中的角速度均匀，不会像欧拉角那样“忽快忽慢”。

A.5 常用希腊字母与含义

符号	读法	本书含义
α, β, γ	alpha, beta, gamma	形状系数、角度
θ	theta	姿态参数（轴角）
λ	lambda	特征值、损失权重
μ	mu	均值
σ	sigma	标准差
ψ	psi	表情参数

符号	读法	本书含义
Ω	Omega	角速度、旋转矩阵
π	pi	相机参数
ϵ	epsilon	噪声

A.6 推荐的自学顺序

1. 3Blue1Brown 的《线性代数的本质》（建立直觉）；
2. 任何一本工科线性代数教材的前四章（矩阵、秩、特征值）；
3. 概率论的前三章（随机变量、期望、方差）；
4. 把第 26 章的代码跑一遍，改参数；
5. 回来读第 3、5、8 章。

数学不是看会的，是用会的。每读一个公式，试着用 NumPy 写一遍，比抄十遍笔记有用。

附录 B 术语中英对照

> 按拼音/字母混排，方便查阅。括号里是本书第一次出现时的章节。

中文	英文	一句话解释
主动形状模型	Active Shape Model (ASM)	用 PCA 约束关键点搜索的经典方法（第 6 章）
主动外观模型	Active Appearance Model (AAM)	同时建模形状与纹理（第 6 章）
仿射变换	affine transformation	线性变换 + 平移，保持平行线
轴角	axis-angle	用“单位轴 + 转角”表示旋转（第 4 章）
绑定姿势	bind pose / rest pose	蒙皮前的参考姿势（第 5 章）
混合权重	blend weights	每个顶点受各关节影响的权重（第 5 章）
blendshape	blend shape	用一组预定义形变的线性组合表示表情/形状（第 6、8 章）
相机内参	camera intrinsics	焦距、主点、畸变（第 4 章）
相机外参	camera extrinsics	相机在世界坐标中的旋转与平移（第 4 章）
蒙皮	skinning	让顶点跟随骨架变形（第 5 章）
线性混合蒙皮	Linear Blend Skinning (LBS)	对多个刚体变换做加权平均（第 5 章）
对偶四元数蒙皮	Dual Quaternion Skinning (DQS)	用四元数插值替代矩阵插值，缓解体积损失
糖纸效应	candy wrapper effect	扭转时肢体被拧细的伪影（第 5 章）
协方差	covariance	两个变量“一起变”的程度（第 3 章）
协方差矩阵	covariance matrix	所有两两协方差排成的对称矩阵（第 3 章）
主成分分析	Principal Component Analysis (PCA)	找方差最大方向的降维方法（第 3 章）
特征向量	eigenvector	被矩阵作用后方向不变的方向（第 2、3 章）
特征值	eigenvalue	特征向量方向上的缩放倍数/方差（第 3 章）
奇异值分解	Singular Value Decomposition (SVD)	任意矩阵分解为旋转—缩放—旋转（第 2 章）

中文	英文	一句话解释
对应	correspondence	不同网格顶点之间的解剖级一一对应（第 6 章）
非刚性配准	non-rigid ICP	让模板变形去贴合扫描的迭代方法（第 6 章）
三维形变模型	3D Morphable Model (3DMM)	人脸形状/纹理的 PCA 参数化模型（第 6、15 章）
双线性模型	bilinear model	用两组系数分别控制身份与表情（第 15 章）
表情基	expression basis	描述表情变化的向量基（第 16 章）
关节回归	joint regression	从形状顶点回归关节位置（第 8 章）
前向运动学	forward kinematics (FK)	由关节角算出关节世界位置（第 5 章）
逆运动学	inverse kinematics (IK)	由末端位置反推关节角
重投影误差	reprojection error	三维点投影到二维后与观测的差（第 4、10 章）
可微渲染	differentiable rendering	对渲染过程求导，把图像误差反传到几何（第 4 章）
分析—合成	analysis-by-synthesis	反复“渲染—比较—更新”的拟合思路（第 10 章）
弱透视相机	weak-perspective camera	用缩放+平移近似透视，忽略深度引起的尺寸变化（第 10 章）
度量尺度	metric scale	以米为单位的真实尺寸（第 11 章）
世界坐标	world coordinates	以场景（重力/地面）为参考的坐标（第 11 章）
足部接触	foot contact	脚是否踩地，用于恢复世界位移（第 11 章）
单阶段多人	one-stage multi-person	一次前向输出所有人参数（第 11 章）
自顶向下	top-down	先检测再逐人拟合（第 11 章）
平均关节位置误差	MPJPE	关节位置的平均欧氏误差（第 12 章）
Procrustes 对齐	Procrustes alignment	去掉旋转/缩放/平移后再比较（第 12 章）
每顶点误差	PVE	网格顶点平均误差（第 12 章）
正确关键点比例	PCK	误差小于阈值的关节占比（第 12 章）
加速度误差	acceleration error	衡量动作抖动的指标（第 11、12 章）
神经辐射场	Neural Radiance Field (NeRF)	用 MLP 表示空间颜色与密度（第 13、25 章）
3D 高斯泼溅	3D Gaussian Splatting (3DGS)	用高斯椭球表示场景，实时渲染（第 13、25 章）
有符号距离场	Signed Distance Field (SDF)	每个点到表面的有符号距离（第 13 章）

中文	英文	一句话解释
隐式表面	implicit surface	用函数零等值面表示表面（第 13 章）
扩散模型	diffusion model	学习“加噪—去噪”的生成模型（第 25 章）
分数蒸馏采样	Score Distillation Sampling (SDS)	用 2D 扩散模型指导 3D 优化（第 25 章）
变分自编码器	Variational Autoencoder (VAE)	学习低维隐空间的生成模型（第 14、25 章）
姿态先验	pose prior	限制姿态“像人”的概率模型（第 10、14 章）
运动先验	motion prior	限制动作连续、符合动力学的模型（第 14 章）
重定向	retargeting	把动作从一套骨架迁移到另一套（第 14 章）
数字人	digital human / avatar	可驱动、可渲染的虚拟人（第 13、18 章）
化身	avatar	同上，常指带外观的个体化模型
服饰位移	clothing displacement	衣服相对裸体表面的偏移（第 13 章）
参数化	parametric	用少量参数描述高维对象
拓扑	topology	顶点/边/面的连接关系（第 4 章）
流形	manifold	局部像欧氏空间的曲面（第 4 章）
测地距离	geodesic distance	沿表面的最短路径长度（第 5 章）
薄板样条	Thin Plate Spline (TPS)	一种平滑的非刚性变换（第 6 章）
高斯混合模型	Gaussian Mixture Model (GMM)	多个高斯叠加的概率模型（第 10 章）
岭回归	ridge regression	带 L2 正则的最小二乘（第 2 章）
伪逆	pseudo-inverse	非方阵/奇异矩阵的“广义逆”（第 2 章）

附录 C 论文时间线与导读

本表按年份列出书中讨论的论文，方便你按时间顺序补课。标有引用编号的，表示正文引用过，可在附录 D 找到完整条目。

C.1 怎么用这张表

1. 先看你关心的对象（人体/人脸/人手/动物），找到它的 3–5 个关键节点；
2. 按年份从早到晚读，注意每篇的“前人局限”；
3. 读到不懂的术语，回附录 B 查；
4. 想复现，先看论文的项目页是否提供代码与模型。

C.2 时间线

年份	论文	发表
2014	FAUST: Dataset and Evaluation for 3D Mesh Registration [15]	CVPR
2014	Generative Adversarial Networks	NeurIPS
2014	Auto-Encoding Variational Bayes	ICLR
2014	MoSh: Motion and Shape Capture from Sparse Markers	ECCV
2015	SMPL: A Skinned Multi-Person Linear Model [5]	SIGGRAPH Asia
2016	Keep It SMPL: Automatic Estimation of 3D Human Pose and Shape from a Single Image [6]	ECCV
2016	Face2Face: Real-time Face Capture and Reenactment of RGB Videos [67]	CVPR
2017	Disentangling Factors of Variation by Mixing Them	SIGGRAPH
2017	Learning a model of facial shape and expression from 4D scans [46]	SIGGRAPH Asia
2017	Embodied Hands: Modeling and Capturing Hands and Bodies Together [16]	SIGGRAPH Asia

年份	论文	发表
2017	MoFA: Model-based Deep Convolutional Face Autoencoder for Unsupervised Monocular Reconstruction [50]	CVPR
2017	BodyNet: Volumetric Inference of 3D Human Body Shapes	ICCV
2017	3D Menagerie: Modeling the 3D Shape and Pose of Animals [87]	CVPR
2018	Video Based Reconstruction of 3D People Models	CVPR
2018	3D Face Morphable Models "In-the-Wild" [52]	ECCV
2018	Total Capture: A 3D Deformation Model for Tracking Faces, Hands, and Bodies	CVPR
2018	End-to-end Recovery of Human Shape and Pose [7]	CVPR
2018	Deep Video Portraits [68]	CVPR
2018	Gaussian Process Morphable Models	TPAMI
2018	Neural Body Fitting: Unifying Deep Learning and Model Based Human Pose and Shape Estimation	ECCV
2019	Tex2Shape: Detailed Full Human Body Geometry from a Single Image [34]	ICCV
2019	3D Morphable Models as Spatial Transformer Networks	ICCV
2019	3D Hand Shape and Pose From Images in the Wild [73]	CVPR
2019	Accurate 3D Face Reconstruction with Weakly-Supervised Learning: From Robust Face Parsing to Dense Landmarks	CVPR
2019	GANFIT: Generative Adversarial Network Fitting for High Fidelity 3D Face Reconstruction [51]	CVPR
2019	Learning Joint Reconstruction of Hands and Manipulated Objects [81]	CVPR
2019	Learning 3D Human Dynamics from Video	CVPR
2019	Convolutional Mesh Regression for Single-Image Human Shape Reconstruction [22]	ICCV
2019	Learning to Reconstruct 3D Human Pose and Shape via Model-fitting in the Loop [8]	ICCV
2019	AMASS: Archive of Motion Capture as Surface Shapes [32]	ICCV
2019	Expressive Body Capture: 3D Hands, Face, and Body from a Single Image [21]	CVPR

年份	论文	发表
2019	Monocular Total Capture: Posing Face, Body, and Hands in the Wild	CVPR
2019	Three-D Safari: Learning to Estimate Zebra Pose, Shape, and Texture from Images "In the Wild"	ICCV
2020	Pushing the Envelope for RGB-based Dense 3D Hand Pose Estimation via Neural Rendering	CVPR
2020	Who Left the Dogs Out? 3D Animal Reconstruction with Expectation Maximization in the Loop [88]	ECCV
2020	3D Morphable Face Models -- Past, Present and Future [45]	IJCV
2020	HOnnotate: A method for 3D Annotation of Hand and Object Poses [80]	CVPR
2020	Denoising Diffusion Probabilistic Models	NeurIPS
2020	VIBE: Video Inference for Human Body Pose and Shape Estimation [23]	CVPR
2020	AvatarMe: Realistically Renderable 3D Facial Reconstruction "in-the-wild" [59]	CVPR
2020	Learning to Dress 3D People in Generative Clothing [33]	CVPR
2020	NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis [93]	ECCV
2020	I2L-MeshNet: Image-to-Lixel Prediction Network for Accurate 3D Human Pose and Mesh Estimation from a Single RGB Image [77]	ECCV
2020	STAR: Sparse Trained Articulated Human Body Regressor [17]	ECCV
2020	GRAB: A Dataset of Whole-Body Human Grasping of Objects [82]	CVPR
2020	StyleRig: Rigging StyleGAN for 3D Control over Portrait Images [69]	CVPR
2020	FaceScape: A Large-Scale High Quality 3D Face Dataset and Detailed Riggable 3D Face Prediction [49]	TPAMI
2021	Emerging Properties in Self-Supervised Vision Transformers	ICCV
2021	DexYCB: A Benchmark for Capturing Hand Grasping of Objects [75]	CVPR
2021	Learning an Animatable Detailed 3D Face Model from In-The-Wild Images [53]	SIGGRAPH
2021	AvatarMe++: Facial Shape and BRDF Inference with Photorealistic Rendering-Aware GANs [60]	TPAMI
2021	Mesh Graphormer [78]	ICCV

年份	论文	发表
2021	The Power of Points for Modeling Humans in Clothing	ICCV
2021	PyMAF: 3D Human Pose and Shape Regression with Pyramidal Mesh Alignment Feedback Loop [79]	ICCV
2021	HuMoR: 3D Human Motion Model for Robust Pose Estimation [42]	ICCV
2021	FrankMocap: A Monocular 3D Whole-Body Pose Estimation System via Regression and Integration [76]	ICCV
2021	RAFT: Recurrent All-Pairs Field Transforms for Optical Flow	ECCV
2022	EMOCA: Emotion Driven Monocular Face Capture and Animation [54]	CVPR
2022	FaceVerse: a Fine-grained and Detail-controllable 3D Face Morphable Model from a Hybrid Dataset [58]	CVPR
2022	MotionDiffuse: Text-Driven Human Motion Generation with Diffusion Model [44]	AAAI
2022	HumanNeRF: Free-viewpoint Rendering of Moving People from Monocular Video [37]	CVPR
2022	Human Motion Diffusion Model [43]	ICLR
2022	Text2Mesh: Text-Driven Neural Stylization for Meshes	CVPR
2022	ArtiBoost: Boosting Articulated 3D Hand-Object Pose Estimation via Online Exploration and Synthesis [85]	CVPR
2022	Animal Kingdom: A Large and Diverse Dataset for Animal Behavior Understanding [91]	ECCV
2023	SMPLer-X: Scaling Up Expressive Human Pose and Shape Estimation	NeurIPS
2023	Learning Neural Parametric Head Models [56]	CVPR
2023	HumanRF: High-Fidelity Neural Radiance Fields for Humans in Motion [39]	ICCV
2023	InstantAvatar: Learning Avatars from Monocular Video in 60 Seconds [38]	CVPR
2023	3D Gaussian Splatting for Real-Time Radiance Field Rendering [70]	SIGGRAPH
2023	Gaussian Avatars: Photorealistic Head Avatars with Rigged 3D Gaussians [71]	arXiv
2023	Segment Anything [94]	ICCV
2023	Reconstructing Hands in 3D with Transformers [74]	CVPR

年份	论文	发表
2023	DreamFusion: Text-to-3D using 2D Diffusion [97]	ICLR
2023	Score-Based Generative Modeling through Stochastic Differential Equations	ICLR
2023	HOLD: Category-agnostic 3D Reconstruction of Interacting Hands and Objects from Video [83]	ICCV
2023	SDFusion: Multimodal 3D Shape Completion, Reconstruction, and Generation	CVPR
2023	EMOTE: Emotional Speech-Driven Animation with Content-Emotion Disentanglement [63]	ICCV
2023	MagicPony: Learning Articulated 3D Animals in the Wild [90]	ICCV
2023	FlashAvatar: High-fidelity Head Avatar with Efficient Gaussian Embedding [72]	arXiv
2023	I M Avatar: Implicit Morphable Head Avatars from Videos	CVPR
2023	GeneFace: Generalized and High-Fidelity Audio-Driven 3D Talking Face Synthesis [65]	ICLR
2023	Metric3D: Towards Zero-shot Metric 3D Prediction from A Single Image	ICCV
2023	SadTalker: Learning Realistic 3D Motion Coefficients for Stylized Audio-Driven Single Image Talking Face Animation [64]	CVPR
2024	Multi-HMR: Multi-Person Whole-Body Human Mesh Recovery in a Single Shot [29]	CVPR
2024	Hand Pose Estimation: A Survey	CVPR
2024	CameraHMR: Aligning People with Perspective [30]	CVPR
2024	MANUS: Markerless Hand-Object Grasp Capture using Articulated 3D Gaussians [86]	CVPR
2024	MonoNPHM: Dynamic Head Reconstruction from Monocular Videos [57]	CVPR
2024	MotionLLM: Understanding Human Behaviors from Human Motions and Videos	arXiv
2024	SplattingAvatar: Realistic Real-Time Human Avatars with Mesh-Embedded Gaussian Splatting [40]	CVPR
2024	DINOv2: Learning Robust Visual Features without Supervision [95]	TMLR

年份	论文	发表
2024	Media2Face: Co-speech Facial Animation Generation With Multi-Modality Guidance [66]	CVPR
2024	GaussianAvatars: Photorealistic Head Avatars with Rigged 3D Gaussians [41]	CVPR
2024	WHAM: Reconstructing World-grounded Humans with Accurate 3D Motion [26]	CVPR
2024	Human Motion Generation: A Survey	arXiv
2024	TRAM: Global Trajectory and Motion of 3D Humans from in-the-wild Videos [27]	ECCV
2025	MHR: Momentum Human Rig [20]	arXiv
2025	ATLAS: Decoupling Skeletal and Shape Parameters for Expressive Parametric Human Modeling [19]	arXiv

C.3 一条推荐的精读路线（人体）

1. Allen 2003：形状空间；
2. Anguelov 2005：SCAPE，形状/姿态分离；
3. Loper 2015：SMPL；
4. Bogo 2016：SMPLify；
5. Kanazawa 2018：HMR；
6. Kolotouros 2019：SPIN；
7. Pavlakos 2019：SMPL-X；
8. Mahmood 2019：AMASS；
9. Kocabas 2020：VIBE；
10. 2021 之后的神经渲染与生成模型。

C.4 一条推荐的精读路线（人脸）

1. Blanz & Vetter 1999：3DMM；
2. Paysan 2009：BFM；
3. Li 2017：FLAME；
4. Tewari 2017 / Deng 2019：拟合与弱监督；
5. Feng 2021：DECA；

6. Feng 2022 / Zhu 2020：FaceVerse / FaceScape；
7. Garbin 2023：NPHM；
8. 2023 之后的 3DGS 头像。

C.5 一条推荐的精读路线（人手）

1. Romero 2017：MANO；
2. Boukhayma 2019：单图手部；
3. Hasson 2019：手物联合；
4. Rong 2021：FrankMocap；
5. Pavlakos 2023：HaMeR；
6. 2024 之后的高斯手物。

C.6 一条推荐的精读路线（动物）

1. Zuffi 2017：SMAL；
2. Zuffi 2019：Three-D Safari；
3. Biggs 2020：Dogs Out；
4. Biggs 2022：BARC；
5. Wu 2023：MagicPony；
6. 2024 之后的 AniMer / LiveStock。

附录 D 参考文献

下列条目按正文首次引用的顺序编号。

[1] Timothy F. Cootes, Christopher J. Taylor, David H. Cooper, et al. *Active Shape Models-Their Training and Application*. CVIU, 1995.

[2] Volker Blanz, Thomas Vetter. *A Morphable Model For The Synthesis Of 3D Faces*. SIGGRAPH, 1999.

[3] *The Space of Human Body Shapes: Reconstruction and Parameterization from Range Scans*. SIGGRAPH, 2003.

[4] Dragomir Anguelov, Praveen Srinivasan, Daphne Koller, et al. *SCAPE: Shape Completion and Animation of People*. SIGGRAPH, 2005.

[5] Matthew Loper, Naureen Mahmood, Javier Romero, et al. *SMPL: A Skinned Multi-Person Linear Model*. SIGGRAPH Asia, 2015.

[6] Bogo, Federica, Kanazawa, Angjoo, Lassner, Christoph, et al. *Keep It SMPL: Automatic Estimation of 3D Human Pose and Shape from a Single Image*. ECCV, 2016. arXiv:1607.08128

[7] Kanazawa, Angjoo, Black, Michael J., Jacobs, David W., et al. *End-to-end Recovery of Human Shape and Pose*. CVPR, 2018. arXiv:1712.06584

[8] *Learning to Reconstruct 3D Human Pose and Shape via Model-fitting in the Loop*. ICCV, 2019. arXiv:1906.05664

[9] *SMALST: Deep Learning Approach to Non-rigid 3D Shape Estimation*. arXiv, 2019.

[10] *Joint-Dependent Local Deformations for Hand Animation and Object Grasping*. Book Chapter, 1988.

[11] Ladislav Kavan, Steven Collins, Jiří Žára, et al. *Geometric Skinning with Approximate Dual Quaternion Blending*. ACM TOG, 2008. DOI:10.1145/1409625.1409627

[12] *Pose Space Deformation: A Unified Approach to Shape Interpolation and Skeleton-Driven Deformation*. SIGGRAPH, 2000.

[13] *Statistical Models of Appearance for Computer Vision*. BMVC, 2001.

[14] *A Statistical Model of Human Pose and Body Shape*. Eurographics, 2009.

[15] Federica Bogo, Javier Romero, Matthew Loper, et al. *FAUST: Dataset and Evaluation for 3D Mesh Registration*. CVPR, 2014.

[16] Javier Romero, Dimitrios Tzionas, Michael J. Black. *Embodied Hands: Modeling and Capturing Hands and Bodies Together*. SIGGRAPH Asia, 2017. arXiv:2201.02610v1

[17] *STAR: Sparse Trained Articulated Human Body Regressor*. ECCV, 2020. arXiv:2008.02616

[18] *GHUM & GHUML: Generative 3D Human Shape and Articulated Pose Models*. 3DV, 2020.

[19] Jinhyung Park, Javier Romero, Shunsuke Saito, et al. *ATLAS: Decoupling Skeletal and Shape Parameters for Expressive Parametric Human Modeling*. arXiv, 2025. arXiv:2508.15767

[20] Aaron Ferguson, Ahmed A. A. Osman, Berta Bescos, et al. *MHR: Momentum Human Rig*. arXiv, 2025. arXiv:2511.15586

[21] Pavlakos, Georgios, Choutas, Vasileios, Ghorbani, Nima, et al. *Expressive Body Capture: 3D Hands, Face, and Body from a Single Image*. CVPR, 2019. arXiv:1904.05866

[22] *Convolutional Mesh Regression for Single-Image Human Shape Reconstruction*. ICCV, 2019. arXiv:1906.05260

[23] Kocabas, Muhammed, Athanasiou, Nikos, Black, Michael J. *VIBE: Video Inference for Human Body Pose and Shape Estimation*. CVPR, 2020. arXiv:1912.05656

[24] *Humans in 4D: Reconstructing and Tracking Humans with Transformers*. ICCV, 2023.

[25] *HMR 2.0: A Strategy For Decoding Non-Linear Structure in Human Modeling*. arXiv, 2024.

[26] Shin, Soyong, Kim, Juyong, Halilaj, Eni, et al. *WHAM: Reconstructing World-grounded Humans with Accurate 3D Motion*. CVPR, 2024. arXiv:2312.07531

[27] Yufu Wang, Ziyun Wang, Lingjie Liu, et al. *TRAM: Global Trajectory and Motion of 3D Humans from in-the-wild Videos*. ECCV, 2024. arXiv:2403.17346v2

[28] *World-Grounded Human Motion Recovery via Gravity-View Coordinates*. arXiv, 2024.

[29] Baradel, Fabien, Armando, Matthieu, Galaaoui, Salma, et al. *Multi-HMR: Multi-Person Whole-Body Human Mesh Recovery in a Single Shot*. CVPR, 2024. arXiv:2402.14654

[30] Patel, Priyanka, Black, Michael J. *CameraHMR: Aligning People with Perspective*. CVPR, 2024. arXiv:2411.08128

[31] *NLF: Neural Local Fields for Metric-Scale Human Reconstruction*. ECCV, 2024.

[32] Mahmood, Naureen, Ghorbani, Nima, Troje, Nikolaus F., et al. *AMASS: Archive of Motion Capture as Surface Shapes*. ICCV, 2019. arXiv:1904.03278

[33] Ma, Qianli, Yang, Jinlong, Ranjan, Anurag, et al. *Learning to Dress 3D People in Generative Clothing*. CVPR, 2020. arXiv:1907.13615

[34] Alldieck, Thiemo, Pons-Moll, Gerard, Theobalt, Christian, et al. *Tex2Shape: Detailed Full Human Body Geometry from a Single Image*. ICCV, 2019. arXiv:1904.08645

[35] *PIFu: Pixel-Aligned Implicit Function for High-Resolution Clothed Human Digitization*. ICCV, 2019.

[36] *SNARF: Differentiable Forward Skinning for Animating Non-Rigid Neural Implicit Shapes*. ICCV, 2021.

[37] *HumanNeRF: Free-viewpoint Rendering of Moving People from Monocular Video*. CVPR, 2022. arXiv:2201.00506

[38] Jiang, Tianjian, Chen, Xu, Song, Jie, et al. *InstantAvatar: Learning Avatars from Monocular Video in 60 Seconds*. CVPR, 2023. arXiv:2212.10550

[39] Mustafa Işık, Martin Rünz, Markos Georgopoulos, et al. *HumanRF: High-Fidelity Neural Radiance Fields for Humans in Motion*. ICCV, 2023. arXiv:2305.06356v2

[40] Shao, Zhijing, Wang, Zhaolong, Li, Zhuang, et al. *SplattingAvatar: Realistic Real-Time Human Avatars with Mesh-Embedded Gaussian Splatting*. CVPR, 2024. arXiv:2403.05087

[41] Qian, Shenhan, Kirschstein, Tobias, Schoneveld, Liam, et al. *GaussianAvatars: Photorealistic Head Avatars with Rigged 3D Gaussians*. CVPR, 2024. arXiv:2312.02069

[42] Rempe, Davis, Birdal, Tolga, Hertzmann, Aaron, et al. *HuMoR: 3D Human Motion Model for Robust Pose Estimation*. ICCV, 2021. arXiv:2105.04668

[43] Tevet, Guy, Raab, Sigal, Gordon, Brian, et al. *Human Motion Diffusion Model*. ICLR, 2022. arXiv:2209.14916

[44] Zhang, Mingyuan, Cai, Zhongang, Pan, Liang, et al. *MotionDiffuse: Text-Driven Human Motion Generation with Diffusion Model*. AAAI, 2022. arXiv:2208.15001

[45] Bernhard Egger, William A. P. Smith, Ayush Tewari, et al. *3D Morphable Face Models -- Past, Present and Future*. IJCV, 2020. arXiv:1909.01815v2

[46] Tianye Li, Timo Bolkart, Michael J. Black, et al. *Learning a model of facial shape and expression from 4D scans*. SIGGRAPH Asia, 2017.

[47] Pascal Paysan, Reinhard Knothe, Brian Amberg, et al. *A 3D Face Model for Pose and Illumination Invariant Face Recognition*. 3DPVT, 2009.

[48] Daniel Vlasic, Matthew Brand, Hanspeter Pfister, et al. *Face Transfer with Multilinear Models*. SIGGRAPH, 2005.

[49] Haotian Yang, Hao Zhu, Yanru Wang, et al. *FaceScape: A Large-Scale High Quality 3D Face Dataset and Detailed Riggable 3D Face Prediction*. TPAMI, 2020. arXiv:2011.12438

[50] Tewari, Ayush, Zollhöfer, Michael, Kim, Hyeongwoo, et al. *MoFA: Model-based Deep Convolutional Face Autoencoder for Unsupervised Monocular Reconstruction*. CVPR, 2017. arXiv:1703.10590

[51] Gecer, Baris, Ploumpis, Stylianos, Kotsia, Irene, et al. *GANFIT: Generative Adversarial Network Fitting for High Fidelity 3D Face Reconstruction*. CVPR, 2019. arXiv:1902.05978

[52] James Booth, Epameinondas Antonakos, Stylianos Ploumpis, et al. *3D Face Morphable Models "In-the-Wild"*. ECCV, 2018. arXiv:1701.05360v1

[53] Feng, Yao, Feng, Haiwen, Black, Michael J., et al. *Learning an Animatable Detailed 3D Face Model from In-The-Wild Images*. SIGGRAPH, 2021. arXiv:2012.04012

[54] Radek Daněček, Michael J. Black, Timo Bolkart. *EMOCA: Emotion Driven Monocular Face Capture and Animation*. CVPR, 2022. arXiv:2204.01050

[55] George Retsinas, Panagiotis P. Filntisis, Radek Danecek, et al. *SMIRK: 3D Facial Meshes at Scale*. ICCV, 2023.

[56] Simon Giebenhain, Tobias Kirschstein, Markos Georgopoulos, et al. *Learning Neural Parametric Head Models*. CVPR, 2023. arXiv:2212.02761v2

[57] Simon Giebenhain, Tobias Kirschstein, Markos Georgopoulos, et al. *MonoNPHM: Dynamic Head Reconstruction from Monocular Videos*. CVPR, 2024. arXiv:2312.06740v2

[58] Lizhen Wang, Zhiyuan Chen, Tao Yu, et al. *FaceVerse: a Fine-grained and Detail-controllable 3D Face Morphable Model from a Hybrid Dataset*. CVPR, 2022. arXiv:2203.14057

[59] Alexandros Lattas, Stylianos Moschoglou, Baris Gecer, et al. *AvatarMe: Realistically Renderable 3D Facial Reconstruction "in-the-wild"*. CVPR, 2020. arXiv:2004.04534

[60] Alexandros Lattas, Stylianos Moschoglou, Stylianos Ploumpis, et al. *AvatarMe++: Facial Shape and BRDF Inference with Photorealistic Rendering-Aware GANs*. TPAMI, 2021.

[61] *MICA: Towards Metrically Accurate 3D Face Reconstruction*. ECCV, 2022.

[62] *SPECTRE: Visual Speech-Aware Perceptual 3D Facial Expression Reconstruction from Videos*. ECCV, 2022.

[63] *EMOTE: Emotional Speech-Driven Animation with Content-Emotion Disentanglement*. ICCV, 2023. arXiv:2306.08990

[64] Wenxuan Zhang, Xiaodong Cun, Xuan Wang, et al. *SadTalker: Learning Realistic 3D Motion Coefficients for Stylized Audio-Driven Single Image Talking Face Animation*. CVPR, 2023.

[65] Zhenhui Ye, Ziyue Jiang, Yi Ren, et al. *GeneFace: Generalized and High-Fidelity Audio-Driven 3D Talking Face Synthesis*. ICLR, 2023.

[66] Qingcheng Zhao, Pengyu Long, Qixuan Zhang, et al. *Media2Face: Co-speech Facial Animation Generation With Multi-Modality Guidance*. CVPR, 2024. arXiv:2401.15687v2

[67] Justus Thies, Michael Zollhöfer, Marc Stamminger, et al. *Face2Face: Real-time Face Capture and Reenactment of RGB Videos*. CVPR, 2016.

[68] Hyeongwoo Kim, Pablo Garrido, Ayush Tewari, et al. *Deep Video Portraits*. CVPR, 2018.

[69] Tewari, Ayush, Elgharib, Mohamed, Bharaj, Gaurav, et al. *StyleRig: Rigging StyleGAN for 3D Control over Portrait Images*. CVPR, 2020. arXiv:2004.00121

[70] Kerbl, Bernhard, Kopanas, Georgios, Leimkühler, Thomas, et al. *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. SIGGRAPH, 2023. arXiv:2308.04079

[71] Shenhan Qian, Tobias Kirschstein, Liam Schoneveld, et al. *Gaussian Avatars: Photorealistic Head Avatars with Rigged 3D Gaussians*. arXiv, 2023.

[72] Jun Xiang, Xuan Gao, Yudong Guo, et al. *FlashAvatar: High-fidelity Head Avatar with Efficient Gaussian Embedding*. arXiv, 2023. arXiv:2312.02214v2

[73] Boukhayma, Adnane, de Bem, Rodrigo, Torr, Philip H. S. *3D Hand Shape and Pose From Images in the Wild*. CVPR, 2019. arXiv:1902.03451

[74] Pavlakos, Georgios, Shan, Dandan, Radosavovic, Ilija, et al. *Reconstructing Hands in 3D with Transformers*. CVPR, 2023. arXiv:2312.05251

[75] Yu-Wei Chao, Wei Yang, Yu Xiang, et al. *DexYCB: A Benchmark for Capturing Hand Grasping of Objects*. CVPR, 2021. arXiv:2104.04631v1

[76] Yu Rong, Takaaki Shiratori, Kripasindhu Sarkar, et al. *FrankMocap: A Monocular 3D Whole-Body Pose Estimation System via Regression and Integration*. ICCV, 2021. arXiv:2010.08244

[77] Gyeongsik Moon, Kyoung Mu Lee. *I2L-MeshNet: Image-to-Lixel Prediction Network for Accurate 3D Human Pose and Mesh Estimation from a Single RGB Image*. ECCV, 2020. arXiv:2008.03713v2

[78] Kevin Lin, Lijuan Wang, Zicheng Liu. *Mesh Graphormer*. ICCV, 2021. arXiv:2104.00272v2

[79] Zhang, Hongwen, Tian, Yating, Zhou, Xinchi, et al. *PyMAF: 3D Human Pose and Shape Regression with Pyramidal Mesh Alignment Feedback Loop*. ICCV, 2021. arXiv:2103.16507

[80] Shreyas Hampali, Mahdi Rad, Markus Oberweger, et al. *HOnnotate: A method for 3D Annotation of Hand and Object Poses*. CVPR, 2020. arXiv:1907.01481v6

[81] Hasson, Yana, Varol, Gül, Tzionas, Dimitrios, et al. *Learning Joint Reconstruction of Hands and Manipulated Objects*. CVPR, 2019. arXiv:1904.05767

[82] Omid Taheri, Nima Ghorbani, Michael J. Black, et al. *GRAB: A Dataset of Whole-Body Human Grasping of Objects*. CVPR, 2020. arXiv:2008.11200v1

[83] Zicong Fan, Maria Parelli, Maria Eleni Kadoglou, et al. *HOLD: Category-agnostic 3D Reconstruction of Interacting Hands and Objects from Video*. ICCV, 2023. arXiv:2311.18448v1

[84] *GANHand: Predicting Human Grasp Affordances in Multi-Object Scenes*. CVPR, 2020.

[85] Kailin Li, Lixin Yang, Xinyu Zhan, et al. *ArtiBoost: Boosting Articulated 3D Hand-Object Pose Estimation via Online Exploration and Synthesis*. CVPR, 2022. arXiv:2109.05488v2

[86] *MANUS: Markerless Hand-Object Grasp Capture using Articulated 3D Gaussians*. CVPR, 2024. arXiv:2312.02137

[87] Zuffi, Silvia, Kanazawa, Angjoo, Jacobs, David, et al. *3D Menagerie: Modeling the 3D Shape and Pose of Animals*. CVPR, 2017. arXiv:1611.07700

[88] *Who Left the Dogs Out? 3D Animal Reconstruction with Expectation Maximization in the Loop*. ECCV, 2020. arXiv:2004.12989

[89] *BARC: Breed-Augmented Regression Using Classification for 3D Dog Reconstruction from Images*. ECCV, 2022.

[90] Wu, Shangzhe, Li, Ruining, Jakab, Tomas, et al. *MagicPony: Learning Articulated 3D Animals in the Wild*. ICCV, 2023. arXiv:2303.16160

[91] Ng, Xun Long, Ong, Kian Eng, Zheng, Qichen, et al. *Animal Kingdom: A Large and Diverse Dataset for Animal Behavior Understanding*. ECCV, 2022. arXiv:2205.05677

[92] *3D-Fauna: A Dataset and Benchmark for 3D Animal Reconstruction*. ICCV, 2023.

[93] Mildenhall, Ben, Srinivasan, Pratul P., Tancik, Matthew, et al. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV, 2020. arXiv:2003.08934

[94] Kirillov, Alexander, Mintun, Eric, Ravi, Nikhila, et al. *Segment Anything*. ICCV, 2023. arXiv:2304.02643

[95] Oquab, Maxime, Darcet, Timothée, Moutakanni, Théo, et al. *DINOv2: Learning Robust Visual Features without Supervision*. TMLR, 2024. arXiv:2304.07193

[96] *Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data*. CVPR, 2024. arXiv:2401.04398

[97] Poole, Ben, Jain, Ajay, Barron, Jonathan T., et al. *DreamFusion: Text-to-3D using 2D Diffusion*. ICLR, 2023. arXiv:2209.14988

后 记

写这本书的念头，来自一次很具体的挫败：想给刚进实验室的学生讲清楚 SMPL 的公式，却发现找不到一份“从一列数字讲起”的材料——要么是论文里三行公式带过，要么是博客停在“10 个形状参数、72 个姿态参数”这一层。于是就有了把这条路重走一遍的打算。

写下来之后才发现，参数化技术真正的门槛不在公式本身，而在因果：SCAPE 为什么要把形变放在三角形上，SMPL 为什么愿意用会“捏细”的 LBS 去换实时，SMPL-X 为什么把 90 维的手指压成 24 维，SMAL 为什么用玩具动物而不是真动物。这些“为什么”串起来，才是一条能自己往下走的线。所以全书尽量按“问题 → 旧方法 → 局限 → 新方法 → 新局限”的顺序写，而不是按论文列表写。

书中所有插图都由程序生成，代码在仓库里，可以逐张改参数重画；所有公式都用 KaTeX 排版，网页上可以直接读，也可以下载 PDF。写作过程中反复提醒自己两件事：不写没核对过的数字，不写自己没想明白的比喻。 如果仍有错漏，那一定是我核对得不够。

这本书不会替读者读论文。它想做的是把那道门槛降低一点：读完第 1–6 章，你能看懂 PCA 与蒙皮；读完第 7–14 章，你能判断一篇新的人体重建论文改了什么；读完第 15–24 章，你能把同一套方法迁移到脸、手和动物上。

技术在往前跑。2020 年之后，神经渲染、扩散模型、视觉基础模型又换了一轮底座，但“用少量参数描述高维对象”这件事没有变。愿这本书在你换方向、换任务、换模型的时候，还能剩下一点可迁移的东西。

Dr. Dezeming

2026 年，于实验室

ABOUT THE BOOK

四类模型，一套方法。

从 1995 年的统计形状模型讲到今天的扩散模型与基础模型：先用平均形状 + 形变基 + 骨架蒙皮把「人」压缩成几十个参数，再用可微渲染把它和图像连起来。人体、人脸、人手、动物四条主线共享同一套数学，只是换了对象与关节。

第一篇　地基　向量与最小二乘 · 概率与 PCA · 三维表示与旋转 · LBS · ASM → 3DMM

第二篇　人体　SCAPE · SMPL 逐项拆公式 · SMPL 家族 · 单图与视频重建 · 着装与动作生成

第三篇　人脸　3DMM 深入与 Basel Face Model · FLAME · 高保真细节与纹理 · 驱动与神经头像

第四篇　人手　手部解剖与 MANO · 单手重建 · 手物交互与接触

第五篇　动物　SMAL 与玩具扫描 · 动物重建四代方法 · 鸟类 / 鱼类与跨物种生成

第六篇　前沿与实战　扩散与基础模型 · 可运行的 PCA / LBS / 拟合 demo · 研究脉络

> 好的模型不是参数越多越好，而是把不该纠缠的因素分开。
>
> —— 第 7 章　SCAPE

Dr. Dezeming

2026 年 · 第一版　·　27 章 + 4 附录